Huawei HCIE Certification Training

HCIE-openGauss

Lab Guide

Issue: 1.0



Huawei Technologies Co., Ltd.

Copyright © Huawei Technologies Co., Ltd. 2024. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions


and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address:	Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China
Website:	https://e.huawei.com

Huawei Certification System

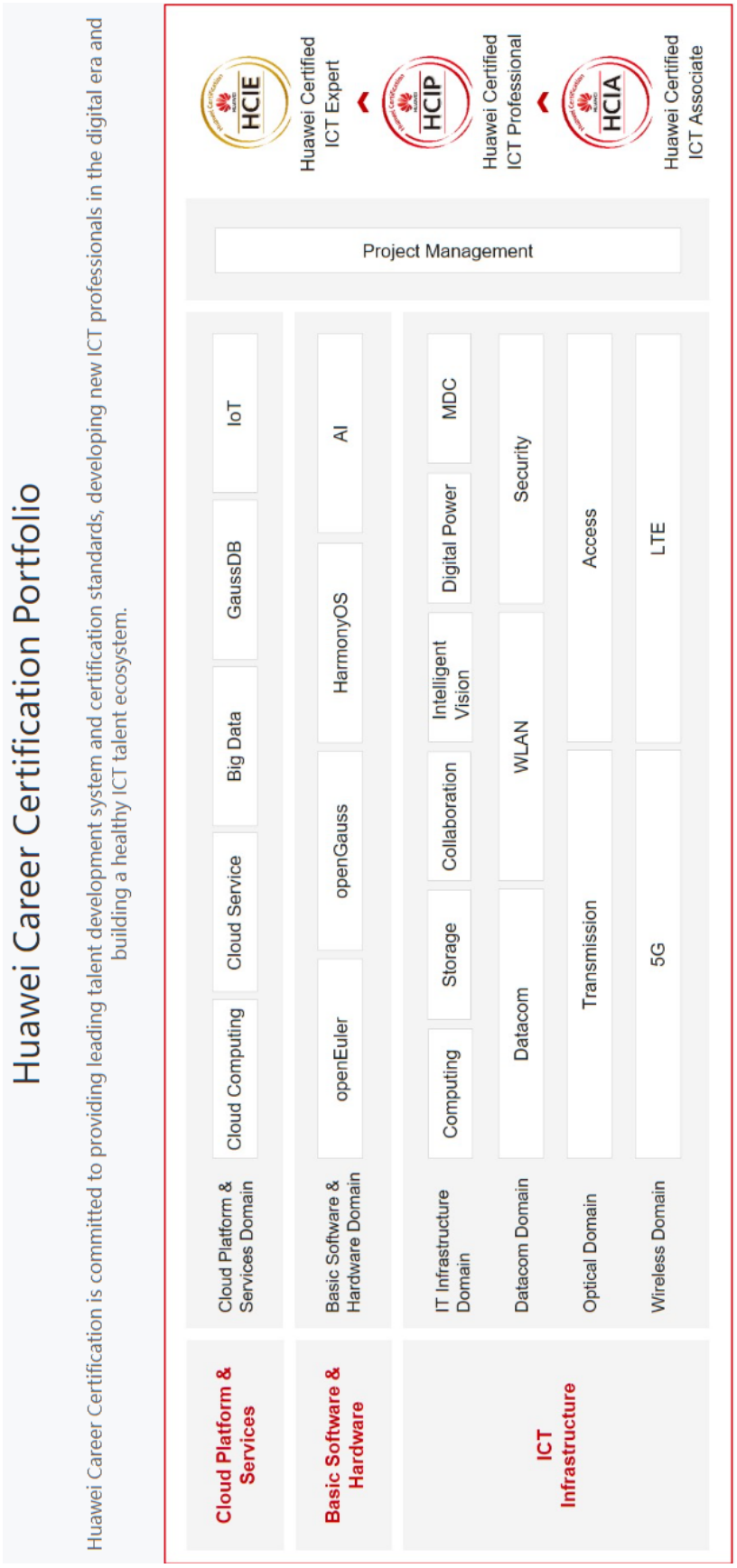
Huawei Certification is an integral part of the company's Platform + Ecosystem strategy. It supports the development of ICT infrastructure that features Cloud-Pipe-Device synergy. Our certification is always evolving to reflect the latest trends in ICT development. Huawei Certification consists of three categories: ICT Infrastructure Certification, Basic Software & Hardware Certification, and Cloud Platform & Services Certification, making it the most extensive technical certification program in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

In keeping with the ICT convergence trend, Huawei Certification covers all ICT fields. Through its leading talent development system and certification standards, Huawei is committed to cultivating new ICT professionals in the digital era and building a healthy ICT talent ecosystem.

HCIE-openGauss certification is particular for database administrators (DBAs), senior database developers, senior database O&M engineers, and other people who want to learn database technologies and have basic knowledge. The certification covers transaction management and lock mechanism, storage engines, log management, SQL engine, database performance optimization, backup, restoration, DR, advanced features, and troubleshooting.

The Huawei certification system introduces the industry, fosters innovation, and imparts cutting-edge database knowledge.


Huawei Career Certification Portfolio
Huawei Career Certification is committed to providing leading talent development system and certification standards, developing new ICT professionals in the digital era and building a healthy ICT talent ecosystem.
HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate
Project Management
Cloud Platform & Services
Basic Software & Hardware
ICT Infrastructure
Cloud Platform & Services Domain
Cloud Computing
Cloud Service
Big Data
GaussDB
IoT
Basic Software & Hardware Domain
openEuler
openGauss
HarmonyOS
AI
IT Infrastructure Domain
Computing
Storage
Collaboration
Intelligent Vision
Digital Power
MDC
Datacom Domain
Datacom
WLAN
Security
Optical Domain
Transmission
Access
Wireless Domain
5G
LTE

About This Document

Overview

This document is designed for the HCIE-openGauss V1.0 certification training. It is intended to help trainees prepare for the HCIE-openGauss V1.0 exam and for those interested in understanding working principles of database transactions and lock mechanisms, storage engine principles, log types and working principles, SQL engine's working principles and performance optimization methods, backup and restoration, DR methods and principles, as well as advanced features and troubleshooting related to database technologies.

Description

This document consists of eight labs, including transaction management and lock, data file storage, log file management, SQL engine, database performance optimization, backup, restoration, and DR, advanced feature implementation, and troubleshooting.

- Lab 1: transaction management and lock mechanism. It helps you understand the working principles of transactions and locks through transaction consistency exercise, subtransaction exercise, and lock reproduction.
- Lab 2: storage management. By compiling a database and installing the pageinspect plug-in to view and analyze the data page structure, you can understand the underlying logic and principles of data storage.
- Lab 3: log management. It focuses on the composition of WAL files and aims to help you understand the composition and working process of WAL files.
- Lab 4: SQL engine. By analyzing the execution processes of different SQL statements, you can master the working principles of different operators and independently interpret SQL execution plans.
- Lab 5: database performance optimization. Through dynamic and static database optimization, you can master common database optimization methods and independently optimize databases in different scenarios.
- Lab 6: backup, restoration, and DR. This lab uses backup and restoration tools, PITR, and flashback to help you master the operations and principles of backup, restoration, and DR.
- Lab 7: advanced features, including AI capabilities, fully-encrypted databases, and MOTs. It helps you have a deeper understanding of the advanced openGauss features.
- Lab 8: troubleshooting, aiming to solve common problems or faults.

Background Knowledge Required

This course is for HCIE certification. To better understand this course, you need to have:

- Database development and O&M knowledge and basic theoretical knowledge of openGauss.
- Ability to compile common shell commands.
- Common knowledge of Linux or Linux-like OSs.
- Knowledge of HCIP-openGauss certification courses.

Lab Environment

Networking Description

This lab environment is intended for database engineers preparing for the HCIE-openGauss V1.0 exam. You need to purchase an Elastic Cloud Server (ECS) on Huawei Cloud and set up an openGauss database on the ECS. Each lab environment consists of one elastic IP address (EIP), one ECS server, openEuler 20.03LST, openGauss V5.0.0, and one gsql Client (for connecting to the database).

In this lab environment, standalone openGauss V5.0.0 (enterprise edition) is used.

To meet the HCIE-openGauss lab requirements, you are advised to use the following configurations in each lab environment:

The mapping between the device, model, and version is as follows.

Device	Model	Software Version
ECS	8 vCPUs | 32 GiB	-
openGauss	-	V5.0.0
openEuler	-	V20.03
EIP	-	-

Contents

About This Document 3
Overview 3
Description 3
Background Knowledge Required 4
Lab Environment 4
1 Transaction Management and Lock Mechanism 1
1.1 Introduction 1
1.1.1 About This Lab 1
1.1.2 Objectives 1
1.2 General Transaction Features 1
1.2.1 Automatic Transaction Commit 1
1.2.2 Transaction Isolation 2
1.2.3 Transaction Consistency 3
1.3 Transaction Isolation Levels 4
1.3.1 READ COMMITTED 5
1.3.2 REPEATABLE READ 6
1.4 Subtransactions 8
1.4.1 BEGIN...END 8
1.4.2 BEGIN...SAVEPOINT...COMMIT/ROLLBACK 8
1.5 Viewing Lock Information 10
1.6 Reproducing a Shared Lock 11
1.7 Reproducing ACCESS SHARE Locks 12
1.8 Reproducing a ROW EXCLUSIVE Lock 14
1.9 Reproducing an ACCESS EXCLUSIVE Lock 15
2 Storage Management 17
2.1 Introduction 17
2.1.1 About This Lab 17
2.1.2 Objectives 17
2.2 Compiling openGauss 17
2.2.1 Lab Description 17
2.2.2 Compiling a Database 18
2.2.3 Starting openGauss 22
2.3 Installing the pageinspect Plug-in 24
2.4 Using the pageinspect Plug-in to Analyze the Page Structure of a Table 25

2.4.1 Viewing the Page Header Structure of a Table 26
2.4.2 Viewing Tuple Data on a Page 28
2.4.3 Viewing Original Data on a Page 28
3 Log Management 29
3.1 Introduction 29
3.1.1 About This Lab 29
3.1.2 Objectives 29
3.2 Lab Tasks 29
3.2.1 Task 1: Viewing WAL Files 30
3.2.2 Task 2: Parsing WAL Files 34
3.2.3 Task 3: Using WALs to Automatically Restore the Database After It Stops Abnormally 41
3.2.4 Task 4: Using WALs to Implement PITR for Databases 49
3.2.5 Task 5: Checking and Clearing System Logs 54
4 SQL Execution Engine 62
4.1 Introduction 62
4.1.1 About This Lab 62
4.1.2 Objectives 62
4.2 Scan Operator Execution Plan 62
4.2.1 Seq Scan Operators 62
4.2.2 Index Scan Operators 64
4.2.3 Subquery Scan Operators 66
4.2.4 TID Scan Operators 68
4.2.5 Function Scan Operators 68
4.3 Join Operator Execution Plan 69
4.3.1 Hash Join Operators 69
4.3.2 Merge Join Operators 71
4.3.3 Nested Loop Join Operators 73
4.4 Control Operator Execution Plan 77
4.4.1 Result Operators 77
4.4.2 Limit Operators 78
4.4.3 Append Operators 78
4.5 Materialization Operator Execution Plan 79
4.5.1 Sort Operators 79
4.5.2 Hash Operators 80
4.5.3 Unique Operators 81
4.5.4 Aggregate Operators 82
4.5.5 Group Operators 82
4.5.6 SetOp Operators 83

5 Database Optimization 85
5.1 Introduction 85
5.1.1 About This Lab 85
5.1.2 Objectives 85
5.2 Querying the Server and openGauss Database Parameters 85
5.2.1 Checking the Server Hardware Architecture 85
5.2.2 Checking the CPU Load 86
5.2.3 Checking the Memory 88
5.2.4 Checking the OS Version 88
5.2.5 Checking the Network Status 88
5.2.6 Checking the I/O Status 89
5.2.7 Querying Key Parameters of the openGauss Database 90
5.3 Vectorized Executor 91
5.3.1 Lab Description 91
5.3.2 Optimizing SQL Queries Using a Vectorized Executor 91
5.4 SMP-related Experiments 93
5.4.1 Lab Description 93
5.4.2 Using SMP to Optimize SQL Queries 94
5.5 GUC Parameter Setting and Plan Hints 97
5.5.1 Lab Description 97
5.5.2 Printing an Execution Plan in Different Formats 97
5.5.3 Printing an Execution Plan in Rollback Mode 104
5.5.4 Setting GUC Parameters to Optimize an Execution Plan 108
5.5.5 Using Plan Hints on an Execution Plan 114
5.5.6 Poor Plan Hints Increase the Time Consumption 115
5.6 Optimizing openGauss Indexes 116
5.6.1 Lab Description 116
5.6.2 Basic Usage of Indexes 116
5.6.3 Selecting a Proper Column as an Index 117
5.6.4 Selecting a Proper Index Column Type 119
5.6.5 Dead Tuple Problem During Index Scans 120
5.6.6 Invalid Indexes 122
5.7 Optimizing Table Structures 124
5.7.1 Lab Description 124
5.7.2 Selecting Proper Storage Types 125
5.7.3 Selecting Proper Partition Types 127
5.7.4 Selecting Proper Partitioned Table Indexes 129
5.7.5 Properly Planning Partitions 133
5.8 CBO 135

5.8.1 Lab Description 135
5.8.2 Identifying and Updating Statistics 135
5.8.3 Modifying Cost Parameters 137
5.9 Diagnosing SQL Performance 139
5.9.1 Lab Description 139
5.9.2 Using WDRs 139
5.10 Optimizing Subqueries 142
5.10.1 Lab Description 142
5.10.2 Correlated Subqueries and Non-correlated Subqueries 143
5.10.3 Using WITH to Rewrite and Optimize Subqueries 145
5.10.4 Rewriting and Improving Subqueries 148
5.11 SQL Rewriting 151
5.11.1 Lab Description 151
5.11.2 Replace UNION with UNION ALL. 151
6 Backup and Restoration 153
6.1 Introduction 153
6.1.1 About This Lab 153
6.1.2 Objectives 153
6.2 Logical Backup and Restoration 153
6.2.1 Using gs_dump to Export Data from the **postgres** Database 153
6.2.2 Using gs_dumpall to Export Data from the **postgres** Database 154
6.2.3 Exporting the **employees** Table in the Public Schema from the **postgres** Database and Saving It to an SQL Text File 155
6.2.4 Using gsql and gs_restore to Perform Logical Restoration 157
6.3 Physical Backup and Restoration 159
6.3.1 Using gs_backup to Back Up Parameter Files 159
6.3.2 Full Backup Using gs_basebackup 160
6.3.3 Restoring Data from Backup Files 161
6.3.4 Using gs_probackup to Perform Incremental Backup and Restoration 163
6.3.5 Point-In-Time Recovery (PITR) 167
6.4 Flashback 169
6.4.1 Flashback Query 169
6.4.2 Flashback Table 170
6.4.3 Flashback DROP/TRUNCATE 172
7 Advanced Features 175
7.1 Introduction 175
7.1.1 About This Lab 175
7.1.2 Objectives 175
7.2 AI Capabilities 175

7.2.1 Installing DBMind 176
7.2.2 Using the Native DB4AI Engine to Train a House Price Prediction Model 184
7.2.3 Using DB4AI-Snapshots to Manage Dataset Versions 188
7.3 Fully-encrypted Database 192
7.3.1 Overview 192
7.3.2 Using gsql to Operate a Fully-encrypted Database to Get Familiar with Its Applications 193
7.3.3 Creating and Executing a Function or Stored Procedure That Involves Encrypted Columns 196
7.4 MOT Management 199
7.4.1 Deleting/Creating an MOT 199
7.4.2 Creating an Index for an MOT 200
7.4.3 Converting a Disk-based Table into an MOT 201
8 Troubleshooting 205
8.1 Introduction 205
8.1.1 About This Lab 205
8.1.2 Objectives 205
8.2 Tasks 205
8.2.1 Task 1: An Error Is Reported or Threads Cannot Be Created in High Concurrency Scenarios 205
8.2.2 Task 2: Failing to Connect to and Start Up a Database Due to a Network Fault 213
8.2.3 Task 3: An Error Reporting Insufficient Space Is Reported When TPC-H Data Is Imported Using a Script 215
8.2.4 Task 4: Core Dump Occurs Due to Insufficient Disk Space During Write 220

1 Transaction Management and Lock Mechanism

1.1 Introduction

1.1.1 About This Lab

This lab provides practical exercises for Chapter 1, allowing you to gain a deeper understanding of transactions, including their definitions, features, and types.

The lab includes transaction-related and lock-related exercises. The transaction-related exercises include general transaction feature exercise, isolation-level transaction exercise, and subtransaction exercise. Lock-related exercises include reproduction of common lock types.

1.1.2 Objectives

Through exercises on transaction features, isolation levels, and subtransactions, you will gain a deeper understanding of transaction properties and working principles. In addition, reproducing common types of locks will help in independently grasping the lock principles and their behavior in different scenarios.

1.2 General Transaction Features

A transaction is an execution unit operated by the database management system. A database transaction usually includes read/write operations of a sequence on a database. When a transaction is committed, the database system needs to ensure that all operations in the transaction are successfully completed and the results are permanently retained. If some operations in the transaction are not completed, all operations in the transaction need to be rolled back to the status before the transaction is executed.

1.2.1 Automatic Transaction Commit

Step 1 Create a table.

```
openGauss=# CREATE TABLE tb1(att1 int,att2 int);
```

Step 2 Insert a data record.

```
openGauss=# insert into tb1(att1,att2) values(1,1);
```

By default, openGauss automatically starts a transaction for the preceding INSERT statement and commits the transaction after the insert operation is performed.

You can also manually control the start and commit of transactions. You can manually start a transaction in any of the following ways: (1) BEGIN; (2) BEGIN TRANSACTION; (3) BEGIN WORK;

You can end a transaction in either of the following ways: (1) COMMIT; (equivalent to COMMIT WORK or COMMIT TRANSACTION) (2) ROLLBACK;

1.2.2 Transaction Isolation

Step 1 Open **session1**, begin a transaction, insert data, and query the data.

```
openGauss=#begin;
openGauss=#delete from tb1;
openGauss=#insert into tb1(att1,att2) values(2,2);
openGauss=#select * from tb1;
att1 | att2
------+------
     2 |    2
(1 row)
```

BEGIN is used to start a transaction. After data is deleted from the table, a record is inserted. When you query the table, the record is displayed.

If you open another database connection and query data in **tb1**, you cannot see the data.

Step 2 Open **session2** and query table **tb1**.

```
openGauss=# select * from tb1;
 att1 | att2
------+------
(0 rows)
```

No data is displayed. The preceding transaction has not been committed, and the transaction isolation makes it impossible to see the uncommitted modifications of other transactions.

Step 3 Commit the transaction in **session1**.

```
openGauss=# commit;
COMMIT
```

Step 4 In **session2**, continue to query table **tb1**.

```
openGauss=# select * from tb1;
 att1 | att2
------+------
     2 |    2
(1 row)
```

After a transaction is committed, other transactions can view the record in **tb1**.

1.2.3 Transaction Consistency

This exercise uses a common scenario in our daily life to further explain the transaction consistency issue. A person wants to use electronic currency to purchase an item worth CNY10 in a store. This process involves at least two operations:

1. The payer's account is debited by CNY10.
2. The store's account is credited by CNY10.

Step 1 Create a transaction order table, insert data, and initialize the payer's account balance to CNY20 and the store's account balance to CNY500.

```
openGauss=# create table orders(name varchar(32), amount integer);
CREATE TABLE
openGauss=# insert into orders values('customer', 20);
INSERT 0 1
openGauss=# insert into orders values('shop', 500);
INSERT 0 1
openGauss=# SELECT * FROM ORDERS;
    name     | amount
----------+--------
 customer |        20
 shop        |      500
(2 rows)
```

Step 2 After initialization, begin a transaction, and simulate a transaction in which the person pays CNY10 and the store's account receives CNY10.

```
openGauss=# begin;
BEGIN
openGauss=# update orders set amount=10 where name='customer';
UPDATE 1
openGauss=# update orders set amount=510 where name='shop';
UPDATE 1
openGauss=# select * from orders;
    name     | amount
----------+--------
 customer |        10
 shop        |      510
(2 rows)
```

After the preceding operations are completed, the balances of the payer and store accounts have been updated. If the transaction is normal, commit the transaction. If the transaction is abnormal, skip step 3 and go to step 4.

Step 3 Commit the transaction.

```
openGauss=# commit;
COMMIT
```

Step 4 If an error occurs during the transaction, roll back the transaction and cancel the transaction. The transaction will be aborted, and the balances of the payer and store accounts will be restored to their values prior to the transaction.

```
openGauss=# rollback;
ROLLBACK
openGauss=# SELECT * FROM ORDERS;
    name     | amount
----------+--------
 customer |        20
 shop        |      500
```

The results show that a database system supporting transactions must ensure that updates to both account balances are completed together or canceled together. Otherwise, inconsistencies in account balances may occur, potentially leading to severe consequences.

1.3 Transaction Isolation Levels

In a database, multiple sessions may attempt to access the same data concurrently. The goal of concurrency control is to ensure that all sessions can access the data efficiently while maintaining its integrity. The database uses multi-version concurrency control (MVCC) to maintain data consistency. Each transaction sees a snapshot of data from a specific point in time. By isolating transactions for each session, inconsistencies caused by updates from concurrent transactions can be avoided.

During SQL execution in a concurrent environment, three issues must be considered:

- Dirty read: A transaction reads data written by another concurrent uncommitted transaction.
- Non-repeatable read: A transaction re-reads data it has previously read and finds that data has been modified by another committed transaction.
- Phantom read: A transaction re-executes a query returning a set of data that satisfies a search condition, only to find that the data has changed due to another committed transaction.

To avoid the preceding issues, the SQL standard defines four transaction isolation levels.

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Not possible	Possible	Possible
REPEATABLE READ	Not possible	Not possible	Possible
SERIALIZABLE	Not possible	Not possible	Not possible

openGauss supports two isolation levels: **READ COMMITTED** and **REPEATABLE READ**.

READ COMMITTED: Only committed data can be read. This is the default value.

REPEATABLE READ: Only the data committed before the transaction starts can be read. Uncommitted data or data committed in other concurrent transactions cannot be read.

1.3.1 READ COMMITTED

Step 1 Create the test table **tb2**.

```
openGauss=# CREATE TABLE tb2(att1 int,att2 int);
CREATE TABLE
```

Step 2 Dirty read.

Open two windows: **session1** and **session2**.

The default isolation level of the database system is **READ COMMITTED**. Therefore, you do not need to reset the isolation level of the database.

1. The current isolation levels of **session1** and **session2** are **READ COMMITTED**.
2. Query table **tb2** in **session1**. The query result is empty.
3. Start a transaction in **session2** and insert data.
4. Query table **tb2** in **session1**. The result is still empty.
5. Roll back the transaction in **session2**.
6. Query table **tb2** in **session1**. The query result is empty.

```
Session1:                Session2:
openGauss=# show transaction_isolation;          openGauss=# show transaction_isolation;
transaction_isolation          transaction_isolation
-----------------------                    -----------------------
read committed           read committed
(1 row)                (1 row)
openGauss=#begin;
BEGIN
openGauss=# select * from tb2;
 att1 | att2
------+------
(0 rows)

                openGauss=# begin;
                BEGIN
                openGauss=# insert into tb2 values(1,1);
                INSERT 0 1
openGauss=# select * from tb2;
 att1 | att2
------+------
(0 rows)
                openGauss=# rollback;
                ROLLBACK
openGauss=# select * from tb2;
 att1 | att2
------+------
(0 rows)
```

When a transaction runs at the **READ COMMITTED** level, a SELECT operation can view data committed before the query starts. Uncommitted data or changes made by other

concurrent transactions during query cannot be viewed. Therefore, dirty read can be avoided at this isolation level and data of uncommitted transactions cannot be read.

Step 3 Non-repeatable read.

Open two windows: **session1** and **session2**. The default isolation level of the database is **READ COMMITTED**. Therefore, you do not need to reset the isolation level of the database.

1. Query table **tb2** in **session1**. The query result is empty.
2. Start a transaction in **session2** and insert data.
3. Query table **tb2** in **session1**. The result is still empty.
4. Commit the transaction in **session2**.
5. Query table **tb2** in **session1**. You can see the data inserted in **session2**.

```
Session1:              Session2:
openGauss=# begin;
BEGIN
openGauss=# select * from tb2;
 att1 | att2
------+------
(0 rows)
                       openGauss=# begin;
                       BEGIN
                       openGauss=# insert into tb2 values(1,1);
                       INSERT 0 1
openGauss=# select * from tb2;
 att1 | att2
------+------
(0 rows)
                       openGauss=# commit;
                       COMMIT
openGauss=# select * from tb2;
 att1 | att2
------+------
    1 |    1
(1 row)
```

It can be seen that when a transaction runs at the **READ COMMITTED** level, a SELECT operation can view data that has been committed by another transaction.

1.3.2 REPEATABLE READ

Set the isolation level of each session to **REPEATABLE READ**.

Step 1 Create the test table **tb3**.

```
openGauss=# CREATE TABLE tb3(att1 int,att2 int);
CREATE TABLE
```

Step 2 Perform dirty read and non-repeatable read.

Open two windows: **session1** and **session2**. The default isolation level of the database is **READ COMMITTED**. Therefore, you do not need to reset the isolation level of the database.

1. Set the isolation levels of **session1** and **session2** to **REPEATABLE READ** and view the isolation levels.
2. Start a transaction in **session1** and query table **tb3**. The query result is empty.
3. Start a transaction in **session2** and insert data.
4. Query table **tb3** in **session1**. The result is still empty.
5. Commit the transaction in **session2**.
6. Query table **tb3** in **session1**. The query result is still empty.

```
Session1:                                          Session2:
openGauss=# set                                    tpch=# set
openGauss-# default_transaction_isolation=         openGauss-# default_transaction_isolation=
openGauss-# 'REPEATABLE READ';                     openGauss-# 'REPEATABLE READ';
openGauss=# show transaction_isolation;            openGauss=# show transaction_isolation;
transaction_isolation                              transaction_isolation
----------------------                                 ----------------------
repeatable read                                    repeatable read
(1 row)                                            (1 row)
openGauss=# begin;
BEGIN
openGauss=# select * from tb3;
 att1 | att2
------+------
(0 rows)
                                                    openGauss=# begin;
BEGIN
openGauss=# insert into tb3 values(1,1);
INSERT 0 1
openGauss=# select * from tb3;
 att1 | att2
------+------
(0 rows)
                                                   openGauss=# commit;
                                                    COMMIT
openGauss=# select * from tb3;
 att1 | att2
------+------
(0 rows)
```

It can be seen that when a transaction runs at the **REPEATABLE READ** isolation level, data query results in the transaction can always keep consist, and are not affected by commit of other transactions.

1.4 Subtransactions

1.4.1 BEGIN...END

Step 1 Create the test table **tb4**.

```
openGauss=# CREATE TABLE tb4(att1 int,att2 int);
CREATE TABLE
```

Step 2 Execute statements to start a subtransaction.

```
 BEGIN
    INSERT INTO tb4 VALUES(1,1);
    BEGIN
       INSERT INTO tb4 VALUES(2,2);
       UPDATE TEST SET att1=3, att2=3 WHERE att1=1;
    EXCEPTION
       WHEN OTHERS THEN
       RAISE INFO 'ERROR OCCURS';
    END;
    INSERT INTO tb4 VALUES(4,4);
    END;
    /
INFO:   ERROR OCCURS
ANONYMOUS BLOCK EXECUTE
```

UPDATE TEST SET att1=3, att2=3 WHERE att1=1; in the subtransaction has a syntax error. When this SQL statement is executed, an error is reported and the subtransaction is rolled back. As a result, the subsequent SQL statement is executed: INSERT INTO tb4 VALUES (4,4);

Step 3 Query the table **tb4**.

```
openGauss=# SELECT * FROM tb4;
 att1 | att2
------+------
    1 |    1
    4 |    4
(2 rows)
```

Therefore, the rollback of a subtransaction does not affect the commit of the parent transaction.

1.4.2 BEGIN...SAVEPOINT...COMMIT/ROLLBACK

Step 1 Create the test table **tb5**.

```
openGauss=# CREATE TABLE tb5(att1 int,att2 int);
CREATE TABLE
```

Step 2 Start a transaction block and insert data.

```
openGauss=# begin;
BEGIN
openGauss=# INSERT INTO tb5 VALUES(1,1);
INSERT 0 1
```

Step 3 Create the first savepoint **S1**.

```
openGauss=# SAVEPOINT S1;
SAVEPOINT
openGauss=# INSERT INTO tb5 VALUES(2,2);
INSERT 0 1
openGauss=# INSERT INTO tb5 VALUES(3,3);
INSERT 0 1
```

Step 4 Roll back to the savepoint **S1**. If the data inserted after the savepoint **S1** does not exist, the rollback to **S1** is successful.

```
openGauss=# ROLLBACK TO SAVEPOINT S1;
ROLLBACK
openGauss=# SELECT * FROM tb5;
 att1 | att2
------+------
    1 |    1
(1 row)
```

Step 5 Create savepoint **S2**.

```
openGauss=# SAVEPOINT S2;
SAVEPOINT
openGauss=# INSERT INTO tb5 VALUES(4,4);
INSERT 0 1
```

Step 6 Release savepoint **S2**.

```
openGauss=# RELEASE SAVEPOINT S2;
RELEASE
openGauss=# SELECT * FROM tb5;
 att1 | att2
------+------
    1 |    1
    4 |    4
(2 rows)
```

Step 7 Create savepoint **S3**.

```
openGauss=# SAVEPOINT S3;
SAVEPOINT
openGauss=# INSERT INTO tb5 VALUES(5,5);
INSERT 0 1
```

Step 8 Create savepoint **S4**.

```
openGauss=# SAVEPOINT S4;
SAVEPOINT
openGauss=# INSERT INTO tb5 VALUES(6,6,6);
ERROR:  INSERT has more expressions than target columns
LINE 1: INSERT INTO tb5 VALUES(6,6,6);
```

Step 9 Roll back to savepoint **S4**.

```
openGauss=# ROLLBACK TO SAVEPOINT S4;
ROLLBACK
openGauss=# INSERT INTO tb5 VALUES(6,6);
INSERT 0 1
```

Step 10 Release savepoint **S3**.

```
openGauss=# RELEASE SAVEPOINT S3;
RELEASE
```

Step 11 Commit the transaction.

```
openGauss=# COMMIT;
COMMIT
openGauss=# SELECT * FROM tb5;
 att1 | att2
------+------
    1 |    1
    4 |    4
    5 |    5
    6 |    6
(4 rows)
```

It can be seen that by creating savepoints, you can roll back to any existing savepoint. The subtransaction has the same features as the parent transaction and can be committed or rolled back as a whole.

1.5 Viewing Lock Information

Step 1 Create table **tb6** and insert data.

```
openGauss=# CREATE TABLE tb6(att1 int,att2 int);
CREATE TABLE
openGauss=# INSERT INTO tb6 VALUES(1,1);
INSERT 0 1
```

Step 2 Check the ID of the current session.

You can run the **SELECT pg_backend_pid();** command to view the ID of the current session. The result is as follows:

```
openGauss=# SELECT pg_backend_pid( );
```

```
 pg_backend_pid
----------------
 281450093779312
(1 row)
```

Step 3 Search for the lock added to the current session.

```
openGauss=# SELECT locktype,database,relation,pid,mode FROM pg_locks WHERE pid=
281450093779312;
   locktype    | database | relation |          pid          |         mode
---------------+----------+----------+-----------------------+---------------------
 relation      |    15730 |    12136 | 281450093779312 | AccessShareLock
 virtualxid    |          |          | 281450093779312 | ExclusiveLock
(2 rows)
```

The preceding information lists locks of the current session. The **locktype** column indicates the type where a lock is placed. The **relation** column indicates the corresponding relation ID. The **mode** column indicates the lock mode.

Step 4 Query the relation ID of table **tb6**.

```
openGauss=# SELECT oid FROM pg_class WHERE relname='tb6';
  oid
-------
 24597
(1 row)
```

The ID of table **tb6** is 24597.

1.6 Reproducing a Shared Lock

A shared lock is placed only when the **CREATE INDEX** command is executed. Since the execution time of this command is very short, the **BEGIN** command is used to start a transaction, and then create an index to obtain the lock information.

Step 1 Start a transaction and place a shared lock.

```
openGauss=# BEGIN;
BEGIN
openGauss=# CREATE INDEX idx_att1 ON tb6(att1);
CREATE INDEX
openGauss=# SELECT locktype,database,relation,pid,mode FROM pg_locks WHERE pid=
281450093779312;
    locktype      | database | relation |          pid          |         mode
------------------+----------+----------+-----------------------+---------------------
 relation         |    15730 |    12136 | 281450093779312 | AccessShareLock
 virtualxid       |          |          | 281450093779312 | ExclusiveLock
 relation         |    15730 |    24597 | 281450093779312 | AccessShareLock
 relation         |    15730 |    24597 | 281450093779312 | ShareLock
 transactionid    |          |          | 281450093779312 | ExclusiveLock
 relation         |    15730 |    24600 | 281450093779312 | AccessExclusiveLock
(6 rows)
```

You can see that a shared lock is placed on table **tb6**. The shared lock mode conflicts with the ROW EXCLUSIVE and ACCESS EXCLUSIVE lock modes. Therefore, operations such as INSERT, ALTER, and DROP cannot be performed. The SELECT operation can be performed.

Step 2 Perform the INSERT operation.

Open another session window and perform the INSERT operation. It is found that the command is blocked and cannot be executed.

```
openGauss=# insert into tb6 values(2,2);
```

Step 3 Perform the SELECT operation.

Open another session window and perform the SELECT operation. The SELECT statement is successfully executed because the shared lock and the INSERT operation in step 2 do not block the SELECT statement.

```
openGauss=# select * from tb6;
 att1 | att2
------+------
    1 |    1
(1 row)
```

Step 4 Commit the transaction and release the shared lock.

```
openGauss=# commit;
COMMIT
```

Commit the transaction in the first session. It is found that the INSERT operation in the second session is successfully executed. The result is as follows:

```
openGauss=# insert into tb6 values(2,2);
INSERT 0 1
```

1.7 Reproducing ACCESS SHARE Locks

ACCESS SHARE locks are placed on common query commands.

Step 1 Query the PID of the current session.

```
openGauss=# SELECT pg_backend_pid( );
 pg_backend_pid
----------------
 281449887013232
(1 row)
```

Step 2 Start a transaction, query the table, and place a lock.

```
openGauss=# BEGIN;
BEGIN
openGauss=# SELECT * FROM tb6;
 att1 | att2
------+------
    1 |    1
    2 |    2
(2 rows)

openGauss=# SELECT locktype,database,relation,pid,mode FROM pg_locks WHERE pid=
281449887013232;
   locktype    | database | relation |          pid          |         mode
---------------+----------+----------+-----------------------+---------------------
 relation      |    15730 |    12136 | 281449887013232 | AccessShareLock
 relation      |    15730 |    24597 | 281449887013232 | AccessShareLock
 virtualxid |          |          | 281449887013232 | ExclusiveLock
(3 rows)
```

You can see that an ACCESS SHARE lock is placed on the **tb6** table. The ACCESS SHARE lock mode conflicts with the ACCESS EXCLUSIVE lock mode. Therefore, table-level operations such as ALTER or DROP cannot be performed. Operations such as SELECT, INSERT, and UPDATE can be performed.

Step 3 Perform the SELECT operation.

Open another session window and perform the SELECT operation. The operation is successful.

```
openGauss=# SELECT * FROM tb6;
 att1 | att2
------+------
    1 |    1
    2 |    2
(2 rows)
```

Step 4 Perform the ALTER operation.

Open another session window and perform the ALTER operation. It is found that the command is blocked and cannot be executed.

```
openGauss=# alter table tb6 add(att3 varchar(10));
```

Step 5 Commit the transaction and release the lock.

```
openGauss=# commit;
COMMIT
```

Commit the transaction in the first session. It is found that ALTER in the third session is successfully executed.

```
openGauss=# alter table tb6 add(att3 varchar(10));
ALTER TABLE
```

1.8 Reproducing a ROW EXCLUSIVE Lock

A ROW EXCLUSIVE lock is placed on a statement that modifies table data. Run the **BEGIN** command to start a transaction, and then modify the table data to obtain the lock information.

Step 1 Query the PID of the current session.

```
openGauss=# SELECT pg_backend_pid( );
 pg_backend_pid
-----------------
 281449887013232
(1 row)
```

Step 2 Start a transaction, modify table data, and place a lock.

```
openGauss=# begin;
BEGIN
openGauss=# UPDATE tb6 SET att1 = 3 ,att2 = 3 where att1=1;
UPDATE 1
openGauss=# SELECT locktype,database,relation,pid,mode FROM pg_locks WHERE pid=
281449887013232;
    locktype      | database | relation |          pid          |          mode
---------------+----------+----------+-----------------+------------------
 relation         |    15730 |    12136 | 281449887013232 | AccessShareLock
 relation         |    15730 |    24597 | 281449887013232 | RowExclusiveLock
 virtualxid       |          |          | 281449887013232 | ExclusiveLock
 transactionid |          |          | 281449887013232 | ExclusiveLock
(4 rows)
```

You can see that a ROW EXCLUSIVE lock is placed on the **tb6** table. The ROW EXCLUSIVE lock mode conflicts with the shared and ACCESS EXCLUSIVE lock modes. Therefore, operations such as CREATE INDEX, ALTER, and DROP cannot be performed.

Step 3 Perform the CREATE INDEX operation.

Open another session window and perform the CREATE INDEX operation. It is found that the command is blocked and cannot be executed.

```
openGauss=# CREATE INDEX idx_att2 ON tb6(att2);
```

Step 4 Commit the transaction in the first session.

```
openGauss=# commit;
COMMIT
```

Commit the transaction in the first session. It is found that the CREATE INDEX operation in the second session is successfully executed.

```
openGauss=# CREATE INDEX idx_att2 ON tb6(att2);
CREATE INDEX
```

Similarly, table-level operations such as ALTER TABLE are blocked. However, common query statements are not blocked.

1.9 Reproducing an ACCESS EXCLUSIVE Lock

An ACCESS EXCLUSIVE lock is placed on a statement that modifies a table structure. Run the **BEGIN** command to start a transaction, and then modify the table to obtain the lock information.

Step 1 Query the PID of the current session.

```
openGauss=# SELECT pg_backend_pid( );
 pg_backend_pid
----------------
 281449983678832
(1 row)
```

Step 2 Start a transaction, modify the table structure, and place a lock.

```
openGauss=# BEGIN;
BEGIN
openGauss=# ALTER TABLE tb6 ADD(att4 varchar(10));
ALTER TABLE
openGauss=# SELECT locktype,database,relation,pid,mode FROM pg_locks WHERE
pid=281449983678832;
    locktype      | database | relation |          pid          |            mode
---------------+----------+----------+-----------------+---------------------
 relation         |    15730 |    12136 | 281449983678832 | AccessShareLock
 virtualxid       |          |          | 281449983678832 | ExclusiveLock
 relation         |    15730 |    24597 | 281449983678832 | AccessExclusiveLock
 object           |    15730 |          | 281449983678832 | ExclusiveLock
 transactionid |          |          | 281449983678832 | ExclusiveLock
(5 rows)
```

You can see that an ACCESS EXCLUSIVE lock is placed on the **tb6** table. The ACCESS EXCLUSIVE lock mode conflicts with all other lock modes. Therefore, operations such as SELECT, INSERT, and CREATE INDEX cannot be performed.

Step 3 Perform the SELECT operation.

Open another session window and perform the SELECT operation. It is found that the command is blocked and cannot be executed.

```
openGauss=# SELECT * FROM tb6;
```

Step 4 Commit the transaction in the first session.

```
openGauss=# commit;
COMMIT
```

Commit the transaction in the first session. It is found that the SELECT operation in the second session is successfully executed.

```
openGauss=# SELECT * FROM tb6;
 att1 | att2 | att3 | att4
------+------+------+------
    2 |    2 |      |
    3 |    3 |      |
(2 rows)
```

2 Storage Management

2.1 Introduction

2.1.1 About This Lab

In this lab, openGauss 3.0.0 is compiled to install the pageinspect plug-in. The pageinspect plug-in is used to analyze the page structure (such as viewing the header structure, tuple data, and original data of the page), helping you understand the underlying data storage mode.

An ECS is used, which is the same as the lab environment described in "About This Document."

2.1.2 Objectives

- Master how to compile openGauss.
- Master how to install the pageinspect plug-in.
- Understand the page structure of data storage.
- Understand the physical storage structure of relational tables in relational databases.

2.2 Compiling openGauss

2.2.1 Lab Description

openGauss 3.0.0 is used for compilation. (This version has compiled third-party library files.)

The openGauss compilation requires two components: **openGauss-server** and **binarylibs**. **openGauss-server** is the main code of openGauss and can be obtained from the openGauss open-source community. **binarylibs** is the third-party library file on which openGauss depends. The process of directly compiling the **binarylibs** source code is complex and time-consuming. You are advised to download the compiled file from the openGauss open-source community.

Run **configure** and **make** for compilation.

- The **configure** script is used to check the system configuration and availability and generate a makefile. It detects information about libraries, header files, and compilers installed in the system and includes the information in the generated makefile.

- The **make** command line tool is used to compile source code based on instructions in the makefile. It reads the makefile and compiles the source code according to the instructions in the file. Generally, the makefile contains all information required for compiling the source code, such as compilation options, source files, and target files.

In this process, you can run **configure** to generate a makefile based on the system configuration, and then run **make** to perform compilation. This method enables the program to automatically adapt to different environments.

2.2.2 Compiling a Database

Step 1 Install the yum packages for the OS to compile a database.

```
[root@ecs-57e4 ~]# sed -i s/gpgcheck=1/gpgcheck=0/g /etc/yum.repos.d/openEuler.repo
[root@ecs-57e4 ~]#yum -y install -y readline-devel
[root@ecs-57e4 ~]# yum -y install libaio-devel
[root@ecs-57e4 ~]# yum -y install bison
[root@ecs-57e4 ~]# yum -y install flex
```

Step 2 Create a user group and user for compiling the database.

```
[root@ecs-57e4 ~]# groupadd gsgrp
[root@ecs-57e4 ~]# useradd -g gsgrp -u 1001 -d /home/gslab gslab
```

Set a password of the **gslab** user.

```
[root@ecs-57e4 ~]# passwd gslab
```

Step 3 Log in as the **gslab** user.

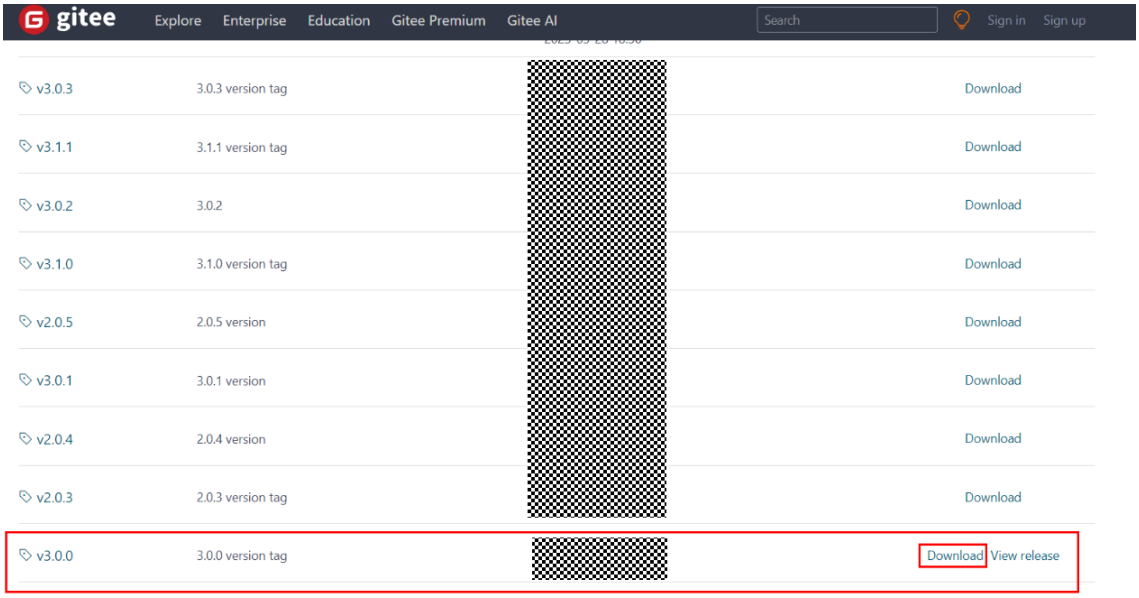
```
[root@ecs-57e4 ~]# su - gslab
```

Step 4 Create the **opengauss-compile** directory for compiling openGauss.

```
[gslab@ecs-57e4 ~]$ mkdir opengauss-compile
[gslab@ecs-57e4 ~]$ cd opengauss-compile/
```

Step 5 Download the source code of openGauss 3.0.0.

Sign in or sign up, and then download the source code at https://gitee.com/opengauss/openGauss-server/tags.



Use a file transfer tool (such as the SFTP client) to upload the **openGauss-server-v3.0.0.zip** package to the **/home/gslab/opengauss-compile/** directory on the ECS.

Step 6 Download the compiled third-party library binary package (tar.gz) provided by the openGauss community.

Download address: https://gitee.com/opengauss/openGauss-server/tree/3.0.0#%E4%B8%8B%E8%BD%BDopengauss

2.0.0	v2.0.0 v2.0.1 v2.0.2 v2.0.3 v2.0.4 v2.0.5	https://opengauss.obs.cn-south-1.myhuaweicloud.com/2.0.0/openGauss-third_party_binarylibs.tar.gz
2.1.0	v2.1.0	https://opengauss.obs.cn-south-1.myhuaweicloud.com/2.1.0/openGauss-third_party_binarylibs.tar.gz
3.0.0	v3.0.0 v3.0.1 v3.0.2	https://opengauss.obs.cn-south-1.myhuaweicloud.com/3.0.0/openGauss-third_party_binarylibs.tar.gz
	v3.0.3	**openEuler_arm:** https://opengauss.obs.cn-south-1.myhuaweicloud.com/3.0.0/binarylibs/openGauss-third_party_binarylibs_openEuler_arm-3.0.3.tar.gz **openEuler_x86:** https://opengauss.obs.cn-south-1.myhuaweicloud.com/3.0.0/binarylibs/openGauss-third_party_binarylibs_openEuler_x86_64-3.0.3.tar.gz **Centos_x86:** https://opengauss.obs.cn-south-1.myhuaweicloud.com/3.0.0/binarylibs/openGauss-third_party_binarylibs_Centos7.6_x86_64-3.0.3.tar.gz

You can run the **wget** command on the ECS to download the package.

```
[gslab@ecs-57e4 opengauss-compile]$ wget    https://opengauss.obs.cn-south-1.myhuaweicloud.com/3.0.0/openGauss-third_party_binarylibs.tar.gz
```

Step 7 Decompress the openGauss source code package and third-party library package.

Decompress the openGauss source code package **openGauss-server-v3.0.0.zip**.

```
[gslab@ecs-57e4 opengauss-compile]$ unzip openGauss-server-v3.0.0.zip
```

Decompress the third-party library binary package **openGauss-third_party_binarylibs.tar.gz**.

```
[gslab@ecs-57e4 opengauss-compile]$ tar -zxvf openGauss-third_party_binarylibs.tar.gz
```

Rename the directory of the third-party library binary package to **binarylibs-v3.0.0**.

```
[gslab@ecs-57e4 opengauss-compile]$ mv openGauss-third_party_binarylibs binarylibs-v3.0.0
```

Step 8 Configure environment variables.

Use the vim editor to edit the **~/.bashrc** file and add environment variables.

```
[gslab@ecs-57e4 opengauss-compile]$ vim ~/.bashrc
```

Configure the following environment variables in the .bashrc file:

```
export CODE_BASE=/home/gslab/opengauss-compile/openGauss-server-v3.0.0 # Path of the openGauss-server file
export BINARYLIBS=$CODE_BASE/../binarylibs-v3.0.0
export GAUSSHOME=$CODE_BASE/dest
export GCC_PATH=$BINARYLIBS/buildtools/openeuler_aarch64/gcc7.3
export CC=$GCC_PATH/gcc/bin/gcc
export CXX=$GCC_PATH/gcc/bin/g++
export LD_LIBRARY_PATH=$GAUSSHOME/lib:$GCC_PATH/gcc/lib64:$GCC_PATH/isl/lib:$GCC_PATH/mpc/lib/:$GCC_PATH/mpfr/lib/:$GCC_PATH/gmp/lib/:$LD_LIBRARY_PATH
export PATH=$GAUSSHOME/bin:$GCC_PATH/gcc/bin:$PATH
```

The environment variables are described as follows:

- *CODE_BASE*: path of the openGauss source code
- *BINARYLIBS*: path of the compiled third-party library file
- *GAUSSHOME*: path for installing openGauss after compilation
- *GCC_PATH*: path of GCC compiler tools
- *CC*: path of GCC compiler commands
- *CXX*: path of G++ compiler commands
- *LD_LIBRARY_PATH*: directory where the system searches for the dynamic link library
- *PATH*: directory where the system searches for executable files

Run the **source** command for the environment variables to take effect.

```
[gslab@ecs-57e4 opengauss-compile]$ source ~/.bashrc
```

Step 9 Compile openGauss.

Go to the **openGauss-server** directory and prepare for compilation.

```
[gslab@ecs-57e4 opengauss-compile]$cd openGauss-server-v3.0.0
```

Run the **configure** script to perform the configuration before compilation. To perform debugging, set the version to **debug**.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$ ./configure --gcc-version=7.3.0 CC=g++ CFLAGS='-O0' --prefix=$GAUSSHOME --3rd=$BINARYLIBS --enable-debug --enable-cassert --enable-thread-safety --with-readline --without-zlib
```

The parameters in the **configure** script are described as follows:

- **--gcc-version=7.3.0**: GCC 7.3.0 is used for compilation.
- **CC=g++**: The G++ compiler is specified.
- **CFLAGS='-O0'**: The compiler optimization level is 0.
- **--prefix=***$GAUSSHOME*: The installation program path is specified by *$GAUSSHOME*.
- **--3rd=***$BINARYLIBS*: The precompiled third-party library file in *$BINARYLIBS* is used.
- **--enable-debug**: The **debug** mode is enabled.
- **--enable-cassert**: The ASSERT statement of the C language is used.
- **--enable-thread-safety**: Thread security is enabled.
- **--with-readline**: The **readline** library is used.
- **--without-zlib**: The **zlib** library is not used.

If an error is reported during the execution of the **configure** script, the possible causes are as follows: The dependent library or tool is missing. The compiler version is incorrect. The environment variable configuration is incorrect. Rectify the fault based on the error message.

Run the **make** command to perform compilation. The **-j4** option of the **make** command specifies parallel compilation using 4 processes.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$ make -j4
```

If no error is reported during the compilation, the compilation is successful and "All of openGauss successfully made. Ready to install." is displayed in the last line.

(Optional) Clear the files generated during compilation to restore the directory to the status before compilation and prepare for the next compilation. If the compilation is successful or files do not need to be cleared, do not run this command.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$ make clean
```

Step 10 Install openGauss.

Install the compiled openGauss binary file.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$ make install
```

If openGauss is successfully installed, "openGauss installation complete." is displayed in the last line.

2.2.3 Starting openGauss

The installed openGauss server can run only after being initialized. You need to run the **gs_initdb** command to initialize openGauss. This command creates the directory structure and configuration files required by the database, and initializes the system directory and configuration files, including creating a database directory, generating a system catalog, and creating the default database and template database. A large number of system catalogs and views are generated during database initialization. The following databases are generated during database initialization:

template0: It is the initial backup database provided by openGauss. You can use **template0** as a template to generate a clean database if necessary.

template1: It is a template database. When a database is created later, all content in the **template1** database is copied to the new database.

postgres: It is a built-in database dedicated to managing system metadata. It contains information about metadata such as all databases, tables, views, rules, and stored procedures in the system. The information is used to maintain the consistency and integrity of databases in openGauss and is maintained and used by the built-in management tool of the system. This database is invisible to administrators and cannot be directly accessed by users.

The **gs_initdb** command is executed in the following format:

gs_initdb [options] [pgdata]

[options] indicates optional parameters, and **[pgdata]** indicates the database storage directory.

Common options of the **gs_initdb** command are as follows:

- **--pgdata** or **-D**: specifies the database storage directory.
- **--nodename**: specifies the name of the initialized node.
- **--encoding** or **-E**: specifies the encoding format for the new database.

Step 1 Initialize the database.

Run the following **gs_initdb** command to initialize openGauss:

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$ gs_initdb --pgdata=$GAUSSHOME/data --
nodename=dblab --encoding=utf-8
```

In the preceding command, **--pgdata=*$GAUSSHOME*/data** specifies the storage directory of the database instance, and *$GAUSSHOME* is an environment variable. The **--nodename=dblab** parameter specifies the name of the database instance. The **--encoding=utf-8** parameter specifies the encoding mode of the database instance. After

this command is executed, a database instance named **dblab** is created in the *$GAUSSHOME*/**data** directory. The encoding mode of the instance is UTF-8.

After openGauss is initialized, the message "Success. You can now start the database server of single node using: "is displayed. You can run the **gaussdb** or **gs_ctl** command to start the database server.

Step 2 Start the database.

Ensure that the database instance has been initialized using **gs_initdb** before the openGauss database server is started. After the database is initialized, you can run a gs_ctl command to start the database server. gs_ctl is a tool provided by openGauss for controlling the database service. It can be used to start, stop, and restart the database service and query the database service status.

Common gs_ctl commands are as follows:

- **gs_ctl start**: starts the database server.
- **gs_ctl stop**: stops the database server.
- **gs_ctl restart**: restarts the database server.
- **gs_ctl status**: checks the database server status.
- **gs_ctl reload**: reloads the configuration file.

These commands can use parameters to specify information such as the database instance directory, log file, and cluster mode. You can run these commands to easily manage the openGauss database server.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$gs_ctl start -D $GAUSSHOME/data -Z single_node -l logfile
```

Start the openGauss single-node database server. The database instance directory is *$GAUSSHOME*/**data**, and run logs are recorded in the log file.

Parameters in the gs_ctl commands are described as follows:

- **-D** *$GAUSSHOME*/**data**: specifies the directory where the database instance is located.
- **-Z single_node**: enables the single-node mode.
- **-l logfile**: specifies the name of a log file. The log content is recorded in this file.

Step 3 Check the running status of the database server.

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$gs_ctl status -D $GAUSSHOME/data
```

Step 4 (Optional) To stop the database server, run the following command:

```
[gslab@ecs-57e4 openGauss-server-v3.0.0]$gs_ctl stop -D $GAUSSHOME/data -Z single_node -l logfile
```

Step 5 Connect to the database.

```
[gslab@ecs-wql openGauss-server-v3.0.0]$ gsql postgres -r
```

```
gsql ((openGauss 3.0.0 build ) compiled at 20XX-XX-XX 17:56:30 commit 0 last mr   debug)
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
openGauss=#
```

postgres is the name of the database to be connected. The **-r** option indicates that the line editing function of the gsql client is enabled. You can use the backspace key to edit commands. If the connection is successful, the gsql interactive CLI is displayed.

Step 6 Exit the database.

```
openGauss=# \q
[gslab@ecs-wql openGauss-server-v3.0.0]$
```

2.3 Installing the pageinspect Plug-in

pageinspect is a plug-in inherited from PostgreSQL. It provides functions for you to observe database pages at a low level.

1. Switch to the **gslab** user and log in to the ECS.
2. Go to the **pageinspect** directory and prepare for compilation.

Step 1 Go to the **pageinspect** directory.

```
[gslab@ecs-wql openGauss-server-v3.0.0]$ cd /home/gslab/opengauss-compile/openGauss-server-v3.0.0/contrib/pageinspect
```

View the file content in the **pageinspect** directory.

```
[gslab@ecs-wql pageinspect]$ ll
total 80K
-rw-r--r-- 1 gslab gsgrp   17K Apr   1   2022 btreefuncs.cpp
-rw-r--r-- 1 gslab gsgrp 1.6K Apr   1   2022 fsmfuncs.cpp
-rw-r--r-- 1 gslab gsgrp 9.2K Apr   1   2022 ginfuncs.cpp
-rw-r--r-- 1 gslab gsgrp 6.7K Apr   1   2022 heapfuncs.cpp
-rw-r--r-- 1 gslab gsgrp   458 Apr   1   2022 Makefile
-rw-r--r-- 1 gslab gsgrp 3.5K Apr   1   2022 pageinspect--1.0.sql
-rw-r--r-- 1 gslab gsgrp   173 Apr   1   2022 pageinspect.control
-rw-r--r-- 1 gslab gsgrp 1.3K Apr   1   2022 pageinspect--unpackaged--1.0.sql
-rw-r--r-- 1 gslab gsgrp   18K Apr   1   2022 rawpage.cpp
```

Step 2 Compile the pageinspect plug-in.

Run the **make** command to compile pageinspect. The warning displayed during the compilation does not affect the compilation.

```
[gslab@ecs-wql pageinspect]$ make
```

Step 3 Install the pageinspect plug-in.

Run the **make install** command to install the pageinspect plug-in.

```
[gslab@ecs-wql pageinspect]$ make install
/usr/bin/mkdir -p '/home/gslab/opengauss-compile/openGauss-server-v3.0.0/dest/lib/postgresql'
/usr/bin/mkdir -p '/home/gslab/opengauss-compile/openGauss-server-
v3.0.0/dest/share/postgresql/extension'
/usr/bin/mkdir -p '/home/gslab/opengauss-compile/openGauss-server-
v3.0.0/dest/share/postgresql/extension'
/bin/sh ../../config/install-sh -c -m 755    pageinspect.so '/home/gslab/opengauss-compile/openGauss-
server-v3.0.0/dest/lib/postgresql/pageinspect.so'
/bin/sh ../../config/install-sh -c -m 644 ./pageinspect.control '/home/gslab/opengauss-
compile/openGauss-server-v3.0.0/dest/share/postgresql/extension/'
/bin/sh ../../config/install-sh -c -m 644 ./pageinspect--1.0.sql ./pageinspect--unpackaged--1.0.sql
'/home/gslab/opengauss-compile/openGauss-server-v3.0.0/dest/share/postgresql/extension/'
[gslab@ecs-wql pageinspect]$
```

If the preceding information is displayed, the installation is successful.

Step 4 Change the password of the **gslab** user. Log in to the **postgres** database and change the password of the **gslab** user.

```
openGauss=# ALTER ROLE gslab password 'openGauss@123';
```

Step 5 Create the pageinspect plug-in in the **postgres** database.

```
openGauss=# CREATE EXTENSION pageinspect;
CREATE EXTENSION
```

Execute the preceding CREATE EXTENSION statement to create a dependency.

Step 6 Check whether pageinspect is successfully installed.

```
openGauss=# SELECT * FROM pg_extension;
```

```
openGauss=# SELECT * FROM pg_extension;
     extname     | extowner | extnamespace | extrelocatable | extversion | extconfig | extcondition
-----------------+----------+--------------+----------------+------------+-----------+--------------
 plpgsql         |       10 |           11 | f              | 1.0        |           |
 dist_fdw        |       10 |           11 | t              | 1.0        |           |
 file_fdw        |       10 |           11 | t              | 1.0        |           |
 hdfs_fdw        |       10 |           11 | t              | 1.0        |           |
 log_fdw         |       10 |           11 | t              | 1.0        |           |
 hstore          |       10 |           11 | t              | 1.1        |           |
 security_plugin |       10 |           11 | t              | 1.0        |           |
 pageinspect     |       10 |         2200 | t              | 1.0        |           |
(8 rows)
```

2.4 Using the pageinspect Plug-in to Analyze the Page Structure of a Table

To use the pageinspect plug-in to analyze the page structure of a table, create table **tb7** in the **postgres** database.

Create the test table **tb7**.

```
openGauss=# CREATE TABLE tb7(att1 int,att2 int);
CREATE TABLE
```

2.4.1 Viewing the Page Header Structure of a Table

Step 1 View the page header structure of an empty table.

```
openGauss=# SELECT* FROM page_header(get_raw_page('tb7', 0));
ERROR:  block number 0 is out of range for relation "tb7"
```

The error message "block number 0 is out of range for relation "tb7"" is displayed, indicating that no data is added to the empty table, no page is created, and page 0 does not exist.

Step 2 Insert the first tuple and view the page header structure.

Run an INSERT statement.

```
openGauss=# INSERT INTO tb7 VALUES(1,1);
INSERT 0 1
```

Run the following statement to view the page header structure:

```
openGauss=# SELECT* FROM page_header(get_raw_page('tb7', 0));
     lsn      | tli | flags | lower | upper | special | pagesize | version | prune_xid
--------------+-----+-------+-------+-------+---------+----------+---------+-----------
 0/2402B08 |    0 |       0 |      44 |  8160 |      8192 |       8192 |          6 |       14547
(1 row)
```

Page 0 has been created. The number of bytes occupied by the page header is analyzed as follows:

lsn: 8 bytes

tli: 2 bytes

flags: 2 bytes

lower: 2 bytes

upper: 2 bytes

special: 2 bytes

pagesize and **version**: 2 bytes

prune_xid: 4 bytes

------------------------- 24 bytes ------------

pd_xid_base: 8 bytes

pd_multi_base: 8 bytes

------------------------ 40 bytes --------------

An **ItemIdData pd_linp** tuple pointer array item: 4 bytes

---------- 44 bytes ----------

The value in the **lower** column is **44**. The page size is 8192 bytes, and the value in the **upper** column is **8160**. That is, the first inserted tuple occupies 32 bytes.

Step 3 Insert the second tuple and view the page header structure.

```
openGauss=# INSERT INTO tb7 VALUES(2,2);
INSERT 0 1
```

Run the following statement to view the page header structure:

```
openGauss=# SELECT* FROM page_header(get_raw_page('tb7', 0));
      lsn      | tli | flags | lower | upper | special | pagesize | version | prune_xid
-----------+-----+-------+-------+-------+---------+----------+---------+-----------
 0/2403968 |   0 |     0 |    48 |  8128 |    8192 |     8192 |       6 |     14547
(1 row)
```

The value in the **lower** column is changed to **48**, which is increased by 4. This indicates that a tuple pointer occupies 4 bytes. The value in the **upper** column is changed to **8128**, which is reduced by 32. This indicates that a tuple in the table occupies 32 bytes.

Step 4 Insert the last seven tuples and check the page header structure.

```
openGauss=# INSERT INTO tb7 VALUES(3,3);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(4,4);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(5,5);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(6,6);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(7,7);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(8,8);
INSERT 0 1
openGauss=# INSERT INTO tb7 VALUES(9,9);
INSERT 0 1
```

Run the following statement to view the page header structure:

```
openGauss=# SELECT* FROM page_header(get_raw_page('tb7', 0));
      lsn      | tli | flags | lower | upper | special | pagesize | version | prune_xid
-----------+-----+-------+-------+-------+---------+----------+---------+-----------
 0/24048A0 |   0 |     0 |    76 |  7904 |    8192 |     8192 |       6 |     14547
(1 row)
```

In this case, there are nine tuples in total. The nine tuple pointers occupy 36 bytes. Therefore, the value of the **lower** column is **76**. The nine tuples inserted into the table occupy 32 x 9 = 288 bytes. Therefore, the value of the **upper** column is 8192 – 648 = **7904**.

2.4.2 Viewing Tuple Data on a Page

Run the following command to view tuple data on a page:

```
openGauss=# SELECT * FROM heap_page_items(get_raw_page('tb7', 0));
```

The result is as follows.

```
openGauss=# SELECT * FROM heap_page_items(get_raw_page('tb7', 0));
 lp | lp_off | lp_flags | lp_len | t_xmin | t_xmax | t_field3 | t_ctid | t_infomask2 | t_infomask | t_hoff | t_bits | t_oid
----+--------+----------+--------+--------+--------+----------+--------+-------------+------------+--------+--------+-------
  1 |   8160 |        1 |     32 |  14550 |      0 |        0 | (0,1)  |           2 |       2048 |     24 |        |
  2 |   8128 |        1 |     32 |  14551 |      0 |        0 | (0,2)  |           2 |       2048 |     24 |        |
  3 |   8096 |        1 |     32 |  14552 |      0 |        0 | (0,3)  |           2 |       2048 |     24 |        |
  4 |   8064 |        1 |     32 |  14553 |      0 |        0 | (0,4)  |           2 |       2048 |     24 |        |
  5 |   8032 |        1 |     32 |  14554 |      0 |        0 | (0,5)  |           2 |       2048 |     24 |        |
  6 |   8000 |        1 |     32 |  14555 |      0 |        0 | (0,6)  |           2 |       2048 |     24 |        |
  7 |   7968 |        1 |     32 |  14556 |      0 |        0 | (0,7)  |           2 |       2048 |     24 |        |
  8 |   7936 |        1 |     32 |  14557 |      0 |        0 | (0,8)  |           2 |       2048 |     24 |        |
  9 |   7904 |        1 |     32 |  14558 |      0 |        0 | (0,9)  |           2 |       2048 |     24 |        |
(9 rows)
```

From the result, you can see the change of the **lp_off** column in the tuple pointer, the **lp_len** column is set to **32**, the transaction ID (**t_xmin**) of the tuple insertion operation increases, and the **t_ctid** tuple identifier (TID) changes.

2.4.3 Viewing Original Data on a Page

Run the following command to view original data on a page:

```
openGauss=# SELECT get_raw_page::text FROM get_raw_page('tb7', 0);
```

The original data of the returned page is output in hexadecimal format. You can view the header and tail of the page. **0** indicates the free space within the page (with the intermediate part omitted due to its length).

The following figure shows the original data (page header) on the page.

```
 \x00000000a0484002000000004c00e01e002006200000000d33800000000000000000000000000e09f4000c09f4000a09f4000809f4000609f4000409
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

3 Log Management

3.1 Introduction

3.1.1 About This Lab

In this lab, you can view the default log segment file size, current log LSN, name of the log file that is being written, current log file name and LSN offset in the WAL segment file, and directory where the WAL segment file is located. Use the mog_xlogdump tool to view WAL segment files and master related lab procedures and methods. In addition, database restoration (triggering database restoration in immediate shutdown mode), database backup, and PITR are verified to enhance the understanding and application of WAL functions. You can learn how to check and clear logs by completing the lab.

3.1.2 Objectives

- Understand the working principle of WAL files.
- Understand the organization of Xlogs.
- Understand the WAL writing process.
- Understand the principle of using WALs to redo Xlog records for database restoration.
- Master the database backup and PITR methods.
- Master the methods of viewing and checking logs.

3.2 Lab Tasks

The following describes how to install a new standalone openGauss database:

If you want the lab result to be the same as that in the lab guide, you need to prepare a clean lab environment (for example, purchase a new ECS), install and deploy a new standalone openGauss database, and then perform the following labs according to the lab guide. It is also feasible for you to perform the following labs based on the existing database environment. However, the execution results (for example, the number of files in a directory) of some tasks or commands are different. This requires you to understand the operations you have performed and to have certain database theory and practical skills.

3.2.1 Task 1: Viewing WAL Files

This task describes how to view the default size, physical path, and naming rules of WAL files, and view LSNs and the relationship between LSNs and WALs.

Step 1 Log in to the **postgres** database.

Log in as the **root** user and switch to the **omm** user.

```
[root@ standalone ~]# su – omm
...
Welcome to 4.19.90-2110.8.0.0119.oe1.aarch64
...
```

Check the startup mode of the database.

```
[omm@standalone ~]$ gs_om -t status --detail
[    Cluster State    ]

cluster_state    : Normal
redistributing   : No
current_az       : AZ_ALL

[   Datanode State    ]

     node         node_ip             port         instance                                  state
-----------------------------------------------------------------------------------------------------
1   standalone 192.168.0.38      26000         6001 /opt/huawei/install/data/dn     P Primary Normal
```

Check that the database is started.

If the database is not started, run the following command to start the database in advance:

```
[omm@standalone ~]$ gs_om -t start
```

Use the gsql client to log in to the **postgres** database by running the following command:

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r
```

26000 indicates the database port number. You can change it to the port number of the current database in the environment.

Step 2 Check the size of the WAL segment file in the current system.

After logging in to the database using gsql, run the following command to query the default size of the WAL segment file of openGauss:

```
openGauss=# show wal_segment_size;
```

The result is as follows:

```
wal_segment_size
------------------
 16MB
(1 row)
```

The default size of the WAL segment file is 16 MB.

Step 3 View the physical form of WAL segment files.

Exit the current gsql client (for example, by running the **\q** command) or create a session window using MobaXterm. Run the following commands to view the path of WAL files in the current system:

```
[omm@standalone ~]$ cd /opt/huawei/install/data/dn/pg_xlog/
[omm@standalone pg_xlog]$ ll
```

Note that **/opt/huawei/install/data/dn/pg_xlog/** is the WAL path of the current database. If your path is not **/opt/huawei/install/data/dn/pg_xlog/**, set it according to the actual situation.

After you run the **ll** command, the command output may be different. The possible results are as follows:

Result 1 (key concern): If the standalone openGauss database is newly installed and no addition, deletion, or modification operation is performed on the database, the following information is displayed:

```
[omm@standalone pg_xlog]$ ll
total 33M
-rw------- 1 omm dbgrp   16M Apr 10 09:24 000000010000000000000001
-rw------- 1 omm dbgrp   16M Apr 10 09:34 000000010000000000000002
drwx------ 2 omm dbgrp 4.0K Apr 10 09:24 archive_status
[omm@standalone pg_xlog]$
```

According to the command output, you can infer that:

1. The name of a WAL segment file consists of 24 hexadecimal characters. Every eight characters form a string.
2. The size of a WAL segment file is 16 MB. The name of a WAL segment file consists of 24 hexadecimal digits. The file name starts from 000000010000000000000001 and increases in ascending order. When the last digit is F, a new WAL segment file is generated, the file name is 000000010000000000000010. In this case, WAL segment files are named in ascending order.

Result 2 (for extension): If the standalone openGauss database has been installed and a large number of addition, deletion, and modification operations have been performed, the possible results are as follows:

```
...
-rw-------. 1 omm dbgrp   16M Mar 12 13:06 000000010000000000000004
-rw-------. 1 omm dbgrp   16M Mar 12 13:06 000000010000000000000005
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 000000010000000000000006
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 000000010000000000000007
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 000000010000000000000008
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 000000010000000000000009
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 00000001000000000000000A
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 00000001000000000000000B
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 00000001000000000000000C
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 00000001000000000000000D
-rw-------. 1 omm dbgrp   16M Mar 12 13:07 00000001000000000000000E
-rw-------. 1 omm dbgrp   16M Mar 12 13:08 00000001000000000000000F
```

```
-rw-------. 1 omm dbgrp   16M Mar 12 13:08 000000010000000000000010
-rw-------. 1 omm dbgrp   16M Mar 12 13:08 000000010000000000000011
-rw-------. 1 omm dbgrp   16M Mar 12 13:08 000000010000000000000012
drwx------. 2 omm dbgrp 4.0K Mar 11 11:49 archive_status
```

The result clearly shows that the conclusion described in result 1 is the same.

Note: A large number of operations have been performed in the environment of this case. Therefore, a large number of files are generated. If a new environment is used, the query result may be different, which is normal.

Step 4 Check the LSN of the current WAL, and the file name and offset of the current WAL segment.

openGauss provides the following LSN-related functions:

a. pg_current_xlog_location() function: reports the current WAL write location (described by the LSN). Note that the data before this point has been written to the kernel, but may not have been synchronized to disk.

b. select pg_current_xlog_insert_location() function: reports the current WAL insertion position (described by the LSN). This function is used for debugging purposes.

c. select gs_current_xlog_insert_end_location() function: reports the end position of the current WAL write (described by the LSN).

The following demonstration is performed after openGauss is installed and no addition, deletion, or modification is performed.

1. Use gsql to log in to the **postgres** database again, run the preceding three functions, and check the result.

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r        -- Login
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# select pg_current_xlog_location();      -- Call the function.
 pg_current_xlog_location
----------------------------------
 0/2461670          --- Location where the current WAL is inserted, that is, the LSN recorded in the WAL.
(1 row)

openGauss=# select pg_current_xlog_insert_location();   -- Call the function.
 pg_current_xlog_insert_location
----------------------------------
 0/2461670
(1 row)

openGauss=# select gs_current_xlog_insert_end_location();   -- Call the function.
 gs_current_xlog_insert_end_location
-------------------------------------
 0/2461670
(1 row)
```

The LSNs of all the three functions are the same when no data is added, deleted, or modified in the database.

Note: The value **0/2461670** is the LSN, which may be different from the queried LSN. Use the actual LSN. The WAL segment file is updated every 60 seconds. Therefore, the results of running the preceding three functions every 60 seconds are different from the last results.

An example of the LSN numbering rule is as follows: 0/2461670 (*high-order 32 bits*/*low-order 32 bits*).

2. Based on the LSN, check the file name of the WAL segment and the offset of the WAL segment file.

```
openGauss=# select pg_xlogfile_name('0/2461670'),pg_xlogfile_name_offset('0/2461670');
        pg_xlogfile_name         |         pg_xlogfile_name_offset
-------------------------+---------------------------------
 000000010000000000000002 | (000000010000000000000002,4593264)
(1 row)
```

In the command output, **000000010000000000000002** indicates the name of the WAL segment file, and **4593264** in **(000000010000000000000002,4593264)** indicates the offset (a decimal integer) of the file name. **4593264** is converted from the hexadecimal number 461670 in the low-order 32 bits of the LSN **0/2461670**. You can run the following command to verify the value:

```
openGauss=# select x'461670'::int;
   int4
---------
 4593264
(1 row)
```

Analyze the results obtained in the preceding two steps.

1. The name of a WAL file consists of 24 hexadecimal digits, which are divided into time line, high-order 32 bits of the LSN, and low-order 32 bits of the LSN/(2^{24})%256. In this case, **000000010000000000000002** is divided into the following three parts:

00000001: time line.

00000000: logic ID, corresponding to the high-order 32 bits of the LSN (**0** in this example).

00000002: logic segment number **logseg**. The value corresponds to the integer part of low-order 32 bits of the LSN/(2^{24}). In this example, the hexadecimal number 2461670 is converted into the decimal number 38147696, which is divided by 2 to the power of 24, and then the modulo operation is performed on the integer part of the quotient to 256. The result is 2.

2. LSN is an unsigned 64-bit integer (uint64). It uniquely identifies a specific location in WALs. That is, LSN is used to identify the location of each database addition, deletion, and modification log in WALs, each transaction has a start LSN and an end LSN. It is mainly used to ensure the transaction durability and consistency of the

database, and to accurately roll back or move to a specific state during fault recovery.

3.2.2 Task 2: Parsing WAL Files

Tool for parsing WAL files:

The mog_xlogdump toolkit is used to parse WAL files. WAL files are encoded binary files, which are inconvenient for readers to read. In addition, openGauss does not have the pg_xlogdump commands similar to PostgreSQL to parse WAL files. Therefore, a third-party tool is required for users to read or view WAL files, helping you better understand the functions of WALs.

The main objectives of this task are as follows:

When performing DML operations on database tables, users can observe changes in WAL files and contents recorded in WALs, and skillfully use the mog_xlogdump tool to parse and view WALs.

Log in to the OS as the omm user and perform the following steps:

Step 1 Download the mog_xlogdump toolkit.

Switch to the **omm** user and run the following commands in the **/home/omm** directory to download the mog_xlogdump toolkit:

```
[omm@standalone ~]$ cd /home/omm
[omm@standalone ~]$ wget https://cdn-mogdb.enmotech.com/mogdb-media/5.0.5/Kylin_arm64/Toolkits-5.0.5-Kylin-arm64.tar.gz
--20XX-XX-10 14:19:59--  https://cdn-mogdb.enmotech.com/mogdb-media/5.0.5/Kylin_arm64/Toolkits-5.0.5-Kylin-arm64.tar.gz
Resolving cdn-mogdb.enmotech.com (cdn-mogdb.enmotech.com)... 42.248.140.58
Connecting to cdn-mogdb.enmotech.com (cdn-mogdb.enmotech.com)|42.248.140.58|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13953858 (13M) [application/gzip]
Saving to: 'Toolkits-5.0.5-Kylin-arm64.tar.gz'

Toolkits-5.0.5-Kylin-arm64.tar.gz
100%[=====================================================================>]
13.31M   14.6MB/s      in 0.9s

20XX-XX-10 14:20:01 (14.6 MB/s) - 'Toolkits-5.0.5-Kylin-arm64.tar.gz' saved [13953858/13953858]
```

Step 2 Decompress the mog_xlogdump toolkit.

In the **/home/omm** directory, run the following command to decompress **Toolkits-5.0.5-Kylin-arm64.tar.gz** to the current directory:

```
[omm@standalone ~]$ tar -zvxf Toolkits-5.0.5-Kylin-arm64.tar.gz     -- Decompress.
...
[omm@standalone ~]$ ll        --- Check the result after the decompression.
total 14M
```

```
drwx------ 8 omm dbgrp 4.0K Apr 10 14:22 toolkits     -- Directory after the decompression.
-rw------- 1 omm dbgrp  14M Dec 26 16:03 Toolkits-5.0.5-Kylin-arm64.tar.gz
```

The name of the decompressed directory is **toolkits**.

Step 3 Copy the mog_xlogdump commands to the **bin** directory in the openGauss installation path.

```
[omm@standalone ~]$ cp /home/omm/toolkits/mog_xlogdump/mog_xlogdump
/opt/huawei/install/app/bin/
```

Step 4 Modify **wal_level** to **logical**.

When **wal_level** is set to **logical**, the most information about WALs is recorded, including WALs required for logical parsing. Therefore, **wal_level** needs to be set to **logical**. Set this parameter by modifying the **postgresql.conf** file.

```
[omm@standalone ~]$ vim /opt/huawei/install/data/dn/postgresql.conf
```

Locate **wal_level** (line 197) and change **hot_standby** to **logical**, as shown in the following figure.

```
197 # wal_level = hot_standby
198 wal_level = logical
```

Step 5 Set the database to allow the remote login.

Set this parameter by modifying the **pg_hba.conf** file.

```
[omm@standalone ~]$ vim /opt/huawei/install/data/dn/pg_hba.conf
```

Add the following content under **# IPv4 local connections** in the file:

host all all 0.0.0.0/0 sha256

For details, see the following figure.

```
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
#host    all    all    192.168.56.26/32    sha256
#host    all    all    192.168.56.27/32    sha256
host    all    all    0.0.0.0/0    sha256
```

Step 6 Restart the database and reload the configuration file.

After the **postgresql.conf** and **pg_hba.conf** files are configured, run the following command to restart the database:

```
[omm@standalone ~]$ gs_om -t restart
```

After the restart command is executed, information similar to the following is displayed if the startup is successful:

```
Stopping cluster.
```

```
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================
[SUCCESS] opengauss
20XX-XX-09 09:19:45.962 661497b1.1 [unknown] 281464907694096 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-09 09:19:45.962 661497b1.1 [unknown] 281464907694096 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-09 09:19:45.966 661497b1.1 [unknown] 281464907694096 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   Failed to initialize the memory protect for
g_instance.attr.attr_storage.cstore_buffers (16 Mbytes) or shared memory (2192 Mbytes) is larger.
=========================================
Successfully started.
```

After the restart is successful, perform the following operations:

Step 7 Log in to the database and create table **t2**.

Run the following SQL statements to create table **t2**:

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# create table t2(id int, money money,inet inet,bool bool,numeric numeric ,text text);
CREATE TABLE
openGauss=#
```

Step 8 Set **Replica Identity** of table **t2** to **FULL**.

In the lab test environment, you may focus on the data of only several tables in the database. To better observe the changes of WAL files in a fine-grained manner, for example, to view the inserted records in WALs, you can set **Replica Identity** to **FULL**. In this mode, all columns in a row of records can be used as replication identifiers (similar to that each column in a table forms a primary key). Run the following statement:

```
openGauss=# alter table t2 replica identity full;
```

After the modification is successful, check the structure of table **t2**.

```
openGauss=# \d+ t2
                                Table "public.t2"
 Column  |  Type   | Modifiers | Storage  | Stats target | Description
---------+---------+-----------+----------+--------------+-------------
 id      | integer |           | plain    |              |
 money   | money   |           | plain    |              |
 inet    | inet    |           | main     |              |
 bool    | boolean |           | plain    |              |
 numeric | numeric |           | main     |              |
 text    | text    |           | extended |              |
```

```
Replica Identity: FULL
Has OIDs: no
Options: orientation=row, compression=no
```

The value of **Replica Identity** is changed to **FULL**.

Step 9 Add, delete, and modify data in the **t2** table.

Run the following SQL statements to add the following data to the **t2** table:

```
openGauss=# insert into t2 values(1, 24.241, '192.168.255.132', true, 3.1415926, 'ljfsodfo29892ifj');
openGauss=# insert into t2 values(2, 928.8271, '10.255.132.101', false, 3.1415926, 'vzvzcxwf2424@');
```

Update the record whose **id** is **1**.

```
openGauss=# update t2 set money=982.371 where id =1;
```

Delete the record whose **id** is **1**.

```
openGauss=# delete from t2 where id=1;
```

In this case, only one record is left in the **t2** table.

```
openGauss=# select * from t2;
 id |   money   |         inet         | bool |   numeric   |       text
----+---------+----------------+------+-----------+---------------
  2 | $928.83 | 10.255.132.101 | f    | 3.1415926 | vzvzcxwf2424@
(1 row)
```

Step 10 View the content of the table in WALs based on the OID of the table.

The mog_xlogdump tool can parse table data of a specified OID based on parameters such as **-o** (OID of the table) and **-R** (column table of the table).

1. Run the following statement to check the OID of the **t2** table:

```
openGauss=# select oid from pg_class where relname='t2';
```

The result is as follows:

```
  oid
-------
 16387
(1 row)
```

2. View the LSN of the current WAL file.

```
openGauss=#   select
pg_current_xlog_location(),pg_current_xlog_insert_location(),gs_current_xlog_insert_end_location();
 pg_current_xlog_location | pg_current_xlog_insert_location | gs_current_xlog_insert_end_location
--------------------------+---------------------------------+-------------------------------------
```

```
 0/273D788                | 0/273D788                | 0/273D788
(1 row)
```

The current LSN is **0/273D788**. The actual LSN may be different.

3. Check the name of the WAL file based on its actual LSN.

```
openGauss=# select pg_xlogfile_name('0/273D788');
        pg_xlogfile_name
--------------------------
 000000010000000000000002
(1 row)
```

The active log file is **000000010000000000000002**, which records the operations performed by users on the **t2** table.

4. Run the **\q** command to exit the gsql client.
5. Go to the **/opt/huawei/install/app/bin** directory and add the OID and WAL files to the following commands:

```
[omm@standalone ~]$ cd /opt/huawei/install/app/bin
[omm@standalone bin]$ ./mog_xlogdump -o 16387 -R int,money,inet,bool,numeric,text
/opt/huawei/install/data/dn/pg_xlog/000000010000000000000002
```

Then, run the preceding commands. The command output is as follows:

```
'insert','tuple':{'int':'1','money':'$24.24','inet':'192.168.255.132','bool':true,'numeric':'3.1415926','text':'ljfs
odfo29892ifj'}
'insert','tuple':{'int':'2','money':'$928.83','inet':'10.255.132.101','bool':false,'numeric':'3.1415926','text':'vzv
zcxwf2424@'}
'update','old_tuple':{'int':'1','money':'$24.24','inet':'192.168.255.132','bool':true,'numeric':'3.1415926','tex
t':'ljfsodfo29892ifj'}
,'new_tuple':{'int':'1','money':'$982.37','inet':'192.168.255.132','bool':true,'numeric':'3.1415926','text':'ljfs
odfo29892ifj'}
'delete','tuple':{'int':'1','money':'$982.37','inet':'192.168.255.132','bool':true,'numeric':'3.1415926','text':'ljf
sodfo29892ifj'}
mog_xlogdump: FATAL:  error in WAL record at 0/273DEC8: invalid record length at 0/273DF68:
wanted 32, got 0
```

According to WALs, when users perform addition, deletion, and modification operations, these operations and data are recorded in WAL segment files. If a database fault occurs, WALs can be redone to restore data.

In the preceding command output:

'insert' indicates that the data is added, and **'tuple'** following **'insert'** is a tuple (inserted record).

'update' indicates that the data is updated, **'old_tuple'** indicates the original tuple (original record), and **'new_tuple'** indicates the new tuple (new record).

'delete' indicates the data is deleted, followed by the tuple to be deleted.

Note: If the last line (containing **FATAL**) is displayed in the execution result, ignore it because the third-party tool may have compatibility issues. Pay attention to the detailed WAL file content (that is, the operations performed on table **t2**) that is displayed.

Step 11 View the header information of WAL files.

A WAL segment file consists of header information and data. You can use the mog_xlogdump tool to view the header information.

1. Check whether WAL segment files exist in the **pg_xlog** directory.

```
[omm@standalone dn]$ cd /opt/huawei/install/data/dn/pg_xlog
[omm@standalone pg_xlog]$ ll
total 33M
-rw------- 1 omm dbgrp   16M Apr 10 09:24 000000010000000000000001
-rw------- 1 omm dbgrp   16M Apr 10 18:10 000000010000000000000002
drwx------ 2 omm dbgrp 4.0K Apr 10 09:24 archive_status
[omm@standalone pg_xlog]$
```

The command output shows that there are two WAL segment files in the current environment. To view the header information of a segment file (for example, **000000010000000000000002**), run the following command:

```
[omm@standalone pg_xlog]$ mog_xlogdump -n 10
/opt/huawei/install/data/dn/pg_xlog/000000010000000000000002
```

The result is as follows:

```
REDO @ 0/2000728; LSN 0/2000780: prev 0/1FFFED8; xid 11968; term 1; len 2; total 82; crc
318218347; desc: Btree - insert leaf: off 138, blkref #0: rel 1663/1/15316 storage HEAP DISK blk 2
lastlsn 0/1FFFED8
REDO @ 0/2000780; LSN 0/2000B98: prev 0/2000728; xid 11968; term 1; len 3; total 1044; crc
1490430647; desc: Heap - XLOG_HEAP_INSERT insert: off 4, blkref #0: rel 1663/1/15314 storage HEAP
DISK blk 105 lastlsn 0/2000728
REDO @ 0/2000B98; LSN 0/2000BF0: prev 0/2000780; xid 11968; term 1; len 2; total 82; crc
2281135433; desc: Btree - insert leaf: off 139, blkref #0: rel 1663/1/15316 storage HEAP DISK blk 2
lastlsn 0/2000780
REDO @ 0/2000BF0; LSN 0/2001420: prev 0/2000B98; xid 11968; term 1; len 11; total 2094; crc
4017518349; desc: Heap - XLOG_HEAP_INSERT insert(init): off 11526, blkref #0: rel 1663/1/15314
storage HEAP DISK blk 106 lastlsn 0/0
REDO @ 0/2001420; LSN 0/2001478: prev 0/2000BF0; xid 11968; term 1; len 2; total 82; crc
1920653999; desc: Btree - insert leaf: off 140, blkref #0: rel 1663/1/15316 storage HEAP DISK blk 2
lastlsn 0/2000BF0
REDO @ 0/2001478; LSN 0/2001A58: prev 0/2001420; xid 11968; term 1; len 3; total 1497; crc
3507899773; desc: Heap - XLOG_HEAP_INSERT insert: off 2, blkref #0: rel 1663/1/15314 storage HEAP
DISK blk 106 lastlsn 0/2001420
REDO @ 0/2001A58; LSN 0/2001AB0: prev 0/2001478; xid 11968; term 1; len 2; total 82; crc
2318262921; desc: Btree - insert leaf: off 141, blkref #0: rel 1663/1/15316 storage HEAP DISK blk 2
lastlsn 0/2001478
REDO @ 0/2001AB0; LSN 0/20022F0: prev 0/2001A58; xid 11968; term 1; len 3; total 2086; crc
648500789; desc: Heap - XLOG_HEAP_INSERT insert: off 3, blkref #0: rel 1663/1/15314 storage HEAP
DISK blk 106 lastlsn 0/2001A58
REDO @ 0/20022F0; LSN 0/2002348: prev 0/2001AB0; xid 11968; term 1; len 2; total 82; crc
3150207505; desc: Btree - insert leaf: off 142, blkref #0: rel 1663/1/15316 storage HEAP DISK blk 2
lastlsn 0/2001AB0
REDO @ 0/2002348; LSN 0/2002A80: prev 0/20022F0; xid 11968; term 1; len 3; total 1846; crc
3572822742; desc: Heap - XLOG_HEAP_INSERT insert: off 4, blkref #0: rel 1663/1/15314 storage HEAP
DISK blk 106 lastlsn 0/20022F0
```

```
Wal Log parse finish!
```

Note: You can use the pg_current_xlog_insert_location() and pg_xlogfile_name() functions to locate **000000010000000000000002** in the actual environment, or directly go to the **pg_xlog** directory to view WAL segment files. The pg_xlogfile_name() function is used to search for the WAL file name corresponding to the LSN based on the LSN.

In the preceding result:

1) **REDO**: location of the redo point.
2) **LSN**: location of the current LSN.
3) **prev**: location of the previous LSN.
4) **xid**: transaction ID.
5) **total**: total length of the record.
6) **desc**: detailed content, for example, the insert operation performed.
7) **lastlsn**: last LSN.

In the output log, the value of **REDO** in the previous record is the same as the values of **pre** and **lastlsn** in the next record. The value of **REDO** in the next record is the same as that of **LSN** in the previous record. Therefore, the header information of WAL files is written in sequence.

Step 12 View the statistics of WAL files.

You can use the mog_xlogdump tool to view WALs.

1. Check the LSN of the current WAL and the WAL file name corresponding to the LSN. You can use the pg_current_xlog_insert_location() and pg_xlogfile_name() functions to query them.

```
openGauss=# select
pg_current_xlog_insert_location(),pg_xlogfile_name(pg_current_xlog_insert_location());
 pg_current_xlog_insert_location |        pg_xlogfile_name
---------------------------------+--------------------------
 0/2742670                       | 000000010000000000000002
(1 row)
```

Note: The actual result may be different.

2. Add the LSN (**0/2742670**) of the current WAL file into the following command and run the command:

```
[omm@standalone dn]$ mog_xlogdump -z
/opt/huawei/install/data/dn/pg_xlog/000000010000000000000002 -s 0/2742670
```

The result is as follows:

```
Type                 N        (%)     Record size     (%)      FPI size      (%)      Combined size
(%)
----                 -        ---     -----------     ---      --------      ---      -------------   ---
XLOG                 2 ( 25.00)              304 ( 47.50)             0 (  0.00)              304 ( 47.50)
Transaction          0 (  0.00)                0 (  0.00)             0 (  0.00)                0 (  0.00)
Storage              0 (  0.00)                0 (  0.00)             0 (  0.00)                0 (  0.00)
```

```
CLOG                 0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
Database             0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
Tablespace           0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
MultiXact            0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
RelMap               0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
Standby              6 ( 75.00)        336 ( 52.50)          0 (  0.00)        336     ( 52.50)
Heap3                0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
Barrier              0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
...
MOT                  0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
UHeap                0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
UHeap2               0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
UndoLog              0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
UHeapUndo            0 (  0.00)          0 (  0.00)          0 (  0.00)          0
(  0.00)
UndoAction           0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
UBtree               0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
UBtree2              0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
SegpageStorage       0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
ReplicationOrigin    0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
CompressionRelation  0 (  0.00)          0 (  0.00)          0 (  0.00)          0     (  0.00)
                     --------            --------            --------            --------
Total                8                 640 [100.00%]         0 [0.00%]         640
[100%]
mog_xlogdump: FATAL:  error in WAL record at 0/2742890: invalid record length at 0/2742930:
wanted 32, got 0
```

In the preceding result:

Type: log type in the Xlog.

N: number and proportion of times.

Record size: record size and proportion.

FPI size: full page image size and proportion.

combined size: total size of records and FPIs.

3.2.3 Task 3: Using WALs to Automatically Restore the Database After It Stops Abnormally

This task simulates the following scenarios:

When the database stops abnormally, for example, the OS breaks down or is powered off, the database may not flush all WALs to disks in time. If these WALs are not completely flushed to disks, the database attempts to redo these unfinished WALs when the database is restarted to ensure database consistency. However, if WALs have been completely flushed to disks, the database does not need to redo WALs when the database is restarted because all change operations have been written to disks and data is consistent.

The main objectives of this task are as follows:

Shut down the database in fast or immediate mode to simulate abnormal database shutdown. When the database is restarted, check the abnormal shutdown mode in which the database depends on redo WALs to implement database self-recovery.

You can shut down the database in one of the following modes:

1. Fast: Quickly shut down the database, disconnect the client, roll back the transactions that are not committed, and proceed with normal database shutdown.
2. Immediate: Immediately stop the database process, and shut down and exit the database. In the next time when the database starts, it will enter the database restoration process and automatically redo WALs.

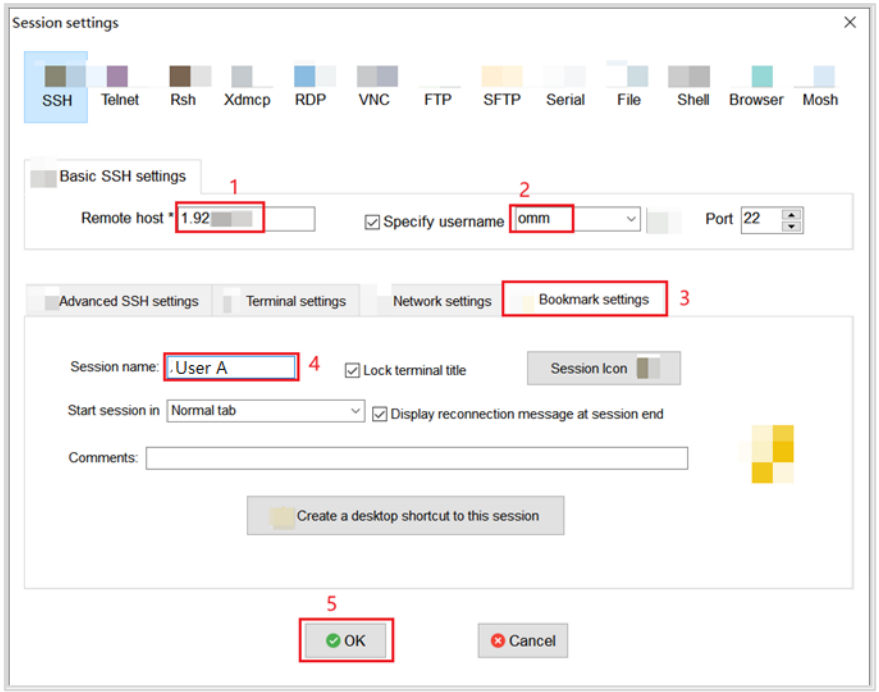
The lab roadmap is as follows:

Use the remote connection tool MobaXterm to create an SSH session to simulate user A, and then use MobaXterm to create another SSH session to simulate user B. When user A is performing a DDL or DML operation on the database, user B shuts down the database in fast or immediate mode. For user A, user B's operations cause abnormal database shutdown. When user A restarts the database, view run logs to check whether the database has restored data using WALs.

The procedure is as follows:

Step 1 Use MobaXterm to create two SSH sessions to simulate user A and user B to log in to the system.

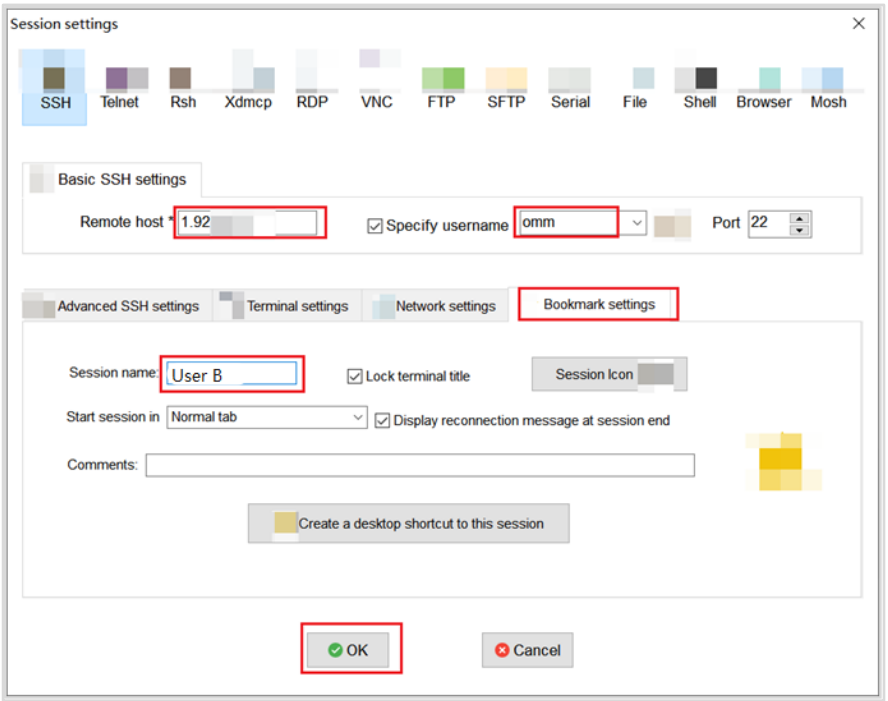
Double-click MobaXterm and create a session. On the session setting page, enter the public IP address, username, and session name, as shown in the following figure.



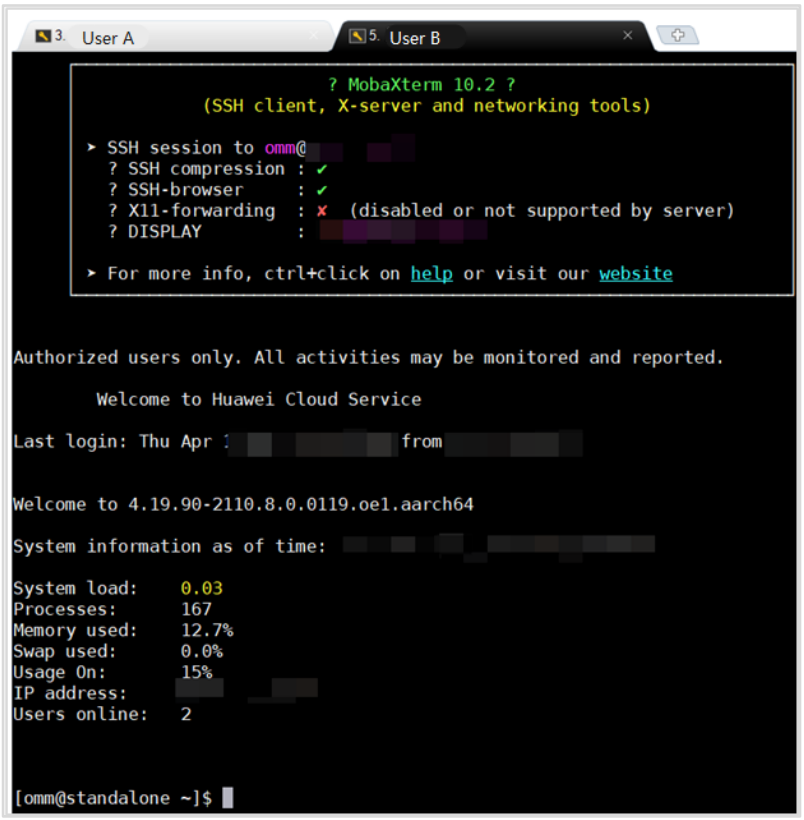
Click **OK** and enter the password of the **omm** user to remotely log in to the server. The password has been set during database installation. After the login is successful, the following page is displayed. Note that the session title is displayed as "User A."



On MobaXterm, create another session. On the session setting page, enter the public IP address, username, and session name, as shown in the following figure.



Click **OK** and enter the password of the **omm** user to remotely log in to the server. The password has been set during database installation. After the login is successful, the following page is displayed. Note that the session title is displayed as "User B."



Step 2 On the GUI of user A, log in to the **postgres** database.

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:09:38 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#
```

Step 3 On the GUI of user A, create test table **t3** and check the OID of table **t3**.

```
openGauss=# CREATE TABLE t3 (id INT, name VARCHAR(50));
CREATE TABLE
openGauss=# select oid from pg_class where relname='t3';
  oid
-------
 16396
(1 row)
```

In this case, remember the OID of the table, which is required when you view WAL files in the next step.

Step 4 View the content of the **t3** table in WAL files based on the OID of the table.

On the GUI of user A, run the following command to view the LSN of the current WAL and the file where the **t3** table is located:

```
openGauss=# select
pg_current_xlog_insert_location(),pg_xlogfile_name(pg_current_xlog_insert_location());
 pg_current_xlog_insert_location |        pg_xlogfile_name
---------------------------------+--------------------------
 0/2B0B7F0                       | 000000010000000000000002
(1 row)
```

The current WAL file is **000000010000000000000002**.

View the content of the WAL file where the **t3** table is located based on the OID of the **t3** table, and run the following commands on the GUI of user B:

```
[omm@standalone ~]$ cd /opt/huawei/install/data/dn/pg_xlog/
[omm@standalone pg_xlog]$ mog_xlogdump -o 16396 -R int,varchar 000000010000000000000002 -f
```

The WAL does not contain the data of table **t3**. Retain the status of user B for future observation.

Step 5 On the GUI of user A, start the transaction and perform insertions.

```
openGauss=# Begin;
openGauss=# insert into t3 values(1, 'test');
INSERT 0 1
openGauss=# insert into t3 values(2, 'fast test');
INSERT 0 1
openGauss=#
```

After the insertions are complete, switch to the GUI of user B. The following information is displayed:

```
[omm@standalone pg_xlog]$ mog_xlogdump -o 16396 -R int,varchar 000000010000000000000002 -f
'insert','tuple':{'int':'1','varchar':'test'}
'insert','tuple':{'int':'2','varchar':'fast test'}
```

The insertion performed on the GUI of user A has been written into the **000000010000000000000002** log file.

Note: The commit or end operation is not performed.

Step 6 Switch to the GUI of user B and shut down the database in fast mode.

Switch to the GUI of user B and press **Ctrl+C** to stop the **mog_xlogdump** command.

```
[omm@standalone pg_xlog]$ mog_xlogdump -o 16396 -R int,varchar 000000010000000000000002 -f
'insert','tuple':{'int':'1','varchar':'test'}
'insert','tuple':{'int':'2','varchar':'fast test'}
^C     --- Press Ctrl+C.
[omm@standalone pg_xlog]$
```

Run the following command to shut down the database in fast mode:

```
[omm@standalone pg_xlog]$ gs_om -t stop -m fast
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
```

The database is shut down.

Note: The operation on the GUI of user A is stopped, but the transaction is not committed. In this case, the database is stopped abnormally.

Step 7 On the GUI of user B, restart the database and check whether the database redoes WALs.

Run the following command to restart the database:

```
[omm@standalone pg_xlog]$ gs_om -t start
Starting cluster.
=========================================
[SUCCESS] standalone
20XX-XX-12 00:00:34.524 66180922.1 [unknown] 281469410934800 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-12 00:00:34.524 66180922.1 [unknown] 281469410934800 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-12 00:00:34.528 66180922.1 [unknown] 281469410934800 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   Failed to initialize the memory protect for
g_instance.attr.attr_storage.cstore_buffers (16 Mbytes) or shared memory (2192 Mbytes) is larger.
=========================================
Successfully started.
```

The database is started properly.

Then, log in to the **postgres** database and check whether the **t3** table contains data.

```
[omm@standalone pg_xlog]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# select * from t3;
 id | name
----+------
(0 rows)

openGauss=#
```

There is no data in the **t3** table, which indicates that:

1. WALs of openGauss record DML-related operations.
2. After openGauss is shut down in fast mode before transactions are committed, WAL redo is not performed when the database is restarted. Instead, the transaction is rolled back when the database is shut down in fast mode to ensure database data consistency.

To further verify that the database does not have redo logs when the database is restarted after the fast database is shut down, you can view the system run logs of the database. The logs are stored in the **/var/log/omm/omm/pg_log/dn_6001** directory. Locate and view the latest log, the logs do not contain log records such as "redo starts at *xxx*" and "redo done at *xxx*". In this case, the database does not redo WALs after the database is shut down and restarted in fast mode. (The system run log contains a large amount of content. You can verify the content based on the actual situation.)

Step 8 Switch to the GUI of user A, log in to the database, start the display transaction, and perform the insert operation.

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# begin;
BEGIN
openGauss=# insert into t3 values(8, 'immediate test8');
INSERT 0 1
openGauss=# insert into t3 values(9, 'immediate test9');
INSERT 0 1
openGauss=#
```

After the insertions are complete, switch to the GUI of user B. The following information is displayed:

```
[omm@standalone pg_xlog]$ mog_xlogdump -o 16396 -R int,varchar 000000010000000000000002 -f
'insert','tuple':{'int':'1','varchar':'test'}
'insert','tuple':{'int':'2','varchar':'fast test'}
'insert','tuple':{'int':'8','varchar':'immediate test8'}
'insert','tuple':{'int':'9','varchar':'immediate test9'}
```

The insertion performed on the GUI of user A has been written into the **000000010000000000000002** log file.

Note: The commit or end operation is not performed.

Step 9 On the GUI of user B, shut down the database in immediate mode, and then start the database.

Shut down the database in immediate mode.

```
[omm@standalone ~]$ gs_om -t stop -m immediate   ## Run the database shutdown command.
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
```

Then, restart the database.

```
[omm@standalone pg_xlog]$ gs_om -t start
Starting cluster.
=========================================
[SUCCESS] standalone
20XX-XX-12 01:00:32.979 66181730.1 [unknown] 281472128516112 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-12 01:00:32.979 66181730.1 [unknown] 281472128516112 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-12 01:00:32.983 66181730.1 [unknown] 281472128516112 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   Failed to initialize the memory protect for
g_instance.attr.attr_storage.cstore_buffers (16 Mbytes) or shared memory (2192 Mbytes) is larger.
=========================================
Successfully started.
[omm@standalone pg_xlog]$
```

The database is started properly.

On the GUI of user B, log in to the **postgres** database and check whether the **t3** table contains data.

```
[omm@standalone pg_xlog]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# select * from t3;
 id | name
----+------
(0 rows)

openGauss=#
```

There is still no data in the **t3** table because the transactions are not committed. You can view the system run logs of the database. The logs are stored in the **/var/log/omm/omm/pg_log/dn_6001** directory. Locate and view the latest log. You can find that the log contains records such as "redo starts at *xxx*" and "redo done at *xxx*". Some key logs are as follows:

```
20XX-XX-12 00:17:42.539 66180d26.6125 [unknown] 281454457952624 dn_6001 0 dn_6001 00000   0
[BACKEND] LOG:   database system was not properly shut down; automatic recovery in progress
20XX-XX-12 00:17:42.555 66180d26.6125 [unknown] 281454457952624 dn_6001 0 dn_6001 00000   0
[BACKEND] LOG:   redo starts at 0/2B0E390
20XX-XX-12 00:17:42.656 66180d26.6125 [unknown] 281454457952624 dn_6001 0 dn_6001 00000   0
[BACKEND] LOG:   redo done at 0/2B0E5D8, end at 0/2B0E638
20XX-XX-12 00:17:42.656 66180d26.6125 [unknown] 281454457952624 dn_6001 0 dn_6001 DB010
0 [REDO] LOG:   [REDO_STATS]redo_unlink_stats_file: unlink redo.state sucessfully!, ret:-1
```

In the preceding information:

"database system was not properly shut down; automatic recovery in progress" indicates that the database stops abnormally. When the database is restarted, the data restoration process automatically starts.

"redo starts at 0/2B0E390" indicates the start position of redo WALs.

"redo done at 0/2B0E5D8, end at 0/2B0E638" indicates the end position of redo WALs.

In this case, after the database is shut down in immediate mode and then restarted, the database redoes WALs and restores the data before the abnormal shutdown.

Note: To ensure that the lab process is the same as that in fast mode, the insertion is performed for the display transaction and information indicating that the transaction is committed is not displayed. Therefore, there is no data in table **t3**, which is normal. In this lab, you need to check whether run logs contain redo-related content.

3.2.4 Task 4: Using WALs to Implement PITR for Databases

Database PITR and functions:

In openGauss, **PITR** is actually a combination of **physical backup** and **WAL-based recovery**. A physical backup is a snapshot of a running database cluster, which is also called a base backup. The PITR function uses the base backup and archive log data to restore the database to any point in time after the base backup. PITR works by redoing WALs in archive logs based on base backups. PITR helps database administrators restore the database to the state before a misoperation. For example, if all data in a table is deleted by mistake, PITR can be used to restore the database to the state before the data is deleted.

Prerequisites for database PITR:

1. Full data files have been physically backed up.
2. WAL files have been archived.

Process of database PITR:

1. Replace the target database directory with the physical backup files.
2. Delete all files in the database directory **pg_xlog**.
3. Copy the archived WAL files to the **pg_xlog** directory.
4. Create the **recovery.conf** file in the database directory and specify the database restoration degree.
5. Restart the database, connect to the database, and check whether it is restored to the expected state.
6. If it is restored to the expected state, use pg_xlog_replay_resume() to enable the primary node to provide services for external systems. In this lab, the primary node is a single-node system.

The procedure is as follows:

Step 1 Before performing physical backup, check the number of database tables.

```
[omm@standalone global]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:09:38 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# \d
                          List of relations
 Schema | Name | Type   | Owner |                Storage
--------+------+-------+-------+-----------------------------------
```

```
 public | t2    | table | omm     | {orientation=row,compression=no}
 public | t3    | table | omm     | {orientation=row,compression=no}
(2 rows)
```

Currently, there are two tables in the database: **t2** and **t3**. This step is used to verify the database restoration.

Step 2 Create a physical backup directory.

Exit the gsql client, create a backup directory, and name it **backup**.

```
openGauss=# \q
[omm@standalone ~]$ mkdir -p /home/omm/backup
```

Step 3 Start physical backup.

openGauss provides the **gs_basebackup** script for full data file backup. Before running the **gs_basebackup** command, run the **ifconfig** command to view the private IP address of the current server.

```
[omm@standalone ~]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.0.38  netmask 255.255.255.0  broadcast 192.168.0.255
        inet6 fe80::f816:3eff:fe46:9ec4  prefixlen 64  scopeid 0x20<link>
        ether fa:16:3e:46:9e:c4  txqueuelen 1000  (Ethernet)
        RX packets 39602  bytes 5374068 (5.1 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 69429  bytes 25834742 (24.6 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
...
```

192.168.0.38 is the private IP address of the server. Add this IP address to the following command to start backup:

```
[omm@standalone ~]$ gs_basebackup -D /home/omm/backup -h 192.168.0.38 -p 26000
```

The result is as follows:

```
INFO:  The starting position of the xlog copy of the full build is: 0/3000028. The slot minimum LSN is: 0/0. The disaster slot minimum LSN is: 0/0. The logical slot minimum LSN is: 0/0.
[20XX-XX-12 10:28:02]:begin build tablespace list
[20XX-XX-12 10:28:02]:finish build tablespace list
[20XX-XX-12 10:28:02]:begin get xlog by xlogstream
[20XX-XX-12 10:28:02]: check identify system success
[20XX-XX-12 10:28:02]: send START_REPLICATION 0/3000000 success
[20XX-XX-12 10:28:02]: keepalive message is received
[20XX-XX-12 10:28:02]: keepalive message is received
[20XX-XX-12 10:28:07]:gs_basebackup: base backup successfully
```

If "gs_basebackup: base backup successfully" is displayed in the command output, the backup is successful.

Go to the backup directory and run the **ll** command to view the files that are successfully backed up. The details are as follows:

```
[omm@standalone ~]$ cd /home/omm/backup/
[omm@standalone backup]$ ll
total 5.0M
-rw------- 1 omm dbgrp   206 Apr 12 10:28 backup_label
drwx------ 5 omm dbgrp 4.0K Apr 12 10:28 base
-rw------- 1 omm dbgrp 4.3K Apr 12 10:28 cacert.pem
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 global
-rw------- 1 omm dbgrp 4.7M Apr 12 10:28 gswlm_userinfo.cfg
-rw------- 1 omm dbgrp   21K Apr 12 10:28 mot.conf
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_clog
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_csnlog
-rw------- 1 omm dbgrp      0 Apr 12 10:28 pg_ctl.lock
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_errorinfo
-rw------- 1 omm dbgrp 4.6K Apr 12 10:28 pg_hba.conf
-rw------- 1 omm dbgrp 4.5K Apr 12 10:28 pg_hba.conf.bak
-rw------- 1 omm dbgrp 1.0K Apr 12 10:28 pg_hba.conf.lock
-rw------- 1 omm dbgrp 1.6K Apr 12 10:28 pg_ident.conf
drwx------ 4 omm dbgrp 4.0K Apr 12 10:28 pg_llog
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_logical
drwx------ 4 omm dbgrp 4.0K Apr 12 10:28 pg_multixact
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_notify
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_replslot
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_serial
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_snapshots
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_stat_tmp
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_tblspc
drwx------ 2 omm dbgrp 4.0K Apr 12 10:28 pg_twophase
-rw------- 1 omm dbgrp      4 Apr 12 10:28 PG_VERSION
drwx------ 3 omm dbgrp 4.0K Apr 12 10:28 pg_xlog
-rw------- 1 omm dbgrp   36K Apr 12 10:28 postgresql.conf
-rw------- 1 omm dbgrp   36K Apr 12 10:28 postgresql.conf.guc.bak
-rw------- 1 omm dbgrp 1.0K Apr 12 10:28 postgresql.conf.lock
-rw------- 1 omm dbgrp      0 Apr 12 10:28 postmaster.pid.lock
-rw------- 1 omm dbgrp 4.3K Apr 12 10:28 server.crt
-rw------- 1 omm dbgrp 1.8K Apr 12 10:28 server.key
-rw------- 1 omm dbgrp    56 Apr 12 10:28 server.key.cipher
-rw------- 1 omm dbgrp    24 Apr 12 10:28 server.key.rand
drwx------ 5 omm dbgrp 4.0K Apr 12 10:28 undo
[omm@standalone backup]$
```

The **backup_label** file exists in the current backup directory, which is different from the database data directory. The file records that **/home/omm/backup** is a backup directory and records backup-related content as follows:

```
[omm@standalone backup]$ cat backup_label
START WAL LOCATION: 0/3000028 (file 000000010000000000000002)
CHECKPOINT LOCATION: 0/30000A8
BACKUP METHOD: streamed
BACKUP FROM: master
START TIME: 20XX-XX-12 10:28:02 CST
LABEL: gs_basebackup base backup
```

```
[omm@standalone backup]$
```

In the preceding information:

START WAL LOCATION: LSN from which the WAL file starts.

CHECKPOINT LOCATION: check point.

BACKUP METHOD: backup method.

BACKUP FROM: backup object, for example, the current master node.

START TIME: time when the backup starts.

LABEL: label that records the type of backup performed by gs_basebackup.

The full data file has been physically backed up, which is the first prerequisite for PITR. Then, archive WAL files.

Step 4 After the backup is complete, add a table and insert data into the table.

After the backup is complete, create the **testPITR1** table in the database and initialize the data. In this case, new data is generated in the database.

The **testPITR1** table is created using a subquery as follows:

```
openGauss=# CREATE TABLE testPITR1 AS SELECT * FROM pg_class;
INSERT 0 803
```

The **testPITR1** table contains 803 records.

```
openGauss=# select count(*) from testpitr1;
 count
-------
   803
(1 row)
```

Step 5 After a table and data are added, create a restoration point **restore_point_1**.

Call the pg_create_restore_point() function to create a restoration point named **restore_point_1**.

The pg_create_restore_point(name Text) function is a system function used to create a restoration point. That is, a named point is defined in WALs, and the restoration process ends at this point. This function receives an input parameter of the text type, indicating the name of the current restoration point, and returns **pg_lsn**, that is the LSN.

Run the following statement to create restoration point **restore_point_1**:

```
openGauss=# SELECT pg_create_restore_point('restore_point_1');
 pg_create_restore_point
-------------------------
 0/404E1C0
(1 row)
```

The return value of the function is **0/404E1C0**, that is, the LSN.

Step 6 Copy WALs to a specified directory.

Copy WAL files to another directory, for example, **/home/omm/archive**.

Run the **\q** command to exit the database.

```
openGauss=# \q
```

Create the **archive** directory for storing backups of **pg_xlog**.

```
[omm@standalone ~]$ mkdir -p /home/omm/archive
```

Run the following command to copy the data in the **pg_xlog** directory to the **/home/omm/archive** directory:

```
[omm@standalone ~]$ cp -r /opt/huawei/install/data/dn/pg_xlog/* /home/omm/archive/
```

Go to the **/home/omm/archive** directory and check that WAL files are copied.

```
[omm@standalone archive]$ ll
total 225M
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000001
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000002
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000003
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000004
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000005
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000006
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000007
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000008
-rw------- 1 omm dbgrp   16M Apr 12 11:26 000000010000000000000009
-rw------- 1 omm dbgrp   16M Apr 12 11:26 00000001000000000000000A
-rw------- 1 omm dbgrp   16M Apr 12 11:26 00000001000000000000000B
-rw------- 1 omm dbgrp   16M Apr 12 11:26 00000001000000000000000C
-rw------- 1 omm dbgrp   16M Apr 12 11:26 00000001000000000000000D
-rw------- 1 omm dbgrp   16M Apr 12 11:26 00000001000000000000000E
drwx------ 2 omm dbgrp 4.0K Apr 12 11:26 archive_status
[omm@standalone archive]$
```

Note: It is normal that the number of WAL files in the directory in the actual environment is different from that in the manual.

WALs has been archived, which is the second prerequisite for PITR.

Step 7 Use WALs and backups to perform PITR.

Shut down the database.

```
[omm@standalone ~]$ gs_om -t stop
```

Clear all files in the data directory (**/opt/huawei/install/data/dn/**) of the current database:

```
[omm@standalone ~]$ rm -rf /opt/huawei/install/data/dn/*
```

Copy the full backup file in **/home/omm/backup/** to the **/opt/huawei/install/data/dn/** directory.

```
[omm@standalone ~]$ cp -r /home/omm/backup/* /opt/huawei/install/data/dn/
```

Copy the WAL files to the directory.

```
[omm@standalone ~]$ cp -r /home/omm/archive/* /opt/huawei/install/data/dn/pg_xlog/
```

Create the **recovery.conf** file in the **/opt/huawei/install/data/dn** directory. This file is used to configure the restoration progress.

```
[omm@standalone dn]$ vim /opt/huawei/install/data/dn/recovery.conf
```

Append the following content to the configuration file:

```
recovery_target_name = 'restore_point_1'
```

In this case, the data file is restored to **restore_point_1** in the configuration file.

Restart the database and log in to the database to check whether the created table exists.

```
[omm@standalone dn]$ gs_om -t start
[omm@standalone dn]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:09:38 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# \d
                          List of relations
 Schema |     Name      | Type  | Owner |                 Storage
--------+---------------+-------+-------+---------------------------------
 public | t2            | table | omm   | {orientation=row,compression=no}
 public | t3            | table | omm   | {orientation=row,compression=no}
 public | testpitr1 | table | omm   | {orientation=row,compression=no}
(3 rows)
```

The **testPITR1** table before **restore_point_1** is restored successfully. When a database fault occurs, openGauss can use WALs and physical backups to restore the database in PITR mode, preventing data loss.

3.2.5 Task 5: Checking and Clearing System Logs

To check and clear system logs (including database installation and deployment logs, system running logs, and logs generated during performance check), perform the following steps: When you go to the log directory, you can see that different directories are used to store different types of logs.

```
[omm@standalone ~]$ cd /var/log/omm/omm
[omm@standalone omm]$ ll
total 40K
drwx------. 3 omm dbgrp 4.0K Mar 25 20:43 asp_data
drwx------. 7 omm dbgrp 4.0K Mar 25 20:43 bin
drwx------. 3 omm dbgrp 4.0K Apr  3 11:32 cm
drwx------. 3 omm dbgrp 4.0K Mar 25 20:43 gs_profile
```

```
drwx------. 3 omm dbgrp 4.0K Mar 25 20:43 mem_log
drwx------. 2 omm dbgrp 4.0K Apr  9 10:21 om
drwx------. 3 omm dbgrp 4.0K Mar 25 20:42 pg_audit
drwx------. 3 omm dbgrp 4.0K Mar 25 20:42 pg_log
drwx------. 3 omm dbgrp 4.0K Mar 25 20:43 pg_perf
drwx------. 3 omm dbgrp 4.0K Mar 25 20:43 sql_monitor
```

In the preceding information:

pg_lo: directory for storing system run logs.

om: directory for storing O&M management logs, such as installation, deployment, and performance logs.

cm: directory for storing cluster management logs.

pg_audit: directory for storing audit logs.

bin: directory for storing logs generated by the **gs_guc** and **gs_ctl** commands.

sql_monitor: directory for storing slow SQL logs.

Step 1 Log in to the database server as the **omm** user and go to the directory where system logs are stored.

Run the following commands to store system logs in the **/var/log/omm/omm/pg_log/dn_6001** directory:

```
[omm@standalone pg_log]$ cd /var/log/omm/omm/pg_log/dn_6001
[omm@standalone dn_6001]$ ll
total 6.6M
-rw-------. 1 omm dbgrp 5.7K Mar 29 11:09 postgresql-2024-03-29_110949.log
-rw-------. 1 omm dbgrp 5.7K Apr  1 09:19 postgresql-2024-04-01_091927.log
-rw-------. 1 omm dbgrp 6.6M Apr  3 10:59 postgresql-2024-04-03_105858.log
```

Step 2 View the content of a system run log.

After the database server is restarted, a new log file, for example, **postgresql-2024-04-03_105858.log**, is generated. Run the following command to view the log content:

```
[omm@standalone dn_6001]$ cat postgresql-2024-04-03_105858.log
...
2024-04-03 11:07:05.098 660cc7d9.5060 metadatabase 140140081239808 gs_clean 0 dn_6001 00000
0 [BACKEND] LOG:  statement: select name, setting from pg_settings where name in
('connection_info')
2024-04-03 11:07:05.103 660cc7d9.5060 metadatabase 140140081239808 gs_clean 0 dn_6001 00000
0 [BACKEND] LOG:  statement: SET connection_info =
'{"driver_name":"libpq","driver_version":"(openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29
03:09:38 commit 0 last mr  "}'
2024-04-03 11:07:05.104 660cc7d9.5060 metadatabase 140140081239808 gs_clean 0 dn_6001 00000
0 [BACKEND] LOG:  statement: SELECT NSPNAME FROM PG_NAMESPACE WHERE NSPNAME LIKE
'pg_temp_%'
2024-04-03 11:07:05.105 660cc7d9.5060 metadatabase 140140081239808 gs_clean 0 dn_6001 00000
0 [BACKEND] LOG:  statement: SELECT SESSIONID, TEMPID, TIMELINEID FROM PG_DATABASE D,
PG_STAT_GET_ACTIVITY_FOR_TEMPTABLE() AS S WHERE S.DATID = D.OID AND D.DATNAME =
'metadatabase'
```

```
2024-04-03 11:07:05.146 660cc7d9.5060 postgres 140140081239808 gs_clean 0 dn_6001 00000    0
[BACKEND] LOG:    statement: select name, setting from pg_settings where name in
('connection_info')
2024-04-03 11:07:05.149 660cc7d9.5060 postgres 140140081239808 gs_clean 0 dn_6001 00000    0
[BACKEND] LOG:    statement: SET connection_info =
'{"driver_name":"libpq","driver_version":"(openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29
03:09:38 commit 0 last mr    "}'
2024-04-03 11:07:05.149 660cc7d9.5060 postgres 140140081239808 gs_clean 0 dn_6001 00000    0
[BACKEND] LOG:    statement: SELECT NSPNAME FROM PG_NAMESPACE WHERE NSPNAME LIKE
'pg_temp_%'
2024-04-03 11:07:05.150 660cc7d9.5060 postgres 140140081239808 gs_clean 0 dn_6001 00000    0
[BACKEND] LOG:    statement: SELECT SESSIONID, TEMPID, TIMELINEID FROM PG_DATABASE D,
PG_STAT_GET_ACTIVITY_FOR_TEMPTABLE() AS S WHERE S.DATID = D.OID AND D.DATNAME =
'postgres'
2024-04-03 11:07:17.444 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 00000    0
[DBL_WRT] LOG:    [batch flush] DW truncate end: file_head[dwn 347, start 14019], total_pages 0
2024-04-03 11:07:17.448 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 01000    0
[BACKEND] WARNING:    replicationSlotMinLSN is InvalidXLogRecPtr!!!
2024-04-03 11:07:17.448 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 01000    0
[BACKEND] WARNING:    replicationSlotMaxLSN is InvalidXLogRecPtr!!!
2024-04-03 11:07:17.449 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 00000    0
[BACKEND] LOG:    CreateCheckPoint PrintCkpXctlControlFile: [checkPoint] oldCkpLoc:D/B16CACC0,
oldRedo:D/B16CAC40, newCkpLoc:D/B16CAE60, newRedo:D/B16CADE0, preCkpLoc:D/B16CAB20
2024-04-03 11:07:17.449 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 00000    0
[BACKEND] LOG:    will update control file (create checkpoint), shutdown:0
2024-04-03 11:07:17.451 660cc5f6.6127 [unknown] 140139860850432 dn_6001 0 dn_6001 00000    0
[BACKEND] LOG:    attempting to remove WAL segments older than log file
000000010000000D0000009F
2024-04-03 11:07:20.794 660cc7e8.5060 postgres 140140081239808 Clean Statement thread 0
dn_6001 00000    0 [BACKEND] LOG:    clean statement thread start
```

Step 3 Delete logs related to system running.

Check whether the logs contain error or fatal information. If the logs do not contain error or fatal information and the disk space usage is about to reach the threshold, delete the logs as follows:

```
[omm@standalone dn_6001]$ rm -f postgresql-2024-04-03_105858.log
[omm@standalone dn_6001]$ ll
-rw-------. 1 omm dbgrp 5.7K Mar 29 11:09 postgresql-2024-03-29_110949.log
-rw-------. 1 omm dbgrp 5.7K Apr   1 09:19 postgresql-2024-04-01_091927.log
```

Step 4 View logs generated during installation and deployment.

Run the following commands to check that the logs generated during database installation and deployment are stored in the **/var/log/omm/omm/om** directory:

```
[omm@standalone dn_6001]$ cd /var/log/omm/omm/om
[omm@standalone om]$ ll
-rw-------. 1 omm dbgrp    11K Mar 25 20:43 gs_install-2024-03-25_204212.log
-rw-------. 1 omm dbgrp 485K Mar 27 23:32 gs_local-2024-03-25_204005.log
-rw-------. 1 omm dbgrp      0 Mar 27 23:33 gs_local-2024-03-27_233301.log
-rw-------. 1 omm dbgrp      0 Mar 27 23:34 gs_local-2024-03-27_233401.log
-rw-------. 1 omm dbgrp      0 Mar 27 23:35 gs_local-2024-03-27_233501.log
```

```
-rw-------. 1 omm dbgrp      0 Mar 27 23:36 gs_local-2024-03-27_233601.log
-rw-------. 1 omm dbgrp   47K Mar 28 09:56 gs_local-2024-03-27_233702.log
-rw-------. 1 omm dbgrp   89K Mar 28 09:32 gs_om-2024-03-25_204324.log
-rw-------. 1 omm dbgrp   39K Mar 29 10:47 gs_om-2024-03-28_100603.log
-rw-------. 1 omm dbgrp 9.0K Apr   3 11:32 gs_om-2024-03-29_110410.log
-rw-------. 1 omm dbgrp 8.4K Mar 25 20:41 gs_preinstall-2024-03-25_203913.log
```

The logs in red and bold are generated during database preinstallation and installation.

Step 5 View logs generated during preinstallation.

Run the following command to view preinstallation logs. The logs help you view, analyze, and locate faults if error information is generated during preinstallation.

```
[omm@standalone om]$ cat gs_preinstall-2024-03-25_203913.log
[2024-03-25
20:39:13.728073][2648][gs_preinstall(initGlobals:431)][gs_preinstall][LOG][Step1]:Parsing the
configuration file.
[2024-03-25
20:39:35.704715][2700][gs_preinstall(initGlobals:447)][gs_preinstall][LOG][Step1]:Successfully parsed
the configuration file
...
[2024-03-25 20:40:19.235874][2700][gs_preinstall][DEBUG]:Successfully created [omm] user on all
nodes.
[2024-03-25 20:40:19.255942][2700][gs_preinstall][LOG]:Generate cluster user password files
successfully.
...
[2024-03-25
20:41:10.662897][2700][PreinstallImpl.py(doPreInstallSucceed:1251)][gs_preinstall][LOG][Step17]:Suc
cessfully set finish flag.
[2024-03-25 20:41:10.663058][2700][gs_preinstall][LOG]:Preinstallation succeeded.
```

Step 6 View the logs generated during the database installation.

Run the following command to view installation logs. The logs help you view, analyze, and locate faults if error information is generated during installation.

```
[omm@standalone om]$ cat gs_install-2024-03-25_204212.log
...
[2024-03-25 20:42:12.708360][5263][gs_install][LOG]:begin deploy..
[2024-03-25 20:42:12.708418][5263][gs_install][DEBUG]:Installing application
...
[2024-03-25 20:42:18.226192][5263][gs_install][LOG]:begin install Cluster..
[2024-03-25 20:42:18.226497][5263][gs_install][LOG]:Installing applications on all nodes.
...
[2024-03-25 20:43:25.587727][5263][gs_install][LOG]:Successfully started cluster.
[2024-03-25
20:43:25.587801][5263][InstallImpl.py(doStart:678)][gs_install][DEBUG][Step7]:Successfully started
the cluster.
[2024-03-25 20:43:25.609800][5263][gs_install][LOG]:Successfully installed application.
[2024-03-25 20:43:25.609938][5263][gs_install][LOG]:end deploy..
```

Step 7 Delete logs generated during preinstallation and installation.

If the disk space is about to reach the threshold or is insufficient, you can perform the deletion.

```
[omm@standalone om]$ rm -f gs_preinstall-2024-03-25_203913.log
[omm@standalone om]$ rm -f gs_preinstall-2024-03-25_203913.log
```

Step 8 View a performance log.

The performance log is stored in the **/var/log/omm/omm/om** directory. You can view the following log in the directory:

```
[omm@standalone om]$ ll
-rw-------. 1 omm dbgrp 9.8K Apr  3 11:32 gs_checkperf-2024-04-03_113205.log
-rw-------. 1 omm dbgrp  11K Mar 25 20:43 gs_install-2024-03-25_204212.log
```

The log is stored in the same directory as the log generated during installation and deployment.

If the environment is newly installed and no performance test is performed, the log does not exist. In this case, you can perform a performance test as follows:

```
[omm@standalone om]$ gs_checkperf -i PMK --detail
Cluster statistics information:
Host CPU usage rate:
     Host total CPU time                    :    14223460.000 Jiffies
     Host CPU busy time                     :     1340070.000 Jiffies
     Host CPU iowait time                   :     5250.000    Jiffies
     Host CPU busy time ratio               :     9.42          %
     Host CPU iowait time ratio             :     .04           %
MPPDB CPU usage rate:
     MPPDB CPU time % in busy time          :     82.63         %
     MPPDB CPU time % in total time         :     7.79          %
Shared buffer hit rate:
     Shared Buffer Reads                    :     813284
     Shared Buffer Hits                     :     45840673
     Shared Buffer Hit ratio                :     98.26         %
In memory sort rate:
     In-memory sort count                   :     0
     In-disk sort count                     :     0
     In-memory sort ratio                   :     0
I/O usage:
     Number of files                        :     145
     Physical Reads                         :     515466
     Physical Writes                        :     566477
     Read Time                              :     11866305    ms
     Write Time                             :     5517317     ms
Disk usage:
     DB size                                :     2608          MB
     Total Physical writes                  :     566477
     Average Physical write                 :     102672.55
     Maximum Physical write                 :     566477
Activity statistics:
     Active SQL count                       :     4
```

```
    Session count                                 :     8
    ...
```

After the performance test is complete, go to the directory where the performance log is stored again. A log similar to **gs_checkperf-*xxx*.log** is displayed.

Step 9 View the content in a performance log.

Run the **cat** or **more** command. In the following example, the **cat** command is used:

```
[omm@standalone om]$ cat gs_checkperf-2024-04-03_113205.log
[2024-04-03 11:32:10.904258][22540][gs_checkperf][DEBUG]:Check cm_ctl is failed msg: .
[2024-04-03 11:32:10.905025][22540][gs_checkperf][DEBUG]:Checking PMK performance.
[2024-04-03 11:32:11.558898][22540][gs_checkperf][DEBUG]:Checking cluster status.
[2024-04-03 11:32:23.724580][22540][gs_checkperf][DEBUG]:Successfully install pmk schema.
[2024-04-03 11:32:24.043230][22540][gs_checkperf][DEBUG]:Getting PMK meta data.
...
[2024-04-03 11:32:30.886789][22540][gs_checkperf][DEBUG]:Successfully parsed session cpu stat of
single host into the file[/opt/huawei/tmp/sessionCpuTempFile_22540_standalone].
[2024-04-03 11:32:31.107596][22540][gs_checkperf][DEBUG]:Successfully parsed session IO stat
of single host into the file[/opt/huawei/tmp/sessionIOTempFile_22540_standalone].
[2024-04-03 11:32:31.359037][22540][gs_checkperf][DEBUG]:Successfully parsed session memory
stat of single host into the file[/opt/huawei/tmp/sessionMemTempFile_22540_standalone].
...
```

Step 10 Delete the performance log.

If the disk space is about to reach the threshold or is insufficient, you can perform the deletion.

```
[omm@standalone om]$ rm -f gs_checkperf-2024-04-03_113205.log
```

Step 11 Check and clear a gs_guc log.

Go to the **/var/log/omm/omm/bin/gs_guc** directory and view the log in the directory.

```
[omm@standalone ~]$ cd /var/log/omm/omm/bin/gs_guc
[omm@standalone gs_guc]$ ll
total 8.0K
-rw-------. 1 omm dbgrp 6.7K Mar 29 14:18 gs_guc-2024-03-25_204312-current.log
[omm@standalone gs_guc]$
```

Run the following command to view the content in a gs_guc log:

```
[omm@standalone gs_guc]$ cat gs_guc-2024-03-25_204312-current.log     # Run the query
command.
## Some parts of the log content is as follows:
[2024-03-25 20:43:12] expected instance path: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: ssl=on: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: ssl_cert_file='server.crt': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: ssl_key_file='server.key': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: ssl_ca_file='cacert.pem': [/opt/huawei/install/data/dn/postgresql.conf]

Total instances: 1. Failed instances: 0.
```

```
Success to perform gs_guc!
...
```

Note: There is only the log **gs_guc-2024-03-25_204312-current.log** in the environment. In the production environment, there are logs that do not contain *xxxx*-**current**. You can delete them.

Run the following command to delete the gs_guc log:

```
[omm@standalone gs_guc]$ rm -f gs_guc-2024-03-25_204312-current.log
```

Step 12 Check and clear a slow SQL log.

Go to the **/var/log/omm/omm/sql_monitor/dn_6001** directory and view the log in the directory.

```
[omm@standalone ~]$ cd /var/log/omm/omm/sql_monitor/dn_6001
[omm@standalone dn_6001]$ ll
total 0
-rw-------. 1 omm dbgrp 0 Mar 26 00:00 slow_query_log-2024-03-26_000000.log
-rw-------. 1 omm dbgrp 0 Mar 28 00:00 slow_query_log-2024-03-28_000000.log
[omm@standalone dn_6001]$
```

Run the following command to view the content in a slow SQL log:

```
[omm@standalone dn_6001]$ cat slow_query_log-2024-03-28_000000.log
```

Note: If no valid slow SQL log is generated in the environment, no content is displayed.

Run the following command to delete the slow SQL log:

```
[omm@standalone dn_6001]$ rm -f slow_query_log-2024-03-28_000000.log
```

Step 13 Check and clear a gs_ctl log.

Go to the **/var/log/omm/omm/bin/gs_ctl** directory and view the log in the directory.

```
[omm@standalone ~]$ cd /var/log/omm/omm/bin/gs_ctl/
[omm@standalone gs_ctl]$ ll
total 12K
-rw-------. 1 omm dbgrp 11K Apr  9 10:29 gs_ctl-2024-03-25_204315-current.log
[omm@standalone gs_ctl]$
```

Run the following command to view the content in a gs_ctl log:

```
[omm@standalone gs_ctl]$ cat gs_ctl-2024-03-25_204315-current.log     # Run the query command.
## Some parts of the log content is as follows:
[2024-03-28 22:08:58.925][19241][][gs_ctl]: gs_ctl stopped ,datadir is /opt/huawei/install/data/dn
[2024-03-28 22:09:01]
[2024-03-28 22:09:01.983][20101][][gs_ctl]: gs_ctl started,datadir is /opt/huawei/install/data/dn
[2024-03-28 22:09:02.031][20101][][gs_ctl]: waiting for server to start...
[2024-03-28 22:09:07.054][20101][][gs_ctl]:    done
```

```
[2024-03-28 22:09:07.054][20101][][gs_ctl]: server started (/opt/huawei/install/data/dn)
[2024-03-28 22:09:08]
[2024-03-28 22:09:08.853][21020][][gs_ctl]: gs_ctl query ,datadir is /opt/huawei/install/data/dn
 HA state:
        local_role                     : Normal
        static_connections             : 0
        db_state                       : Normal
        detail_information             : Normal

 Senders info:
No information
 Receiver info:
No information
...
```

Note: There is only the log **gs_guc-2024-03-25_204312-current.log** in the environment. In the production environment, there are logs that do not contain *xxxx*-**current**. You can delete them.

Run the following command to delete the gs_ctl log:

```
[omm@standalone gs_ctl]$ rm -f gs_ctl-2024-03-25_204315-current.log
```

4 SQL Execution Engine

4.1 Introduction

4.1.1 About This Lab

This lab offers exercises for *Chapter 4 SQL Engine* to deepen the understanding of the SQL execution plan and working principles of key operators.

The execution process of SQL statements includes lexical parsing, syntax parsing, semantic parsing, query rewriting, path generation, plan generation, and plan execution. In the plan execution phase, a series of operators are used to perform actual database operations. You can use the EXPLAIN syntax to view information about the execution plan, including the operators used in the execution plan.

This exercise uses different SQL statements to cover different types of operators and interprets them in detail through execution plans.

4.1.2 Objectives

Understand the functions and application scenarios of different types of operators.

Interpret SQL execution plans that contain different types of operators.

4.2 Scan Operator Execution Plan

This section describes the application scenarios and execution plans of SQL statements of common scan operators, such as Seq Scan, Index Scan, Subquery Scan, TID Scan, and Function Scan.

4.2.1 Seq Scan Operators

Sequential scan is one of the most basic scan types. During a sequential scan, the database engine reads the entire table row by row and checks each row to determine whether the query conditions are met. Sequential scans do not depend on indexes. Therefore, sequential scans may be the most effective choice when the table is small or the query requires a large amount of data in the table.

Step 1 Create the **test** table.

```
create table test(
id int,
name text,
```

```
age int);
```

Step 2 Use the generate_series function to insert 1000 rows of data into the table.

```
openGauss=# insert into test select
generate_series(1,1000),
'test'::text,
generate_series(1,1000);
INSERT 0 1000
openGauss=# select * from test limit 5;
 id | name | age
----+------+-----
  1 | test |   1
  2 | test |   2
  3 | test |   3
  4 | test |   4
  5 | test |   5
(5 rows)
```

Step 3 Perform simple queries using EXPLAIN.

```
openGauss=# explain select id from test where id > 50;
                  QUERY PLAN
-----------------------------------------------------
 Seq Scan on test   (cost=0.00..18.50 rows=950 width=4)
   Filter: (id > 50)
(2 rows)
```

Step 4 Interpret the execution plan.

Seq Scan indicates that openGauss performs a full table scan. **test** indicates the name of the table to be scanned.

cost indicates a measure of the estimated cost of performing the operation. It usually consists of two values: initial cost and total cost. **0.00** is the estimated cost before the data is returned. In this case, the cost is 0, indicating that data may be returned from the start of the scan. **18.50** is the estimated total cost for the entire scan. (Note that part of data in EXPLAIN in this document is the estimated result, which may be different from the actual result due to various factors.)

rows indicates the estimated number of rows that meet the conditions. In this example, the number of rows is **950**. This is estimated by the optimizer based on statistics. It is estimated that the value of the **id** column in 950 rows of data is greater than 50.

width indicates the average number of bytes in each row of data. In this case, the value is **4**, in bytes. This number helps the database estimate the memory consumption of the query.

Filter: (id > 50) indicates that this filter condition is used when a sequential scan checks each row of data to determine whether the row is included in the result.

4.2.2 Index Scan Operators

Indexes are used to quickly find table rows that meet specific conditions. When performing an index scan, the database first traverses the index to find an index entry that matches the query condition, and then accesses a corresponding row in the table according to a pointer in the index entry.

Step 1 Create indexes.

```
openGauss=# create index test_id on test(id);
CREATE INDEX
```

Step 2 Execute the query statement using EXPLAIN.

The query statement for the condition **id > 50** is as follows:

```
openGauss=# explain select id from test   where id > 50;
                  QUERY PLAN
-----------------------------------------------------------
 Seq Scan on test   (cost=0.00..18.50 rows=950 width=4)
   Filter: (id > 50)
(2 rows)
```

The query statement for the condition **id < 20** is as follows:

```
openGauss=# explain select id from test   where id < 20;
                          QUERY PLAN
------------------------------------------------------------------------------
 [Bypass]
 Index Only Scan using test_id on test   (cost=0.00..8.60 rows=20 width=4)
   Index Cond: (id < 20)
(3 rows)
```

Step 3 Interpret and compare execution plans.

When **id < 20** is used to query, **Seq Scan** changes to **Index Only Scan**. **using test_id on test** indicates that the created index **test_id** is used. Accordingly, **Filter** changes to **Index**.

The **[Bypass]** mark indicates that some conventional processing steps are bypassed during query execution, and a better method or technology is directly used to improve query efficiency. Generally, it occurs in the following scenarios:

- When indexes fully cover query requirements, all data required for query can be directly obtained from indexes without accessing data pages of tables.
- When indexes are efficient, accessing an index is more efficient than performing any type of table scan for very small result sets.

For the second point, when the condition **id > 50** is used for queries, sequential scans are still used because the query covers 95% of the data in the table. In this case, if indexes are used, the optimizer expects to retrieve most data pages and may involve a large number of random I/O operations. Therefore, despite its perceived inefficiency, a full table scan may actually be a more efficient way. **id < 20** covers only about 2% of the

data. In this case, indexes can be used to quickly locate these rows, greatly reducing the amount of data that must be read.

Step 4 Add the AND query condition to EXPLAIN.

```
openGauss=# explain select id from test where id < 20 and age > 0;
                        QUERY PLAN
------------------------------------------------------------------
 Index Scan using test_id on test  (cost=0.00..8.65 rows=20 width=4)
   Index Cond: (id < 20)
   Filter: (age > 0)
(3 rows)
```

Step 5 Interpret the execution plan.

Compared with **where id < 20**, **where id < 20 and age > 0** does not have **[Bypass]** and **Index Only Scan** changes to **Index Scan**. Only the **test_id** index of **id** is created. However, the **id** and **age** columns are used in the condition, the index does not cover both the two columns. In addition, **Index Cond: (id < 20)** is used for columns with indexes, and **Filter: (age > 0)** is used for columns without indexes.

Step 6 Delete the **id** column index and add the **(id,age)** index.

```
openGauss=# drop index test_id;
DROP INDEX
openGauss=# create index test_id_age on test(id, age);
CREATE INDEX
```

Step 7 Perform the AND query again.

```
openGauss=# explain select id from test where id < 20 and age > 0;
                              QUERY PLAN
-----------------------------------------------------------------------
 Bitmap Heap Scan on test  (cost=4.46..10.75 rows=20 width=4)
   Recheck Cond: ((id < 20) AND (age > 0))
   ->  Bitmap Index Scan on test_id_age  (cost=0.00..4.45 rows=20 width=0)
         Index Cond: ((id < 20) AND (age > 0))
(4 rows)
```

Step 8 Interpret the execution plan.

Bitmap Index Scan is the first step of executing a plan. The database uses the **test_id_age** index to search for rows that meet the conditions **id < 20** and **age > 0**. The cost starts at **0.00** and ends at **4.45**. It is estimated that 20 rows are found, which is less than the query result for estimated **8.65** when the **where id < 20 and age > 0** condition is used for queries for the first time.

Bitmap Heap Scan is the second step of the execution plan. It uses the bitmap generated by **Bitmap Index Scan** to access the actual data page and retrieve the rows that meet the conditions. The cost starts at **4.46** and ends at **10.75**. The expected operation result is also 20 rows. **Recheck Cond**: A bitmap index scan may only roughly filter rows when creating a bitmap. Therefore, when accessing an actual data row, you need to recheck the conditions to ensure that the row meets all the conditions.

When a query involves multiple conditions that are supported by the same index or multiple indexes, **Bitmap Index Scan** can effectively combine the results of the indexes. This method is more effective than a single index scan when processing multi-condition queries, especially when each condition alone is not enough to limit the result set to the extreme, but the combination of conditions can significantly reduce the number of target rows.

Step 9 Drop the **test** table.

```
openGauss=# drop table test;
DROP TABLE
```

4.2.3 Subquery Scan Operators

Subquery Scan is usually used in queries involving subqueries. A subquery can be part of a SELECT statement and is usually used as a filter condition, a data source, or a column value calculated in a SELECT clause.

Step 1 Create the **employees** table.

```
openGauss=# CREATE TABLE employees (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100),
        age INTEGER,
        salary DECIMAL(10, 2),
        department VARCHAR(50));

NOTICE:  CREATE TABLE will create implicit sequence "employees_id_seq" for serial column
"employees.id"
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "employees_pkey" for table
"employees"
CREATE TABLE
```

Step 2 Insert data.

```
openGauss=# INSERT INTO employees (name, age, salary, department) VALUES
                ('Alice', 30, 50000.00, 'HR'),
                ('Bob', 22, 40000.00, 'Marketing'),
                ('Charlie', 28, 45000.00, 'IT'),
                ('David', 35, 55000.00, 'Finance'),
                ('Eve', 45, 62000.00, 'IT'),
                ('Frank', 32, 48000.00, 'Marketing');
INSERT 0 6
```

Step 3 Execute the subquery statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT *
                FROM (
                    SELECT name, salary
                    FROM employees
                    WHERE salary > (
```

```
                        SELECT AVG(salary)
                        FROM employees
                    )
                    ORDER BY salary DESC
                    LIMIT 10
                ) AS top_earners
                WHERE salary > (
                    SELECT AVG(salary)
                    FROM employees
                );
                        QUERY PLAN
------------------------------------------------------------------------------
 Subquery Scan on top_earners  (cost=39.38..39.53 rows=3 width=234)
   Filter: (top_earners.salary > $0)
   InitPlan 1 (returns $0)
     ->  Aggregate  (cost=12.62..12.62 rows=1 width=48)
           ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=16)
   ->  Limit  (cost=26.75..26.78 rows=10 width=234)
         InitPlan 2 (returns $1)
           ->  Aggregate  (cost=12.62..12.62 rows=1 width=48)
                 ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=16)
         ->  Sort  (cost=14.13..14.30 rows=70 width=234)
               Sort Key: public.employees.salary DESC
               ->  Seq Scan on employees  (cost=0.00..12.61 rows=70 width=234)
                     Filter: (salary > $1)
(13 rows)
```

Step 4 Interpret the execution plan.

A query statement containing subqueries is executed. That is, select is nested in **select * from**. The query is complex, and the EXPLAIN output is complex. Many operators that have yet described are displayed. This is because subqueries are usually optimized, especially in scenarios where simpler execution plans can be used to solve the query problem. To ensure that Subquery Scan appears in a query plan, you need to create a scenario where the result of a subquery is directly used as the data source for another query. Therefore, a complex SQL statement is constructed so that the subquery is not optimized, which reflects the power of query optimization. Only the Subquery Scan operator in the first row is concerned. Other operators will be described later.

Subquery Scan on top_earners indicates that openGauss scans the result of the internal query **top_earners**. This internal query generates a temporary result set through a more complex operation. The result set contains the top 10 employees whose salaries are higher than the average salary of the company.

The values of cost are from 39.38 to 39.53. It is estimated that three rows will be found. This cost estimate includes the cost of internal queries and the cost of filtering these results to include only those records whose salaries are higher than the company's average.

Filter: The filter condition is **top_earners.salary > $0**. In this case, **$0** indicates the value generated by an initial plan (**InitPlan 1 (returns $0)**), that is, the average salary of a company.

4.2.4 TID Scan Operators

Tuple identifier scan (TID Scan) is a special scan method that directly uses TIDs to access rows in a table. A TID is an internal identifier of the location of a row in its physical page and is usually used for low-level operations, such as row-level locking or specific system management functions.

Step 1 Execute the query statement using EXPLAIN ctid.

```
openGauss=# EXPLAIN SELECT * FROM employees WHERE ctid = '(0,1)';
                        QUERY PLAN
-----------------------------------------------------------
 Tid Scan on employees   (cost=0.00..4.01 rows=1 width=360)
   TID Cond: (ctid = '(0,1)'::tid)
(2 rows)
```

Step 2 Interpret the execution plan.

To enable TID Scan in the query plan, you need to manually specify a TID to select rows in a query. This can be achieved by using the **ctid** column, which is a system column that stores the physical location of a row.

In practice, using a TID to directly access a row is rare because it depends on the physical location of the row, which is usually opaque and volatile in databases.

TID Scan indicates that a query plan accesses data in a table based on the physical location (TID) of a row. This is an efficient data access mode because it avoids regular index scans or sequential scans and directly jumps to the physical location of the data row.

rows=1 indicates that one row is expected to be found because the query specifies an accurate row position through ctid.

The TID Cond section describes the TID condition, that is, the **ctid** value used for query. **(ctid = '(0,1)'::tid)** indicates that the query is searching for the row whose **ctid** value is **(0,1)**. **::tid** is a type conversion of openGauss, indicating that a character string is converted to the TID type.

4.2.5 Function Scan Operators

Function Scan is used in an execution plan, indicating that the database is calling one or more functions and the return values of these functions are processed as rows in the table. To construct an EXPLAIN statement that displays Function Scan, you can use a built-in function that returns a set of tables (set-returning function).

Step 1 Call the generate_series() function.

```
openGauss=# SELECT * FROM generate_series(1, 10);
 generate_series
-----------------
               1
               2
               3
               4
```

```
             5
             6
             7
             8
             9
             10
(10 rows)
```

The generate_series function is called, which is a built-in function that returns a table set. The sequence table set of 1 to 10 is returned as the result. This function has been used to fill in the table. Function Scan can also be used for more complex function calls, for example, dynamically pulling data from external data sources.

Step 2 Execute the function query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT * FROM generate_series(1, 10);
                          QUERY PLAN
------------------------------------------------------------------------
 Function Scan on generate_series  (cost=0.00..10.00 rows=1000 width=4)
(1 row)
```

A Function Scan is displayed when executing the EXPLAIN command because the query plan needs to scan the sequence rows generated by the generate_series function.

Step 3 Interpret the execution plan.

The **rows** value indicates that the query optimizer expects that this function call will generate about 1000 rows of data. This number is an estimated value, which is usually based on the historical execution statistics of the function or the definition feature of the function. In practice, only 10 rows of data are generated using **generate_series(1, 10)**. Therefore, 1000 rows are the default estimate or a conservative preset of the optimizer.

4.3 Join Operator Execution Plan

This section describes the execution plans of SQL statements of the hash join, merge join, and nested loop join operators, and explains the application scenarios and execution plans of related scan operators.

4.3.1 Hash Join Operators

Hash join is one of the common join modes in SQL databases. It is especially suitable for processing large-scale data sets that cannot be joined directly through indexes. In openGauss, the optimizer usually selects hash join when the following conditions are met:

- Equi-join condition: Hash join requires that the condition be equi-join (for example, **A = B**), which allows the database to build a hash table for one of the tables (or both).
- Large datasets: Hash join is applicable to processing a large amount of data, especially when the sizes of two tables differ greatly.

- Without valid indexes: Hash join is more effective when there is no valid index on the column to be joined or the index does not improve performance.
- Cost effectiveness: Compared with other join methods (such as nested loop join or merge join), hash join is more efficient in processing a large amount of data because it reduces the time complexity of data search by creating a hash table.

Step 1 Create the **departments** table to implement the join operation.

```
openGauss=# CREATE TABLE departments (
                    department_id SERIAL PRIMARY KEY,
                    department_name VARCHAR(50)
                );
NOTICE:  CREATE TABLE will create implicit sequence "departments_department_id_seq" for serial column "departments.department_id"
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "departments_pkey" for table "departments"
CREATE TABLE
```

Step 2 Execute the hash join query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT e.name, d.department_name FROM employees e JOIN departments d ON e.department = d.department_name;
                          QUERY PLAN
------------------------------------------------------------------------------
 Hash Join  (cost=14.70..37.70 rows=561 width=336)
   Hash Cond: ((d.department_name)::text = (e.department)::text)
   ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118)
   ->  Hash  (cost=12.09..12.09 rows=209 width=336)
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336)
(5 rows)
```

Step 3 Interpret the execution plan.

The hash join result set contains 561 rows, and the average byte width of each row in the result set is 336.

Hash Cond indicates the join condition used by hash join. In this case, it indicates the department name, which is compared in text format.

Seq Scan on departments d scans the **departments** table in sequence.

Hash (cost=12.09..12.09 rows=209 width=336) indicates data used to construct a hash table. The data is in the **employees** table.

-> Seq Scan on employees e indicates the sequential scan of the **employees** table before a hash table is generated, reflecting the cost and number of rows of data read from the table.

The estimated number of rows is greatly different. Actually, the two tables contain only single-digit rows, and the actual query result contains only six rows. It can be seen that this is only an estimated result, especially when there is a small amount of actual data, which may cause a large deviation. In the actual production environment, when there is a large amount of data, the error of the estimated value is generally within an acceptable range, and is still meaningful as a reference for selecting an execution plan. You can add

the analyze keyword to obtain the actual cost and number of rows of the query statement.

Step 4 Use EXPLAIN analyze to print an execution plan.

```
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=14.70..37.70 rows=561 width=336) (actual time=0.211..0.217 rows=6 loops=1)
   Hash Cond: ((d.department_name)::text = (e.department)::text)
   ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.010..0.010 rows=4 loops=1)
   ->  Hash  (cost=12.09..12.09 rows=209 width=336) (actual time=0.021..0.021 rows=6 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 257kB
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336) (actual
time=0.006..0.009 rows=6 loops=1)
 Total runtime: 0.556 ms
(7 rows)
```

After EXPLAIN analyze is used, the actual cost and number of rows of the query statement are displayed.

4.3.2 Merge Join Operators

Merge join is an efficient SQL join method. It is usually used to process sorted data sets or optimize joins through sorting. In openGauss, the selection of merge join depends on the following conditions:

- Sorted keys: Merge join is best suited for those join keys that have been sorted or can be sorted efficiently.
- Cost and data volume: When the dataset is large and the pre-sorting cost is low, merge join is more efficient than other join types (such as nested loop join or hash join).
- Valid indexes: The optimizer may prefer to use merge join if there is a valid index on the joined column, especially if the index has been sorted by join key.

Merge join requires that the columns involved in the join in the two tables be sorted in advance. If these columns are not sorted in advance, the database automatically sorts them and then performs the join.

Step 1 Disable hash join and enable merge join.

You can use parameters to disable hash join and enable merge join, use the same query statement as hash join, and observe the execution plan.

Note: The default values of **enable_mergejoin**, **enable_hashjoin**, and **enable_nestloop** are **off**, **on**, and **off**, respectively.

```
openGauss=# SET enable_hashjoin = off;
SET
openGauss=# set enable_mergejoin = on;
SET
```

Step 2 Execute the merge join query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                          QUERY PLAN
------------------------------------------------------------------------------
 Merge Join  (cost=59.86..69.32 rows=561 width=336)
   Merge Cond: ((e.department)::text = (d.department_name)::text)
   ->  Sort  (cost=20.14..20.67 rows=209 width=336)
         Sort Key: e.department
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336)
   ->  Sort  (cost=39.72..41.06 rows=537 width=118)
         Sort Key: d.department_name
         ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118)
(8 rows)
```

Step 3 Interpret the execution plan.

After hash join is disabled and merge join is enabled, merge join is selected in the execution plan. The estimation is inaccurate. The reason is the same as that of hash join. The estimated cost of merge join is **69.32**, which is higher than the cost **37.70** of hash join. Therefore, hash join is selected when hash join is not disabled.

In addition, merge join requires that the columns involved in the join in the two tables be in the sorting state in advance. In the execution plan, the sorting operation is performed on both tables.

Step 4 Use EXPLAIN analyze to print an execution plan.

If no operation is performed for a long time, the session is reconnected and hash join is disabled because the parameters set in set mode take effect only in the session. After the reconnection, the default values are restored. By default, hash join is enabled. Step 1 needs to be performed again.

```
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                                                QUERY PLAN
-----------------------------------------------------------------------------------------
 Merge Join  (cost=59.86..69.32 rows=561 width=336) (actual time=0.093..0.109 rows=6 loops=1)
   Merge Cond: ((e.department)::text = (d.department_name)::text)
   ->  Sort  (cost=20.14..20.67 rows=209 width=336) (actual time=0.033..0.033 rows=6 loops=1)
         Sort Key: e.department
         Sort Method: quicksort  Memory: 25kB
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336) (actual
time=0.009..0.012 rows=6 loops=1)
   ->  Sort  (cost=39.72..41.06 rows=537 width=118) (actual time=0.054..0.054 rows=5 loops=1)
         Sort Key: d.department_name
         Sort Method: quicksort  Memory: 25kB
         ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.005..0.007 rows=4 loops=1)
 Total runtime: 0.236 ms
(11 rows)
```

The actual execution cost and time are displayed. Merge join is faster than hash join. As mentioned above, hash join has a better effect when there is a large amount of data. In this case, the data volume is small, but the estimated number of rows is inaccurate. As a result, the optimizer selects hash join. It can also be seen that the optimizer does not always select the optimal execution plan, and is affected by data features.

4.3.3 Nested Loop Join Operators

Nested loop join is one of the simplest join types in database queries. It loops a table and searches for matched rows in each iteration to join tables. This type of join operation is applicable to the following cases:

- Small table join: Nested loop join can be very efficient when at least one of the tables involved in the join is relatively small, especially when the small table can be completely stored in the memory.
- Lack of indexes: Nested loop join is preferred by the optimizer when the column of the join condition does not have a proper index or the index cannot be used to reduce data access traffic.
- Join conditions with high selectivity: When join conditions can significantly reduce the number of rows that need to be further checked, nested loop join can effectively reduce query costs.

Step 1 Disable hash join and merge join.

You can use parameters to disable hash join and merge join, use the same query statement as hash join, and observe the execution plan.

```
openGauss=# SET enable_hashjoin = off;
SET
openGauss=# SET enable_mergejoin = off;
SET
```

Step 2 Execute the query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                           QUERY PLAN
-----------------------------------------------------------------------------
 Nested Loop  (cost=0.00..1711.48 rows=561 width=336)
   Join Filter: ((e.department)::text = (d.department_name)::text)
   ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118)
   ->  Materialize  (cost=0.00..13.13 rows=209 width=336)
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336)
(5 rows)
```

Step 3 Interpret the execution plan.

After hash join and merge join are disabled, nested loop join is used for the database. It can be seen that the estimation is also inaccurate. The reason is the same as above. The estimated cost of nested loop join is **1711.48**, which is much higher than that of hash join

and merge join. Therefore, when nested loop is not disabled, the database selection sequence is hash join, merge join, and nested loop join.

Other contents are similar. The following describes Materialize in EXPLAIN. It caches data in the **employees** table in the memory, which helps reduce the cost of accessing the same data for multiple times.

Step 4 Use EXPLAIN analyze to print an execution plan.

If no operation is performed for a long time, the session is reconnected and hash join and merge join are disabled because the parameters set in set mode take effect only in the session. After the reconnection, the default values are restored. By default, hash join and merge join are enabled. Step 1 needs to be performed again.

```
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
QUERY PLAN
------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.00..1711.48 rows=561 width=336) (actual time=0.048..0.067 rows=6 loops=1)
   Join Filter: ((e.department)::text = (d.department_name)::text)
   Rows Removed by Join Filter: 18
   ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.023..0.023 rows=4 loops=1)
   ->  Materialize  (cost=0.00..13.13 rows=209 width=336) (actual time=0.012..0.015 rows=24
loops=4)
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336) (actual
time=0.007..0.009 rows=6 loops=1)
 Total runtime: 0.212 ms
(7 rows)
```

Step 5 Interpret the execution plan.

According to the actual execution cost and execution time, nested loop join is the fastest. As mentioned above, the nested loop join is suitable for a small amount of data, which is consistent with the actual scenario. However, the estimated number of rows has a large error. As a result, the optimizer selects hash join first, and then merge join. It can also be seen that the optimizer does not always select the optimal execution plan, and is affected by data features. In this case, the actual priority result is the opposite. This reflects a limitation of the estimation algorithm. However, it is impractical to require that the estimation algorithm be completely consistent with an actual result before the estimation algorithm is actually executed. This case is only an extreme one. When the data volume of the test case is small, the absolute time difference between the three operators is small. The data in the manual is obtained in the debug and non-performance environment. The difference in the actual production environment is smaller. In openGauss, the estimation algorithm is optimized to meet the actual situation in more scenarios. Insert a large amount of data and test again.

Step 6 Insert 100,000 data records.

```
openGauss=# DO $$
            DECLARE
                iter INTEGER;
```

```
BEGIN
    FOR iter IN 1..100000 LOOP
        INSERT INTO employees (name, age, salary, department)
        VALUES ('Alice', 30, (RANDOM() * (100000 - 10000) + 10000)::INT, 'HR');
    END LOOP;
END $$;
```

Step 7 Use hash join, merge join, and nested loop join to perform the test.

Hash join connection test:

```
openGauss=# SET enable_hashjoin = on;
SET enable_mergejoin = on;
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=22.08..5073.34 rows=268516 width=124) (actual time=0.228..74.308 rows=100006
loops=1)
   Hash Cond: ((e.department)::text = (d.department_name)::text)
   ->  Seq Scan on employees e  (cost=0.00..1741.06 rows=100006 width=9) (actual
time=0.011..11.212 rows=100006 loops=1)
   ->  Hash  (cost=15.37..15.37 rows=537 width=118) (actual time=0.019..0.019 rows=4 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 257kB
         ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.006..0.008 rows=4 loops=1)
 Total runtime: 85.516 ms
(7 rows)
```

Merge join connection test:

```
openGauss=# SET enable_hashjoin = off;
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=10086.14..14116.57 rows=268516 width=124) (actual time=63.514..98.509
rows=100006 loops=1)
   Merge Cond: ((d.department_name)::text = (e.department)::text)
   ->  Sort  (cost=39.72..41.06 rows=537 width=118) (actual time=0.031..0.033 rows=4 loops=1)
         Sort Key: d.department_name
         Sort Method: quicksort  Memory: 25kB
         ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.009..0.011 rows=4 loops=1)
   ->  Sort  (cost=10046.42..10296.44 rows=100006 width=9) (actual time=63.473..68.107
rows=100006 loops=1)
         Sort Key: e.department
         Sort Method: quicksort  Memory: 13229kB
         ->  Seq Scan on employees e  (cost=0.00..1741.06 rows=100006 width=9) (actual
time=0.005..22.563 rows=100006 loops=1)
 Total runtime: 108.746 ms
(11 rows)
```

Nested loop join connection test:

```
openGauss=# SET enable_hashjoin = off;
SET enable_mergejoin = off;
openGauss=# EXPLAIN analyze SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.department = d.department_name;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.00..807306.10 rows=268516 width=124) (actual time=0.027..193.270
rows=100006 loops=1)
   Join Filter: ((e.department)::text = (d.department_name)::text)
   Rows Removed by Join Filter: 300018
   ->  Seq Scan on employees e  (cost=0.00..1741.06 rows=100006 width=9) (actual
time=0.009..11.195 rows=100006 loops=1)
   ->  Materialize  (cost=0.00..18.05 rows=537 width=118) (actual time=5.198..28.425 rows=400024
loops=100006)
         ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118) (actual
time=0.003..0.004 rows=4 loops=1)
 Total runtime: 202.968 ms
(7 rows)
```

Step 8 Interpret the execution plan.

After the data volume is increased, the actual execution time sorting has the same priority as the estimated sorting and selection. Hash join is faster than merge join and nested loop join. The data in this document is obtained in the debug version and non-performance environment. If the result is inconsistent or the difference is small, you can perform step 4 to increase the data volume and change the number in the loop, for example, 1 million.

Considering the execution efficiency and unnecessary troubles, the single-digit data volume is restored. The experiment is only for qualitative demonstration. The estimation deviation may still occur in the following sections.

Step 9 Restore the data volume of the single digit and the hash join connection.

```
openGauss=# SET enable_hashjoin = on;
openGauss=# drop table employees;
DROP TABLE
openGauss=# CREATE TABLE employees (
      id SERIAL PRIMARY KEY,
      name VARCHAR(100),
      age INTEGER,
      salary DECIMAL(10, 2),
      department VARCHAR(50));

openGauss=# INSERT INTO employees (name, age, salary, department) VALUES
              ('Alice', 30, 50000.00, 'HR'),
              ('Bob', 22, 40000.00, 'Marketing'),
              ('Charlie', 28, 45000.00, 'IT'),
              ('David', 35, 55000.00, 'Finance'),
              ('Eve', 45, 62000.00, 'IT'),
              ('Frank', 32, 48000.00, 'Marketing');
```

4.4 Control Operator Execution Plan

This section describes the execution plans of SQL statements of the control operators Result, Limit, and Append, and explains the application scenarios and execution plans of related scan operators.

4.4.1 Result Operators

In the EXPLAIN output of an SQL query, the Result operation usually indicates that a calculation or condition filtering occurs when no data is actually read from the physical table. It is usually used in the following cases:

- The query involves constant or expression calculation. If the query involves only returning a set of calculated values or constants, instead of retrieving data from a table, the Result node may appear.
- The query uses a function to return a result when the query depends on a function that returns a dynamic calculation result (for example, a function that generates data or a condition selection function).
- The WHERE clause filters all rows. If the WHERE clause does not contain any rows that meet the condition, the query returns an empty result set or a calculated value.

Step 1 Execute the query statement 1 using EXPLAIN.

```
openGauss=# EXPLAIN SELECT 'Hello World' AS greeting;
                        QUERY PLAN
------------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
(1 row)
```

This query does not involve any table, but returns a constant string **'Hello World'** as the **greeting** column.

Step 2 Execute the query statement 2 using EXPLAIN.

```
openGauss=# EXPLAIN SELECT CASE WHEN random() > 0.5 THEN 'Greater than 0.5' ELSE 'Less than
or equal to 0.5' END AS random_result;
                        QUERY PLAN
------------------------------------------
 Result  (cost=0.00..0.01 rows=1 width=0)
(1 row)
```

Step 3 Interpret the execution plan.

This query uses the random() function to generate a random floating-point number between 0 and 1, and then uses the CASE statement to return different strings based on the random number value. This type of query is based on the return result of the function and does not depend on any foreign table data.

4.4.2 Limit Operators

In the EXPLAIN output of an SQL query, the LIMIT operation usually indicates that the query plan contains an operation that limits the number of rows to be returned. It is usually used in the following cases:

- LIMIT clause: The maximum number of rows returned by a query is limited when the query contains the LIMIT clause.
- OFFSET clause: It is often used with LIMIT. **OFFSET** specifies the number of rows to skip before rows are returned.
- Query optimization: In some cases, even if LIMIT is not explicitly used in a query, the optimizer may implicitly use LIMIT due to some internal optimization policies (for example, when it is estimated that no more rows are required for the query).

Step 1 Execute the query statement using EXPLAIN LIMIT.

```
openGauss=# EXPLAIN SELECT * FROM employees LIMIT 10;
                          QUERY PLAN
---------------------------------------------------------------------
 Limit  (cost=0.00..0.58 rows=10 width=360)
   ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=360)
(2 rows)
```

Step 2 Interpret the execution plan.

The query plan contains LIMIT, indicating that the database performs the LIMIT operation to ensure that the result contains only a specified number of rows. This operation is useful for controlling the results of large dataset queries and can significantly improve performance, especially when only partial data is required.

4.4.3 Append Operators

In the EXPLAIN output of an SQL query, the APPEND operation is performed in the following scenarios:

- Union set operation involving in a query: When a query needs to merge result sets from multiple tables or subqueries, for example, the UNION ALL operation is performed.
- Partitioned table querying: When a query involves partitioned tables and data needs to be retrieved from multiple partitions, the APPEND operation is used to combine the results from each partition.
- Inherited table structure usage: When a table uses the table inheritance feature and the parent table is queried, APPEND is performed to merge the results from all child tables.

Step 1 Execute the query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT name FROM employees
                UNION ALL
                SELECT name FROM employees WHERE department = 'HR';
                        QUERY PLAN
```

```
------------------------------------------------------------------------
 Result  (cost=0.00..24.71 rows=210 width=218)
   ->  Append  (cost=0.00..24.71 rows=210 width=218)
         ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=218)
         ->  Seq Scan on employees  (cost=0.00..12.61 rows=1 width=218)
               Filter: ((department)::text = 'HR'::text)
(5 rows)
```

Step 2 Interpret the execution plan.

This query selects the names of all employees from the **employees** table and uses UNION ALL to combine the names of employees whose department is **'HR'** in the same table. The query plan includes an APPEND step, indicating that the database performs a merge operation to directly combine two independent query result sets.

4.5 Materialization Operator Execution Plan

This section describes the execution plans of SQL statements of the materialization operators Sort, Hash, Unique, Aggregate, Group, and SetOp, and explains the application scenarios and execution plans of related scan operators.

4.5.1 Sort Operators

In the EXPLAIN output of an SQL query, the reasons that SORT operation occurs are as follows:

- Sorting: When the query contains the ORDER BY clause, the result set needs to be sorted to meet the specified sorting order.
- Aggregation: Some aggregate functions (such as GROUP BY) may need to sort data first to calculate aggregation more efficiently.
- Window functions: When a window function is used, the data involved in window calculation may need to be sorted.
- Optimized joins: When some types of joins (such as merge join) are performed, if the related columns of the joined tables are not sorted in advance, the columns need to be sorted before the join. For details, see "Merge Join Operators."

Step 1 Execute the query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT name, department, salary
FROM employees
ORDER BY salary DESC;
                          QUERY PLAN
---------------------------------------------------------------
 Sort  (cost=20.14..20.67 rows=209 width=352)
   Sort Key: salary DESC
   ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=352)
(3 rows)
```

Step 2 Interpret the execution plan.

The query plan contains a sorting step, indicating that the data must be sorted to fit the ORDER BY clause of the query. The sorting typically involves loading data into memory and adjusting the order of rows using appropriate sort algorithms, such as fast sort or merge sort. If the data volume is large, sorting may involve using disk space to store temporary data.

The estimated value is also quite different. You can run the **analyze** command to view the actual situation.

Step 3 Use EXPLAIN analyze to print query statements.

```
openGauss=# EXPLAIN analyze SELECT name, department, salary
FROM employees
ORDER BY salary DESC;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Sort  (cost=20.14..20.67 rows=209 width=352) (actual time=0.093..0.094 rows=6 loops=1)
   Sort Key: salary DESC
   Sort Method: quicksort  Memory: 25kB
   ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=352) (actual time=0.010..0.013
rows=6 loops=1)
 Total runtime: 0.221 ms
(5 rows)
```

4.5.2 Hash Operators

In the EXPLAIN output of an SQL query, hash operations are usually performed during hash join or hash table construction for quick search and comparison. The details are as follows:

- Hash join: When a hash join operation is performed, the system constructs a hash table for one or two joined tables. This type of join is suitable for equivalent joins, especially when no valid index is available or the table size is uneven.
- Aggregation: If the hash method is used for grouping when grouping and aggregation are performed (for example, using GROUP BY), the hash operation also occurs. In this case, the hash table is used for aggregation calculation of fast grouping key values.

Step 1 Execute the query statement using EXPLAIN, which is the same as hash join.

```
openGauss=# EXPLAIN SELECT e.name, d.department_name
                FROM employees e
                JOIN departments d ON e.department = d.department_name;
                            QUERY PLAN
------------------------------------------------------------------------------
 Hash Join  (cost=14.70..37.70 rows=561 width=336)
   Hash Cond: ((d.department_name)::text = (e.department)::text)
   ->  Seq Scan on departments d  (cost=0.00..15.37 rows=537 width=118)
   ->  Hash  (cost=12.09..12.09 rows=209 width=336)
         ->  Seq Scan on employees e  (cost=0.00..12.09 rows=209 width=336)
(5 rows)
```

Step 2 Interpret the execution plan.

Hash (cost=12.09..12.09 rows=209 width=336) indicates data used to construct a hash table. The data is in the **employees** table.

The hash table is constructed to support hash join. In this process, the database scans the **employees** table, applies the hash function to the **department** column in each row, and stores the result in the hash table in the memory. In this case, when the **departments** table is scanned, each department name can be quickly matched with an entry in the hash table to determine which **employees** rows are connected to the **departments** rows.

4.5.3 Unique Operators

In the EXPLAIN output of an SQL query, the unique operation usually indicates that the database system is performing deduplication. It is usually used in the following cases:

- SELECT DISTINCT: It is used to ensure that the returned result does not contain duplicate rows when the query contains the DISTINCT keyword.
- Aggregate queries: In some cases, even if DISTINCT is not explicitly used, deduplication may be implicitly performed to ensure correct execution of aggregate functions, such as COUNT(DISTINCT column).
- Window functions: Deduplication may be a necessary step when calculating some window functions, especially when the window functions depend on unique values.

Step 1 Execute the query statement 1 using EXPLAIN.

```
openGauss=# EXPLAIN SELECT DISTINCT department FROM employees;
                          QUERY PLAN
------------------------------------------------------------------------
 HashAggregate   (cost=12.61..14.61 rows=200 width=118)
   Group By Key: department
   ->  Seq Scan on employees   (cost=0.00..12.09 rows=209 width=118)
(3 rows)
```

Step 2 Interpret the execution plan 1.

In modern database systems, especially when processing large data sets, HashAggregate is considered to be a more efficient way to deduplicate. Compared with traditional Unique filtering, HashAggregate groups data by building a hash table, which is usually faster than sorting and then deduplication.

Step 3 Execute the query statement 2 using EXPLAIN.

```
openGauss=# EXPLAIN SELECT DISTINCT department
                FROM (
                     SELECT department
                     FROM employees
                     ORDER BY department
                ) sub;
QUERY PLAN
------------------------------------------------------------------------------
 Unique   (cost=20.14..23.28 rows=200 width=118)
   ->  Sort   (cost=20.14..20.67 rows=209 width=118)
```

```
            Sort Key: employees.department
            ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=118)
(4 rows)
```

Step 4 Interpret the execution plan 2.

In this query, the subquery first sorts the data by department, and then the external query applies DISTINCT to the sorted result. This structure can sometimes cause the optimizer to use the Unique step, especially when the result of a subquery is explicitly deduplicated by an external query.

4.5.4 Aggregate Operators

In the EXPLAIN output of an SQL query, the AGGREGATE operation usually indicates that the query contains the use of aggregate functions. Aggregate functions include but are not limited to SUM(), AVG(), COUNT(), MAX(), and MIN(), and are used to perform statistical calculation. They are usually used in the following cases:

- Aggregate query: contains aggregate functions and needs to collect statistics on data.
- GROUP BY clause: used together with the GROUP BY clause to group specified columns or expressions and calculate the aggregate value of each group.

Step 1 Execute the aggregate function query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT AVG(salary) AS average_salary FROM employees;
                        QUERY PLAN
------------------------------------------------------------------
 Aggregate  (cost=12.62..12.62 rows=1 width=48)
   ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=16)
(2 rows)
```

Step 2 Interpret the execution plan.

The query plan includes an AGGREGATE step, indicating that the database performs an aggregation operation to calculate the average salary. This operation is completed in the final phase of data processing, and usually involves scanning an entire table or a dataset of multiple rows to calculate a result.

4.5.5 Group Operators

In the EXPLAIN output of an SQL query, the GROUP or GroupAggregate operation usually indicates that the query plan involves a grouping operation, which is used to process the column or expression specified in the GROUP BY clause.

Step 1 Execute the GROUP BY query statement 1 using EXPLAIN.

```
openGauss=# EXPLAIN SELECT department, AVG(salary) AS average_salary
                FROM employees
                GROUP BY department;
                                  QUERY PLAN
------------------------------------------------------------------
 HashAggregate  (cost=13.14..15.64 rows=200 width=166)
```

```
    Group By Key: department
    ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=134)
(3 rows)
```

Step 2 Interpret the execution plan 1.

The optimizer usually prefers to use the HashAggregate policy for grouping and aggregation operations, because these methods are usually more efficient than traditional GROUP operations (sort-based grouping). In particular, the optimizer prefers hash aggregation because it is usually faster and more resource-efficient, in cases where there is no obvious indication that sorting and grouping are required.

Step 3 Execute the GROUP BY query statement 2 using EXPLAIN and disable HashAggregate.

```
openGauss=# SET enable_hashagg = off;
SET
openGauss=# EXPLAIN SELECT department, AVG(salary) AS average_salary
                FROM employees
                GROUP BY department;
                                    QUERY PLAN
------------------------------------------------------------------------
 GroupAggregate  (cost=20.14..24.21 rows=200 width=166)
   Group By Key: department
   ->  Sort  (cost=20.14..20.67 rows=209 width=134)
         Sort Key: department
         ->  Seq Scan on employees  (cost=0.00..12.09 rows=209 width=134)
(5 rows)
```

Step 4 Interpret the execution plan 2.

Before performing GroupAggregate, you must sort the data because GroupAggregate needs to process the data in the order of the **department** column to effectively group the data.

Step 5 Cancel the disabling.

```
openGauss=# RESET enable_hashagg;
RESET
```

4.5.6 SetOp Operators

In the EXPLAIN output of SQL query, SetOp and HashSetOp indicate set operations, which usually involve set theory operations on datasets, such as deduplication (DISTINCT), set union (UNION), intersection (INTERSECT), and difference set (EXCEPT). These operations are common when multiple datasets need to be processed and combined or duplicate data needs to be removed. They are usually used in the following cases:

- DISTINCT: When the query contains SELECT DISTINCT, SetOp may be used to remove duplicate rows.

- Union, intersection, and difference sets: When SQL keywords such as UNION, INTERSECT, and EXCEPT are used in a query, SetOp may be used to perform these set operations.

Step 1 Execute the query statement using EXPLAIN.

```
openGauss=# EXPLAIN SELECT name FROM employees WHERE department = 'Sales'
EXCEPT
SELECT name FROM employees WHERE department = 'HR';
                              QUERY PLAN
-------------------------------------------------------------------------------
 HashSetOp Except    (cost=0.00..25.25 rows=1 width=218)
   ->   Append    (cost=0.00..25.25 rows=2 width=218)
          ->   Subquery Scan on "*SELECT* 1"    (cost=0.00..12.62 rows=1 width=218)
                 ->   Seq Scan on employees    (cost=0.00..12.61 rows=1 width=218)
                        Filter: ((department)::text = 'Sales'::text)
          ->   Subquery Scan on "*SELECT* 2"    (cost=0.00..12.62 rows=1 width=218)
                 ->   Seq Scan on employees    (cost=0.00..12.61 rows=1 width=218)
                        Filter: ((department)::text = 'HR'::text)
(8 rows)
```

Step 2 Interpret the execution plan.

The result of the query statement is an empty set, which is used only for SetOp demonstration. The difference set operation is performed on two subqueries. Two Subquery Scan operations are performed on each subquery. After being appended, the result is processed by HashSetOp Except.

5 Database Optimization

5.1 Introduction

5.1.1 About This Lab

This lab instructs you to perform the database optimization and practice related cases in Chapter 5 of the theoretical textbook.

This lab includes querying performance parameters of the server and openGauss database system, setting the vectorized executor, setting the SMP, optimizing indexes, optimizing the table structure, setting the CBO, diagnosing SQL performance, optimizing subqueries, and rewriting SQL statements.

5.1.2 Objectives

Through setting performance-related parameters and practicing database optimization cases, you can have a deeper understanding of the related knowledge of theoretical textbooks.

5.2 Querying the Server and openGauss Database Parameters

The running performance of the database depends on the resources of the server. This lab describes how to view server resources, including the CPU, memory, disk, and network, by running commands, so that you can learn about the running status of the current system.

The running performance of the openGauss database depends on the overall parameter configuration. Therefore, it is important to check whether key parameters are properly configured before optimization. In this lab, you only need to understand how to view and set GUC parameters. For the parameters that need to be restarted, viewing them is enough. You can check and set them when it comes to actual optimization scenarios.

5.2.1 Checking the Server Hardware Architecture

Step 1 Log in to the Linux server and run the **uname** command.

```
uname -a
```

Enter the preceding command and press **Enter**. The architecture information of the server is displayed.

```
[root@server1 ~]# uname -a
Linux server1 5.10.0-182.0.0.95.oe2203sp3.aarch64 #1 SMP Sat Dec 30 13:16:24 CST 2023 aarch64 aarch64 aarch64 GNU/Linux
[root@server1 ~]#
```

aarch64 indicates that the server platform architecture is Arm. The platform version of the openGauss database must be consistent with the platform architecture of the server. Otherwise, the database cannot be installed.

5.2.2 Checking the CPU Load

Step 1 Log in to the Linux server and run the following command:

```
top
```

The following information is displayed, in which **Tasks** indicates the current task status.

The first part is the overview information, and each line indicates the CPU and memory usage of a process.

1. **Total** indicates the total number of tasks, **running** indicates the number of running tasks, and **stopped** indicates the number of stopped tasks.
2. In the row **%Cpu(s)**, **us** indicates the total CPU usage in user mode, and **sy** indicates the CPU usage of the system kernel.
3. **MiB Mem** indicates the memory information, in MB. **total** indicates the total memory, **free** indicates the free memory, and **used** indicates the used memory.

```
top - 14:26:42 up 17 days, 22:35,  5 users,  load average: 0.03, 0.01, 0.01
Tasks: 1266 total,   1 running, 1265 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 514417.1 total,  72030.5 free, 315800.2 used, 129549.2 buff/cache
MiB Swap:   4096.0 total,   4095.5 free,      0.5 used. 198616.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4655 openvsw+  10 -10 8745024 288052  14056 S   3.6   0.1     16,32 ovs-vswitchd
 375006 root      20   0   27660   5620   3100 R   1.0   0.0   0:00.56 top
      1 root      20   0  168804  11840   7860 S   0.0   0.0   0:40.89 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:02.35 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-kblockd
      9 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
     10 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_rude_
     11 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_tasks_trace
```

Before starting the database, you need to check the CPU usage of the system to prevent other services from occupying too many CPU resources and affecting the database performance.

During the running of the database, you can check the CPU usage to determine whether the CPU has a bottleneck.

Step 2 Run the **top** command and press **1** to view the usage of each CPU.

The output is as follows:

1. **us**: CPU usage in user mode.
2. **ni**: percentage of CPU time in user mode consumed by the process whose scheduling priority has been changed through **nice**.
3. **sy**: percentage of CPU time consumed in system mode.
4. **wa**: percentage of the CPU idle time on waiting for disk I/O.
5. **st**: percentage of time for waiting other virtual CPU computing by using OS virtualization technology such as Xen.
6. **id**: percentage of the CPU idle time.

```
top - 15:29:56 up 17 days, 23:38,  5 users,  load average: 0.00, 0.00, 0.11
Tasks: 1271 total,   1 running, 1270 sleeping,   0 stopped,   0 zombie
%Cpu0  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu1  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu2  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu3  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu4  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu5  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu6  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu7  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu8  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu9  :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu10 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu11 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu12 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu13 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu14 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu15 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu16 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu17 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
%Cpu18 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
```

Step 3 After running the **top** command, press **Shift+M** to sort the results by memory.

```
Tasks: 1274 total,   1 running, 1273 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 514417.1 total,  72084.5 free, 315745.8 used, 129549.7 buff/cache
MiB Swap:   4096.0 total,   4095.5 free,      0.5 used. 198671.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 4655 openvsw+  10 -10 8745024 288052  14056 S   3.6   0.1  16,34 ovs-vswitchd
 3681 root      20   0  435296 107940  79152 S   0.0   0.0  66:53.19 tuned
 2041 root      20   0  106472  74760  73416 S   0.0   0.0   0:05.41 systemd-journal
 3998 root      20   0  246028  42028  38572 S   0.0   0.0   0:14.13 rsyslogd
 3598 root      20   0  275404  19176  15080 S   0.0   0.0   0:30.35 NetworkManager
 6198 root      20   0   54980  14252  12480 S   0.0   0.0   0:00.04 virtlogd
    1 root      20   0  168804  11840   7860 S   0.0   0.0   0:40.94 systemd
 2077 root      20   0   30588   9164   6160 S   0.0   0.0   0:01.26 systemd-udevd
```

5.2.3 Checking the Memory

Log in to the Linux server and run the following command:

```
[root@server1 ~]# free -h
               total        used        free      shared  buff/cache   available
Mem:           502Gi       310Gi        68Gi        52Mi       126Gi       192Gi
Swap:          4.0Gi       524Ki       4.0Gi
```

The output is as follows:

1. The **-h** parameter indicates that the system automatically displays the information in proper units. For example, in this operation, the unit is automatically converted to GB or MB.
2. In the preceding information, **Mem** indicates the system memory status, **total** indicates the total memory, **used** indicates the used memory, and **free** indicates the free memory. **shared** indicates the shared memory.
3. The **Swap** row indicates the usage of the memory swap space. The first column indicates the total swap space size, the second column indicates the used swap space size, and the third column indicates the remaining swap space size.

Before starting the database, you should check the memory usage of the system. Try to release the memory space for the database system to improve overall performance.

During the running of the database, you can check the memory usage to determine whether any memory bottleneck exists. For example, if the free memory is insufficient and the swap space is used, a bottleneck may exist.

5.2.4 Checking the OS Version

Log in to the Linux server and run the following command:

```
[root@server1 ~]# cat    /etc/os-release
NAME="openEuler"
VERSION="20.03 (LTS)"
ID="openEuler"
VERSION_ID="20.03"
PRETTY_NAME="openEuler 20.03 (LTS)"
ANSI_COLOR="0;31"
```

The output of the openEuler OS is as follows:

NAME indicates the OS name, and **VERSION** indicates the OS version.

The version of the openGauss database must be the same as that of the OS on the server. Otherwise, the database cannot be used.

5.2.5 Checking the Network Status

Step 1 Install the missing command.

Install Nginx.

```
[root@server1 ~]#sudo yum install nginx
```

Install the missing command.

```
[root@server1 ~]# sudo yum install sysstat
```

Step 2 Check the network status.

```
[root@ecs-444 ~]# sar -n DEV 3 2
Linux 4.19.90-2110.8.0.0119.oe1.aarch64 (ecs-444)         05/11/20XX        _aarch64_        (8 CPU)

10:18:37 PM       IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
10:18:40 PM          lo      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
10:18:40 PM        eth0      4.00      9.00      0.26      1.02      0.00      0.00      0.00      0.00

10:18:40 PM       IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
10:18:43 PM          lo      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
10:18:43 PM        eth0      3.00      8.67      0.18      0.99      0.00      0.00      0.00      0.00

Average:          IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
Average:             lo      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:           eth0      3.50      8.83      0.22      1.00      0.00      0.00      0.00      0.00
```

Meaning of the command:

1. The parameter **3** indicates that the information is printed once every 3 seconds.
2. The parameter **2** indicates that the information is printed twice and can be omitted. If this argument is omitted, the information keeps being always printed. To stop the printing, press **Ctrl+C**.

The output example is as follows:

1. **IFACE**: NIC name.
2. **rxpck/s**: number of data packets read per second.
3. **txpck/s**: number of packets sent per second.
4. **rxkB/s**: size of data packets read per second.
5. **txkB/s**: size of data packets sent per second.
6. **%ifutil**: network bandwidth usage. If the usage is too high, there is a network bottleneck.

5.2.6 Checking the I/O Status

Step 1 Check I/O status.

```
[root@server1 ~]# iostat -xm 3 2
Linux 5.10.0-182.0.0.95.oe2203sp3.aarch64 (server1)       05/10/20XX        _aarch64_        (128
CPU)

avg-cpu:   %user    %nice %system %iowait   %steal    %idle
             0.11     0.00     0.32     0.00     0.00    99.57

Device              r/s       rMB/s     rrqm/s   %rrqm r_await rareq-sz       w/s       wMB/s
wrqm/s   %wrqm w_await wareq-sz       d/s       dMB/s     drqm/s   %drqm d_await dareq-sz       f/s
f_await   aqu-sz   %util
```

```
dm-0              0.39      0.01      0.00      0.00      8.02     36.79      0.41      0.00      0.00
0.00      2.51      9.86      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.13
dm-1              0.00      0.00      0.00      0.00     24.39     21.51      0.00      0.00      0.00
0.00      0.00      4.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.00
dm-2              0.05      0.02      0.00      0.00      8.22    416.83      0.28      0.06      0.00
0.00      6.88    208.35      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.07
dm-3              0.00      0.00      0.00      0.00     20.47     20.47      0.00      0.00      0.00
0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.00
dm-4              0.00      0.00      0.00      0.00     21.10     20.47      0.00      0.00      0.00
0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.00
dm-5              0.00      0.00      0.00      0.00     31.37     20.47      0.00      0.00      0.00
0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.00
sda               0.00      0.00      0.00      0.00      6.50     11.17      0.00      0.00      0.00
0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.00      0.00
sdb               0.78      0.04      0.01      0.89     10.93     46.50      1.30      0.06      0.28
17.62     10.84     47.83      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
0.02      0.21
```

Meaning of command parameters:

1. The parameter **3** indicates that the information is printed once every 3 seconds.
2. The parameter **2** indicates that the information is printed twice. If this parameter is omitted, the printing continues.

The output is as follows:

1. **Device**: device name.
2. **rrqm/s**: number of device-related read requests merged per second.
3. **%util**: disk bandwidth usage. If the usage is high, a bottleneck may exist.

5.2.7 Querying Key Parameters of the openGauss Database

Check the configuration of the maximum memory, shared memory, and working memory.

```
openGauss=# show max_process_memory;
 max_process_memory
--------------------
 12GB
(1 row)

openGauss=# show shared_buffers;
 shared_buffers
----------------
 1GB
(1 row)

openGauss=# show work_mem;
```

```
 work_mem
----------
 64MB
(1 row)
```

5.3 Vectorized Executor

5.3.1 Lab Description

The openGauss database supports the row executor and vectorized executor for processing row-store tables and column-store tables, respectively.

The vectorized executor highlights:

- More data is read in one batch at a time, saving I/O resources.
- There are many records in a batch, and the CPU cache hit rate increases.
- In pipeline mode, functions are called infrequently.
- Data is processed in batches, which is efficient.

Column-store tables perform better in complex analysis queries, but do not perform well in inserting and updating data. This means such tables cannot be used for services that need to insert and update data frequently.

You can set the GUC parameter **try_vector_engine_strategy** to convert query statements containing row-store tables into vectorized execution plans for execution.

5.3.2 Optimizing SQL Queries Using a Vectorized Executor

Step 1 Log in to the openGauss database as the **omm** user and create tables.

```
openGauss=# create table vector_t1(a int, b int);
CREATE TABLE
openGauss=# create table vector_t2(a int, b int);
CREATE TABLE
```

Step 2 Generate lab data.

```
openGauss=# insert into vector_t1 values (generate_series(0, 1000000), 1);
INSERT 0 1000001
openGauss=# insert into vector_t2 values (generate_series(0, 1000000), 1);
INSERT 0 1000001
```

Step 3 Enable the time consumption calculation of the openGauss database.

```
openGauss=# \timing on
Timing is on.
```

Step 4 Disable the vectorized executor parameter.

Run the following command to check whether the parameter is disabled. If the value is **off**, the parameter is disabled. Otherwise, the parameter is enabled. If the following information is displayed, it is disabled:

```
openGauss=# show try_vector_engine_strategy;
 try_vector_engine_strategy
----------------------------
 off
(1 row)
```

If the value is not **off**, disable the vectorized executor.

```
openGauss=# set try_vector_engine_strategy to off;
SET
Time: 0.420 ms
```

Step 5 Run the query statement in normal mode.

```
openGauss=# select count(1) from (select t1.a from vector_t1 t1 left join vector_t2 t2 on t1.a = t2.a);
  count
---------
 1000001
(1 row)

Time: 1154.044 ms
```

As shown in the preceding information, the time consumed is 1154.044 ms.

Note: The time consumed varies depending on the server and system load.

Step 6 Enable the vectorized executor.

```
openGauss=# set try_vector_engine_strategy to force;
SET
Time: 1.231 ms
```

Step 7 Use the vectorized executor to execute a query statement.

```
openGauss=# select count(1) from (select t1.a from vector_t1 t1 left join vector_t2 t2 on t1.a = t2.a);
  count
---------
 1000001
(1 row)

Time: 309.164 ms
```

As shown in the preceding information, the time is changed to 309.164 ms.

Step 8 Restore parameter settings.

```
openGauss=# \timing off
Timing is off.
openGauss=# reset try_vector_engine_strategy;
```

```
RESET
```

Step 9 Drop data.

```
openGauss=# drop table vector_t1;
DROP TABLE
openGauss=# drop table vector_t2;
DROP TABLE
openGauss=#
```

Compare the time consumed for executing the query SQL statement in different modes.

According to the preceding lab results, it takes 1154.044 ms to execute the query SQL statement in normal mode and 309.164 ms when the vectorized executor is used.

In this test case, using the vectorized executor improves the performance by nearly three times compared with that in normal mode.

Note: During the execution of the query SQL statement, the normal mode is not completely isolated from the vectorized mode. The two modes use different tables and are executed alternately for multiple times to obtain the values of average time consumption.

5.4 SMP-related Experiments

5.4.1 Lab Description

The Symmetric Multi-Processing (SMP) technology of openGauss uses the computer's multi-core CPU architecture to implement multi-thread parallel computing, capitalizing on CPU resources to improve query performance.

Complex queries take a long time, and the system concurrency is low. SMP is used to implement operator-level parallel execution, which effectively speeds up queries, improving the query performance and resource utilization.

SMP implementation: For query operators that can be executed in parallel, data is sliced, multiple worker threads are started for computation, and then the results are summarized and returned to the front end.

A data interaction operator named Stream is added to the SMP architecture to implement data interaction between multiple worker threads, ensuring the query's accuracy and integrity.

Operators that support parallelism, as detailed in the plan:

- Scan: Sequentially scans ordinary row-store tables, partitioned row-store tables, ordinary column-store tables, and partitioned column-store tables.
- Join: hash join and nested loop join
- Agg: HashAgg, SortAgg, PlainAgg, and WindowAgg (which supports only PARTITION BY and does not support ORDER BY)
- Stream: Local Redistribute and Local Broadcast
- Others: Result, Subqueryscan, Unique, Material, Setop, Append, and VectoRow

SMP-specific operators: To execute queries in parallel, Stream operators are added for SMP data exchanges. These new operators can be considered as the subtypes of Stream operators.

- Local Gather aggregates data of parallel threads within an instance.
- Local Redistribute redistributes data based on the distribution key across threads within an instance.
- Local Broadcast broadcasts data to each thread within an instance.
- Local RoundRobin distributes data in polling mode across threads within an instance.

5.4.2 Using SMP to Optimize SQL Queries

Step 1 Create a table structure.

```
openGauss=# create table smp_t1(a int, b int);
CREATE TABLE
```

Step 2 Generate lab data.

```
openGauss=# insert into smp_t1 values (generate_series(0, 1000000), 1);
INSERT 0 1000001
```

Step 3 Enable the time consumption calculation of the openGauss database.

```
openGauss=# \timing on
Timing is on.
```

Step 4 Reset the **query_dop** parameter to **1**.

```
openGauss=# reset query_dop;
RESET
Time: 1.214 ms
openGauss=# show query_dop;
 query_dop
-----------
 1
(1 row)

Time: 0.453 ms
```

If the preceding information is displayed, the parameter is disabled.

Step 5 Run the query statement in normal mode.

Run the statement for three consecutive times. The command output is as follows:

```
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)
```

```
Time: 383.755 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 270.792 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 270.665 ms
```

Among three executed queries, the first takes a longer time than the last two.

Generally, the first query takes a longer time than the subsequent queries because data is not warmed up.

Therefore, pay attention to the impact of this aspect during the test.

The time consumed by last two queries, about 271 ms, is taken as the result.

Note: The time consumed varies depending on the server and system load.

Step 6 Set query_dop to **4** to enable parallel execution.

```
openGauss=# set query_dop = 4;
SET
Time: 0.392 ms
```

Step 7 Run a query statement.

Run the statement for three consecutive times. If the following information is displayed, the execution is successful:

```
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 77.947 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 73.743 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
```

```
(1 row)

Time: 73.959 ms
```

As shown in the preceding information, the time consumed in this mode is much shorter than that in normal mode. The first execution takes a longer time than the last two.

Similarly, the time consumed by the last two executions, about 74 ms, is taken as the result.

Step 8 Set **query_dop** to **8** and run the query SQL statement again.

```
openGauss=# set query_dop = 8;
SET
Time: 0.441 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 41.963 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 41.612 ms
openGauss=# select count(1) from smp_t1 where a > 100000;
 count
--------
 900000
(1 row)

Time: 41.686 ms
```

The time is reduced to about 42 ms.

Step 9 Reset the parameter.

```
openGauss=# \timing off
Timing is off.
openGauss=# reset query_dop;
RESET
```

Step 10 Drop data.

```
openGauss=# drop table smp_t1;
DROP TABLE
```

Step 11 Compare performance data.

query_dop	Time Consumed (ms)	Improvement Ratio
1	271	1
4	74	3.7
8	42	6.5

Summary: According to the preceding lab results, in this test case, parallel computing can greatly improve the query performance.

5.5 GUC Parameter Setting and Plan Hints

5.5.1 Lab Description

An execution plan of openGauss is generated based on query conditions and table structures when the database system executes SQL statements. It provides guidance for the database system to perform query operations.

An execution plan includes information such as each operation step, operation sequence, used index, and table access mode of a query statement. It is the key to optimizing query performance. By analyzing the execution plan, you can learn about the execution status of query statements, find the query performance bottleneck, and optimize the query performance.

You can use GUC parameters to control the printing format, display information about different execution plans, and select different execution operators. You can also use plan hints on execution plans.

5.5.2 Printing an Execution Plan in Different Formats

Step 1 Create tables.

```
openGauss=# create table exp_t1(a int, b int);
CREATE TABLE
openGauss=# create table exp_t2(a int, b int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into exp_t1 values (generate_series(0, 1000000), 1);
INSERT 0 1000001
openGauss=# insert into exp_t2 values (generate_series(0, 1000000), 1);
INSERT 0 1000001
```

Step 3 Run **explain** to print an execution plan.

```
openGauss=# explain select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where t1.a < 10000;
                                        QUERY PLAN
-------------------------------------------------------------------------------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 width=4)
   Hash Cond: (t2.a = t1.a)
   ->  Seq Scan on exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4)
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4)
         ->  Seq Scan on exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4)
               Filter: (a < 10000)
(6 rows)
```

Step 4 Disable the **costs** display.

```
openGauss=# explain(costs off) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where t1.a
< 10000;
              QUERY PLAN
---------------------------------
 Hash Right Join
   Hash Cond: (t2.a = t1.a)
   ->  Seq Scan on exp_t2 t2
   ->  Hash
         ->  Seq Scan on exp_t1 t1
               Filter: (a < 10000)
(6 rows)
```

This makes the command output more concise and clear.

It is suitable for scenarios where the focus is solely on the execution plan operator, without regard to the associated costs.

Step 5 Print an execution plan in verbose mode.

```
openGauss=# explain(verbose) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where t1.a
< 10000;
                                        QUERY PLAN
-------------------------------------------------------------------------------------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 distinct=[200, 10056] width=4)
   Output: t1.a
   Hash Cond: (t2.a = t1.a)
   ->  Seq Scan on public.exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4)
         Output: t2.a, t2.b
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4)
         Output: t1.a
         ->  Seq Scan on public.exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4)
               Output: t1.a
               Filter: (t1.a < 10000)
(10 rows)
```

Step 6 Print an execution plan in **analyze** mode.

```
openGauss=# explain analyze select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where t1.a <
10000;
                                                        QUERY PLAN
```

```
------------------------------------------------------------------------------------------------------------------------
-------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 width=4) (actual time=312.886..686.170
rows=10000 loops=1)
   Hash Cond: (t2.a = t1.a)
   ->  Seq Scan on exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4) (actual
time=0.035..216.820 rows=1000001 loops=1)
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4) (actual time=312.793..312.793
rows=10000 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 608kB
         ->  Seq Scan on exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4) (actual
time=0.058..310.435 rows=10000 loops=1)
               Filter: (a < 10000)
               Rows Removed by Filter: 990001
 Total runtime: 687.150 ms
(9 rows)
```

The command output contains information such as actual time and memory usage, and the **runtime** field.

Note: If the **analyze** parameter is used, the SQL statement is actually executed. The preceding information displays the actual execution results.

Step 7 Print the CPU usage.

The prerequisite for printing the CPU usage is that **analyze** is used. Otherwise, an error is reported. If you run the following statement, an error is reported:

```
openGauss=# explain(CPU on) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where t1.a <
10000;
ERROR:  EXPLAIN option CPU requires ANALYZE
```

Based on the error message, modify the statement as follows:

```
openGauss=# explain(analyze on, CPU on) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a
where t1.a < 10000;
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
-------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 width=4) (actual time=200.804..462.344
rows=10000 loops=1)
   Hash Cond: (t2.a = t1.a)
   (CPU: ex c/r=-1100847248513626, ex row=1010001, ex cyc=-111185682184601154352, inc
cyc=11231977254166200320)
   ->  Seq Scan on exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4) (actual
time=0.009..99.195 rows=1000001 loops=1)
         (CPU: ex c/r=1123086552928208, ex row=1000001, ex cyc=1123087676014761213952, inc
cyc=1123087676014761213952)
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4) (actual time=200.731..200.731
rows=10000 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 608kB
         (CPU: ex c/r=-1123085396774339, ex row=10000, ex cyc=-11230853967743391744, inc
cyc=1123085416577088)
         ->  Seq Scan on exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4) (actual
time=0.024..198.256 rows=10000 loops=1)
```

```
                    Filter: (a < 10000)
                    Rows Removed by Filter: 990001
                    (CPU: ex c/r=11231965821194, ex row=1000001, ex cyc=11231977053159968768, inc
cyc=11231977053159968768)
 Total runtime: 463.281 ms
(13 rows)
```

The preceding command output displays CPU-related data, where execution in analyze mode does not.

Step 8 Print the usage of the buffer.

To print the buffer usage, you also need to enable **analyze**. Otherwise, an error is reported. For example:

```
openGauss=# explain( buffers on) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where
t1.a < 10000;
ERROR:  EXPLAIN option BUFFERS requires ANALYZE
```

Add the **analyze** parameter as follows:

```
openGauss=# explain(analyze on, buffers on) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a =
t2.a where t1.a < 10000;
                                                                QUERY PLAN
------------------------------------------------------------------------------------------------------------------
-------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 width=4) (actual time=200.942..487.442
rows=10000 loops=1)
   Hash Cond: (t2.a = t1.a)
   (Buffers: shared hit=8850)
   ->  Seq Scan on exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4) (actual
time=0.010..100.901 rows=1000001 loops=1)
         (Buffers: shared hit=4425)
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4) (actual time=200.873..200.873
rows=10000 loops=1)
          Buckets: 32768  Batches: 1  Memory Usage: 608kB
         (Buffers: shared hit=4425)
         ->  Seq Scan on exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4) (actual
time=0.028..198.198 rows=10000 loops=1)
               Filter: (a < 10000)
               Rows Removed by Filter: 990001
               (Buffers: shared hit=4425)
 Total runtime: 488.624 ms
(13 rows)
```

You can view the cache hit information in the **Buffers** row. This is useful for analyzing the usage of the cache.

Step 9 Specify the output format of the execution plan.

```
openGauss=#  set explain_perf_mode = 'normal';
openGauss=# explain(format json) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where
t1.a < 10000;
                        QUERY PLAN
```

```
----------------------------------------------
 [                                             +
   {                                           +
     "Plan": {                                 +
       "Node Type": "Hash Join",               +
       "Join Type": "Right",                   +
       "Startup Cost": 17050.71,               +
       "Total Cost": 35326.29,                 +
       "Plan Rows": 10056,                     +
       "Plan Width": 4,                        +
       "Inner Unique": false,                  +
       "Hash Cond": "(t2.a = t1.a)",           +
       "Plans": [                              +
         {                                     +
           "Node Type": "Seq Scan",            +
           "Parent Relationship": "Outer",     +
           "Relation Name": "exp_t2",          +
           "Alias": "t2",                      +
           "Startup Cost": 0.00,               +
           "Total Cost": 14425.01,             +
           "Plan Rows": 1000001,               +
           "Plan Width": 4                     +
         },                                    +
         {                                     +
           "Node Type": "Hash",                +
           "Parent Relationship": "Inner",     +
           "Startup Cost": 16925.01,           +
           "Total Cost": 16925.01,             +
           "Plan Rows": 10056,                 +
           "Plan Width": 4,                    +
           "Plans": [                          +
             {                                 +
               "Node Type": "Seq Scan",        +
               "Parent Relationship": "Outer",+
               "Relation Name": "exp_t1",      +
               "Alias": "t1",                  +
               "Startup Cost": 0.00,           +
               "Total Cost": 16925.01,         +
               "Plan Rows": 10056,             +
               "Plan Width": 4,                +
               "Filter": "(a < 10000)"         +
             }                                 +
           ]                                   +
         }                                     +
       ]                                       +
     }                                         +
   }                                           +
 ]
(1 row)

openGauss=# explain(analyze, format json) select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a
where t1.a < 10000;
                          QUERY PLAN
----------------------------------------------
 [                                             +
```

```
{                                                +
  "Plan": {                                      +
    "Node Type": "Hash Join",                    +
    "Join Type": "Right",                        +
    "Startup Cost": 17050.71,                    +
    "Total Cost": 35326.29,                      +
    "Plan Rows": 10056,                          +
    "Plan Width": 4,                             +
    "Actual Startup Time": 199.484,              +
    "Actual Total Time": 457.752,                +
    "Actual Rows": 10000,                        +
    "Actual Loops": 1,                           +
    "Inner Unique": false,                       +
    "Hash Cond": "(t2.a = t1.a)",                +
    "Plans": [                                   +
      {                                          +
        "Node Type": "Seq Scan",                 +
        "Parent Relationship": "Outer",          +
        "Relation Name": "exp_t2",               +
        "Alias": "t2",                           +
        "Startup Cost": 0.00,                    +
        "Total Cost": 14425.01,                  +
        "Plan Rows": 1000001,                    +
        "Plan Width": 4,                         +
        "Actual Startup Time": 0.009,            +
        "Actual Total Time": 99.898,             +
        "Actual Rows": 1000001,                  +
        "Actual Loops": 1                        +
      },                                         +
      {                                          +
        "Node Type": "Hash",                     +
        "Parent Relationship": "Inner",          +
        "Startup Cost": 16925.01,                +
        "Total Cost": 16925.01,                  +
        "Plan Rows": 10056,                      +
        "Plan Width": 4,                         +
        "Actual Startup Time": 199.418,          +
        "Actual Total Time": 199.418,            +
        "Actual Rows": 10000,                    +
        "Actual Loops": 1,                       +
        "Hash Buckets": 32768,                   +
        "Original Hash Buckets": 32768,          +
        "Hash Batches": 1,                       +
        "Original Hash Batches": 1,              +
        "Peak Memory Usage": 608,                +
        "Plans": [                               +
          {                                      +
            "Node Type": "Seq Scan",             +
            "Parent Relationship": "Outer",      +
            "Relation Name": "exp_t1",           +
            "Alias": "t1",                       +
            "Startup Cost": 0.00,                +
            "Total Cost": 16925.01,              +
            "Plan Rows": 10056,                  +
            "Plan Width": 4,                     +
```

```
                    "Actual Startup Time": 0.023,     +
                    "Actual Total Time": 196.968,     +
                    "Actual Rows": 10000,               +
                    "Actual Loops": 1,                  +
                    "Filter": "(a < 10000)",          +
                    "Rows Removed by Filter": 990001+
                  }                                     +
                ]                                       +
              }                                         +
            ]                                           +
        },                                              +
        "Triggers": [                                  +
        ],                                              +
        "Total Runtime": 458.696                        +
    }                                                    +
 ]
(1 row)
```

Step 10 Print an execution plan in **performance** mode.

```
openGauss=# explain performance select t1.a from exp_t1 t1 full join exp_t2 t2 on t1.a = t2.a where
t1.a < 10000;
                                                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
-----------------
 Hash Right Join  (cost=17050.71..35326.29 rows=10056 distinct=[200, 10056] width=4) (actual
time=200.913..482.454 rows=10000 loops=1)
   Output: t1.a
   Hash Cond: (t2.a = t1.a)
   (Buffers: shared hit=8850)
   (CPU: ex c/r=-1101041048251915, ex row=1010001, ex cyc=-1112052559775483035648, inc
cyc=11233954588370022400)
   ->  Seq Scan on public.exp_t2 t2  (cost=0.00..14425.01 rows=1000001 width=4) (actual
time=0.012..102.240 rows=1000001 loops=1)
         Output: t2.a, t2.b
         (Buffers: shared hit=4425)
         (CPU: ex c/r=1123284267796455, ex row=1000001, ex cyc=112328539108072279 2448, inc
cyc=112328539108072279 2448)
   ->  Hash  (cost=16925.01..16925.01 rows=10056 width=4) (actual time=200.841..200.841
rows=10000 loops=1)
         Output: t1.a
          Buckets: 32768  Batches: 1  Memory Usage: 608kB
         (Buffers: shared hit=4425)
         (CPU: ex c/r=-1123283110397747, ex row=10000, ex cyc=-1123283110397747 6096, inc
cyc=1123283130202384)
         ->  Seq Scan on public.exp_t1 t1  (cost=0.00..16925.01 rows=10056 width=4) (actual
time=0.025..198.012 rows=10000 loops=1)
               Output: t1.a
               Filter: (t1.a < 10000)
               Rows Removed by Filter: 990001
               (Buffers: shared hit=4425)
               (CPU: ex c/r=11233943153164, ex row=1000001, ex cyc=11233954387107678208, inc
cyc=11233954387107678208)
 Total runtime: 483.627 ms
(21 rows)
```

If the **performance** mode is used, the SQL statement is also executed.

Step 11 Drop data.

```
openGauss=# drop table exp_t1;
DROP TABLE
openGauss=# drop table exp_t2;
DROP TABLE
```

5.5.3 Printing an Execution Plan in Rollback Mode

When the **analyze** or **performance** mode is used to print an execution plan, an SQL statement is actually executed. When you need to perform operations such as UPDATE and INSERT in **analyze** or **performance** mode, you need to execute the statements in transactions and roll back the statements after the execution to prevent services from being affected.

Step 1 Create a table structure.

```
openGauss=# create table exp_fallback_t1(a int, b int);
CREATE TABLE
```

Step 2 Print an execution plan using INSERT in normal mode.

```
openGauss=# explain insert into exp_fallback_t1 values(1, 1);
                                QUERY PLAN
-----------------------------------------------------------------
 [Bypass]
 Insert on exp_fallback_t1  (cost=0.00..0.01 rows=1 width=0)
   ->  Result  (cost=0.00..0.01 rows=1 width=0)
(3 rows)

openGauss=# select * from exp_fallback_t1;
 a | b
---+---
(0 rows)
```

If the execution plan using INSERT is printed in normal mode, the SQL statement is not executed. As a result, no data can be queried.

Step 3 Print an execution plan using UPDATE in normal mode.

```
openGauss=# insert into exp_fallback_t1 values (1, 1);
INSERT 0 1
openGauss=# explain update exp_fallback_t1 set a = 2, b = 2;
                                QUERY PLAN
-------------------------------------------------------------------------
 Update on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6)
(2 rows)
```

```
openGauss=# select * from exp_fallback_t1;
 a | b
---+---
 1 | 1
(1 row)
```

If the execution plan using UPDATE is printed in normal mode, the SQL statement is not executed. Therefore, the queried data does not change.

Step 4 Print an execution plan using DELETE in normal mode.

```
openGauss=# insert into exp_fallback_t1 values (1, 1);
INSERT 0 1
openGauss=# explain delete from exp_fallback_t1;
                                  QUERY PLAN
------------------------------------------------------------------------------
 Delete on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6)
(2 rows)

openGauss=# select * from exp_fallback_t1;
 a | b
---+---
 1 | 1
 1 | 1
(2 rows)
```

When the execution plan using DELETE is printed in normal mode, the SQL statement is not executed and the data is not deleted.

Step 5 Print the execution plan using INSERT in **analyze** mode.

Delete historical data.

```
openGauss=# delete from exp_fallback_t1;
DELETE 2
```

Print the execution plan using INSERT in **analyze** mode.

```
openGauss=# explain analyze insert into exp_fallback_t1 values(100, 100);
                                                        QUERY PLAN
---------------------------------------------------------------------------------------------------------------
 [Bypass]
 Insert on exp_fallback_t1  (cost=0.00..0.01 rows=1 width=0) (actual time=0.055..0.056 rows=1
loops=1)
   ->  Result  (cost=0.00..0.01 rows=1 width=0) (actual time=0.002..0.003 rows=1 loops=1)
 Total runtime: 0.155 ms
(4 rows)

openGauss=# select * from exp_fallback_t1;
  a  |  b
-----+-----
 100 | 100
```

```
(1 row)
```

When the execution plan using INSERT is printed in **analyze** mode, the SQL statement is executed and data is inserted into the table.

Step 6 Print an execution plan using UPDATE in **analyze** mode.

```
openGauss=# explain analyze update exp_fallback_t1 set a = 200, b = 200;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Update on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.052..0.054 rows=1
loops=1)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.013..0.014
rows=1 loops=1)
 Total runtime: 0.206 ms
(3 rows)

openGauss=# select * from exp_fallback_t1;
  a  |  b
-----+-----
 200 | 200
(1 row)
```

When the execution plan using UPDATE is printed in **analyze** mode, the SQL statement is executed and data is modified.

Step 7 Print an execution plan using DELETE in **analyze** mode.

```
openGauss=# explain analyze delete from exp_fallback_t1;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Delete on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.042..0.044 rows=1
loops=1)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.009..0.010
rows=1 loops=1)
 Total runtime: 0.228 ms
(3 rows)

openGauss=# select * from exp_fallback_t1;
 a | b
---+---
(0 rows)
```

When the execution plan using DELETE is printed in **analyze** mode, the SQL statement is executed and data is deleted.

Step 8 Print an execution plan using INSERT in **analyze+rollback** mode.

Delete data.

```
openGauss=# delete from exp_fallback_t1;
DELETE 0
```

Start a transaction and print an execution plan.

```
openGauss=# START TRANSACTION;
START TRANSACTION
openGauss=# explain analyze insert into exp_fallback_t1 values(1000, 1000);
                                                          QUERY PLAN
------------------------------------------------------------------------------------------------------------
 [Bypass]
 Insert on exp_fallback_t1  (cost=0.00..0.01 rows=1 width=0) (actual time=0.065..0.066 rows=1
loops=1)
   ->  Result  (cost=0.00..0.01 rows=1 width=0) (actual time=0.002..0.002 rows=1 loops=1)
 Total runtime: 0.176 ms
(4 rows)

openGauss=# ROLLBACK;
ROLLBACK
openGauss=# select * from exp_fallback_t1;
 a | b
---+---
(0 rows)
```

When the **analyze+rollback** mode is used, the actual execution status is printed and the real data remains unchanged.

Step 9 Use the **analyze+rollback** mode to print an execution plan using UPDATE.

```
openGauss=# insert into exp_fallback_t1 values (1, 1);
INSERT 0 1
openGauss=# START TRANSACTION;
START TRANSACTION
openGauss=# explain analyze update exp_fallback_t1 set a = 2000, b = 2000;
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Update on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.061..0.063 rows=1
loops=1)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.017..0.018
rows=1 loops=1)
 Total runtime: 0.257 ms
(3 rows)

openGauss=# ROLLBACK;
ROLLBACK
openGauss=#
openGauss=# select * from exp_fallback_t1;
 a | b
---+---
 1 | 1
(1 row)
```

Step 10 Print an execution plan using DELETE in **analyze+rollback** mode.

```
openGauss=# insert into exp_fallback_t1 values (2, 2);
INSERT 0 1
openGauss=# START TRANSACTION;
START TRANSACTION
openGauss=# explain analyze delete exp_fallback_t1;
```

```
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Delete on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.042..0.048 rows=2 loops=1)
   ->  Seq Scan on exp_fallback_t1  (cost=0.00..31.49 rows=2149 width=6) (actual time=0.015..0.017 rows=2 loops=1)
 Total runtime: 0.226 ms
(3 rows)

openGauss=# ROLLBACK;
ROLLBACK
openGauss=#
openGauss=# select * from exp_fallback_t1;
 a | b
---+---
 1 | 1
 2 | 2
(2 rows)
```

Step 11 Drop data.

```
openGauss=# drop table exp_fallback_t1;
DROP TABLE
```

5.5.4 Setting GUC Parameters to Optimize an Execution Plan

5.5.4.1 enable_bitmapscan

Step 1 Create tables.

```
openGauss=# create table btm_t1(a numeric,b numeric,c numeric);
CREATE TABLE
openGauss=# create table btm_t2(a numeric,b numeric,c numeric);
CREATE TABLE
openGauss=# create table btm_t3(a numeric,b numeric,c numeric);
CREATE TABLE
```

Step 2 Prepare lab data.

```
declare
 n1 numeric := 100;
 n2 numeric := 0;
 n3 numeric := 100;
begin
 while n1 > 0 loop
  n2 := 0;
  while n2 < 100 loop
   n3 := 100;
   while n3 > 0 loop
    insert into btm_t1 values(n1, n2, n3);
    insert into btm_t2 values(n1, n3, n2);
    insert into btm_t3 values(n2, n1, n3);
```

```
      n3 := n3 - 1;
    end loop;
    n2 := n2 + 1;
  end loop;
  n1 := n1 - 1;
 end loop;
end;
/
```

If the following information is displayed, the execution is successful:

```
ANONYMOUS BLOCK EXECUTE
```

Step 3 Create indexes.

```
create index idx_btm_11 on btm_t1 using btree (a);
create index idx_btm_12 on btm_t1 using btree (b);
create index idx_btm_13 on btm_t1 using btree (c);

create index idx_btm_21 on btm_t2 using btree (a);
create index idx_btm_22 on btm_t2 using btree (b);
create index idx_btm_23 on btm_t2 using btree (c);

create index idx_btm_31 on btm_t3 using btree (a);
create index idx_btm_32 on btm_t3 using btree (b);
create index idx_btm_33 on btm_t3 using btree (c);
```

Step 4 Disable **enable_bitmapscan** and print an execution plan.

```
openGauss=# set enable_bitmapscan to off;
SET
openGauss=# explain (costs off) select * from btm_t1 t1, btm_t2 t2, btm_t3 t3
where t1.a = t2.b and t2.a = t3.c and t3.a = t1.c and t1.a = 1 and t2. a = 1 and t3.a = 1;
                              QUERY PLAN
-------------------------------------------------------------
 Nested Loop
   ->  Nested Loop
         ->  Index Scan using idx_btm_21 on btm_t2 t2
               Index Cond: (a = 1::numeric)
               Filter: (b = 1::numeric)
         ->  Materialize
               ->  Index Scan using idx_btm_11 on btm_t1 t1
                     Index Cond: (a = 1::numeric)
                     Filter: (c = 1::numeric)
   ->  Materialize
         ->  Index Scan using idx_btm_33 on btm_t3 t3
               Index Cond: (c = 1::numeric)
               Filter: (a = 1::numeric)
(13 rows)
```

The execution plan does not contain the bitmap scan operator.

Step 5 Enable **enable_bitmapscan** and print an execution plan.

```
openGauss=# set enable_bitmapscan to on;
SET
openGauss=# explain (costs off) select * from btm_t1 t1, btm_t2 t2, btm_t3 t3
where t1.a = t2.b and t2.a = t3.c and t3.a = t1.c and t1.a = 1 and t2. a = 1 and t3.a = 1;
                                    QUERY PLAN
------------------------------------------------------------------------
 Nested Loop
   ->  Nested Loop
         ->  Index Scan using idx_btm_21 on btm_t2 t2
               Index Cond: (a = 1::numeric)
               Filter: (b = 1::numeric)
         ->  Materialize
               ->  Index Scan using idx_btm_11 on btm_t1 t1
                     Index Cond: (a = 1::numeric)
                     Filter: (c = 1::numeric)
   ->  Materialize
         ->  Bitmap Heap Scan on btm_t3 t3
               Recheck Cond: ((c = 1::numeric) AND (a = 1::numeric))
               ->  BitmapAnd
                     ->  Bitmap Index Scan on idx_btm_33
                           Index Cond: (c = 1::numeric)
                     ->  Bitmap Index Scan on idx_btm_31
                           Index Cond: (a = 1::numeric)
(17 rows)
```

The bitmap heap scan operator is displayed in the execution plan.

Note: This lab only tests the impact of GUC parameters on the execution plan. In actual services, you need to determine whether to enable GUC parameters based on the actual time consumption. In this test case, enabling bitmap scan on the test server can reduce the time consumed by about 10 ms.

Step 6 Drop data.

```
openGauss=# drop table btm_t1;
DROP TABLE
openGauss=# drop table btm_t2;
DROP TABLE
openGauss=# drop table btm_t3;
DROP TABLE
```

5.5.4.2 enable_tidscan

Step 1 Create a table.

```
openGauss=# create table tid_t1(a numeric,b numeric,c numeric);
CREATE TABLE
```

Step 2 Prepare lab data.

```
declare
  n1 numeric := 10;
  n2 numeric := 0;
  n3 numeric := 100;
```

```
begin
 while n1 > 0 loop
  n2 := 0;
  while n2 < 100 loop
   n3 := 100;
   while n3 > 0 loop
    insert into tid_t1 values(n1, n2, n3);
    n3 := n3 - 1;
   end loop;
   n2 := n2 + 1;
  end loop;
  n1 := n1 - 1;
 end loop;
end;
/
```

If the following information is displayed, the execution is successful:

```
ANONYMOUS BLOCK EXECUTE
```

Step 3 Disable **enable_tidscan** and print an execution plan.

```
openGauss=# set enable_tidscan to off;
SET
openGauss=# explain analyze select * from tid_t1 where ctid = '(0,10)'::tid;
                                                        QUERY PLAN
---------------------------------------------------------------------------------------------------------
 Seq Scan on tid_t1  (cost=0.00..979.39 rows=1 width=96) (actual time=0.047..28.243 rows=1
loops=1)
   Filter: (ctid = '(0,10)'::tid)
   Rows Removed by Filter: 99999
 Total runtime: 28.320 ms
(4 rows)
```

Step 4 Enable **enable_tidscan** to print an execution plan.

```
openGauss=# set enable_tidscan to on;
SET
openGauss=# explain analyze select * from tid_t1 where ctid = '(0,10)'::tid;
                                                      QUERY PLAN
-----------------------------------------------------------------------------------------------------
 Tid Scan on tid_t1  (cost=0.00..4.01 rows=1 width=96) (actual time=0.009..0.009 rows=1 loops=1)
   TID Cond: (ctid = '(0,10)'::tid)
 Total runtime: 0.047 ms
(3 rows)
```

According to the comparison of the two execution plans, the TidScan operator can greatly improve the query performance. In this test case, the performance is improved by up to three orders of magnitude.

Step 5 Drop data.

```
openGauss=# drop table tid_t1;
```

```
DROP TABLE
openGauss=# reset enable_tidscan;
RESET
```

5.5.4.3 enable_indexonlyscan

Step 1 Create a table.

```
openGauss=# create table testidx_t1 (a int, b int);
CREATE TABLE
openGauss=# create index idx_testidx_t1 on testidx_t1 (a, b);
CREATE INDEXCREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into testidx_t1 values(generate_series(1, 1000), generate_series(1, 1000));
INSERT 0 1000
```

Step 3 Disable **enable_indexonlyscan** and print an execution plan.

```
openGauss=# set enable_indexonlyscan to off;
SET
openGauss=# explain analyze select a, b from testidx_t1 where a = 10 and b = 20;
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
-----
 [Bypass]
 Index Scan using idx_testidx_t1 on testidx_t1  (cost=0.00..8.27 rows=1 width=8) (actual
time=0.013..0.013 rows=0 loops=1)
   Index Cond: ((a = 10) AND (b = 20))
 Total runtime: 0.061 ms
(4 rows)
```

Step 4 Enable **enable_indexonlyscan** to print an execution plan.

```
openGauss=# set enable_indexonlyscan to on;
SET
openGauss=# explain analyze select a, b from testidx_t1 where a = 10 and b = 20;
                                                                   QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
----------
 [Bypass]
 Index Only Scan using idx_testidx_t1 on testidx_t1  (cost=0.00..8.27 rows=1 width=8) (actual
time=0.012..0.012 rows=0 loops=1)
   Index Cond: ((a = 10) AND (b = 20))
   Heap Fetches: 0
 Total runtime: 0.062 ms
(5 rows)
```

In this test case, index scan is used. Therefore, the actual time consumption is the same and within the fluctuation range.

Note that in different environments, although the related GUC parameter is enabled, the database system may not use the corresponding operator.

Step 5 Drop data.

```
openGauss=# drop table testidx_t1;
DROP TABLE
openGauss=# reset enable_indexonlyscan;
RESET
```

5.5.4.4 Scenarios Where GUC Parameters Cannot Affect an Execution Plan

Step 1 Create table structures.

```
openGauss=# create table employee(id int, deptid int);
CREATE TABLE
openGauss=# create table manager(id int, deptid int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into employee values(1, 1), (2,1),(3,2),(4, 1), (5,2);
INSERT 0 5
openGauss=# insert into manager values(1,1), (2,2),(3,1),(4,2);
INSERT 0 4
```

Step 3 Disable **enable_nestloop** and print an execution plan.

```
openGauss=# set   enable_nestloop to off;
SET
openGauss=# explain SELECT * FROM employee e JOIN manager m ON e.deptid < m.deptid;
                                  QUERY PLAN
------------------------------------------------------------------------------
 Nested Loop   (cost=0.00..69341.37 rows=1539400 width=16)
   Join Filter: (e.deptid < m.deptid)
   ->  Seq Scan on employee e   (cost=0.00..31.49 rows=2149 width=8)
   ->  Materialize   (cost=0.00..42.23 rows=2149 width=8)
         ->  Seq Scan on manager m   (cost=0.00..31.49 rows=2149 width=8)
(5 rows)
```

Therefore, disabling the **enable_nestloop** parameter only prevents the system from using the nested loop execution plan. If there is no other feasible execution plan, the nested loop plan is still used.

Step 4 Drop data.

```
openGauss=# reset enable_nestloop;
RESET
openGauss=# drop table employee;
DROP TABLE
```

5.5.5 Using Plan Hints on an Execution Plan

Step 1 Create table structures.

```
openGauss=# create table t1(c1 number,c2 number,c3 number);
CREATE TABLE
openGauss=# create table t2(c1 number,c2 number,c3 number);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values(generate_series(1,10000), 2 ,3);
INSERT 0 10000
openGauss=# insert into t2 values(generate_series(1,10000), 2, 3);
INSERT 0 10000
```

Step 3 View the default execution plan.

```
openGauss=# explain analyze select * from t1 join t2 on t1.c1 = t2.c2;
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=135.21..950.45 rows=63546 width=192) (actual time=5.444..13.828 rows=10000
loops=1)
   Hash Cond: (t1.c1 = t2.c2)
   ->  Seq Scan on t1  (cost=0.00..90.65 rows=3565 width=96) (actual time=0.044..2.452
rows=10000 loops=1)
   ->  Hash  (cost=90.65..90.65 rows=3565 width=96) (actual time=5.331..5.331 rows=10000
loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 715kB
         ->  Seq Scan on t2  (cost=0.00..90.65 rows=3565 width=96) (actual time=0.033..2.469
rows=10000 loops=1)
 Total runtime: 14.777 ms
(7 rows)
```

Step 4 Use plan hints to change an execution plan.

```
openGauss=# explain analyze select/*+ mergejoin(t1 t2) */ * from t1 join t2 on t1.c1 = t2.c2;
                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Merge Join  (cost=1638.77..1838.77 rows=10000 width=30) (actual time=8.899..13.089 rows=10000
loops=1)
   Merge Cond: (t1.c1 = t2.c2)
   ->  Sort  (cost=819.39..844.39 rows=10000 width=15) (actual time=3.672..3.673 rows=3 loops=1)
         Sort Key: t1.c1
         Sort Method: quicksort  Memory: 1010kB
         ->  Seq Scan on t1  (cost=0.00..155.00 rows=10000 width=15) (actual time=0.013..1.227
rows=10000 loops=1)
   ->  Sort  (cost=819.39..844.39 rows=10000 width=15) (actual time=5.217..5.708 rows=10000
loops=1)
         Sort Key: t2.c2
         Sort Method: quicksort  Memory: 1010kB
         ->  Seq Scan on t2  (cost=0.00..155.00 rows=10000 width=15) (actual time=0.008..1.135
rows=10000 loops=1)
```

```
 Total runtime: 14.072 ms
(11 rows)
```

Step 5 Drop data.

```
openGauss=# Drop table t1;
DROP TABLE
openGauss=# Drop table t2;
DROP TABLE
```

5.5.6 Poor Plan Hints Increase the Time Consumption

Step 1 Create table structures.

```
openGauss=# create table t1(c1 number,c2 number,c3 number);
CREATE TABLE
openGauss=# create table t2(c1 number,c2 number,c3 number);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values(generate_series(1,100000), 2 ,3);
INSERT 0 100000
openGauss=# insert into t2 values(generate_series(1,10000), 2, 3);
INSERT 0 10000
openGauss=# delete from t1;
DELETE 100000
openGauss=# insert into t1 values(generate_series(1,1000), 2 ,3);
INSERT 0 1000
```

Step 3 View the default execution plan.

```
openGauss=# explain analyze select * from t1 join t2 on t1.c1 = t2.c2;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=135.21..9578.16 rows=736066 width=192) (actual time=21.043..25.166 rows=10000
loops=1)
   Hash Cond: (t1.c1 = t2.c2)
   ->  Seq Scan on t1  (cost=0.00..1049.94 rows=41294 width=96) (actual time=15.675..15.930
rows=1000 loops=1)
   ->  Hash  (cost=90.65..90.65 rows=3565 width=96) (actual time=5.283..5.283 rows=10000
loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 715kB
         ->  Seq Scan on t2  (cost=0.00..90.65 rows=3565 width=96) (actual time=0.034..2.352
rows=10000 loops=1)
 Total runtime: 26.187 ms
(7 rows)
```

Step 4 Use plan hints to change an execution plan.

```
openGauss=# explain analyze select/*+ nestloop(t1 t2) */ * from t1 join t2 on t1.c1 = t2.c2;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------
 Nested Loop  (cost=0.00..2209346.15 rows=736066 width=192) (actual time=11.419..3882.750
rows=10000 loops=1)
   Join Filter: (t1.c1 = t2.c2)
   Rows Removed by Join Filter: 9990000
   ->  Seq Scan on t1  (cost=0.00..1049.94 rows=41294 width=96) (actual time=4.810..5.285
rows=1000 loops=1)
   ->  Materialize  (cost=0.00..108.48 rows=3565 width=96) (actual time=0.150..823.231
rows=10000000 loops=1000)
         ->  Seq Scan on t2  (cost=0.00..90.65 rows=3565 width=96) (actual time=0.011..1.231
rows=10000 loops=1)
 Total runtime: 3883.752 ms
(7 rows)
```

Step 5 Drop data.

```
openGauss=# Drop table t1;
DROP TABLE
openGauss=# Drop table t2;
DROP TABLE
```

5.6 Optimizing openGauss Indexes

5.6.1 Lab Description

Indexes are an important method to improve query performance. Indexes use data in some common query columns of a table to construct a tree structure, and the performance of each query is optimized by trading off space for time. This experiment uses cases related to indexes to explore the impact of indexes on query performance.

5.6.2 Basic Usage of Indexes

Step 1 Create a table.

```
openGauss=# create table t1(a int, b int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values (generate_series(1, 100000), 1);
INSERT 0 100000
```

Step 3 Run a query SQL statement.

```
penGauss=# \timing on
Timing is on.
```

```
openGauss=# select * from t1 where a > 1000 and a < 1003;
   a    | b
--------+---
 1001 | 1
 1002 | 1
(2 rows)

Time: 34.666 ms
openGauss=# \timing off
Timing is off.
```

Step 4 Create an index.

```
openGauss=# create index idx_t1_a on t1(a);
CREATE INDEX
```

Step 5 Run the query SQL statement again.

```
openGauss=# \timing on
Timing is on.
openGauss=# select * from t1 where a > 1000 and a < 1003;
   a    | b
--------+---
 1001 | 1
 1002 | 1
(2 rows)

Time: 1.084 ms
openGauss=# \timing off
Timing is off.
```

After the index is used, the execution speed is improved by more than one order of magnitude.

The larger the data volume, the more obvious the improvement.

Step 6 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
```

5.6.3 Selecting a Proper Column as an Index

Step 1 Create a table.

```
openGauss=# create table t1(a int, b int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values (generate_series(1, 4), generate_series(1, 10000));
```

```
INSERT 0 10000
```

Step 3 Create an index for column **a**.

```
openGauss=# create index idx_t1_a on t1(a);
CREATE INDEX
```

Step 4 Run a query SQL statement.

```
openGauss=# \timing on
Timing is on.
openGauss=# select count(*)    from t1 where a > 2 and b < 10;
 count
-------
       4
(1 row)

Time: 3.009 ms
openGauss=# select count(*)    from t1 where a > 2 and b < 10;
 count
-------
       4
(1 row)

Time: 2.243 ms
openGauss=# \timing off
Timing is off.
```

Step 5 Drop the old index and create another.

```
openGauss=# drop index idx_t1_a;
DROP INDEX
openGauss=# create index idx_t1_b on t1(b);
CREATE INDEX
```

Step 6 Run the query SQL statement again.

```
openGauss=# \timing on
Timing is on.
openGauss=# select count(*)    from t1 where a > 2 and b < 10;
 count
-------
       4
(1 row)

Time: 1.180 ms
openGauss=# select count(*)    from t1 where a > 2 and b < 10;
 count
-------
       4
(1 row)

Time: 0.506 ms
```

```
openGauss=# \timing off
Timing is off.
```

Result: If column **b** is selected to create an index, the query performance is improved because there are few or no duplicate values in column **b**.

Step 7 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
```

5.6.4 Selecting a Proper Index Column Type

Step 1 Create tables.

```
openGauss=# create table t1(a int, b int);
CREATE TABLE
openGauss=# create table t2(a numeric, b numeric);
CREATE TABLE
openGauss=# create table t3(a text, b text);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values (generate_series(1, 10000), 1);
INSERT 0 10000
openGauss=# insert into t2 values (generate_series(1, 10000), 1);
INSERT 0 10000
openGauss=# insert into t3 values (generate_series(1, 10000), 1);
INSERT 0 10000
```

Step 3 Create indexes.

```
openGauss=# create index idx_t1_a on t1(a);
CREATE INDEX
openGauss=# create index idx_t2_a on t2(a);
CREATE INDEX
openGauss=# create index idx_t3_a on t3(a);
CREATE INDEX
```

Step 4 Run a query SQL statement.

```
openGauss=# \timing on
Timing is on.
openGauss=# select count(1) from t1 where a > 100 and a < 1000;
 count
-------
   899
(1 row)
```

```
Time: 3.288 ms
openGauss=# select count(1) from t2 where a > 100 and a < 1000;
 count
-------
   899
(1 row)

Time: 5.258 ms
openGauss=# select count(1) from t3 where a > 100 and a < 1000;
 count
-------
   899
(1 row)

Time: 7.118 ms
openGauss=# \timing off
Timing is off.
```

In this test case, the time consumption of the types is ranked as follows: int < numeric < text.

Step 5 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
openGauss=# drop table t2;
DROP TABLE
openGauss=# drop table t3;
DROP TABLE
```

5.6.5 Dead Tuple Problem During Index Scans

In the openGauss database, dead tuples represent deleted data rows in a table. Although these data rows are no longer used, the disk space is still reserved for future reuse. During index scan, dead tuples may cause performance problems because they increase the scan time.

Solution:

Periodically perform the VACUUM operation, which can clear dead tuples and release space for future use. You can set a scheduled task or manually run the **VACUUM FULL** command to clear dead tuples.

Perform the AUTOVACUUM operation on the database to automatically clear dead tuples. You can adjust related parameters to optimize the AUTOVACUUM behavior.

Design a proper deletion policy: Avoid frequent data deletion, or use a logical deletion policy to mark whether a record has been deleted with a field instead of physically deleting the record.

Periodically run the **ANALYZE** statement to update system statistics about tables and indexes, helping the query optimizer make a better execution plan.

Step 1 Create a table.

```
openGauss=# create table t1(a int, b int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values (generate_series(1, 1000000), 1);
INSERT 0 1000000
```

Step 3 Create an index.

```
openGauss=# create index idx_t1_a on t1(a);
CREATE INDEX
```

Step 4 Run a query SQL statement.

```
openGauss=# Explain analyze select * from t1 where a > 200000 and a < 500000;
                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
-------
 Bitmap Heap Scan on t1  (cost=107.50..4817.88 rows=5000 width=8) (actual time=26.044..69.908
rows=299999 loops=1)
   Recheck Cond: ((a > 200000) AND (a < 500000))
   Heap Blocks: exact=1329
   ->  Bitmap Index Scan on idx_t1_a  (cost=0.00..106.25 rows=5000 width=0) (actual
time=25.800..25.800 rows=299999 loops=1)
         Index Cond: ((a > 200000) AND (a < 500000))
 Total runtime: 92.887 ms
(6 rows)
```

Step 5 Update the index column.

```
openGauss=# update t1 set a = a + 1 where a %2 = 0;
UPDATE 500000
```

Step 6 Run the query SQL statement again.

A large amount of data is output. Therefore, the execution plan is used to run the statement.

```
openGauss=# Explain analyze select * from t1 where a > 200000 and a < 500000;
                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
-------
 Bitmap Heap Scan on t1  (cost=189.14..7255.27 rows=7501 width=8) (actual time=35.088..126.692
rows=300000 loops=1)
   Recheck Cond: ((a > 200000) AND (a < 500000))
   Heap Blocks: exact=1993
   ->  Bitmap Index Scan on idx_t1_a  (cost=0.00..187.26 rows=7501 width=0) (actual
time=34.684..34.684 rows=449999 loops=1)
         Index Cond: ((a > 200000) AND (a < 500000))
 Total runtime: 149.776 ms
(6 rows)
```

The result shows that the process takes about 150 ms.

Step 7 Run the **VACUUM FULL** command.

```
openGauss=# vacuum full t1;
VACUUM
```

Step 8 Run the query SQL statement again.

```
openGauss=# Explain analyze select * from t1 where a > 200000 and a < 500000;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
-------
 Bitmap Heap Scan on t1  (cost=107.50..4817.88 rows=5000 width=8) (actual time=26.329..70.186
rows=300000 loops=1)
   Recheck Cond: ((a > 200000) AND (a < 500000))
   Heap Blocks: exact=1330
   ->  Bitmap Index Scan on idx_t1_a  (cost=0.00..106.25 rows=5000 width=0) (actual
time=26.078..26.078 rows=300000 loops=1)
         Index Cond: ((a > 200000) AND (a < 500000))
 Total runtime: 93.187 ms
(6 rows)
```

The result shows that the time is reduced to 90 ms.

Step 9 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
```

5.6.6 Invalid Indexes

Step 1 Create a table.

```
openGauss=# create table t1(a int, b int);
CREATE TABLE
```

Step 2 Prepare lab data.

```
openGauss=# insert into t1 values (generate_series(1, 100000), 1);
INSERT 0 100000
```

Step 3 Create an index.

```
openGauss=# create index idx_t1_a on t1(a);
CREATE INDEX
```

Step 4 Run SQL statement 1 to check whether an index is invalid in the query.

```
openGauss=# explain analyze select * from t1 where a + 1 = 100;
```

```
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on t1  (cost=0.00..1943.00 rows=500 width=8) (actual time=0.044..22.473 rows=1 loops=1)
   Filter: ((a + 1) = 100)
   Rows Removed by Filter: 99999
 Total runtime: 22.534 ms
(4 rows)
```

According to the preceding execution plan, the SQL statement does not use the index for query, indicating that the index is invalid. This is because when expressions or functions are used in condition columns, indexes become invalid, and each row of data is calculated, leading to unnecessary performance overhead.

Step 5 After SQL 1 is modified, the index takes effect.

```
openGauss=# explain analyze select * from t1 where a = 100 - 1;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 [Bypass]
 Index Scan using idx_t1_a on t1  (cost=0.00..8.27 rows=1 width=8) (actual time=0.021..0.022 rows=1
loops=1)
   Index Cond: (a = 99)
 Total runtime: 0.070 ms
(4 rows)

openGauss=# explain analyze select * from t1 where a = 99;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 [Bypass]
 Index Scan using idx_t1_a on t1  (cost=0.00..8.27 rows=1 width=8) (actual time=0.020..0.021 rows=1
loops=1)
   Index Cond: (a = 99)
 Total runtime: 0.061 ms
(4 rows)
```

The result shows that the SQL statement uses the index for query and the index takes effect.

Step 6 Run SQL statement 2 to check whether an index is invalid in the query.

```
openGauss=# explain analyze select * from t1 where a = 99.0;
                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on t1  (cost=0.00..1943.00 rows=500 width=8) (actual time=0.065..34.651 rows=1 loops=1)
   Filter: ((a)::numeric = 99.0)
   Rows Removed by Filter: 99999
 Total runtime: 34.716 ms
(4 rows)
```

According to the preceding execution plan, the SQL statement does not use the index for query, indicating that the index is invalid. The reason is that the numeric type does not match the column type in the WHERE condition.

Step 7 After SQL 2 is modified, the index takes effect.

```
openGauss=# explain analyze select * from t1 where a = 99.0::integer;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------
 [Bypass]
 Index Scan using idx_t1_a on t1  (cost=0.00..8.27 rows=1 width=8) (actual time=0.019..0.020 rows=1
loops=1)
   Index Cond: (a = 99)
 Total runtime: 0.062 ms
(4 rows)
```

According to the preceding execution plan, after the data type of the WHERE condition is changed, the SQL statement uses the index for query again and the index takes effect.

Step 8 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
```

5.7 Optimizing Table Structures

5.7.1 Lab Description

Table structure optimization is a crucial SQL process that involves the following:

1. Selecting data types: Select the most suitable data type for storage. For example, if a column stores only values ranging from 0 to 255, using tinyint saves more storage space than using int.
2. Using suitable indexes: Indexes can significantly speed up queries. However, too many indexes slow down insertion, update, and deletion operations because indexes need to be updated each time data changes. Select the appropriate indexes based on your service requirements.
3. Normalization and denormalization: Normalization eliminates data redundancy but may increase query complexity. Denormalization can reduce query complexity but may introduce data redundancy. Decide the one that works best for your service requirements.
4. Table partitioning and splitting: When a table contains a lot of data, you can use table partitioning or splitting to speed up queries. Partitioning is to physically divide a table into multiple parts, each of which can be independently indexed and queried. Splitting is to divide a large table into multiple small tables. Each small table has its own index.
5. Use primary keys and foreign keys properly: Primary keys and foreign keys can ensure data consistency and integrity. However, too many foreign key constraints

may affect performance. Determine whether to use foreign keys based on the actual situation.

5.7.2 Selecting Proper Storage Types

Step 1 Enable timing.

```
openGauss=# \timing on
Timing is on.
```

Step 2 Create tables.

```
openGauss=# create table row_t(a int, b int, c int, d int, e int, f int, g int, h int, i int, j int, k int, l int, m int, n int, o int, p int, q int);
CREATE TABLE
Time: 2.905 ms
openGauss=# create table col_t(a int, b int, c int, d int, e int, f int, g int, h int, i int, j int, k int, l int, m int, n int, o int, p int, q int) with(orientation=column);
CREATE TABLE
Time: 8.134 ms
```

As shown in the preceding information, it takes a longer time to create a column-store table.

Step 3 Write data for comparison 1.

```
openGauss=# insert into row_t values (generate_series(1, 10000), generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000));
INSERT 0 10000
Time: 28.571 ms
openGauss=# insert into col_t values (generate_series(1, 10000), generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000) , generate_series(1, 10000));
INSERT 0 10000
Time: 1785.605 ms
```

Step 4 Write data for comparison 2.

```
openGauss=# insert into row_t values (generate_series(1, 10000));
INSERT 0 10000
Time: 22.619 ms
openGauss=# insert into col_t values (generate_series(1, 10000));
INSERT 0 10000
Time: 1833.143 ms
```

As shown in the preceding information, inserting data into a column-store table takes a longer time than inserting data into a row-store table.

Step 5 Run a query SQL statement.

```
openGauss=# explain analyze select * from row_t where a > 100 and a < 1000;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on row_t  (cost=0.00..385.88 rows=1367 width=68) (actual time=0.064..7.252 rows=1798
loops=1)
   Filter: ((a > 100) AND (a < 1000))
   Rows Removed by Filter: 18202
 Total runtime: 7.466 ms
(4 rows)

Time: 8.554 ms
openGauss=# explain analyze select * from col_t where a > 100 and a < 1000;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------
 Row Adapter  (cost=15.00..15.00 rows=4 width=68) (actual time=11.425..313.109 rows=1798
loops=1)
   ->  CStore Scan on col_t  (cost=0.00..15.00 rows=4 width=68) (actual time=11.419..312.494
rows=1798 loops=1)
         Filter: ((a > 100) AND (a < 1000))
         Rows Removed by Filter: 18202
 Total runtime: 313.901 ms
(5 rows)

Time: 315.305 ms
openGauss=# explain analyze select a from row_t where a > 100 and a < 1000;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on row_t  (cost=0.00..385.88 rows=1367 width=4) (actual time=0.048..4.935 rows=1798
loops=1)
   Filter: ((a > 100) AND (a < 1000))
   Rows Removed by Filter: 18202
 Total runtime: 5.170 ms
(4 rows)

Time: 6.160 ms
openGauss=# explain analyze select a from col_t where a > 100 and a < 1000;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------
 Row Adapter  (cost=15.00..15.00 rows=4 width=4) (actual time=1.299..110.268 rows=1798 loops=1)
   ->  CStore Scan on col_t  (cost=0.00..15.00 rows=4 width=4) (actual time=1.297..109.917
rows=1798 loops=1)
         Filter: ((a > 100) AND (a < 1000))
         Rows Removed by Filter: 18202
 Total runtime: 110.706 ms
(5 rows)

Time: 111.776 ms
```

As shown in the preceding information, querying the column-store table takes a longer time.

Step 6 Drop data.

```
openGauss=# drop table row_t;
DROP TABLE
```

```
openGauss=# drop table col_t;
DROP TABLE
```

5.7.3 Selecting Proper Partition Types

Step 1 Create tables.

```
create table student_t1 (
  id int,
  name varchar(20),
  score int
) partition by range(id) (
  partition s1p1 values less than (100000),
  partition s1p2 values less than (200000),
  partition s1p3 values less than (300000),
  partition s1p4 values less than (400000),
  partition s1p5 values less than (500000)
);

create table student_t2 (
  id int,
  name varchar(20),
  score int
) partition by hash(id) (
  partition s1p1,
  partition s1p2,
  partition s1p3,
  partition s1p4,
  partition s1p5
);
```

Step 2 Prepare lab data.

```
openGauss=# insert into student_t1 values (generate_series(1,499999), 1, generate_series(50, 100));
INSERT 0 25499949
openGauss=# insert into student_t2 values (generate_series(1, 499999), 1, generate_series(50, 100));
INSERT 0 25499949
```

Note: The number is the least common multiple of the respective range of the two generate_series functions.

Step 3 Run SQL statements for equality query.

```
openGauss=# explain analyze select * from student_t1 where id = 9999;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
--------------
 Partition Iterator  (cost=0.00..174166.59 rows=58639 width=66) (actual time=3.435..1605.848
rows=51 loops=1)
   Iterations: 1
```

```
    ->  Partitioned Seq Scan on student_t1  (cost=0.00..174166.59 rows=58639 width=66) (actual
time=3.429..1605.764 rows=51 loops=1)
            Filter: (id = 9999)
            Rows Removed by Filter: 5099898
            Selected Partitions:  1
 Total runtime: 1606.021 ms
(7 rows)

openGauss=# explain analyze select * from student_t2 where id = 9999;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
--------------
 Partition Iterator  (cost=0.00..284438.59 rows=58639 width=66) (actual time=0.745..1618.139
rows=51 loops=1)
    Iterations: 1
    ->  Partitioned Seq Scan on student_t2  (cost=0.00..284438.59 rows=58639 width=66) (actual
time=0.740..1618.049 rows=51 loops=1)
            Filter: (id = 9999)
            Rows Removed by Filter: 5078427
            Selected Partitions:  1
 Total runtime: 1618.247 ms
(7 rows)
```

As shown in the preceding information, the equality query of the partition keys is performed in similar speed for the two types of partitions, and the partition pruning is successful.

Step 4 Run SQL statement for range query.

```
openGauss=# explain analyze select * from student_t1 where id > 100 and id < 1000;
                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
-----------------
 Partition Iterator  (cost=0.00..203486.30 rows=58639 width=66) (actual time=0.042..1168.077
rows=45849 loops=1)
    Iterations: 1
    ->  Partitioned Seq Scan on student_t1  (cost=0.00..203486.30 rows=58639 width=66) (actual
time=0.038..1162.872 rows=45849 loops=1)
            Filter: ((id > 100) AND (id < 1000))
            Rows Removed by Filter: 5054100
            Selected Partitions:  1
 Total runtime: 1172.223 ms
(7 rows)

openGauss=# explain analyze select * from student_t2 where id > 100 and id < 1000;
                                                                    QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
-----------------
 Partition Iterator  (cost=0.00..313758.30 rows=58639 width=66) (actual time=0.030..8941.876
rows=45849 loops=1)
    Iterations: 5
    ->  Partitioned Seq Scan on student_t2  (cost=0.00..313758.30 rows=58639 width=66) (actual
time=0.390..8935.616 rows=45849 loops=5)
            Filter: ((id > 100) AND (id < 1000))
            Rows Removed by Filter: 25454100
```

```
             Selected Partitions:  1..5
 Total runtime: 8946.228 ms
(7 rows)

openGauss=# explain analyze select * from student_t1 where id > 100 and id < 1000;
                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
----------------
 Partition Iterator  (cost=0.00..410057.44 rows=46738 width=10) (actual time=0.076..1346.945
rows=45849 loops=1)
   Iterations: 1
   ->  Partitioned Seq Scan on student_t1  (cost=0.00..410057.44 rows=46738 width=10) (actual
time=0.069..1341.058 rows=45849 loops=1)
             Filter: ((id > 100) AND (id < 1000))
             Rows Removed by Filter: 5054100
             Selected Partitions:  1
 Total runtime: 1351.198 ms
(7 rows)

openGauss=# explain analyze select * from student_t2 where id > 100 and id < 1000;
                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------
----------------
 Partition Iterator  (cost=0.00..520338.48 rows=54843 width=10) (actual time=0.030..6359.986
rows=45849 loops=1)
   Iterations: 5
   ->  Partitioned Seq Scan on student_t2  (cost=0.00..520338.48 rows=54843 width=10) (actual
time=0.195..6353.904 rows=45849 loops=5)
             Filter: ((id > 100) AND (id < 1000))
             Rows Removed by Filter: 25454100
             Selected Partitions:  1..5
 Total runtime: 6364.346 ms
(7 rows)
```

As shown in the preceding information, for a range query, the range partition can be pruned, so that the result can be returned after only one partition is scanned.

However, hash partitions cannot be pruned. As such, all partitions are scanned, and the time required is more than five to eight times longer compared with scanning range partitions.

Step 5 Drop data.

```
openGauss=# drop table student_t1;
DROP TABLE
openGauss=# drop table student_t2;
DROP TABLE
```

5.7.4 Selecting Proper Partitioned Table Indexes

Step 1 Create tables.

```
create table student_t1 (
  id int,
  name varchar(50),
  score int
) partition by range(id) (
  partition s1p1 values less than (100000),
  partition s1p2 values less than (200000),
  partition s1p3 values less than (300000),
  partition s1p4 values less than (400000),
  partition s1p5 values less than (500000)
);

create table student_t2 (
  id int,
  name varchar(50),
  score int
) partition by range(id) (
  partition s2p1 values less than (100000),
  partition s2p2 values less than (200000),
  partition s2p3 values less than (300000),
  partition s2p4 values less than (400000),
  partition s2p5 values less than (500000)
);
```

Step 2 Prepare lab data.

```
openGauss=# insert into student_t1 values (generate_series(1,499999), 'zhangsan' || random(),
generate_series(50, 100));
INSERT 0 25499949
openGauss=# insert into student_t2 values (generate_series(1, 499999), 'zhangsan' || random(),
generate_series(50, 100));
INSERT 0 25499949
```

Step 3 Create indexes.

```
openGauss=# create index idx_student_t1_id on student_t1(id) global;
CREATE INDEX
openGauss=# create index idx_student_t2_id on student_t2(id) local;
CREATE INDEX
```

Step 4 Run SQL statements for equality query.

```
openGauss=# explain analyze select * from student_t1 where id = 9999;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
------------
 Index Scan using idx_student_t1_id on student_t1    (cost=0.00..23.79 rows=49 width=33) (actual
time=0.134..0.576 rows=51 loops=1)
    Index Cond: (id = 9999)
 Total runtime: 0.645 ms
(3 rows)

openGauss=# explain analyze select * from student_t2 where id = 9999;
                                                                QUERY PLAN
```

```
------------------------------------------------------------------------------------------------------------------------
-----------------------------
 Partition Iterator  (cost=0.00..23.25 rows=47 width=33) (actual time=0.095..0.523 rows=51 loops=1)
   Iterations: 1
   ->  Partitioned Index Scan using idx_student_t2_id on student_t2  (cost=0.00..23.25 rows=47
width=33) (actual time=0.089..0.505 rows=51 loops=1)
         Index Cond: (id = 9999)
         Selected Partitions:  1
 Total runtime: 0.661 ms
(6 rows)

openGauss=# explain analyze select * from student_t1 where id = 300000;
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
------------
 Index Scan using idx_student_t1_id on student_t1  (cost=0.00..23.79 rows=49 width=33) (actual
time=0.133..0.605 rows=51 loops=1)
   Index Cond: (id = 300000)
 Total runtime: 0.670 ms
(3 rows)

openGauss=# explain analyze select * from student_t2 where id = 300000;
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
-----------------------------
 Partition Iterator  (cost=0.00..23.25 rows=47 width=33) (actual time=0.140..0.449 rows=51 loops=1)
   Iterations: 1
   ->  Partitioned Index Scan using idx_student_t2_id on student_t2  (cost=0.00..23.25 rows=47
width=33) (actual time=0.133..0.435 rows=51 loops=1)
         Index Cond: (id = 300000)
         Selected Partitions:  4
 Total runtime: 0.560 ms
(6 rows)
```

In this test case, the equality queries take similar time.

Step 5 Perform range queries.

```
openGauss=# explain analyze select id, sum(score)  from student_t1 where id < 9999 group by id;
                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
-------------------------
 HashAggregate  (cost=55252.40..55348.02 rows=9562 width=16) (actual time=313.945..315.946
rows=9998 loops=1)
   Group By Key: id
   ->  Bitmap Heap Scan on student_t1  (cost=4713.04..52885.44 rows=473392 width=8) (actual
time=40.333..161.385 rows=509898 loops=1)
         Recheck Cond: (id < 9999)
         Heap Blocks: exact=4276
         ->  Bitmap Index Scan on idx_student_t1_id  (cost=0.00..4594.69 rows=473392 width=0)
(actual time=39.431..39.431 rows=509898 loops=1)
               Index Cond: (id < 9999)
 Total runtime: 317.096 ms
(8 rows)
```

```
openGauss=# explain analyze select id, sum(score)  from student_t2 where id < 9999 group by id;
                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=56172.12..56278.61 rows=10649 width=16) (actual time=349.460..351.601 rows=9998 loops=1)
   Group By Key: id
   ->  Partition Iterator  (cost=5047.69..53638.28 rows=506767 width=8) (actual time=37.361..203.077 rows=509898 loops=1)
         Iterations: 1
         ->  Partitioned Bitmap Heap Scan on student_t2  (cost=5047.69..53638.28 rows=506767 width=8) (actual time=37.350..147.598 rows=509898 loops=1)
               Recheck Cond: (id < 9999)
               Heap Blocks: exact=4274
               Selected Partitions:  1
               ->  Partitioned Bitmap Index Scan on idx_student_t2_id  (cost=0.00..4921.00 rows=506767 width=0) (actual time=36.569..36.569 rows=509898 loops=1)
                     Index Cond: (id < 9999)
                     Selected Partitions:  1
 Total runtime: 352.865 ms
(12 rows)

openGauss=# explain analyze select id, sum(score)  from student_t1 group by id;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=593783.37..598934.12 rows=515075 width=16) (actual time=18271.163..18447.428 rows=499999 loops=1)
   Group By Key: id
   ->  Partition Iterator  (cost=0.00..466280.91 rows=25500491 width=8) (actual time=0.032..8433.038 rows=25499949 loops=1)
         Iterations: 5
         ->  Partitioned Seq Scan on student_t1  (cost=0.00..466280.91 rows=25500491 width=8) (actual time=0.263..5637.812 rows=25499949 loops=5)
               Selected Partitions:  1..5
 Total runtime: 18498.084 ms
(7 rows)

openGauss=# explain analyze select id, sum(score)  from student_t2 group by id;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=593768.84..599126.92 rows=535808 width=16) (actual time=18130.639..18307.829 rows=499999 loops=1)
   Group By Key: id
   ->  Partition Iterator  (cost=0.00..466271.89 rows=25499389 width=8) (actual time=0.044..8302.528 rows=25499949 loops=1)
         Iterations: 5
         ->  Partitioned Seq Scan on student_t2  (cost=0.00..466271.89 rows=25499389 width=8) (actual time=0.286..5533.438 rows=25499949 loops=5)
               Selected Partitions:  1..5
 Total runtime: 18358.708 ms
(7 rows)
```

In the first group of SQL statements, the tables with global indexes do not use partition iteration, which consumes little time. The second group of SQL statements uses partition iteration, which takes similar amount of time.

Step 6 Drop data.

```
openGauss=# drop table student_t1;
DROP TABLE
openGauss=# drop table student_t2;
DROP TABLE
```

5.7.5 Properly Planning Partitions

Step 1 Create tables.

```
create table t1(
id int,
tim date
) partition by range (tim) interval ('1 month ') (
  Partition t1p0001 values less than ('0001-01-01'),
  Partition t1p0002 values less than ('0001-02-01')
);

create table t2(
id int,
tim date
) partition by range (tim) interval ('1 year ') (
  Partition t2p0001 values less than ('0001-01-01'),
  Partition t2p0002 values less than ('0002-01-02')
);
```

Step 2 Create a function for inserting data.

```
create or replace function insertvalues_t1() returns int
as $$
declare
  years int := 2000;
  months int := 1;
  cnt int := 0;
  sql text;
begin
  while years <= 2010 loop
    months := 1;
    while months <= 9 loop
insert into t1 values (generate_series(1, 100), concat(years, '-0',months ,'-01'));
      months := months + 1;
      cnt := cnt + 100;
    end loop;
    years := years + 1;
  end loop;
  return cnt;
```

```
end;
$$ language plpgsql;

create or replace function insertvalues_t2() returns int
as $$
declare
 years int := 2000;
 months int := 1;
 cnt int := 0;
 sql text;
begin
 while years <= 2010 loop
  months := 1;
  while months <= 9 loop
insert into t2 values (generate_series(1, 100), concat(years, '-0',months ,'-01'));
    months := months + 1;
    cnt := cnt + 100;
  end loop;
  years := years + 1;
 end loop;
 return cnt;
end;
$$ language plpgsql;
```

Step 3 Test the time required for generating data.

```
openGauss=# \timing on
Timing is on.
openGauss=# select insertvalues_t1();
 insertvalues_t1
-----------------
                9900
(1 row)

Time: 166.181 ms
openGauss=# select insertvalues_t2();
 insertvalues_t2
-----------------
                9900
(1 row)

Time: 72.897 ms
openGauss=# \timing off
Timing is off.
```

The result shows that the inserting data into the **t2** table is faster. This is because the partition interval of the **t2** table is one year, and that of the **t1** table is one month. With the same amount of data to be inserted, the **t1** table needs to be split to more partitions, which takes longer time. Therefore, frequent partitioning increases the time consumption.

Step 4 Drop data.

```
openGauss=# drop table t1;
```

```
DROP TABLE
openGauss=# drop table t2;
DROP TABLE
openGauss=# drop function insertvalues_t1();
DROP FUNCTION
openGauss=# drop function insertvalues_t2();
DROP FUNCTION
```

5.8 CBO

5.8.1 Lab Description

openGauss generates execution plans based on cost estimation. The CBO estimates the execution cost based on data statistics, cardinality estimation, and operator cost models, and selects the plan with the lowest cost for SQL execution.

Data statistics include the number of rows in a table, column width, NULL record ratio, distinct values, and MCVs. Cardinality estimation estimates the number of rows in the intermediate result of each operator, including the number of rows with filter criteria and in the join result. The two parts are obtained based on correct selectivity.

5.8.2 Identifying and Updating Statistics

Step 1 Create a table.

```
create table test1 (
a int,
b double precision,
c varchar(100)
);
```

Step 2 Generate initial data.

```
openGauss=# insert into test1 values (generate_series(1, 5000000), generate_series(1, 10000), 'hello'
|| random());
INSERT 0 5000000
```

Step 3 Run a query SQL statement.

```
openGauss=# explain analyze select a, b, c from test1 where a > 20000 and a < 200000;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..61816.70 rows=6600 width=230) (actual time=6.967..1794.469
rows=179999 loops=1)
   Filter: ((a > 20000) AND (a < 200000))
   Rows Removed by Filter: 4820001
 Total runtime: 1809.735 ms
(4 rows)
```

As shown in the preceding information, the estimated number of rows differs greatly from the actual one.

Note: openGauss supports AUTO VACUUM, which automatically and periodically updates statistics. As such, the estimated number of rows may not be greatly different from the actual one, which also applies to the result of the following query.

Step 4 Perform the VACUUM operation to update statistics.

```
openGauss=# vacuum analyze test1;
VACUUM
```

Step 5 Run the query SQL statement again.

```
openGauss=# explain analyze select a, b, c from test1 where a > 20000 and a < 200000;
                                                                  QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..117016.82 rows=184046 width=33) (actual time=4.556..1188.771
rows=179999 loops=1)
   Filter: ((a > 20000) AND (a < 200000))
   Rows Removed by Filter: 4820001
 Total runtime: 1204.338 ms
(4 rows)
```

The result shows that the estimated number of rows is similar to the actual one.

Step 6 Repeat the addition and deletion operations.

```
openGauss=# Delete from test1;
DELETE 5000000
openGauss=# insert into test1 values (generate_series(1, 5000), generate_series(1, 1000), 'hello' ||
random());
INSERT 0 5000
openGauss=# Delete from test1;
DELETE 5000
openGauss=# insert into test1 values (generate_series(1, 5000), generate_series(1, 1000), 'hello' ||
random());
INSERT 0 5000
openGauss=# Delete from test1;
DELETE 5000
openGauss=# insert into test1 values (generate_series(1, 5000), generate_series(1, 1000), 'hello' ||
random());
INSERT 0 5000
openGauss=# Delete from test1;
DELETE 5000
openGauss=# insert into test1 values (generate_series(1, 5000), generate_series(1, 1000), 'hello' ||
random());
INSERT 0 5000
openGauss=# Delete from test1;
DELETE 5000
openGauss=# insert into test1 values (generate_series(1, 5000), generate_series(1, 1000), 'hello' ||
random());
INSERT 0 5000
```

Step 7 Run the query SQL statement again.

```
openGauss=# explain analyze select a, b, c from test1 where a > 20000 and a < 200000;
                                                          QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..117601.67 rows=184966 width=33) (actual time=44.390..44.390
rows=0 loops=1)
   Filter: ((a > 20000) AND (a < 200000))
   Rows Removed by Filter: 5000
 Total runtime: 44.498 ms
(4 rows)
```

The estimated number of rows differs greatly from the actual one.

Step 8 Perform the VACUUM operation to update statistics.

```
openGauss=# vacuum analyze test1;
VACUUM
```

Step 9 Run the query SQL statement again.

```
openGauss=# explain analyze select a, b, c from test1 where a > 20000 and a < 200000;
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..42304.93 rows=1 width=33) (actual time=39.688..39.688 rows=0
loops=1)
   Filter: ((a > 20000) AND (a < 200000))
   Rows Removed by Filter: 5000
 Total runtime: 39.754 ms
(4 rows)
```

After the VACUUM ANALYZE operation, the estimated number of rows becomes normal.

Step 10 Drop data.

```
openGauss=# drop table test1;
DROP TABLE
```

5.8.3 Modifying Cost Parameters

Step 1 Create a table.

```
create table test1 (
a int,
b double precision,
c varchar(100)
);
```

Step 2 Generate initial data.

```
openGauss=# insert into test1 values (generate_series(1, 5000000), generate_series(1, 10000), 'hello'
|| random());
INSERT 0 5000000
```

Step 3 Run an SQL statement to query the value of **seq_page_cost**.

```
openGauss=# reset seq_page_cost;
RESET
openGauss=# show seq_page_cost;
 seq_page_cost
---------------
 1
(1 row)

openGauss=# explain select a, b, c from test1 where a > 20000 and a < 200000;
                                QUERY PLAN
-----------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..61816.70 rows=6600 width=230)
   Filter: ((a > 20000) AND (a < 200000))
(2 rows)
```

Step 4 Modify the **seq_page_cost** parameter.

```
openGauss=# set seq_page_cost to 10;
SET
```

Step 5 Run the query SQL statement again.

```
openGauss=# explain select a, b, c from test1 where a > 20000 and a < 200000;
                                QUERY PLAN
-----------------------------------------------------------------
 Seq Scan on test1  (cost=0.00..439969.70 rows=6600 width=230)
   Filter: ((a > 20000) AND (a < 200000))
(2 rows)
```

Note that the cost is different from that in step 4.

In actual service scenarios, based on different resource environments such as disks and CPUs, you can adjust the values of related GUC parameters to affect the query estimation of the optimizer and the selection of the execution plan.

Step 6 Drop data.

```
openGauss=# drop table test1;
DROP TABLE
openGauss=# reset seq_page_cost;
SET
```

5.9 Diagnosing SQL Performance

5.9.1 Lab Description

openGauss supports the SQL performance diagnosis, which shows you the running status of the database system.

This lab describes how to enable, generate, and query WDRs.

5.9.2 Using WDRs

Step 1 Enable the WDR snapshot function.

```
openGauss=# alter system set enable_wdr_snapshot to on;
ALTER SYSTEM SET
```

Step 2 Query the WDR list.

```
openGauss=#   select * from snapshot.snapshot;
 snapshot_id |              start_ts                |                end_ts
-------------+---------------------------------+-------------------------------
           3 | 20XX-XX-14 21:22:10.990639+08 | 20XX-XX-14 21:22:11.395643+08
(1 row)
```

As shown in the preceding information, after the WDR function is enabled, the system automatically generates a snapshot.

Step 3 Create a table.

```
create table test1 (
a int,
   b double precision,
c varchar(100)
);
```

Step 4 Generate lab data.

```
openGauss=# insert into test1 values (generate_series(1, 5000000), generate_series(1, 10000), 'hello'
|| random());
INSERT 0 5000000
```

Step 5 Run a query SQL statement.

```
openGauss=# select count(1) from test1;
   count
---------
 5000000
(1 row)
openGauss=# select count(1) , b, max(a) from test1 group by b order by b limit 5;
 count | b |     max
```

```
-------+---+---------
    500 | 1 | 4990001
    500 | 2 | 4990002
    500 | 3 | 4990003
    500 | 4 | 4990004
    500 | 5 | 4990005
(5 rows)
```

Step 6 Generate a snapshot.

```
openGauss=# select create_wdr_snapshot();
                create_wdr_snapshot
-----------------------------------------
 WDR snapshot request has been submitted
(1 row)
```

Step 7 View the snapshot.

```
openGauss=# select * from snapshot.snapshot;
 snapshot_id |              start_ts                |                end_ts
-------------+--------------------------------------+--------------------------------------
           3 | 20XX-05-14 21:22:10.990639+08 | 20XX-05-14 21:22:11.395643+08
           4 | 20XX-05-14 21:28:21.753525+08 | 20XX-05-14 21:28:22.171824+08
(2 rows)
```

Note that this output will be used in the next step.

Step 8 Adjust the output format and specify the WDR output file.

Run the following command:

```
\a \t \o ~/my_wdr_report.html
```

~/my_wdr_report.html is the output file path. In Linux OS, **~** indicates the **home** directory of the user.

The output is as follows:

```
openGauss=# \a \t \o ~/my_wdr_report.html
Output format is unaligned.
Showing only tuples.
```

Step 9 Generate a WDR.

Run the following command:

Note: The first and second parameters are the IDs of the start and end snapshots.

That is, the values in the **snapshot_id** column in step 7. Change the values based on the result of the previous step.

```
select generate_wdr_report(3, 4, 'all', 'cluster', null);
```

Note: No information is displayed in the CLI because the information has been exported to a file.

Step 10 Restore to the initial output format.

```
openGauss=# \o \a \t
Output format is aligned.
Tuples only is off.
```

Step 11 View the WDR.

View the WDR file in the path specified in step 7. Use a browser to open the file.

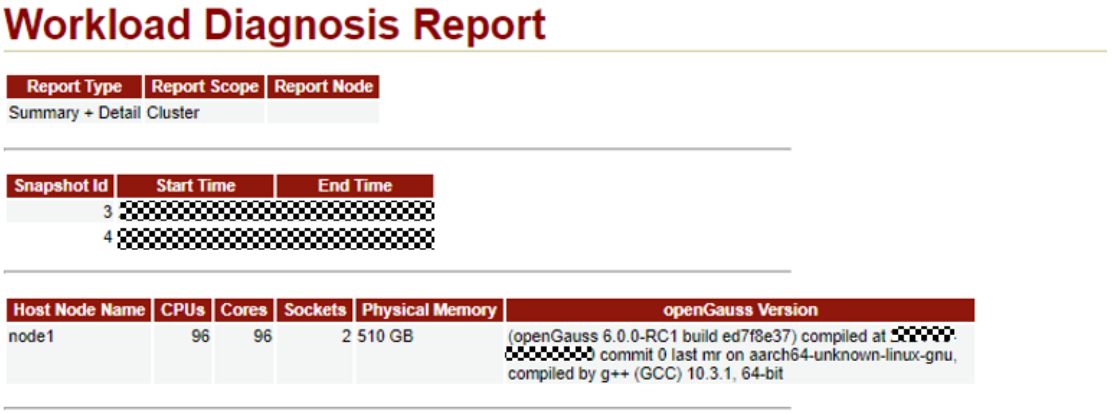
The following figure shows only part of the WDR.

The summary section includes:

1. Most time-consuming wait events and event types.
2. CPU, I/O, and memory statistics.

The detail section includes:

1. Time model, SQL statistics, SQL details, and wait events.
2. Statistics on cache, I/Os, applications, and objects (tables, indexes, and bad blocks).

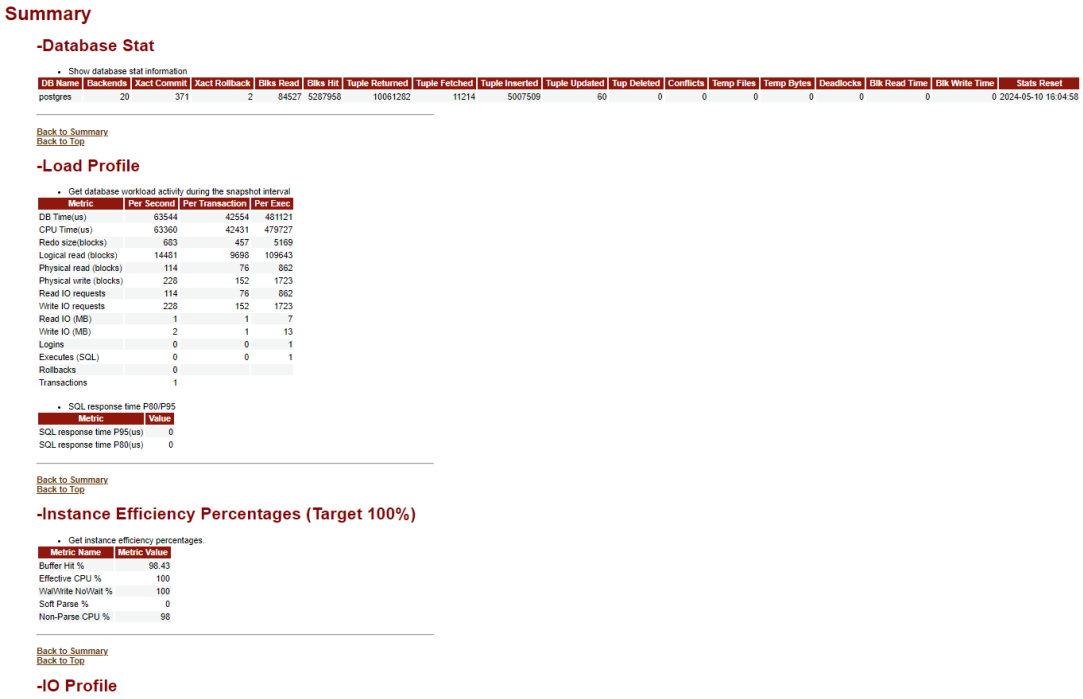


Workload Diagnosis Report

Report Type	Report Scope	Report Node
Summary + Detail	Cluster	

Snapshot Id	Start Time	End Time
3		
4		

Host Node Name	CPUs	Cores	Sockets	Physical Memory	openGauss Version
node1	96	96	2	510 GB	(openGauss 6.0.0-RC1 build ed7f8e37) compiled at commit 0 last mr on aarch64-unknown-linux-gnu, compiled by g++ (GCC) 10.3.1, 64-bit



Summary

-Database Stat

- Show database stat information

DB Name	Backends	Xact Commit	Xact Rollback	Blks Read	Blks Hit	Tuple Returned	Tuple Fetched	Tuple Inserted	Tuple Updated	Tup Deleted	Conflicts	Temp Files	Temp Bytes	Deadlocks	Blk Read Time	Blk Write Time	Stats Reset
postgres	20	371	2	84527	5287958	10061282	11214	5007509	60	0	0	0	0	0	0	0	2024-05-10 16:04:58

Back to Summary
Back to Top

-Load Profile

- Get database workload activity during the snapshot interval.

Metric	Per Second	Per Transaction	Per Exec
DB Time(us)	63544	42554	481121
CPU Time(us)	63360	42431	479727
Redo size(blocks)	683	457	5169
Logical read (blocks)	14481	9698	109643
Physical read (blocks)	114	76	862
Physical write (blocks)	228	152	1723
Read IO requests	114	76	862
Write IO requests	228	152	1723
Read IO (MB)	1	1	7
Write IO (MB)	2	1	13
Logins	0	0	1
Executes (SQL)	0	0	1
Rollbacks	0		
Transactions	1		

- SQL response time P80/P95

Metric	Value
SQL response time P95(us)	0
SQL response time P80(us)	0

Back to Summary
Back to Top

-Instance Efficiency Percentages (Target 100%)

- Get instance efficiency percentages.

Metric Name	Metric Value
Buffer Hit %	98.43
Effective CPU %	100
WalWrite NoWait %	100
Soft Parse %	0
Non-Parse CPU %	98

Back to Summary
Back to Top

-IO Profile

Step 12 Disable WDR snapshot.

```
openGauss=# alter system set enable_wdr_snapshot to off;
ALTER SYSTEM SET
```

Step 13 Drop data.

```
openGauss=# drop table test1;
DROP TABLE
```

5.10 Optimizing Subqueries

5.10.1 Lab Description

When an application runs an SQL statement to operate the database, many subqueries are used because they are clearer than table join. Especially in complicated query statements, subqueries have more complete and independent semantics, which makes SQL statements clearer and easier to understand. Therefore, subqueries are very popular.

In openGauss, subqueries can also be called sublinks based on where they are in SQL statements.

1. Subquery:

Corresponds to a range table (RangeTblEntry) in the query parse tree. That is, a subquery is a SELECT statement following immediately after the FROM keyword.

2. Sublink:

Corresponds to an expression in the query parsing tree. That is, a sublink is a statement in the WHERE or ON clause or in the target list.

5.10.2 Correlated Subqueries and Non-correlated Subqueries

Step 1 Create tables.

```
openGauss=# drop table if exists t1,t2;
DROP TABLE
openGauss=# create table t1 (a int, b int);
CREATE TABLE
openGauss=# create table t2 (a int, b int);
CREATE TABLE
```

Step 2 Generate lab data.

```
openGauss=# insert into t1 values(generate_series(1, 100000), generate_series(1, 1000));
INSERT 0 100000
openGauss=# insert into t2 values(generate_series(1, 100000), generate_series(1, 1000));
INSERT 0 100000
```

Step 3 Enable the time consumption calculation.

```
openGauss=# \timing on
Timing is on.
```

Step 4 Perform a non-correlated subquery.

```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.b in (1, 2, 3, 4, 5));
 a | b
---+---
 1 | 1
 2 | 2
 3 | 3
 4 | 4
 5 | 5
(5 rows)

Time: 20.324 ms
openGauss=# explain select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.b in (1, 2, 3, 4,
5));
                                  QUERY PLAN
------------------------------------------------------------------------
 Nested Loop Semi Join  (cost=0.00..4139.32 rows=5 width=8)
   Join Filter: (t1.a = t2.b)
   ->  Seq Scan on t1  (cost=0.00..2068.00 rows=5 width=8)
         Filter: (a = ANY ('{1,2,3,4,5}'::integer[]))
   ->  Materialize  (cost=0.00..2070.61 rows=521 width=4)
         ->  Seq Scan on t2  (cost=0.00..2068.00 rows=521 width=4)
               Filter: (b = ANY ('{1,2,3,4,5}'::integer[]))
(7 rows)

Time: 1.335 ms
```

Step 5 Perform a correlated subquery.

```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.a = t1.a and t2.b in (1,
2, 3, 4, 5));
 a | b
---+---
 1 | 1
 2 | 2
 3 | 3
 4 | 4
 5 | 5
(5 rows)

Time: 1758393.406 ms
openGauss=# explain select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.a = t1.a and
t2.b in (1, 2, 3, 4, 5));
                                        QUERY PLAN
---------------------------------------------------------------------------
 Seq Scan on t1  (cost=0.00..115901818.00 rows=50000 width=8)
   Filter: (SubPlan 1)
   SubPlan 1
     ->  Seq Scan on t2  (cost=0.00..2318.00 rows=1 width=4)
           Filter: ((a = t1.a) AND (b = ANY ('{1,2,3,4,5}'::integer[])))
(5 rows)

Time: 1.854 ms
```

Compared with step 4, although the results of the two statements are the same, the correlated subquery cannot be pulled up or optimized and takes a long time.

The non-correlated subquery takes only 20 ms.

The correlated subquery takes up to 29 minutes, which is nearly 90,000 times longer than that of non-correlated subqueries.

Therefore, avoid correlated subqueries and transform them into non-correlated subqueries if possible.

Step 6 Disables time consumption calculation.

```
openGauss=# \timing off
Timing is off.
```

Step 7 Drop data.

```
openGauss=# drop table t1;
DROP TABLE
openGauss=# drop table t2;
DROP TABLE
```

5.10.3 Using WITH to Rewrite and Optimize Subqueries

Step 1 Create tables.

```
create table t1 (a int, b int);
create table t2 (a int, b int);
create table t3 (a int, b int);
```

Step 2 Generate lab data.

```
openGauss=# insert into t1 values(generate_series(1, 1000000), generate_series(1, 100000));
INSERT 0 1000000
openGauss=# insert into t2 values(generate_series(1, 1000000), generate_series(1, 100000));
INSERT 0 1000000
openGauss=# insert into t2 values(generate_series(1, 100000), generate_series(1, 100000));
INSERT 0 100000
```

Step 3 Run a query SQL statement.

```
openGauss=# Explain analyze select distinct t1.a, t2.a from t1
openGauss-# left join t2 on t1.a=t2.a
openGauss-# and not exists (select a,b from t3 where t3.a=t1.a and t3.b=t2.a);
                                                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
---------------
 HashAggregate  (cost=210238431828.38..210238432228.38 rows=40000 width=8) (actual
time=2201.060..2542.118 rows=1000000 loops=1)
   Group By Key: t1.a, t2.a
 Temp File Num: 48
   ->  Hash Right Join  (cost=25825.69..210225991319.04 rows=2488101869 width=8) (actual
time=550.357..1538.001 rows=1100000 loops=1)
         Hash Cond: (t2.a = t1.a)
         Join Filter: (NOT (alternatives: SubPlan 1 or hashed SubPlan 2))
         ->  Seq Scan on t2  (cost=0.00..15331.64 rows=1046364 width=4) (actual
time=0.037..256.348 rows=1100000 loops=1)
         ->  Hash  (cost=13936.42..13936.42 rows=951142 width=4) (actual time=549.632..549.632
rows=1000000 loops=1)
               Buckets: 1048576  Batches: 1  Memory Usage: 43349kB
               ->  Seq Scan on t1  (cost=0.00..13936.42 rows=951142 width=4) (actual
time=0.045..279.979 rows=1000000 loops=1)
         SubPlan 1
           ->  Seq Scan on t3  (cost=0.00..42.23 rows=1 width=0) (Actual time: never executed)
                 Filter: ((a = t1.a) AND (b = t2.a))
         SubPlan 2
           ->  Seq Scan on t3  (cost=0.00..31.49 rows=2149 width=8) (actual time=0.001..0.001
rows=0 loops=1)
 Total runtime: 2642.345 ms
(16 rows)
```

Step 4 Optimize the subquery.

```
Explain analyze with temp as
```

```
(
        select * from (select t1.a as a, t2.a as b from t1 left join t2 on t1.a=t2.a)
)
select distinct a,b from temp
where not exists (select a,b from t3 where temp.a=t3.a and temp.b=t3.b);
```

The output is as follows:

```
openGauss=# Explain analyze with temp as
openGauss-# (
openGauss(#             select * from (select t1.a as a, t2.a as b from t1 left join t2 on t1.a=t2.a)
openGauss(# )
openGauss-# select distinct a,b from temp
openGauss-# where not exists (select a,b from t3 where temp.a=t3.a and temp.b=t3.b);
                                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------
-----------------
 HashAggregate  (cost=140932984.62..140933384.62 rows=40000 width=8) (actual
time=2268.189..2615.097 rows=1000000 loops=1)
   Group By Key: t1.a, t2.a
 Temp File Num: 48
   ->  Hash Anti Join  (cost=25889.42..122272220.60 rows=3732152804 width=8) (actual
time=422.738..1601.983 rows=1100000 loops=1)
         Hash Cond: ((t1.a = t3.a) AND (t2.a = t3.b))
         ->  Hash Right Join  (cost=25825.69..56026444.61 rows=4976203738 width=8) (actual
time=422.635..1276.488 rows=1100000 loops=1)
               Hash Cond: (t2.a = t1.a)
               ->  Seq Scan on t2  (cost=0.00..15331.64 rows=1046364 width=4) (actual
time=0.010..136.095 rows=1100000 loops=1)
               ->  Hash  (cost=13936.42..13936.42 rows=951142 width=4) (actual
time=421.958..421.958 rows=1000000 loops=1)
                     Buckets: 1048576  Batches: 1  Memory Usage: 43349kB
                     ->  Seq Scan on t1  (cost=0.00..13936.42 rows=951142 width=4) (actual
time=0.023..159.601 rows=1000000 loops=1)
         ->  Hash  (cost=31.49..31.49 rows=2149 width=8) (actual time=0.003..0.003 rows=0
loops=1)
               Buckets: 32768  Batches: 1  Memory Usage: 256kB
               ->  Seq Scan on t3  (cost=0.00..31.49 rows=2149 width=8) (actual time=0.001..0.001
rows=0 loops=1)
 Total runtime: 2714.719 ms
(15 rows)
```

Compared with step 3, although the subquery is eliminated after rewriting, the time consumption is longer.

Step 5 Set the **query_dop** parameter to execute an SQL parallel query.

```
openGauss=# set query_dop = 4;
SET
openGauss=# Explain analyze select distinct t1.a, t2.a from t1
openGauss-# left join t2 on t1.a=t2.a
openGauss-# and not exists (select a,b from t3 where t3.a=t1.a and t3.b=t2.a);
                                                                                QUERY
PLAN
```

```
------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------
 Streaming(type: LOCAL GATHER)  (cost=52558483841.80..52558484005.28 rows=40000 width=8)
(actual time=[8041.112,8857.076]..[8041.112,8857.076], rows=1000000)
   ->  HashAggregate  (cost=52558483841.80..52558483941.80 rows=40000 width=8) (actual
time=[2063.149,2430.947]..[2063.149,2430.947], rows=1000000)
         Group By Key: t1.a, t2.a
         ->  Hash Right Join  (cost=26872.17..52555373714.47 rows=2488101869 width=8) (actual
time=[460.814,1426.356]..[460.814,1426.356], rows=1100000)
               Hash Cond: (t2.a = t1.a)
               Join Filter: (NOT (alternatives: SubPlan 1 or hashed SubPlan 2))
               ->  Streaming(type: LOCAL REDISTRIBUTE dop: 1/4)  (cost=0.00..25691.86
rows=1046364 width=4) (actual time=[0.039,236.582]..[0.039,236.582], rows=1100000)
                     ->  Seq Scan on t2  (cost=0.00..6832.91 rows=1046364 width=4) (actual
time=[0.014,41.080]..[0.019,43.304], rows=1100000)
               ->  Hash  (cost=23899.85..23899.85 rows=951142 width=4) (actual
time=[460.004,460.004]..[460.004,460.004], rows=1000000)
                     Max Buckets: 1048576  Max Batches: 1  Max Memory Usage: 43349kB
                     Min Buckets: 1048576  Min Batches: 1  Min Memory Usage: 43349kB
                     ->  Streaming(type: LOCAL REDISTRIBUTE dop: 1/4)  (cost=0.00..23899.85
rows=951142 width=4) (actual time=[0.034,189.868]..[0.034,189.868], rows=1000000)
                           ->  Seq Scan on t1  (cost=0.00..6484.10 rows=951142 width=4) (actual
time=[0.013,37.029]..[0.019,38.147], rows=1000000)
               SubPlan 1
                 ->  Seq Scan on t3  (cost=0.00..42.23 rows=1 width=8) (Actual time: never
executed)
                       Filter: ((a = t1.a) AND (b = t2.a))
               SubPlan 2
                 ->  Seq Scan on t3  (cost=0.00..31.49 rows=2149 width=8) (actual
time=[0.001,0.001]..[0.001,0.001], rows=0)
 Total runtime: 8930.868 ms
(19 rows)

openGauss=# Explain analyze with temp as
openGauss-# (
openGauss(#             select * from (select t1.a as a, t2.a as b from t1 left join t2 on t1.a=t2.a)
openGauss(# )
openGauss-# select distinct a,b from temp
openGauss-# where not exists (select a,b from t3 where temp.a=t3.a and temp.b=t3.b);

QUERY PLAN
------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------
 Streaming(type: LOCAL GATHER dop: 1/4)  (cost=109535861.04..109536024.52 rows=40000
width=8) (actual time=[677.751,911.177]..[677.751,911.177], rows=1000000)
   ->  HashAggregate  (cost=109535861.04..109535961.04 rows=40000 width=8) (actual
time=[667.020,786.812]..[668.542,788.495], rows=1000000)
         Group By Key: t1.a, t2.a
         ->  Hash Anti Join  (cost=30009.26..104870670.04 rows=3732152804 width=8) (actual
time=[216.281,505.778]..[217.804,508.395], rows=1100000)
               Hash Cond: ((t1.a = t3.a) AND (t2.a = t3.b))
               ->  Streaming(type: LOCAL REDISTRIBUTE dop: 4/4)  (cost=26872.17..88306104.88
rows=4976203738 width=8) (actual time=[216.233,425.028]..[217.750,428.418], rows=1100000)
                     ->  Hash Right Join  (cost=26872.17..12882516.98 rows=4976203738
width=8) (actual time=[216.178,498.345]..[222.921,542.297], rows=1100000)
```

```
                                    Hash Cond: (t2.a = t1.a)
                                    ->  Streaming(type: LOCAL REDISTRIBUTE dop: 4/4)
(cost=0.00..25691.86 rows=1046364 width=4) (actual time=[0.040,110.887]..[0.195,148.941],
rows=1100000)
                                          ->  Seq Scan on t2  (cost=0.00..6832.91 rows=1046364 width=4)
(actual time=[0.010,41.650]..[0.016,44.190], rows=1100000)
                                    ->  Hash  (cost=23899.85..23899.85 rows=951142 width=4) (actual
time=[215.907,215.907]..[222.672,222.672], rows=1000000)
                                          Max Buckets: 262144  Max Batches: 1  Max Memory Usage:
10814kB
                                          Min Buckets: 262144  Min Batches: 1  Min Memory Usage:
10814kB
                                          ->  Streaming(type: LOCAL REDISTRIBUTE dop: 4/4)
(cost=0.00..23899.85 rows=951142 width=4) (actual time=[0.034,153.669]..[0.053,160.892],
rows=1000000)
                                                ->  Seq Scan on t1  (cost=0.00..6484.10 rows=951142
width=4) (actual time=[0.016,37.425]..[0.020,38.155], rows=1000000)
                        ->  Hash  (cost=3129.03..3129.03 rows=2149 width=8) (actual
time=[0.004,0.004]..[0.005,0.005], rows=0)
                              Max Buckets: 32768  Max Batches: 1  Max Memory Usage: 256kB
                              Min Buckets: 32768  Min Batches: 1  Min Memory Usage: 256kB
                              ->  Streaming(type: LOCAL REDISTRIBUTE dop: 4/1)  (cost=0.00..3129.03
rows=2149 width=8) (actual time=[0.002,0.002]..[0.003,0.003], rows=0)
                                    ->  Seq Scan on t3  (cost=0.00..31.49 rows=2149 width=8) (actual
time=[0.002,0.002]..[0.002,0.002], rows=0)
 Total runtime: 985.284 ms
(21 rows)
```

As shown in the preceding information, parallel query is only supported in the outer layer of the SQL query, and the SMP technology cannot be used in the subquery.

The rewritten SQL statement can completely use the SMP technology because there is no subquery.

The performance of the rewritten SQL statement is improved significantly. The performance is improved by more than 9 times.

Step 6 Restore the **query_dop** parameter.

```
openGauss=# reset query_dop;
RESET
```

Step 7 Drop data.

```
Drop table t1;
Drop table t2;
Drop table t3;
```

5.10.4 Rewriting and Improving Subqueries

Step 1 Create tables.

```
openGauss=# create table t1 (c1 int, c2 int);
CREATE TABLE
openGauss=# create table t2 (c1 int, c2 int);
CREATE TABLE
```

Step 2 Generate lab data.

```
openGauss=# insert into t1 values(generate_series(1, 10000), generate_series(1, 1000));
INSERT 0 10000
openGauss=# insert into t2 values(generate_series(1, 10000), generate_series(1, 1000));
INSERT 0 10000
```

Step 3 Compile a subquery SQL statement that contains IN and run the statement.

```
\timing on
explain select count(1) from t1 where t1.c1 in (select t2.c1 from t2 where t1.c1 = t2.c2);
select count(1) from t1 where t1.c1 in (select t2.c1 from t2 where t1.c1 = t2.c2);
\timing off
```

The output is as follows:

```
openGauss=# \timing on
Timing is on.
openGauss=# explain select count(1) from t1 where t1.c1 in (select t2.c1 from t2 where t1.c1 = t2.c2);
                                  QUERY PLAN
------------------------------------------------------------------------------
 Aggregate  (cost=203807.61..203807.63 rows=1 width=12)
   ->  Aggregate  (cost=203807.61..203807.62 rows=1 width=12)
         ->  Seq Scan on t1  (cost=0.00..203795.52 rows=4836 width=4)
               Filter: (SubPlan 1)
               SubPlan 1
                 ->  Seq Scan on t2  (cost=0.00..165.91 rows=48 width=8)
                       Filter: (t1.c1 = c2)
(7 rows)

Time: 1.138 ms
openGauss=# select count(1) from t1 where t1.c1 in (select t2.c1 from t2 where t1.c1 = t2.c2);
 count
-------
  1000
(1 row)

Time: 16833.010 ms
openGauss=# \timing off
Timing is off.
```

Step 4 Change the subquery SQL statement to the join statement by equality rewriting.

```
\timing on
explain select count(1) from t1 where exists (select t2.c1 from t2 where t1.c1 = t2.c2 and t1.c1 =
t2.c1);
select count(1) from t1 where exists (select t2.c1 from t2 where t1.c1 = t2.c2 and t1.c1 = t2.c1);
```

```
\timing off
```

The output is as follows:

```
openGauss=# \timing on
Timing is on.
openGauss=# explain select count(1) from t1 where exists (select t2.c1 from t2 where t1.c1 = t2.c2
and t1.c1 = t2.c1);
                                        QUERY PLAN
------------------------------------------------------------------------------
 Aggregate  (cost=346.69..346.70 rows=1 width=8)
   ->  Hash Join  (cost=166.27..334.60 rows=4836 width=0)
         Hash Cond: (t1.c1 = t2.c2)
         ->  Seq Scan on t1  (cost=0.00..141.73 rows=9673 width=4)
         ->  Hash  (cost=166.20..166.20 rows=5 width=8)
               ->  HashAggregate  (cost=166.15..166.20 rows=5 width=8)
                     Group By Key: t2.c2, t2.c1
                     ->  Seq Scan on t2  (cost=0.00..165.91 rows=48 width=8)
                           Filter: (c2 = c1)
(9 rows)

Time: 1.172 ms
openGauss=# select count(1) from t1 where exists (select t2.c1 from t2 where t1.c1 = t2.c2 and t1.c1
= t2.c1);
 count
-------
  1000
(1 row)

Time: 5.134 ms
openGauss=# \timing off
Timing is off.
```

Compared with the original SQL statement, the performance is improved by more than 3000 times. When the data volume increases, the improvement is more obvious.

Step 5 Drop data.

```
openGauss=# Drop table t1;
DROP TABLE
openGauss=# Drop table t2;
DROP TABLE
```

5.11 SQL Rewriting

5.11.1 Lab Description

You can adjust SQL statements based on certain rules to improve the SQL execution efficiency while ensuring that the result is correct. You can follow these rules to improve service query efficiency.

5.11.2 Replace UNION with UNION ALL.

Step 1 Create table structures.

```
create table t1 (c1 int, c2 int);
create table t2 (c1 int, c2 int);
```

Step 2 Generate lab data.

```
openGauss=# insert into t1 values(generate_series(1, 10000), generate_series(1, 1000));
INSERT 0 10000
openGauss=# insert into t2 values(generate_series(20000, 30000), generate_series(20000, 30000));
INSERT 0 10001
```

Step 3 Print the SQL execution plan.

```
openGauss=# Explain analyze select c1 from t1 union select c1 from t2;
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------
 HashAggregate  (cost=525.29..718.75 rows=19346 width=4) (actual time=13.993..17.269
rows=20001 loops=1)
   Group By Key: t1.c1
   ->  Append  (cost=0.00..476.92 rows=19346 width=4) (actual time=0.048..7.671 rows=20001
loops=1)
         ->  Seq Scan on t1  (cost=0.00..141.73 rows=9673 width=4) (actual time=0.048..2.685
rows=10000 loops=1)
         ->  Seq Scan on t2  (cost=0.00..141.73 rows=9673 width=4) (actual time=0.041..2.723
rows=10001 loops=1)
 Total runtime: 19.416 ms
(6 rows)

openGauss=# Explain analyze select c1 from t1 union all select c1 from t2;
                                                                 QUERY PLAN
---------------------------------------------------------------------------------------------------------------------
 Result  (cost=0.00..283.46 rows=19346 width=4) (actual time=0.018..8.089 rows=20001 loops=1)
   ->  Append  (cost=0.00..283.46 rows=19346 width=4) (actual time=0.016..5.211 rows=20001
loops=1)
         ->  Seq Scan on t1  (cost=0.00..141.73 rows=9673 width=4) (actual time=0.016..1.498
rows=10000 loops=1)
         ->  Seq Scan on t2  (cost=0.00..141.73 rows=9673 width=4) (actual time=0.015..1.559
rows=10001 loops=1)
 Total runtime: 9.795 ms
(5 rows)
```

The result shows that UNION ALL is more effective and takes less time. Therefore, UNION ALL must be used unless duplicate rows need to be eliminated.

Step 4 Drop data.

```
Drop table t1;
Drop table t2;
```

6 Backup and Restoration

6.1 Introduction

6.1.1 About This Lab

This lab offers practical exercises for *Chapter 6 Backup, Restoration, and Disaster Recovery*.

In this lab, you can practice using logical backup and restoration tools, such as gs_dump, gs_dumpall, and gs_restore provided by openGauss.

As for physical backup and restoration tools, you can practice using gs_backup, gs_basebackup, and gs_probackup offered by openGauss. In addition, you can learn to restore data from backup files.

6.1.2 Objectives

- Understand the basic principles of gs_dump, gs_dumpall, and gs_restore.
- Master the usage of gs_dump, gs_dumpall, and gs_restore.
- Master the usage of gs_backup, gs_basebackup, and gs_probackup.
- Master the usage of flashback table, flashback query, and flashback DROP/TRUNCATE.

6.2 Logical Backup and Restoration

gs_dump, provided by openGauss, is used to export database information. You can export a database or its objects (such as schemas, tables, and views). The database can be the default **postgres** database or a user-defined database.

When gs_dump is used to export data, other users can still access (read or write) the database.

gs_dump can export complete and consistent data. For example, if gs_dump is enabled to export database A at T1, the database data at T1 will be exported, but the modifications after T1 will not.

6.2.1 Using gs_dump to Export Data from the **postgres** Database

Step 1 Use gs_dump to export the full information from the **postgres** database. The exported **MPPDB_backup.sql** file is in plain-text format.

Plain-text SQL script: It contains the SQL statements required to restore the database.

You can use gsql to execute the SQL script. With only a little modification, the SQL script can rebuild a database on other hosts or database products.

-f,–file=FILENAME sends the output to a specified file or directory. If this parameter is omitted, the standard output is generated. If the output format is **-F c**, **-F d**, or **-F t**, the **-f** parameter must be specified. If the value of the **-f** option contains a directory, the current user must have the read and write permissions on the directory, and the directory cannot be an existing one.

-F, –format=p outputs a text SQL script file (default setting).

```
[omm@opengauss01 script]$ gs_dump -U omm    -f /home/omm/MPPDB_backup.sql -p 26000
postgres -F p
gs_dump[port='26000'][postgres][20XX-XX-27 10:11:11]: The total objects number is 431.
gs_dump[port='26000'][postgres][20XX-XX-27 10:11:11]: [100.00%] 431 objects have been dumped.
gs_dump[port='26000'][postgres][20XX-XX-27 10:11:11]: dump database postgres successfully
gs_dump[port='26000'][postgres][20XX-XX-27 10:11:11]: total time: 1758    ms
```

Step 2 Use gs_dump to export the full information of the **postgres** database. The exported **MPPDB_backup.tar** file is in tar format.

-F, –format=t outputs a tar archive format as the input information of gs_restore.

The tar format is compatible with the directory format. Extracting a tar format archive form will generate a valid directory format archive form. However, the .tar archive cannot be further compressed and has an 8-GB limitation on the size of a single table. The order of table data items cannot be changed during restoration.

```
[omm@opengauss01 script]$ gs_dump -U omm    -f /home/omm/MPPDB_backup.tar -p 26000
postgres -F t
gs_dump[port='26000'][postgres][20XX-XX-27 10:16:18]: The total objects number is 431.
gs_dump[port='26000'][postgres][20XX-XX-27 10:16:18]: [100.00%] 431 objects have been dumped.
gs_dump[port='26000'][postgres][20XX-XX-27 10:16:18]: dump database postgres successfully
gs_dump[port='26000'][postgres][20XX-XX-27 10:16:18]: total time: 1555    ms
```

Step 3 Use gs_dump to export the full information of the **postgres** database. The exported **MPPDB_backup** file is in directory format.

-F, –format=d creates a directory containing directory files and the data files of tables and BLOBs.

```
[omm@opengauss01 script]$ gs_dump -U omm    -f /home/omm/MPPDB_backup -p 26000 postgres -
F d
gs_dump[port='26000'][postgres][20XX-XX-27 10:17:15]: The total objects number is 431.
gs_dump[port='26000'][postgres][20XX-XX-27 10:17:15]: [100.00%] 431 objects have been dumped.
gs_dump[port='26000'][postgres][20XX-XX-27 10:17:15]: dump database postgres successfully
gs_dump[port='26000'][postgres][20XX-XX-27 10:17:15]: total time: 1700    ms
```

6.2.2 Using gs_dumpall to Export Data from the postgres Database

gs_dumpall exports data from all openGauss databases in two parts:

gs_dumpall exports all global objects, including information about database users and groups, tablespaces, and attributes (for example, global access permissions).

gs_dumpall calls gs_dump to export SQL scripts from each openGauss database, which contain all the SQL statements required to restore databases.

Step 1 Use gs_dumpall to export all openGauss databases at a time.

```
[omm@opengauss01 script]$ gs_dumpall -f /home/omm/MPPDB_backup.sql -p 26000
```

gs_dumpall can only export plain text. Therefore, only gsql can be used to restore a file exported using gs_dumpall.

6.2.3 Exporting the **employees** Table in the Public Schema from the **postgres** Database and Saving It to an SQL Text File

-n –schema=SCHEMA dumps schemas matching the schema names. This option contains the schema and all its contained objects.

-t –table=TABLE specifies a list of tables, views, sequences, or foreign tables not to be dumped. You can use multiple **-t** parameters or wildcard characters to specify tables.

Step 1 Create the tables named **employees** and **departments**.

```
[omm@opengauss01 script]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# CREATE TABLE employees (
            employee_id integer NOT NULL, /*Employee ID,Primary key*/
            first_name VARCHAR(20), /*First name*/
        last_name VARCHAR(25) NOT NULL, /*Last name*/
        hire_date timestamp(0) without time zone NOT NULL, /*Hiring date*/
        salary numeric(8,2), /*Salary*/
        department_id integer, /*Department ID*/
        CONSTRAINT emp_salary_min CHECK (salary > (0))
);
CREATE TABLE

openGauss=# CREATE TABLE departments (
        department_id integer NOT NULL, /*Department ID,Primary key*/
        department_name varchar(30) NOT NULL, /*Department name*/
        manager_id integer, /*Department location ID*/
        location_id integer /*Department location ID*/
 );
CREATE TABLE
```

Step 2 Insert data into the **employees** table.

```
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
  VALUES (101,'Neena','Kochhar','2005-09-21 00:00:00',17000.00,90);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
```

```
 VALUES (111,'Ismael','Sciarra','2005-09-30 00:00:00',7700.00,100);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (113,'Luis','Popp','2007-12-07 00:00:00',6900.00,100);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (114,'Den','Raphaely','2002-12-07 00:00:00',11000.00,30);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (105,'David','Austin','2005-06-25 00:00:00',4800.00,60);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (106,'Valli','Pataballa','2006-02-05 00:00:00',4800.00,60);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (117,'Sigal','Tobias','2005-07-24 00:00:00',2800.00,30);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (185,'Alexis','Bull','2005-02-20 00:00:00',4100.00,50);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (186,'Julia','Dellinger','2006-06-24 00:00:00',3400.00,50);
INSERT INTO employees (employee_id,first_name,last_name,hire_date,salary,department_id)
 VALUES (145,'John','Russell','2004-10-01 00:00:00',14000.00,80);
```

Step 3 Insert data into the **departments** table.

```
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (10,'Administration',200,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (20,'Marketing',201,1800);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (30,'Purchasing',114,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (40,'Human Resources',203,2400);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (50,'Shipping',121,1500);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (60,'IT',103,1400);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (70,'Public Relations',204,2700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (80,'Sales',145,2500);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (90,'Executive',100,1700);
INSERT INTO departments (department_id,department_name,manager_id,location_id)
 VALUES (100,'Finance',108,1700);
```

Step 4 Query data in the **employees** table and **departments** table.

```
openGauss=# select * from employees;
 employee_id | first_name | last_name    |         hire_date         |   salary   | department_id
-------------+------------+--------------+---------------------------+------------+---------------
-
         101 | Neena      | Kochhar      | 2005-09-21 00:00:00       | 17000.00 |    90
         111 | Ismael     | Sciarra      | 2005-09-30 00:00:00       |  7700.00 |   100
         113 | Luis       | Popp         | 2007-12-07 00:00:00       |  6900.00 |   100
         114 | Den        | Raphaely     | 2002-12-07 00:00:00       | 11000.00 |    30
         105 | David      | Austin       | 2005-06-25 00:00:00       |  4800.00 |    60
```

```
        106 | Valli      | Pataballa     | 2006-02-05 00:00:00 |  4800.00 |  60
        117 | Sigal      | Tobias        | 2005-07-24 00:00:00 |  2800.00 |  30
        185 | Alexis     | Bull          | 2005-02-20 00:00:00 |  4100.00 |  50
        186 | Julia      | Dellinger     | 2006-06-24 00:00:00 |  3400.00 |  50
        145 | John       | Russell       | 2004-10-01 00:00:00 | 14000.00 |  80

openGauss=# select   * from departments;
 department_id | department_name   | manager_id | location_id
---------------+-------------------+------------+-------------
            10 | Administration    |        200 |        1700
            20 | Marketing         |        201 |        1800
            30 | Purchasing        |        114 |        1700
            40 | Human Resources   |        203 |        2400
            50 | Shipping          |        121 |        1500
            60 | IT                |        103 |        1400
            70 | Public Relations  |        204 |        2700
            80 | Sales             |        145 |        2500
            90 | Executive         |        100 |        1700
           100 | Finance           |        108 |        1700
(10 rows)
```

Step 5 Export the **employees** table in the public schema from the **postgres** database and save it to an SQL text file.

```
openGauss-# \q
[omm@opengauss01 script]$ gs_dump -U omm   -f /home/omm/MPPDB_backup_employees.sql -p
26000 postgres -n public -t employees -F p

gs_dump[port='26000'][postgres][20XX-XX-27 10:23:39]: The total objects number is 422.
...
gs_dump[port='26000'][postgres][20XX-XX-27 10:23:39]: total time: 1405   ms
```

Step 6 Export the **departments** table in the public schema from the **postgres** database and save it to a tar file.

```
[omm@opengauss01 script]$ gs_dump -U omm   -f /home/omm/MPPDB_backup_departments.tar -p
26000 postgres -n public -t departments -F t
gs_dump[port='26000'][postgres][20XX-XX-27 10:24:15]: The total objects number is 421.
...
gs_dump[port='26000'][postgres][20XX-XX-27 10:24:15]: total time: 1495   ms
```

6.2.4 Using gsql and gs_restore to Perform Logical Restoration

Step 1 Drop the **employees** table.

```
[omm@opengauss01 script]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=# drop table employees;
```

```
DROP TABLE
openGauss=# \q
```

Step 2 Use gsql to restore the **employees** table.

```
[omm@opengauss01 script]$ gsql -p 26000 postgres -r -f /home/omm/MPPDB_backup_employees.sql
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
total time: 6  ms
```

Step 3 Log in to the database and view data in the **employees** table.

```
[omm@opengauss01 ~]$ gsql -d postgres -p 26000 -r
openGauss=# select * from employees;
 employee_id | first_name | last_name |         hire_date         |   salary   | department_id
-------------+------------+-----------+---------------------------+------------+---------------
         101 | Neena      | Kochhar   | 2005-09-21 00:00:00 | 17000.00 |            90
         111 | Ismael     | Sciarra   | 2005-09-30 00:00:00 |  7700.00 |           100
         113 | Luis       | Popp      | 2007-12-07 00:00:00 |  6900.00 |           100
         114 | Den        | Raphaely  | 2002-12-07 00:00:00 | 11000.00 |            30
         105 | David      | Austin    | 2005-06-25 00:00:00 |  4800.00 |            60
         106 | Valli      | Pataballa | 2006-02-05 00:00:00 |  4800.00 |           60
         117 | Sigal      | Tobias    | 2005-07-24 00:00:00 |  2800.00 |            30
         185 | Alexis     | Bull      | 2005-02-20 00:00:00 |  4100.00 |           50
         186 | Julia      | Dellinger | 2006-06-24 00:00:00 |  3400.00 |           50
         145 | John       | Russell   | 2004-10-01 00:00:00 | 14000.00 |            80
(10 rows)
```

Step 4 Drop the **departments** table.

```
openGauss=# drop table departments;
DROP TABLE
openGauss=# \q
```

Step 5 Use the gs_restore tool to restore the **departments** table.

```
[omm@opengauss01 script]$ gs_restore -p 26000 -d postgres -n public -t departments
/home/omm/MPPDB_backup_departments.tar
start restore operation ...
table departments complete data imported !
```

```
Finish reading 4 SQL statements!
end restore operation ...
restore operation successful
total time: 14    ms
```

Step 6 Log in to the database and view the data in the **departments** table.

```
openGauss=# select   * from departments;
 department_id | department_name   | manager_id | location_id
---------------+------------------+------------+-------------
            10 | Administration    |        200 |        1700
            20 | Marketing         |        201 |        1800
            30 | Purchasing        |        114 |        1700
            40 | Human Resources   |        203 |        2400
            50 | Shipping          |        121 |        1500
            60 | IT                |        103 |        1400
            70 | Public Relations  |        204 |        2700
            80 | Sales             |        145 |        2500
            90 | Executive         |        100 |        1700
           100 | Finance           |        108 |        1700
(10 rows)
```

6.3 Physical Backup and Restoration

Physical backup is classified as either full or incremental backup. Full backup includes the full data of the database at the backup time point, requires an extended timeframe (in direct proportion to the total data volume of the database), and enables a complete database to be restored. In contrast, an incremental backup involves only incremental data modified after a specified time point. It requires a shorter timeframe (in direct proportion to the incremental data volume and not related to the total data volume). However, a complete database can be restored only after both the incremental backup and full backup are performed.

6.3.1 Using gs_backup to Back Up Parameter Files

--parameter backs up parameter files. If **--parameter**, **--binary**, and **--all** are not specified, only parameter files are backed up.

--binary backs up the binary files in the **app** directory.

--all backs up the binary files, **pg_hba.conf** and **postgresql.conf** in the **app** directory.

Step 1 Use gs_backup to back up parameter files.

```
[omm@opengauss01 script]$ gs_backup -t backup --backup-dir=/home/omm/ --parameter
Parsing configuration files.
Successfully parsed the configuration file.
Performing remote backup.
Remote backup succeeded.
Successfully backed up cluster files.
```

Step 2 View the backup parameter file.

```
[omm@opengauss01 ~]$ cd /home/omm
[omm@opengauss01 ~]$ tar -vxf parameter.tar
After the decompression, run the ll command to view the parameter_opengauss01.tar file.
[omm@opengauss01 ~]$ tar -xvf parameter_opengauss01.tar
[omm@opengauss01 ~]$ cd parameter_opengauss01
[omm@opengauss01 parameter_opengauss01]$ ls -l
total 48
-rw------- 1 omm dbgrp  4583 Mar 27 10:04 6001_pg_hba.conf
-rw------- 1 omm dbgrp 36070 Mar 27 10:04 6001_postgresql.conf
-rw------- 1 omm dbgrp     6 Mar 27 10:29 HOSTNAME
```

6.3.2 Full Backup Using gs_basebackup

gs_basebackup supports only full backup of the primary and standby nodes.

gs_basebackup supports hot backup and compressed backup.

If the incremental checkpoint and doublewrite features are enabled, gs_basebackup also backs up doublewrite files.

If you use gs_basebackup to back up a tablespace that contains an absolute path on the same server, you can use tablespace mapping to redirect the tablespace path or use the archive mode to back up the tablespace.

If the **pg_xlog** directory is a soft link, no soft link is created during backup. Data is directly backed up to the **pg_xlog** directory in the destination path.

If the backup permission is revoked during backup, the backup may fail, or the backup data may be unavailable.

If the server cannot respond due to a temporary network fault, gs_basebackup will exit after waiting for a maximum of 120 seconds.

Step 1 View the current database information.

```
openGauss=# \l
                                  List of databases
   Name    | Owner | Encoding | Collate | Ctype | Access privileges
-----------+-------+----------+---------+-------+-------------------
 postgres  | omm   | UTF8     | C       | C     |
 template0 | omm   | UTF8     | C       | C     | =c/omm               +
           |       |          |         |       | omm=CTc/omm
 template1 | omm   | UTF8     | C       | C     | =c/omm
```

Step 2 Create the database **testdb**.

```
openGauss=# create database testdb;
CREATE DATABASE
```

Step 3 Switch to the **testdb** database and create the **table_in_mytbs_ts** table.

```
openGauss=# \c testdb
testdb=# create table table_in_mytbs_ts (col1 char(10));
testdb=# insert into table_in_mytbs_ts values('test');
```

```
testdb=# \q
```

Step 4 Use gs_basebackup to fully back up the database.

```
[omm@opengauss01 ~]$ mkdir /home/omm/backup
[omm@opengauss01 ~]$ gs_basebackup -D /home/omm/backup/ -p 26000
INFO:  The starting position of the xlog copy of the full build is: 0/3000028. The slot minimum LSN is: 0/0. The disaster slot minimum LSN is: 0/0. The logical slot minimum LSN is: 0/0.
[20XX-XX-27 10:36:14]:begin build tablespace list
...
[20XX-XX-27 10:36:19]:gs_basebackup: base backup successfully
```

Step 5 View the backup information in the directory to which backup files are exported.

```
[omm@opengauss01 backup]$ cd /home/omm/backup
[omm@opengauss01 backup]$ ls -l
total 5028
-rw------- 1 omm dbgrp      206 Mar 27 10:36 backup_label
drwx------ 6 omm dbgrp     4096 Mar 27 10:36 base
...
-rw------- 1 omm dbgrp     4402 Mar 27 10:36 server.crt
-rw------- 1 omm dbgrp     1766 Mar 27 10:36 server.key
-rw------- 1 omm dbgrp       56 Mar 27 10:36 server.key.cipher
-rw------- 1 omm dbgrp       24 Mar 27 10:36 server.key.rand
drwx------ 5 omm dbgrp     4096 Mar 27 10:36 undo
```

6.3.3 Restoring Data from Backup Files

When the database is faulty, you need to restore data from backup files. gs_basebackup backs up the database in binary mode. As such, you can directly copy and replace the original files or start the database from the backup.

Step 1 Log in to the database, drop the **testdb** database, and run the **\q** command to exit the database.

```
openGauss=# drop database testdb;
DROP DATABASE
```

Step 2 Check the node location and stop the database.

```
[omm@opengauss01 backup]$ gs_om -t status --detail
```

```
[omm@opengauss01 backup]$ gs_om -t status --detail
[   Cluster State   ]

cluster_state   : Normal
redistributing  : No
current_az      : AZ_ALL

[  Datanode State   ]

    node       node_ip         port      instance                            state
-----------------------------------------------------------------------------------------
1  opengauss01 192.168.0.111   26000      6001 /opt/huawei/install/data/dn   P Primary Normal
```

```
[omm@opengauss01 backup]$ gs_ctl    stop    -D /opt/huawei/install/data/dn
[20XX-XX-01 10:48:51.295][982559][][gs_ctl]: gs_ctl stopped ,datadir is /opt/huawei/install/data/dn
waiting for server to shut down.... done
server stopped
```

Step 3 Back up the original data directory.

```
[omm@opengauss01 backup]$ cd    /opt/huawei/install/data/
[omm@opengauss01 data]$ mv    dn dn_bak
```

Step 4 Copy the backup file to the original data directory.

```
[omm@opengauss01 data]$ mv /home/omm/backup .
[omm@opengauss01 data]$ ls -l
total 8
drwx------ 19 omm dbgrp 4096 Mar 27 10:36 backup
drwx------ 20 omm dbgrp 4096 Mar 27 10:42 dn_bak
```

Step 5 Rename the backup directory with the name of the original data directory.

```
[omm@opengauss01 data]$    mv backup dn
[omm@opengauss01 data]$ ls -l
total 8
drwx------ 19 omm dbgrp 4096 Mar 27 10:36 dn
drwx------ 20 omm dbgrp 4096 Mar 27 10:42 dn_bak
```

Step 6 Restart the database.

```
 [omm@opengauss01 data]$ gs_ctl start    -D /opt/huawei/install/data/dn
[20XX-XX-27 10:48:52.084][20856][][gs_ctl]: gs_ctl started,datadir is /opt/huawei/install/data/dn
[20XX-XX-27 10:48:52.102][20856][][gs_ctl]: waiting for server to start
...
[20XX-XX-27 10:48:56.159][20856][][gs_ctl]:    done
[20XX-XX-27 10:48:56.159][20856][][gs_ctl]: server started(/opt/huawei/install/data/dn)
```

Step 7 View the restored **testdb** database.

```
openGauss=# \l
                                  List of databases
    Name      | Owner | Encoding | Collate | Ctype | Access privileges
-----------+-------+----------+---------+-------+-------------------
 postgres   | omm     | UTF8       | C          | C       |
 template0 | omm     | UTF8       | C          | C       | =c/omm               +
              |          |             |             |          | omm=CTc/omm
 template1 | omm     | UTF8       | C          | C       | =c/omm               +
              |          |             |             |          | omm=CTc/omm
 testdb      | omm     | UTF8       | C          | C       |

openGauss=# \c testdb
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "testdb" as user "omm".
testdb=# \d
                                      List of relations
```

```
 Schema |          Name           | Type  | Owner |                    Storage
--------+-------------------------+-------+-------+---------------------------------
 public | table_in_mytbs_ts | table | omm     | {orientation=row,compression=no}
(1 row)

testdb=# select * from table_in_mytbs_ts;
      col1
------------
 test
(1 row)
```

6.3.4 Using gs_probackup to Perform Incremental Backup and Restoration

The tool supports physical backup for databases deployed in standalone or primary/standby mode.

It supports the backup of contents in external directories, such as script files, configuration files, log files, and dump files.

It supports incremental backup, periodic backup, and remote backup.

It supports settings on the backup retention policy.

To use PTRACK incremental backup, manually add **enable_cbm_tracking = on** to **postgresql.conf**.

To prevent Xlogs from being cleared before the transmission is complete, increase the value of **wal_keep_segments** in the **postgresql.conf** file.

Step 1 Set the parameter **enable_cbm_tracking** to **on**.

```
[omm@opengauss01 data]$ gs_guc reload -N all -I all -c "enable_cbm_tracking=on"

The gs_guc run with the following arguments: [gs_guc -N all -I all -c enable_cbm_tracking=on
reload ].
NOTICE: Turn on cbm tracking function.
Begin to perform the total nodes: 1.
Popen count is 1, Popen success count is 1, Popen failure count is 0.
Begin to perform gs_guc for datanodes.
Command count is 1, Command success count is 1, Command failure count is 0.

Total instances: 1. Failed instances: 0.
ALL: Success to perform gs_guc!

[omm@opengauss01 data]$ gsql -d postgres -p 26000 -r
openGauss=# show enable_cbm_tracking;
 enable_cbm_tracking
---------------------
 on
(1 row)
```

Step 2 Initialize the backup directory.

Run **gs_probackup init -B backup-path [–help]** to initialize the backup directory in **backup-path**. This directory stores backup content. If the **backup-path** backup path exists, it must be empty.

```
[omm@opengauss01 data]$    mkdir -p /home/omm/gs_probackup
[omm@opengauss01 data]$ gs_probackup init -B /home/omm/gs_probackup/
INFO: Backup catalog '/home/omm/gs_probackup' successfully inited
```

Step 3 Add a backup instance.

Initialize a new backup instance in the backup directory of **backup-path** and generate the **pg_probackup.conf** configuration file, which saves the gs_probackup settings of the specified data directory **pgdata-path**.

-B backup-path, --backup-path=backup-path specifies the backup directory. **-D pgdata-path, --pgdata=pgdata-path** specifies the data directory.

```
[omm@opengauss01 data]$ gs_probackup add-instance --instance og1 -B
/home/omm/gs_probackup/ -D /opt/huawei/install/data/dn
INFO: Instance 'og1' successfully inited
```

Step 4 Add related settings to the **pg_probackup.conf** file.

Add the specified connection, compression, and log-related settings to the **pg_probackup.conf** file or modify the existing settings. You are advised not to manually edit the **pg_probackup.conf** file.

```
[omm@opengauss01 data]$ gs_probackup set-config --instance=og1 -B /home/omm/gs_probackup/ -
d postgres -p 26000
```

Step 5 Create test data.

```
openGauss=# \c testdb
testdb=# insert into table_in_mytbs_ts values('openGauss');
testdb=# insert into table_in_mytbs_ts values('GaussDB');
testdb=# select * from table_in_mytbs_ts;
      col1
------------
 test
 openGauss
 GaussDB
(3 rows)
```

Step 6 Perform a full backup.

-b backup-mode, --backup-mode=backup-mode specifies the backup mode, which can be **FULL** or **PTRACK**.
FULL: creates a full backup that contains all data files. **PTRACK**: creates a PTRACK incremental backup.

```
[omm@opengauss01 data]$ gs_probackup backup -B /home/omm/gs_probackup/ --instance og1 -b
full
```

```
INFO: Backup start, gs_probackup version: 2.4.2, instance: og1, backup ID: SAZK7K, backup mode:
FULL, wal mode: STREAM, remote: false, compress-algorithm: none, compress-level: 1
LOG: Backup destination is initialized
...
INFO: Backup SAZK7K resident size: 632MB
INFO: Backup SAZK7K completed
```

Step 7 View the backup information.

```
[omm@opengauss01 data]$ gs_probackup show -B   /home/omm/gs_probackup/

BACKUP INSTANCE 'og1'
================================================================================
=========================================================
 Instance   Version   ID         Recovery Time              Mode   WAL Mode   TLI   Time    Data
WAL   Zratio   Start LSN   Stop LSN    Type   Status
================================================================================
=========================================================
 og1          9.2         SAZK7K   2024-03-27 10:57:25+08   FULL   STREAM      1/0    18s   616MB
16MB     1.00   0/5000028   0/50001E8   FILE   OK
```

Step 8 Add data to the test table.

```
testdb=# insert into table_in_mytbs_ts values('openEuler');
testdb=# insert into table_in_mytbs_ts values('Euler');
testdb=# select * from table_in_mytbs_ts;
      col1
------------
 test
 openGauss
 GaussDB
 openEuler
 Euler
(5 rows)
```

Step 9 Perform incremental backup.

```
[omm@opengauss01 data]$ gs_probackup backup -B /home/omm/gs_probackup/ --instance og1 -b
PTRACK
INFO: Backup start, gs_probackup version: 2.4.2, instance: og1, backup ID: SAZKDB, backup mode:
PTRACK, wal mode: STREAM, remote: false, compress-algorithm: none, compress-level: 1
LOG: Backup destination is initialized
...
INFO: Backup S34PAS resident size: 273MB
INFO: Backup S34PAS completed
```

Step 10 View the backup information.

```
[omm@opengauss01 data]$ gs_probackup show -B   /home/omm/gs_probackup/

BACKUP INSTANCE 'og1'
```

```
========================================================================================
=========================================================
 Instance  Version  ID        Recovery Time              Mode     WAL Mode  TLI  Time    Data
WAL   Zratio  Start LSN   Stop LSN    Type   Status
========================================================================================
=========================================================
 og1         9.2        SAZKDB   2024-03-27 11:00:48+08   PTRACK   STREAM     1/1     7s   257MB
16MB     1.00   0/7000028   0/70001E8   FILE   OK
 og1         9.2        SAZK7K   2024-03-27 10:57:25+08   FULL     STREAM     1/0     18s   616MB
16MB     1.00   0/5000028   0/50001E8   FILE   OK
```

In the command output, **BACKUP INSTANCE 'og1'** indicates the backup information of an openGauss instance named **og1**. The backup list contains two different backup operations:

First record:

Version: version number of the database during the backup. The value is **9.2**.

ID: unique identifier of this backup. The value is **SAZKDB**.

Recovery Time: backup completion time. The value is **2024-03-27 11:00:48+08** (Beijing time).

Mode: backup mode. **PTRACK** indicates that the backup is a track backup (continuous or incremental).

WAL Mode: WAL backup mode. The value **STREAM** indicates that the WAL streaming replication mode is used for backup.

TLI: timeline ID, which is related to the recovery of the database system. The value is **1/1**.

Time: backup time. The value is **7s**.

Data: data file size. The value is **257MB**.

WAL: size of a WAL file. The value is **16MB**.

Zratio: compression ratio. The value is **1.00**, indicating that no compression is performed.

Start LSN: backup start log sequence number (LSN). The value is **0/7000028**.

Stop LSN: The backup end LSN. The value is **0/70001E8**.

Type: backup type. The value **FILE** indicates that the backup is a file-level backup.

Status: backup status. The value **OK** indicates that the backup status is normal.

Second record:

ID: unique identifier of this backup. The value is **SAZK7K**.

Recovery Time: backup completion time. The value is **2024-03-27 10:57:25+08** (earlier than the first backup).

Mode: backup mode. **FULL** indicates a full backup.

The meanings of other columns are the same as those of the preceding columns. Note that the value of **TLI** is **1/0**, which usually occurs during the first full backup. In addition, the **Start LSN** is far from **Stop LSN**, indicating that the complete database status is backed up.

Incremental backup (referring to the backup after the second record in this scenario) is based on the previous full or incremental backup. Only the data that has been changed

in the database since the last backup is backed up. In WAL mode, only WAL files generated during this period are backed up as incremental backup to reduce storage space usage and backup time. In the preceding output, the backup identified by **SAZKDB** is performed after the full backup identified by **SAZK7K**. As such, the former can be considered as an incremental backup based on the latter.

Step 11 Simulate the deletion of the test table.

```
openGauss=# \c testdb
testdb=# drop table table_in_mytbs_ts;
DROP TABLE
testdb=# \q
```

Step 12 Shut down the database. **-D** indicates the node position.

```
[omm@opengauss01 data]$ gs_ctl stop    -D /opt/huawei/install/data/dn
[20XX-XX-27 11:03:57.102][21665][][gs_ctl]: gs_ctl stopped ,datadir is /opt/huawei/install/data/dn
waiting for server to shut down.... done
server stopped
```

6.3.5 Point-In-Time Recovery (PITR)

When a database breaks down or needs to be rolled back to a previous state, the PITR feature of openGauss can be used to restore the database to any point in time after the backup and generate archive data.

PITR can only be restored to a point in time after the physical backup data is generated. The following uses gs_basebackup as an example.

Step 1 Switch to the **root** user, create an archive log directory, and change the corresponding permission.

```
[omm@opengauss01 /]$ exit
[root@opengauss01 ~]# mkdir /ogarchive
[root@opengauss01 ~]# chmod 777 /ogarchive/
```

Step 2 Switch to user **omm** and modify the **postgresql.conf** file in the data directory. Change **archive_mode** to **on** and **archive_command** to **'cp %p /ogarchive/%f'** (the # before the two parameters must be deleted), press **Esc**, and run **:wq** to save the modification and exit.

```
[root@opengauss01 ~]# su omm
[omm@opengauss01 ~]$ cd /ogarchive/
[omm@opengauss01 ogarchive]$ ll
total 0
[omm@opengauss01 ogarchive]$ vi /opt/huawei/install/data/dn/postgresql.conf
```

```
# - Archiving -

archive_mode = on               # allows archiving to be done
                                # (change requires restart)
archive_command = 'cp %p /ogarchive/%f'         # command to use
                                # placeholders: %p = path of fil
                                #               %f = file name o
                                # e.g. 'test ! -f /mnt/server/ar
ir/%f'
#archive_timeout = 0            # force a logfile segment switch
                                # number of seconds; 0 disables
```

Step 3 After the configuration is modified, restart the database for the modification to take effect and log in to the database to view the modification.

```
[omm@opengauss01 root]$ gs_om -t restart
[omm@opengauss01 root]$ gsql -d postgres -p 26000 -r

openGauss=#    show archive_mode;
 archive_mode
--------------
 on
(1 row)

openGauss=# show archive_command;
    archive_command
--------------------
 cp %p /ogarchive/%f
(1 row)
```

Step 4 Use gs_basebackup to fully back up the database.

```
[omm@opengauss01 ogarchive]$ mkdir /home/omm/backup1
[omm@opengauss01 ogarchive]$    gs_basebackup -D /home/omm/backup1/ -p 26000
```

Step 5 If the system breaks down and data is lost, clear all data in the **pg_xlog** directory and use PITR to restore the data.

```
[omm@opengauss01 ogarchive]$ cd /opt/huawei/install/data/dn/pg_xlog/
[omm@opengauss01 pg_xlog]$ rm -rf *
```

Step 6 Before PITR, ensure that the database service has been stopped to prevent new transactions from being written to the database, which affects the recovery.

```
[omm@opengauss01 ~]# gs_om -t stop
```

Step 7 Back up the original data directory, copy the backup file to the original data directory, and rename the backup directory with the name of the original data directory.

```
[omm@opengauss01 ogarchive]$ cd /opt/huawei/install/data/
[omm@opengauss01 data]$ mv    dn dn_bak
[omm@opengauss01 data]$ mv /home/omm/backup1 .
[omm@opengauss01 data]$ mv backup1 dn
[omm@opengauss01 data]$ ll
total 8.0K
drwx------ 19 omm dbgrp 4.0K Apr 11 14:21 dn
drwx------ 20 omm dbgrp 4.0K Apr 11 14:18 dn_bak
```

Step 8 Copy the archive log files to the **pg_xlog** directory and restart the database to implement PITR.

```
[omm@opengauss01 ogarchive]$ cd /ogarchive/
[omm@opengauss01 ogarchive]$ cp * /opt/huawei/install/data/dn/pg_xlog/
[omm@opengauss01 ogarchive]$ gs_om -t start
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================
[SUCCESS] opengauss01
20XX-XX-11 14:39:24.788 6617859c.1 [unknown] 281458751766544 [unknown] 0 dn_6001 01000    0
[BACKEND] WARNING:    could not create any HA TCP/IP sockets
20XX-XX-11 14:39:24.788 6617859c.1 [unknown] 281458751766544 [unknown] 0 dn_6001 01000    0
[BACKEND] WARNING:    could not create any HA TCP/IP sockets
=========================================
Successfully started.
```

6.4 Flashback

This lab consists of three modules: flashback query, flashback table, and flashback DROP/TRUNCATE.

Flashback is part of a database recovery technology. You can use it to selectively cancel the impact of a committed transaction and restore data if incorrect manual operations are performed. Before flashback, committed database modifications could only be restored using backup and restoration or PITR, which would take several minutes or even hours. Using flashback, you can restore committed data to its previous state in just seconds, regardless of the database size.

6.4.1 Flashback Query

Flashback is part of a database recovery technology. You can use it to selectively cancel the impact of a committed transaction and restore data if incorrect manual operations are performed. Before flashback, committed database modifications could only be restored using backup and restoration or PITR, which would take several minutes or even hours.

Using flashback, you can restore committed data to its previous state in just seconds, regardless of the database size.

Data restoration based on MVCC (only supported in Ustore): applicable to the query and restoration of data that is deleted, updated, or inserted by mistake. You can configure the retention period of historical data and run the corresponding command to query or restore data to a specified time point or CSN.

Recycle bin-based restoration (only supported in Ustore): applicable to the restoration of tables that are dropped or truncated by mistake. You can configure the recycle bin function and run the corresponding command to restore the tables that are erroneously dropped or truncated. Note: The recycle bin is not supported in Astore (flashback DROP/TRUNCATE).

6.4.2 Flashback Table

With flashback query, you can query a snapshot of a table at a certain point in time. You can use this feature to view and logically rebuild data that was erroneously deleted or modified. The flashback query uses the MVCC mechanism, allowing you to retrieve and query data from an earlier version of the database.

Step 1 Set parameters based on the database configuration file **postgresql.conf**.

enable_default_ustore_table=on ### Ustore is enabled by default.

undo_zone_count=16384 ### Number of undo zones that can be allocated from the memory. The value **0** indicates that undo and Ustore tables are disabled. The recommended value is **max_connections** multiplied by 4.

undo_retention_time=2000 ### Sets the retention period of old undo versions. The default value is **0**, in seconds.

```
[omm@opengauss01 db2]$ gs_guc set   -N all -I all   -c "enable_default_ustore_table=on"
[omm@opengauss01 db2]$ gs_guc set   -N all -I all   -c "undo_zone_count=16384"
[omm@opengauss01 db2]$ gs_guc set   -N all -I all   -c "undo_retention_time=2000"
```

Restart the database. Note that the DN instance needs to be started.

```
 [omm@ecs-5a5b data]$ gs_om -t start
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================================================
Successfully started primary instance. Wait for standby instance.
=========================================================================
Successfully started cluster.
=========================================================================
```

Step 2 View the modified parameter values.

```
[omm@opengauss01 db2]$ gsql -d postgres -p 26000 -r
openGauss=# show enable_default_ustore_table;
```

```
 enable_default_ustore_table
----------------------------
 on
(1 row)
openGauss=# show undo_zone_count;
 undo_zone_count
-----------------
 16384
(1 row)
openGauss=# show undo_retention_time;
 undo_retention_time
---------------------
 2000s
(1 row)
```

Step 3 Create a test table and insert two data records.

```
openGauss=# create table u_tb (id int,c1 varchar(10));
openGauss=# insert into u_tb values (1,'Aso'),(2,'Hocx');
```

Step 4 Check the current timestamp **t1**.

```
openGauss=#    select clock_timestamp();
            clock_timestamp
-------------------------------
 20XX-XX-27 11:10:04.785096+08
(1 row)
```

Step 5 Insert new data.

```
openGauss=# insert into u_tb values (3,'Ysad');
openGauss=# select * from u_tb;
 id |   c1
----+------
  1 | Aso
  2 | Hocx
  3 | Ysad
(3 rows)
```

Step 6 Check the current timestamp **t2**.

```
openGauss=#    select clock_timestamp();
            clock_timestamp
-------------------------------
 20XX-XX-27 11:10:42.024302+08
(1 row)
```

Step 7 Perform a timestamp-based flashback query.

Query data in the **u_tb** table at **t1**.

```
openGauss=# SELECT * FROM u_tb TIMECAPSULE TIMESTAMP to_timestamp ('2024-03-27 11:10:42',
'YYYY-MM-DD HH24:MI:SS');
```

```
 id |   c1
----+------
  1 | Aso
  2 | Hocx
  3 | Ysad
(3 rows)
```

Step 8 Perform a CSN-based flashback query.

```
openGauss=#   select clock_timestamp();
           clock_timestamp
-------------------------------
 20XX-XX-27 11:12:50.767138+08
(1 row)

openGauss=#   update u_tb set id = 4 where id =3;
openGauss=# select * from u_tb;
 id |   c1
----+------
  1 | Aso
  2 | Hocx
  4 | Ysad
(3 rows)
```

Step 9 Query the CSN corresponding to the timestamp.

```
openGauss=# select snptime,snpcsn from gs_txn_snapshot where snptime between ' 2024-03-27
11:12:23' and '2024-03-27 11:15:06';
                  snptime                  | snpcsn
-------------------------------+--------
 2024-03-27 11:12:23.073448+08 |    2323
 2024-03-27 11:12:26.096588+08 |    2325
 2024-03-27 11:12:29.120286+08 |    2327
...
openGauss=#   SELECT * FROM u_tb TIMECAPSULE CSN 2323;
 id |   c1
----+------
  1 | Aso
  2 | Hocx
  3 | Ysad
(3 rows)
```

6.4.3 Flashback DROP/TRUNCATE

Flashback DROP restores tables that are dropped by mistake and their auxiliary structures, such as indexes and table constraints, from the recycle bin. Flashback DROP is based on the recycle bin mechanism. You can restore physical table files from the recycle bin to restore the dropped tables.

Flashback TRUNCATE restores tables that are truncated by mistake by restoring the physical data of the truncated tables and indexes from the recycle bin. Flashback TRUNCATE is based on the recycle bin mechanism, so you can restore physical table files from the recycle bin to restore the truncated tables.

Step 1 Set parameters based on the database configuration file **postgresql.conf**.

```
enable_recyclebin=on ### Enable the recyclebin feature.
recyclebin_retention_time=30min ### Specifies the retention period of objects in the recycle bin. The objects will be automatically deleted after the retention period expires.
[omm@ecs-5a5b data]$gs_guc set    -N all -I all    -c "enable_recyclebin=on"
[omm@ecs-5a5b data]$gs_guc set    -N all -I all    -c "recyclebin_retention_time=30min"
[omm@ecs-5a5b data]$ gs_om -t restart
openGauss=# show enable_recyclebin;
 enable_recyclebin
-------------------
 on
(1 row)

openGauss=# show    recyclebin_retention_time;
 recyclebin_retention_time
---------------------------
 30min
(1 row)
```

Step 2 Create a test table.

```
openGauss=# create table u_tb2 (id int,c1 varchar(10));
openGauss=# insert into u_tb2 values (1,'Aso'),(2,'Hocx');
openGauss=# select * from u_tb2;
id | c1
----+------
1 | Aso
2 | Hocx
```

Step 3 Truncate the table.

```
openGauss=# truncate u_tb2;
TRUNCATE TABLE

openGauss=# select * from u_tb2;
id | c1
----+----
(0 rows)
```

Step 4 Flash back the TRUNCATE operation.

```
openGauss=# timecapsule table u_tb2 to before truncate;
TimeCapsule Table
openGauss=# select * from u_tb2;
id | c1
----+------
1 | Aso
2 | Hocx
(2 rows)
```

Step 5 Drop the table.

```
openGauss=# drop table u_tb2;
DROP TABLE
```

Step 6 View the recycle bin.

```
openGauss=# SELECT rcyname,rcyoriginname,rcytablespace FROM GS_RECYCLEBIN;
rcyname                                    | rcyoriginname | rcytablespace
-------------------------------------------------------+-------------------+---------------
BIN$3C774EBC071$23FB27078==$0     | u_tb2           | 0
```

Step 7 Flash back the dropped table and rename it.

```
openGauss=# timecapsule table u_tb2 to before drop rename to u_tb2_bak;
TimeCapsule Table

openGauss=# select * from u_tb2_bak;
id | c1
----+------
1 | Aso
2 | Hocx
```

Step 8 Drop table without moving it to the recycle bin.

```
openGauss=# drop table u_tb2_bak purge;
DROP TABLE
```

Step 9 Check the recycle bin.

```
openGauss=# SELECT rcyname,rcyoriginname,rcytablespace FROM GS_RECYCLEBIN;
rcyname | rcyoriginname | rcytablespace
---------+---------------+---------------
(0 rows)
```

7 Advanced Features

7.1 Introduction

7.1.1 About This Lab

This lab offers practical exercises for *Chapter 7 Advanced Features*. It consists of three parts: AI capabilities, fully-encrypted database, and MOT management.

The AI capabilities of openGauss integrate multiple intelligent functions to implement self-management and optimization of the database system, improving database performance, stability, and O&M efficiency.

openGauss fully-encrypted database is a high-level data security solution that ensures data security throughout its lifecycle. It encrypts data during generation, transmission, processing, and storage to prevent unauthorized access and leakage.

openGauss allows applications to freely combine MOTs and standard disk-based tables. MOT is particularly effective for handling highly active, contention-intensive, and performance-sensitive tables that have been identified as bottlenecks. It is widely used among applications.

7.1.2 Objectives

- Master the AI4DB feature of the openGauss database.
- Master the method of modifying openGauss database parameters.
- Master how to use gsql to operate a fully-encrypted database that integrates software and hardware.
- Master the application of the fully-encrypted database.
- Master the methods of creating and deleting an MOT.
- Master how to convert disk-based tables.

7.2 AI Capabilities

This course introduces AI capabilities from two aspects: AI4DB for autonomous database O&M and native DB4AI engine. Through basic operations and configurations, this course helps you understand the basic functions and usage of AI capabilities in the openGauss database.

7.2.1 Installing DBMind

Step 1 Run the **su - omm** command to switch to user **omm** and create the **dbmind_monitor** database user and metadatabase as user **omm**.

```
[root@opengauss01 software]# chown omm:dbgrp -R /opt/software/
[root@opengauss01 ~]# su omm
[omm@opengauss ~]$ gs_guc reload -D /opt/huawei/install/data/dn -c 'password_encryption_type=1'
```

```
[omm@opengauss ~]$ gs_guc reload -D /opt/huawei/install/data/dn -c 'password_encryption_type=1'
gsql -p 26000 -d postgres -c 'grant all privileges to dbmind_monitor;'
gs_guc reload -D /opt/huawei/install/data/dn -c 'password_encryption_type=2'
gsql -p 26000 -d postgres -c 'create database metadatabase owner dbmind_monitor;'The gs_guc run w
s: [gs_guc -D /opt/huawei/install/data/dn -c password_encryption_type=1 reload ].
expected instance path: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc reload: password_encryption_type=1: [/opt/huawei/install/data/dn/postgresql.conf]
server signaled

Total instances: 1. Failed instances: 0.
Success to perform gs_guc!
```

```
[omm@opengauss ~]$ gsql -p 26000 -d postgres -c 'create user dbmind_monitor password
"openEuler@1234";'
[omm@opengauss ~]$ gsql -p 26000 -d postgres -c 'alter user dbmind_monitor monadmin;'
[omm@opengauss01 openGauss]$ gsql -p 26000 -d postgres -c 'grant all privileges to
dbmind_monitor;'
[omm@opengauss01 openGauss]$ gs_guc reload -D /opt/huawei/install/data/dn -c
'password_encryption_type=2'
[omm@opengauss01 openGauss]$ gsql -p 26000 -d postgres -c 'create database metadatabase
owner dbmind_monitor;'
```

Step 2 Install DBMind as user **omm**. Upload the DBMind installation package to the **/home/omm** directory. Ensure that the user of the file is **omm** belonging to the **dbgrp** user group. The corresponding commands are as follows:

```
[omm@opengauss01 openGauss]$ exit
[root@opengauss01 ~]# cd    /home/omm/
[root@opengauss01 ~]# wget https://opengauss.obs.cn-south-
1.myhuaweicloud.com/5.0.0/dbmind/arm/dbmind-installer-aarch64-python3.10.sh.tar.gz
[root@opengauss01 openGauss]# chown omm:dbgrp /home/omm/dbmind-installer-aarch64-
python3.10.sh.tar.gz
[root@opengauss01 omm]# su omm
[omm@opengauss01 ~]$ cd /home/omm/
[omm@opengauss01 omm]# tar -zxvf dbmind-installer-aarch64-python3.10.sh.tar.gz
```

Run the script to start installation.

```
[omm@opengauss01 ~]$ sh dbmind-installer-aarch64-python3.10.sh

Welcome to DBMind Installer

DBMind will now be installed into this location:
/home/omm/openGauss-DBMind

   - Press ENTER to confirm the location
   - Press CTRL-C to abort the installation
```

```
   - Or specify a different location below

>>>      ----- Press Enter.
PREFIX=/home/omm/openGauss-DBMind
Unpacking payload ...
Do you wish the installer to append the path /home/omm/openGauss-DBMind into the environment
variable? [yes|no]
>>>yes     ------ Enter yes.
Installed successfully.
```

Step 3 Make the environment variables take effect.

```
[omm@opengauss01 ~]$ source ~/.bashrc
```

Step 4 Verify the installation.

```
[omm@opengauss01 ~]$ gs_dbmind --help
NOTICE: You have set the variable PYTHONPATH. DBMind ignored it.
usage: [-h] [-v] {service,set,component} ...

openGauss DBMind: An autonomous platform for openGauss

options:
  -h, --help            show this help message and exit
  -v, --version         show program's version number and exit

available subcommands:
  {service,set,component}
                        type '<subcommand> -h' for help on a specific subcommand
    service             send a command to DBMind to change the status of the service
    set                 set a parameter
    component           pass command line arguments to each sub-component.
```

Step 5 Configure the Python resource library.

```
[omm@opengauss01 ~]$ mkdir ~/.pip/
[omm@opengauss01 ~]$ vim ~/.pip/pip.conf
```

Add the following content:

```
[global]
index-url = http://mirrors.aliyun.com/pypi/simple/
trusted-host = mirrors.aliyun.com
timeout = 120
```

Step 6 Install a third-party plug-in and start it.

```
[omm@opengauss01 ~]$ /home/omm/openGauss-DBMind/python/bin/python -m pip install -r
/home/omm/openGauss-DBMind/requirements-aarch64.txt
[omm@opengauss01 ~]$ /home/omm/openGauss-DBMind/python/bin/python -m pip install python-
multipart
```

Install node-exporter and upload it to the home directory **/home/omm** of user **omm**. Ensure that the user and its user group are **omm:dbgrp**.

Decompress the package.

```
[omm@opengauss01 ~]# cd /home/omm/
[omm@opengauss01 omm]# wget
https://github.com/prometheus/node_exporter/releases/download/v1.7.0/node_exporter-1.7.0.linux-arm64.tar.gz
[omm@opengauss01 ~]$ tar -zxvf /home/omm/node_exporter-1.7.0.linux-arm64.tar.gz -C /opt/software/
[omm@opengauss01 ~]$ cd /opt/software/node_exporter-1.7.0.linux-arm64/
```

Start node_exporter. The default port number is **9100**.

```
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ ./node_exporter > node_exporter.log 2>&1 &

[1] 19660
```

Check whether the port is in listening state.

```
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ netstat -anp|grep 9100
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
tcp6       0      0 :::9100                 :::*                    LISTEN
19660/./node_export
```

Start cmd_exporter to collect the cluster status.

```
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ gs_dbmind component cmd_exporter --web.listen-address 0.0.0.0 --web.listen-port 9180 --disable-https
NOTICE: You have set the variable PYTHONPATH. DBMind ignored it.
The process is starting.
```

Start opengauss_exporter. opengauss_exporter, a DBMind component, reads data from system catalogs (or system views) in the openGauss database and saves the data using Prometheus. It supports remote deployment and must be installed on each database node. The port number of opengauss_exporter ranges from 9187 to 9197. If a port is occupied, you can use another port.

Note: For the DSN in the URL format, URL encoding is required because special characters such as the at sign (@) are used to separate the content of each part in the URL string. For example, if the password of user **dbmind** is **openEuler@1234**, the DSN in the URL format is **postgresql:dbmind:Test%40123@127.0.0.1:15432**, that is, the at sign (@) is encoded as **%40**. Change the IP address to your private IP address.

```
 [omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ gs_dbmind component opengauss_exporter --url postgresql://dbmind_monitor:openEuler%401234@192.168.0.111:26000/postgres --web.listen-address 0.0.0.0 --web.listen-port 9187 --log.level info --disable-https
```

After it starts up, run the following command; if the command output similar to the following is displayed, the startup is normal:

```
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ curl -vv http://localhost:9187/metrics
```

Step 7 Install Prometheus and upload it to the home directory **/home/omm** of user **omm**. Then, change the user and its user group to **omm:dbgrp**. Prometheus is a popular open-source monitoring system in the industry and is also a time series database. It needs to be installed on the main control node. The default port number is **9090**.

```
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ cd /home/omm/
[omm@opengauss01 ~]$ wget
https://github.com/prometheus/prometheus/releases/download/v2.51.1/prometheus-2.51.1.linux-
arm64.tar.gz
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ tar -zxvf /home/omm/prometheus-
2.51.1.linux-arm64.tar.gz  -C /opt/software/
[omm@opengauss01 node_exporter-1.7.0.linux-arm64]$ cd /opt/software/prometheus-2.51.1.linux-
arm64/
```

Modify the configuration file.

```
[omm@opengauss01 prometheus-2.51.1.linux-arm64]$ vi ./prometheus.yml
```

Find the [scrape_configs] section in the file that contains the configuration information about all targets to be captured and modify the configuration information based on the actual number of nodes and port usage. In this lab, a standalone system is used. Perform the following configurations and make sure that the information in red has been replaced with your private IP address.

```
 # A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["192.168.0.111:9090"]
  - job_name: "opengauss_exporter"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["192.168.0.111:9187"]
  - job_name: "node_exporter"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["192.168.0.111:9100"]
  - job_name: "reprocessing_exporter"
```

```
    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["192.168.0.111:8181"]
  - job_name: "cmd_exporter"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["192.168.0.111:9180"]
```

Start Prometheus and import logs to the **prometheus.log** file in the same directory.

```
[omm@opengauss01 prometheus-2.51.1.linux-arm64]$ ./prometheus --web.enable-admin-api --
web.enable-lifecycle --storage.tsdb.retention.time=1w >./prometheus.log 2>&1 &
```

Restart reprocessing_exporter.

```
[omm@opengauss01 prometheus-2.51.1.linux-arm64]$ gs_dbmind component reprocessing_exporter
192.168.0.111 9090 --web.listen-address 0.0.0.0 --web.listen-port 8181 --disable-https
NOTICE: You have set the variable PYTHONPATH. DBMind ignored it.
The process is starting.
```

Step 8 Generate the DBMind configuration file. Run the following command in the **openGauss-DBMind** directory to create the **dbmindconf/dbmind.conf** configuration file in the **openGauss-DBMind** directory:

```
[omm@opengauss01 prometheus-2.51.1.linux-arm64]$ cd /home/omm/openGauss-DBMind
[omm@opengauss01 openGauss-DBMind]$ gs_dbmind service setup -c dbmindconf
```

Modify the DBMind configuration file as follows:

```
vim /home/omm/openGauss-DBMind/dbmindconf/dbmind.conf
```

The mandatory configuration items are as follows (highlighted in red):

```
[TSDB]
name = prometheus # The type of time-series database. Options: prometheus.
host = 192.168.0.111 # Address of the time series database.
port = 9090 # Port for connecting to the time series database.
username = (null) # Username for connecting to the time series database.
password = (null) # Password for connecting to the time series database.
ssl_certfile = (null) # The certificate file for ssl connections.
ssl_keyfile = (null) # Certificate private key file.
ssl_keyfile_password = (null) # Password for ssl keyfile.
ssl_ca_file = (null)    # CA certificate to validate requests.

[METADATABASE]
dbtype = opengauss # Database type. Options: sqlite, opengauss, postgresql.
host = 192.168.0.111 # Address of the metadatabase.
port = 26000 # Port for connecting to the metadatabase.
```

```
username = dbmind_monitor # Username for connecting to the metadatabase.
password = openEuler@1234   # Password for connecting to the metadatabase.
database = metadatabase # Name of the metadatabase to be connected.

[WORKER]
process_num = 2   # Number of worker processes on a local node. Less than or equal to zero means adaptive.

[AGENT]
master_url = # You don't need to configure this parameter, then the autodiscovery mode will be started. And all agents registered in TSDB will be found, but you need to configure a set of unified username/password/SSL connection information. Otherwise, you can type the agent URL of the master node. e.g., https://127.0.0.1:9187, if you have multiple sets of clusters, you should separate them by comma (,).
username = dbmind_monitor # Username for logging in to the monitoring database. Credential for agent. If you configure and monitor multiple sets of clusters, and the usernames and passwords of these clusters are different, you need to enter multiple configuration usernames, which are separated by commas (,), and need to correspond to the above master_url option one by one.
password = openEuler@1234 # Password for logging in to the monitoring database. Credential for agent. If you configure to monitor multiple clusters, they need to be separated by comma(,). The specific configuration method is the same as the above parameters.
ssl_certfile = (null) # The certificate file for ssl connections. This is optional. If you configure to monitor multiple clusters, they need to be separated by comma(,). The specific configuration method is the same as the above parameters.
ssl_keyfile = (null) # Certificate private key file. This is optional. If you configure to monitor multiple clusters, they need to be separated by comma(,). The specific configuration method is the same as the above parameters.
ssl_keyfile_password = (null) # Password for ssl keyfile. This is optional. If you configure to monitor multiple clusters, they need to be separated by comma(,). The specific configuration method is the same as the above parameters.
ssl_ca_file = (null)   # CA certificate to validate requests. This is optional. If you configure to monitor multiple clusters, they need to be separated by comma(,). The specific configuration method is the same as the above parameters.

[TIMED_TASK]
task = discard_expired_results,self_monitoring,daily_inspection,
weekly_inspection,slow_sql_diagnosis,knob_recommend,slow_query_killer
self_monitoring_interval = 30   # unit: second.
slow_sql_diagnosis_interval = 30   # unit: second.
index_recommend_interval = 30   # unit: second.
knob_recommend_interval = 30   # unit: second.
slow_query_killer_interval = 30   # unit: second.
discard_expired_results_interval = 30   # unit: second.

[WEB-SERVICE]
host = 192.168.0.111   # Which host to bind for web service. '0.0.0.0' means to bind all.
port = 8080   # Port for listening on the web service.
ssl_certfile = (null) # The certificate file for ssl connections.
ssl_keyfile = (null) # Certificate private key file.
ssl_keyfile_password = (null) # Password for ssl keyfile.
ssl_ca_file = (null)   # CA certificate to validate requests.

[LOG]
maxbytes = 10485760 # Default is 10Mb. Maximum size of a single log file. If maxbytes is zero, the file grows indefinitely.
```

```
backupcount = 1 # Number of backups of log files.
level = INFO    # Options: DEBUG, INFO, WARNING, ERROR.
log_directory = logs    # Directory to log.

[COMMENT]
worker = The form of executing compute-intensive tasks. Tasks can be executed locally or distributed to multiple nodes for execution.
tsdb = Configure the data source for time series data, which come from monitoring the openGauss instance.
metadatabase = Configure the database to record meta-data, which the database can store meta-data for the forecasting and diagnosis process. The database should be an openGauss instance.
self-monitoring = Set up parameters for monitoring and diagnosing openGauss instance.
self-optimization = Set up parameters for openGauss optimization.
```

Initialize the DBMind configuration. That is, 30 tables are created in the metadatabase.

```
[omm@opengauss01 openGauss-DBMind]$ cd /home/omm/openGauss-DBMind
[omm@opengauss01 openGauss-DBMind]$ gs_dbmind service setup --initialize -c dbmindconf
NOTICE: You have set the variable PYTHONPATH. DBMind ignored it.
Starting to generate a dynamic config file...
WARN: default PostgreSQL connector (psycopg2-binary) does not support openGauss.
It would help if you compiled psycopg2 with openGauss manually or created a connection user after setting the GUC password_encryption_type to 1.
Notice: Without explicitly setting agent configurations, the automatic detection mechanism is used.
Starting to encrypt the plain-text passwords in the config file...
Starting to initialize and check the essential variables...
Starting to connect to meta-database and create tables...
The setup process finished successfully.
```

You can log in to the openGauss database to check whether 30 tables are generated.

```
[omm@opengauss01 openGauss-DBMind]$ gsql -d metadatabase -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:08:05 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

metadatabase=# \d
                                                 List of relations
 Schema |                          Name                          |    Type    |       Owner        |
Storage

--------+--------------------------------------------------------+----------+----------------+--------------------------
--------
 public | tb_existing_indexes                                    | table      | dbmind_monitor |
{orientation=row,compress
ion=no}
 public | tb_existing_indexes_id_seq                             | sequence | dbmind_monitor |
 public | tb_future_alarms                                       | table      | dbmind_monitor |
{orientation=row,compress
ion=no}
 public | tb_future_alarms_future_alarm_id_seq                   | sequence | dbmind_monitor |
 public | tb_healing_records                                     | table      | dbmind_monitor |
{orientation=row,compress
ion=no}
```

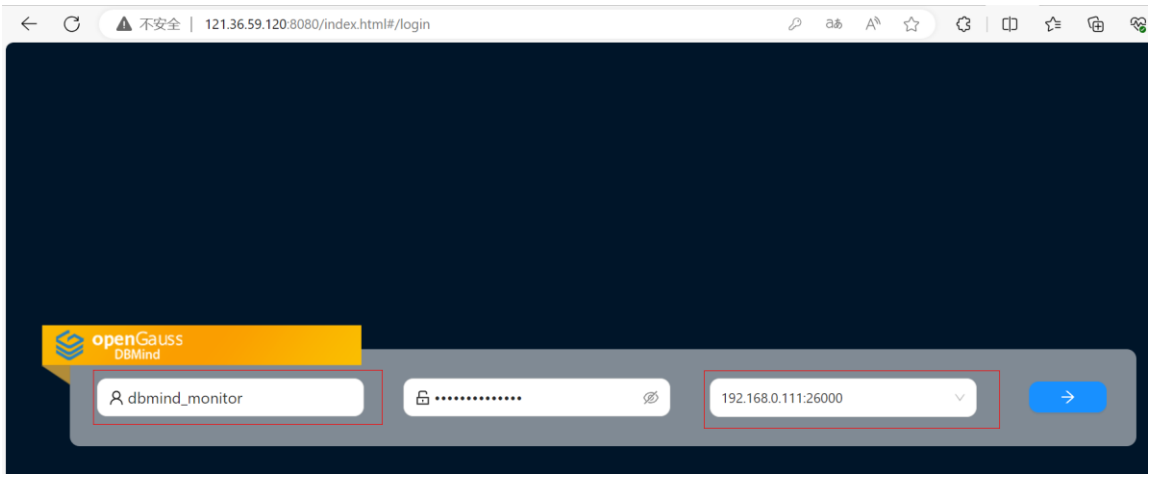
Step 9 Start the DBMind service.

```
metadatabase-# \q
[omm@opengauss01 openGauss-DBMind]$ gs_dbmind service start -c dbmindconf
NOTICE: You have set the variable PYTHONPATH. DBMind ignored it.
The process is starting.
```

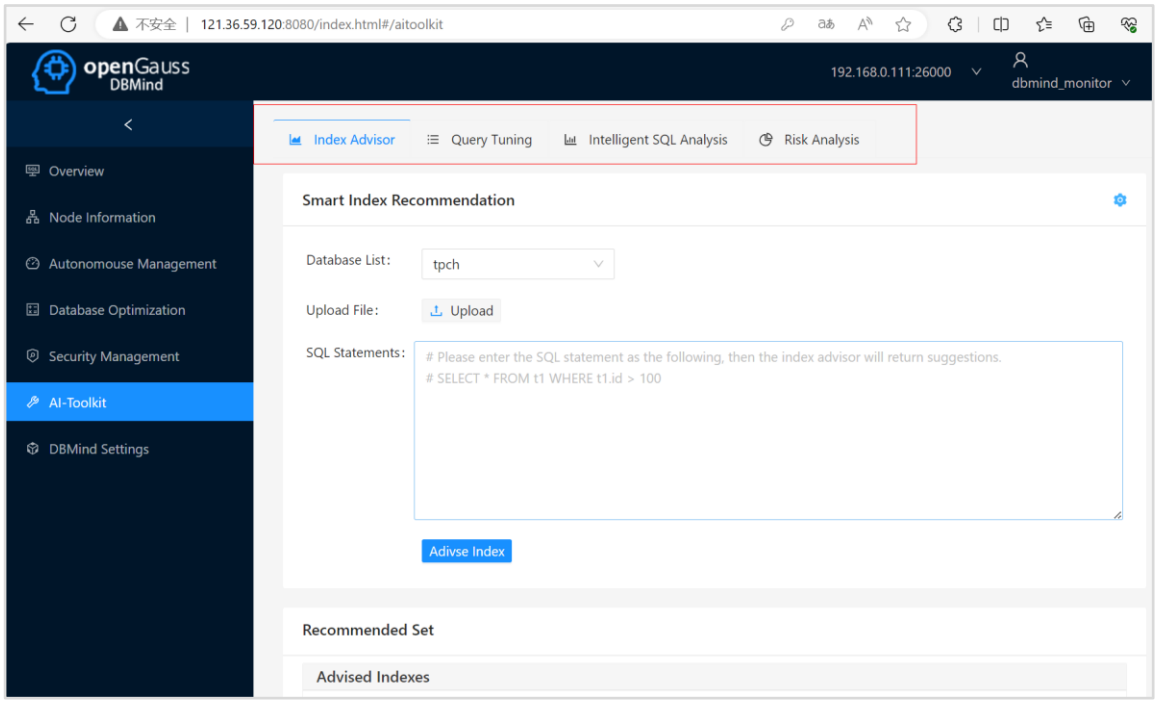
Check whether port **8080** is listened on.

```
[omm@opengauss01 openGauss-DBMind]$ netstat -anp|grep 8080
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
tcp        0      0 192.168.0.111:8080      0.0.0.0:*               LISTEN      28631/DBMind
[Maste
```

Open the browser and access http://*public IP address*:8080/. In normal cases, the following page is displayed.



Click **AI-Toolkit** on the left navigation bar and view functions such as **Index Advisor** and **Intelligent SQL Analysis** on the right pane. You can try to create a table in the database and insert data before using these functions. For example, you can enter SQL statements in the **SQL Statements** text box, and click **Advise Index** to generate a recommended index. You can also run **select * from gs_index_advise ('sql statement')** in the database to generate a recommended index.



That's all about how to download and use the AI capabilities.

7.2.2 Using the Native DB4AI Engine to Train a House Price Prediction Model

Step 1 Create a table named **houses** in the database for data training and prediction.

Log in to the database as user **omm**.

```
[omm@opengauss01 data]$ gsql -d postgres -p 26000 -r
```

Step 2 Insert data into the table.

```
openGauss=# DROP TABLE IF EXISTS houses;
openGauss=# CREATE TABLE houses (id INT, tax INT, bedroom INT, bath FLOAT, price INT,   size INT, lot INT);
INSERT INTO houses VALUES
(1 ,  590 ,        2 ,     1 ,  50000 ,   770 , 22100),
(2 , 1050 ,        3 ,     2 ,  85000 , 1410 , 12000),
(3 ,   20 ,        3 ,     1 ,  22500 , 1060 ,  3500),
(4 ,  870 ,        2 ,     2 ,  90000 , 1300 , 17500),
(5 , 1320 ,        3 ,     2 , 133000 , 1500 , 30000),
(6 , 1350 ,        2 ,     1 ,  90500 ,  820 , 25700),
(7 , 2790 ,        3 ,   2.5 , 260000 , 2130 , 25000),
(8 ,  680 ,        2 ,     1 , 142500 , 1170 , 22000),
(9 , 1840 ,        3 ,     2 , 160000 , 1500 , 19000),
(10 , 3680 ,       4 ,     2 , 240000 , 2790 , 20000),
(11 , 1660 ,       3 ,     1 ,  87000 , 1030 , 17500),
(12 , 1620 ,       3 ,     2 , 118600 , 1250 , 20000),
(13 , 3100 ,       3 ,     2 , 140000 , 1760 , 38000),
(14 , 2070 ,       2 ,     3 , 148000 , 1550 , 14000),
```

```
(15 ,  650 ,       3 ,  1.5 ,  65000 , 1450 , 12000);
```

Step 3 Run the **\d** command to check the structure of the newly created table **houses**.

```
openGauss=# \d houses
```

The table structure information is as follows:

```
openGauss=# \d houses
         Table "public.houses"
 Column  |       Type       | Modifiers
---------+------------------+-----------
 id      | integer          |
 tax     | integer          |
 bedroom | integer          |
 bath    | double precision |
 price   | integer          |
 size    | integer          |
 lot     | integer          |
```

Step 4 Run the **CREATE MODEL** statement to create a binary model based on the SVM algorithm.

```
openGauss=# CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath, size TARGET price < 100000 FROM houses;
```

In the preceding command:

The CREATE MODEL statement is used to train and save a model.

USING specifies the algorithm name.

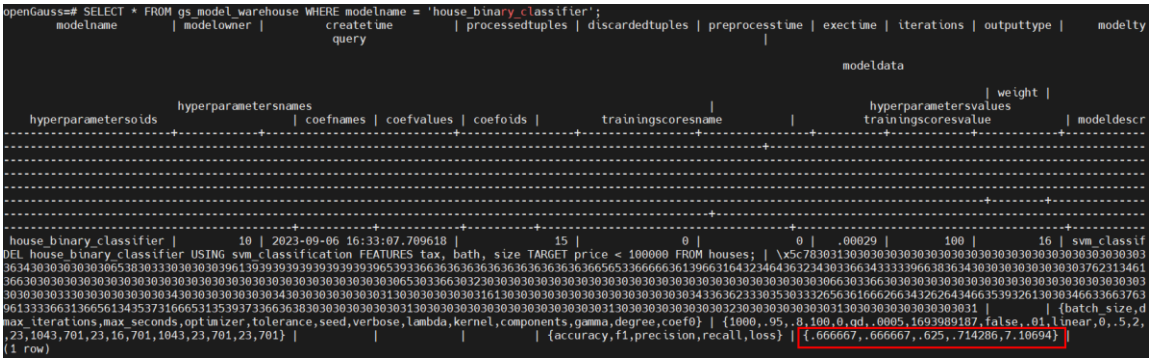
FEATURES specifies the features of the training model and needs to be added based on the column name of the training data table.

TARGET specifies the training target of the model. It can be the column name of the data table required for training or an expression, for example, **price > 10000**.

After the **CREATE MODEL** statement is executed, model training starts and the hyperparameter information used during the training is output.

Step 5 View the trained model information in the gs_model_warehouse system catalog and take a screenshot of the execution result.

```
openGauss=# SELECT * FROM gs_model_warehouse WHERE modelname = 'house_binary_classifier';
```



The information returned from the system catalog shows that the prediction accuracy during the training process is 0.666667.

It is not an ideal accuracy result for binary classification. We will try to improve the model performance by tuning hyperparameters.

Step 6 Run the **DROP TABLE** statement to drop the model.

```
openGauss=# DROP MODEL house_binary_classifier;
```

Step 7 Tune hyperparameters to improve the training performance of the SVM algorithm.

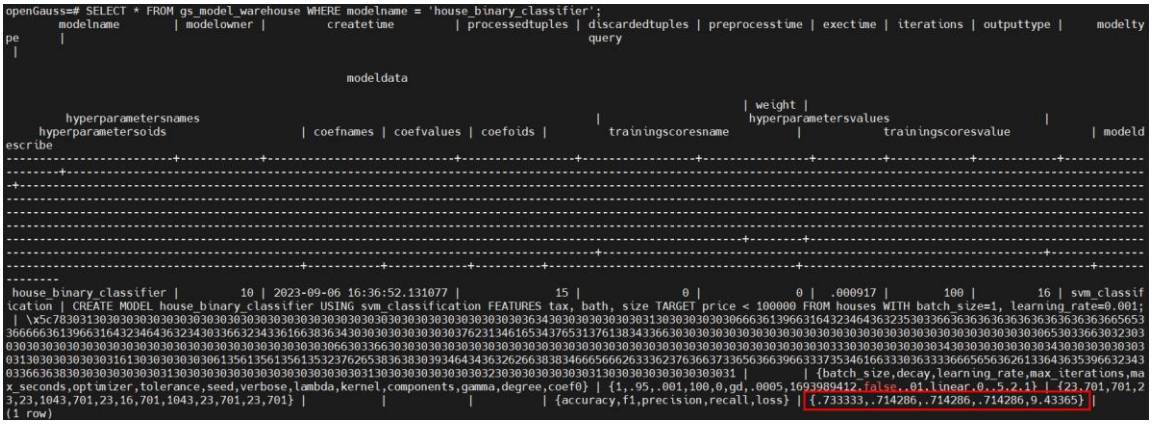
```
openGauss=# CREATE MODEL house_binary_classifier USING svm_classification FEATURES tax, bath,
size TARGET price < 100000 FROM houses WITH batch_size=1, learning_rate=0.001;
```

In the preceding command:

WITH specifies the hyperparameters used for model training. When no hyperparameter is set, the framework uses the default value. In this step, set **learning_rate** to **0.001** and **batch_size** to **1**. Observe the information about the new model and take a screenshot of the execution result.

```
openGauss=# SELECT * FROM gs_model_warehouse WHERE modelname = 'house_binary_classifier';
```

In the returned information, you can see the performance of the new model.



The information returned from the system catalog shows that the prediction accuracy during the training process is 0.733333.

The overall accuracy is improved compared to that using the default hyperparameters.

Step 8 Run the **PREDICT BY** statement to predict sample data.

```
openGauss=# SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY
house_binary_classifier (FEATURES tax, bath, size) AS price_pred FROM houses;
```

tax	bath	size	price	price_actual	price_pred
590	1	770	50000	t	t
1050	2	1410	85000	t	t
20	1	1060	22500	t	t
870	2	1300	90000	t	t
1320	2	1500	133000	f	t
1350	1	820	90500	t	f
2790	2.5	2130	260000	f	f
680	1	1170	142500	f	t
1840	2	1500	160000	f	f
3680	2	2790	240000	f	f
1660	1	1030	87000	t	f
1620	2	1250	118600	f	f
3100	2	1760	140000	f	f
2070	3	1550	148000	f	f
650	1.5	1450	65000	t	t

(15 rows)

The output in the **Price_pred** column is the prediction result, and the output in the **price_actual** column is the actual value.

Step 9 Run the **CREATE MODEL** statement to create a logistic regression model.

```
openGauss=# CREATE MODEL house_logistic_classifier USING logistic_regression FEATURES tax, bath,
size target price < 100000 FROM houses WITH learning_rate=0.001;
```

Step 10 Use the trained logistic regression model to predict data, compare the result with that using the SVM algorithm, and take a screenshot of the execution result.

```
openGauss=# SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY
house_binary_classifier (FEATURES tax, bath, size) AS price_svm_pred, PREDICT BY
house_logistic_classifier (FEATURES tax, bath, size) AS price_logistic_pred FROM houses;
```

```
openGauss=# SELECT tax, bath, size, price, price < 100000 AS price_actual, PREDICT BY house_binary_cl
assifier (FEATURES tax, bath, size) AS price_svm_pred, PREDICT BY house_logistic_classifier (FEATURES
 tax, bath, size) AS price_logistic_pred FROM houses;
 tax  | bath | size | price  | price_actual | price_svm_pred | price_logistic_pred
------+------+------+--------+--------------+----------------+---------------------
  590 |    1 |  770 |  50000 | t            | t              | t
 1050 |    2 | 1410 |  85000 | t            | t              | t
   20 |    1 | 1060 |  22500 | t            | t              | t
  870 |    2 | 1300 |  90000 | t            | t              | t
 1320 |    2 | 1500 | 133000 | f            | t              | t
 1350 |    1 |  820 |  90500 | t            | f              | f
 2790 |  2.5 | 2130 | 260000 | f            | f              | f
  680 |    1 | 1170 | 142500 | f            | t              | t
 1840 |    2 | 1500 | 160000 | f            | f              | f
 3680 |    2 | 2790 | 240000 | f            | f              | f
 1660 |    1 | 1030 |  87000 | t            | f              | f
 1620 |    2 | 1250 | 118600 | f            | f              | f
 3100 |    2 | 1760 | 140000 | f            | f              | f
 2070 |    3 | 1550 | 148000 | f            | f              | f
  650 |  1.5 | 1450 |  65000 | t            | t              | t
(15 rows)
```

7.2.3 Using DB4AI-Snapshots to Manage Dataset Versions

Step 1 Create a table and insert data into it.

Log in to the database as user **omm**.

```
[omm@opengauss01 data]$ gsql -d postgres -p 26000 -r
```

Create a data table named **t1** and insert test data into the table.

```
openGauss=# drop table t1 ;
openGauss=# create table t1 (id int, name varchar);
openGauss=# insert into t1 values (1, 'zhangsan');
openGauss=# insert into t1 values (2, 'lisi');
openGauss=# insert into t1 values (3, 'wangwu');
openGauss=# insert into t1 values (4, 'lisa');
openGauss=# insert into t1 values (5, 'jack');
```

Run the following SQL statement to query the content of the data table:

```
SELECT * FROM t1;
```

```
openGauss=# SELECT * FROM t1;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

Step 2 Create a DB4AI snapshot.

The default version separator is an at sign (@), and the default subversion separator is a period (.). You can set the separator by setting the **db4ai_snapshot_version_delimiter** and **db4ai_snapshot_version_separator** parameters.

```
openGauss=# create snapshot s1@1.0 comment is 'first version' as select * from t1;
```

```
openGauss=# create snapshot s1@1.0 comment is 'first version' as select * from t1;
 schema |  name
--------+--------
 public | s1@1.0
(1 row)
```

The command output indicates that a snapshot has been created for data table **s1**, and the snapshot version is **1.0**. A created data table snapshot can be queried in the same way as a common view but cannot be updated using the INSERT INTO statement. You can use any of the following statements to query the content of data table snapshot **s1** of version 1.0:

```
openGauss=# SELECT * FROM s1@1.0;
openGauss=# SELECT * FROM public.s1@1.0;
openGauss=# SELECT * FROM public . s1 @ 1.0;
```

```
openGauss=# SELECT * FROM s1@1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)

openGauss=# SELECT * FROM public.s1@1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)

openGauss=# SELECT * FROM public . s1 @ 1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

Step 3 You can run the following SQL statement to modify the content of the **t1** data table:

```
openGauss=# UPDATE t1 SET name = 'tom' where id = 4;
openGauss=# insert into t1 values (6, 'john');
openGauss=# insert into t1 values (7, 'tim');
```

```
openGauss=# UPDATE t1 SET name = 'tom' where id = 4;
UPDATE 1
openGauss=# insert into t1 values (6, 'john');
INSERT 0 1
openGauss=# insert into t1 values (7, 'tim');
INSERT 0 1
```

Then, retrieve the content corresponding to **s1** of version 1.0.

```
openGauss=# SELECT * FROM s1@1.0;
```

```
openGauss=# SELECT * FROM s1@1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)

openGauss=# SELECT * FROM public.s1@1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)

openGauss=# SELECT * FROM public . s1 @ 1.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

Although the content of data table **t1** has changed, the query result of data table snapshot **s1@1.0** will not change. Because data in data table **t1** has changed, to use the content of the current data table as version 2.0, you can create snapshot **s1@2.0**.

Step 4 Create snapshot **s1@2.0**.

```
openGauss=# create snapshot s1@2.0 as select * from t1;
```

```
openGauss=# create snapshot s1@2.0 as select * from t1;
 schema |  name
--------+--------
 public | s1@2.0
(1 row)
```

Then, retrieve the content corresponding to **s1** of version 2.0.

```
openGauss=# SELECT * FROM s1@2.0;
```

```
openGauss=# SELECT * FROM s1@2.0;
 id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  5 | jack
  4 | tom
  6 | john
  7 | tim
(7 rows)
```

According to the preceding example, the data table snapshot can solidify the content of a data table to avoid instability during training of a machine learning model caused by

data modification in the process. It can also prevent a lock conflict that is caused when multiple users access and modify the same table at the same time.

You can run an SQL statement to inherit a created data table snapshot and generate a new data table snapshot based on data modification.

Step 5 Create snapshot **s1@3.0**.

You can run an SQL statement to inherit a created data table snapshot and generate a new data table snapshot based on data modification.

```
openGauss=# create snapshot s1@3.0 from @1.0 comment is 'inherits from @1.0' using (INSERT VALUES(6, 'john'), (7, 'tim'); DELETE WHERE id = 1);
```

Based on the **s1@1.0** snapshot, insert two pieces of data and delete one piece of data to generate a new snapshot **s1@3.0**. Then, retrieve **s1@3.0**.

```
openGauss=# SELECT * FROM s1@3.0;
```

```
openGauss=# create snapshot s1@3.0 from @1.0 comment is 'inherits from @1.0' using (INSERT VALUES(6, 'john'), (7, 'tim'); DELETE WHERE id = 1);
 schema |  name
--------+--------
 public | s1@3.0
(1 row)

openGauss=# SELECT * FROM s1@3.0;
 id |  name
----+--------
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
  6 | john
  7 | tim
(6 rows)
```

Step 6 Delete the data table snapshot **SNAPSHOT**.

```
openGauss=# purge snapshot s1@3.0;
```

```
openGauss=# purge snapshot s1@3.0;
 schema |  name
--------+--------
 public | s1@3.0
(1 row)
```

Step 7 Sample from snapshots of a data table.

```
openGauss=# sample snapshot s1@2.0 stratify by name as nick at ratio .5;
```

```
openGauss=# sample snapshot s1@2.0 stratify by name as nick at ratio .5;
 schema |    name
--------+------------
 public | s1nick@2.0
(1 row)
```

You can use this feature to create a training set and a test set. For example:

```
openGauss=# SAMPLE SNAPSHOT s1@2.0   STRATIFY BY name AS _test AT RATIO .2, AS _train AT RATIO .8 COMMENT IS 'training';
```

```
openGauss=# SAMPLE SNAPSHOT s1@2.0  STRATIFY BY name AS _test AT RATIO .2, AS _train AT RATIO .8 COMMENT IS 'training';
 schema |     name
--------+-------------
 public | s1_test@2.0
 public | s1_train@2.0
(2 rows)
```

Step 8 Publish a data table snapshot.

Run the following statement to mark a data table snapshot as archived:

```
openGauss=# archive snapshot s1@2.0;
```

```
openGauss=# archive snapshot s1@2.0;
 schema |  name
--------+--------
 public | s1@2.0
(1 row)
```

You can query the db4ai-snapshots view to view the status of the current data table snapshot and other information.

```
openGauss=# select * from db4ai.snapshot;
```

```
openGauss=# select * from db4ai.snapshot;
 id | parent_id | matrix_id | root_id | schema |     name     | owner |          commands          |    comment    | published | archived |          created          | row_count
----+-----------+-----------+---------+--------+--------------+-------+----------------------------+---------------+-----------+----------+---------------------------+-----------
  0 |           |           |       0 | public | s1@1.0       | omm   | {"select *","from t1",NULL} | first version | t         | f        |          55:53.699935 |         5
  3 |         1 |           |       1 | public | s1nick@2.0   | "omm" | {"SAMPLE nick .5 {name}"}  |               | f         | f        |          11:03.345174 |         5
  8 |         1 |           |       1 | public | s1_test@2.0  | "omm" | {"SAMPLE _test .2 {name}"} |               | f         | f        |          15:41.876365 |         3
  9 |         1 |           |       1 | public | s1_train@2.0 | "omm" | {"SAMPLE _train .8 {name}"} | training      | f         | f        |          15:41.94506  |         6
  1 |           |           |       1 | public | s1@2.0       | omm   | {"select *","from t1",NULL} |               | f         | t        |          59:43.974823 |         7
(5 rows)
```

Step 9 Query the db4ai.snapshot view to view the status of the current data table in the DB4AI schema and take a screenshot of the execution result.

```
openGauss=# \d db4ai.snapshot
```

```
openGauss=# \d db4ai.snapshot
                  Table "db4ai.snapshot"
  Column   |            Type             |         Modifiers
-----------+-----------------------------+---------------------------
 id        | bigint                      |
 parent_id | bigint                      |
 matrix_id | bigint                      |
 root_id   | bigint                      |
 schema    | name                        | not null
 name      | name                        | not null
 owner     | name                        | not null
 commands  | text[]                      | not null
 comment   | text                        |
 published | boolean                     | not null default false
 archived  | boolean                     | not null default false
 created   | timestamp without time zone | default pg_systimestamp()
 row_count | bigint                      | not null
Indexes:
    "snapshot_pkey" PRIMARY KEY, btree (schema, name) TABLESPACE pg_default
    "snapshot_id_key" UNIQUE CONSTRAINT, btree (id) TABLESPACE pg_default
```

7.3 Fully-encrypted Database

7.3.1 Overview

A fully encrypted database aims to protect privacy throughout the data lifecycle. Data is always encrypted during transmission, computing, and storage regardless of the service scenario or environment. After the data owner encrypts data on a client and sends the encrypted data to the server, attackers cannot obtain valuable information even if they steal user data by exploiting system vulnerabilities. This ensures data privacy is protected.

The applications of the fully-encrypted database include: key creation, encrypted table creation, and fully-encrypted equality query.

7.3.2 Using gsql to Operate a Fully-encrypted Database to Get Familiar with Its Applications

7.3.2.1 Key Creation

Step 1 Log in to the host as the OS user **omm**.

Before creating a key, you need to plan the local key storage path and ensure that the database user has the permission to edit the path. Configure the path in the environment variable configuration file of the database user and make the environment variable take effect. Add **export LOCALKMS_FILE_PATH=/opt/software/openGauss** to the end of the **/etc/profile** file.

```
[root@opengauss1 script]# vi /etc/profile
export LOCALKMS_FILE_PATH=/opt/software/openGauss
[root@opengauss1 script]# source /etc/profile
[root@opengauss1 script]# chmod 777 -R /opt/software/openGauss/
```

Step 2 Enable the encryption function and connect to the encrypted database. **-C** indicates that the encryption function is enabled.

```
[root@opengauss1 script]# su omm
[omm@opengauss01 ~]$ gsql -p 26000    postgres -r -C
```

Step 3 Create a CMK and a CEK. A CMK is a master key used to encrypt a CEK. A CEK is a data key used to encrypt data. The creation of a CEK depends on its CMK. Therefore, you need to create a CMK before creating a CEK. Note: Currently, **KEY_STORE** can only be set to **localkms**, and **ALGORITHM** can be set to **RSA_2048**, **RSA3072**, or **SM2**. SM2, SM3, and SM4 are standard Chinese cryptographic algorithms. To avoid legal risks, they must be used together. If you specify the SM4 algorithm to encrypt CEKs when creating a CMK, you must specify the SM3 and SM4 algorithms (SM4_SM3) to encrypt data when creating CEKs.

```
-- Create a CMK.
openGauss=# CREATE CLIENT MASTER KEY ImgCMK1 WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value1", ALGORITHM = RSA_2048);
openGauss=# CREATE CLIENT MASTER KEY ImgCMK WITH (KEY_STORE = localkms, KEY_PATH =
"key_path_value2", ALGORITHM = RSA_2048);

After the command is executed successfully, a key file is generated on the local PC as follows (the file
starts with key_path):
[omm@opengauss1 script]$ ll /opt/software/openGauss/
```

```
[omm@opengauss01 root]$  ll /opt/software/openGauss/
total 520M
-rwxrwxrwx  1 omm dbgrp 1.2K Mar 29 21:33 cluster_config.xml
-rwxrwxrwx  1 omm dbgrp 155M Mar 29  2023 dbmind-installer-aarch64-python3.10.sh
-rw-------  1 omm dbgrp 1.8K Apr  1 10:04 key_path_value1.priv
-rw-------  1 omm dbgrp   84 Apr  1 10:04 key_path_value1.priv.rand
-rw-------  1 omm dbgrp  536 Apr  1 10:04 key_path_value1.pub
-rw-------  1 omm dbgrp   84 Apr  1 10:04 key_path_value1.pub.rand
-rw-------  1 omm dbgrp 1.8K Apr  1 10:05 key_path_value2.priv
-rw-------  1 omm dbgrp   84 Apr  1 10:05 key_path_value2.priv.rand
-rw-------  1 omm dbgrp  536 Apr  1 10:05 key_path_value2.pub
-rw-------  1 omm dbgrp   84 Apr  1 10:05 key_path_value2.pub.rand
drwxrwxrwx 15 omm dbgrp 4.0K Mar 29 21:35 lib
drwxrwxrwx  9 omm dbgrp 4.0K Mar 29 21:36 libcgroup
-rwxrwxrwx  1 omm dbgrp 121M Mar 29  2023 openGauss-5.0.0-openEuler-64bit-all.tar.gz
```

```
-- Create a CEK.
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY =
ImgCMK1, ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK WITH VALUES (CLIENT_MASTER_KEY =
ImgCMK, ALGORITHM  = AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```

Step 4 The query result of the system catalog that stores the key information is as follows.

```
openGauss=# SELECT * FROM gs_client_global_keys;
 global_key_name | key_namespace | key_owner | key_acl |          create_date
-----------------+---------------+-----------+---------+---------------------------
 imgcmk1         |          2200 |        10 |         | 2021-04-21 11:04:00.656617
 imgcmk          |          2200 |        10 |         | 2021-04-21 11:04:05.389746
(2 rows)

openGauss=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner  FROM
gs_column_keys;
 column_key_name | column_key_distributed_id | global_key_id | key_owner
-----------------+---------------------------+---------------+-----------
 imgcek1         |                 760411027 |         16392 |        10
 imgcek          |                3618369306 |         16398 |        10
(2 rows)
```

7.3.2.2 Encrypted Table Creation

```
openGauss=# CREATE TABLE creditcard_info (id_number      int, name            text encrypted with
(column_encryption_key = ImgCEK, encryption_type = DETERMINISTIC),credit_card   varchar(19)
encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));
```

Step 1 Query the detailed information about the table. If the value of **Modifiers** is **encrypted**, the queried columns are encrypted. Insert data into the encrypted table.

```
openGauss=# \d creditcard_info
          Table "public.creditcard_info"
   Column     |         Type          | Modifiers
--------------+-----------------------+-----------
 id_number    | integer               |
 name         | text                  |  encrypted
```

```
 credit_card | character varying |    encrypted

openGauss=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
```

7.3.2.3 Fully-encrypted Equality Query

Step 1 Enable the encryption function.

To enable the encryption function, specify the **-C** parameter for connection. When the JDBC is used for connection, set **enable_ce** to **1**.

When the encryption function is enabled, data can be queried normally as follows:

```
openGauss=# select * from creditcard_info where name = 'joe';
 id_number | name |        credit_card
-----------+------+--------------------
           1 | joe    | 6217986500001288393
(1 row)

openGauss=# select id_number,name from creditcard_info;
 id_number | name
-----------+------
           1 | joe
           2 | joy
(2 rows)
```

Step 2 Disable the encryption function.

Run **gsql -p 26000 -d postgres -r**. The query result is displayed in ciphertext.

Note: When a non-encrypted client is used (with the **-C** parameter not specified) to view the encrypted table data, the data is displayed in ciphertext.

```
openGauss=# \q
[omm@opengauss01j openGauss-DBMind]$ gsql -p 26000 postgres -r
openGauss=# select id_number,name from creditcard_info;
 id_number |                                                
```

```
openGauss=# ALTER TABLE creditcard_info ADD COLUMN age int ENCRYPTED WITH
(COLUMN_ENCRYPTION_KEY = ImgCEK, ENCRYPTION_TYPE = DETERMINISTIC);
ALTER TABLE

openGauss=# \d creditcard_info
           Table "public.creditcard_info"
    Column      |           Type            | Modifiers
-------------+-------------------+------------
 id_number     | integer                   |
 name          | text                      |   encrypted
 credit_card | character varying |   encrypted
 age           | integer                   |   encrypted

openGauss=# ALTER TABLE creditcard_info DROP COLUMN age;
ALTER TABLE

openGauss=# update creditcard_info set credit_card = '80000000011111111' where name = 'joy';
UPDATE 1

openGauss=# select * from creditcard_info   where name = 'joy';
 id_number | name |      credit_card
-----------+------+-------------------
           2 | joy   | 80000000011111111
(1 row)
```

Note that a user with the query permission cannot query encrypted data even if the encryption function is enabled. The error message "ERROR(CLIENT): failed to decrypt column encryption key" is displayed. The edit permission necessary. However, it is normal to query non-encrypted columns.

```
openGauss=> select * from creditcard_info;
ERROR(CLIENT): failed to decrypt column encryption key
```

Encrypted databases provide more choices for protecting users' sensitive privacy data. As information protection and data security become more and more important, encrypted databases will serve as an important feature of openGauss and provide important technical foundation and support for the growth of openGauss. That's all about the application of openGauss fully-encrypted database.

7.3.3 Creating and Executing a Function or Stored Procedure That Involves Encrypted Columns

Step 1 Create a column-encrypted table.

```
openGauss=# CREATE TABLE creditcard_info1 (
    id_number int,
    name text,
    credit_card varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type =
DETERMINISTIC)
) WITH (orientation=row);
CREATE TABLE
```

Step 2 Insert data.

```
openGauss=# insert into creditcard_info1 values(1, 'Avi', '1234567890123456');
INSERT 0 1
openGauss=#    insert into creditcard_info1 values(2, 'Eli', '2345678901234567');
INSERT 0 1
```

Step 3 Create a function supporting encrypted equality query.

```
openGauss=# CREATE FUNCTION f_encrypt_in_sql(val1 text, val2 varchar(19)) RETURNS text AS
'SELECT name from creditcard_info1 where name=$1 or credit_card=$2 LIMIT 1' LANGUAGE SQL;
CREATE FUNCTION
```

This function receives two parameters: **val1** of the text type and **val2** of the varchar(19) type. The return type of the function is also text. The function executes an SQL query to search the **creditcard_info1** table for the first record whose **name** field is **val1** or **credit_card** field is **val2**, and returns the value of the **name** field.

```
openGauss=# CREATE FUNCTION f_encrypt_in_plpgsql (val1 text, val2 varchar(19))
openGauss-# RETURNS text AS $$
openGauss$# DECLARE
openGauss$# c text;
openGauss$# BEGIN
openGauss$#    SELECT into c name from creditcard_info1 where name=$1 or credit_card =$2 LIMIT 1;
openGauss$#     RETURN c;
openGauss$# END; $$
openGauss-# LANGUAGE plpgsql;
CREATE FUNCTION
```

Similar to f_encrypt_in_sql, this function receives two parameters and returns the **name** field value of the matched record. However, it is written in the PL/pgSQL language, uses the variable **c** to store the query result, and then returns the variable.

Step 4 Execute the function. After the two functions are created, call them twice for testing:

```
openGauss=# SELECT f_encrypt_in_sql('Avi','1234567890123456');
 f_encrypt_in_sql
------------------
 Avi
(1 row)
```

In this query, **'Avi'** and **'1234567890123456'** are passed to the function as parameters. The function returns the value of the **name** field in the first matching condition in the **creditcard_info1** table. The result is **'Avi'**.

```
openGauss=# SELECT f_encrypt_in_plpgsql('Avi', val2=>'1234567890123456');
 f_encrypt_in_plpgsql
----------------------
 Avi
(1 row)
```

This query also passes the same parameters. However, because the f_encrypt_in_plpgsql function accepts named parameters, the name of the second parameter **val2** is explicitly specified. The return result is also **'Avi'**.

The preceding piece of code is used to create two functions in the openGauss encrypted database and test the entire process of calling the functions.

Precautions:

Because the query, that is, the dynamic query statement executed in a function or stored procedure, is compiled during execution, the table name and column name in the function or stored procedure must be known in the creation phase. The input parameter cannot be used as a table name or column name, or any connection mode.

Among the **RETURNS**, **IN**, and **OUT** parameters, encrypted and non-encrypted parameters cannot be used together. Although the parameter types are all original, the actual types are different.

In advanced package APIs, for example, dbe_output.print_line(), decryption is not performed on the APIs whose output is printed on the server. This is because when the encrypted data type is forcibly converted into the original plaintext data type, the default value of the data type is printed.

In the current version, **LANGUAGE** of functions and stored procedures can only be **SQL** and **PL/pgSQL**, and does not support other procedural languages such as C and Java.

Other functions or stored procedures for querying encrypted columns cannot be executed in a function or stored procedure.

In the current version, default values cannot be assigned to variables in DEFAULT or DECLARE statements, and return values in DECLARE statements cannot be decrypted. You can use input parameters and output parameters instead when executing functions.

gs_dump cannot be used to back up functions involving encrypted columns.

Keys cannot be created in functions or stored procedures.

In this version, encrypted functions and stored procedures do not support triggers.

Encrypted equality query functions and stored procedures do not support the escape of the PL/pgSQL syntax. The CREATE FUNCTION AS *'Syntax body'* syntax whose syntax body is marked with single quotation marks ('') can be replaced with the CREATE FUNCTION AS *'Syntax body'* syntax.

The definition of an encrypted column cannot be modified in an encrypted equality query function or stored procedure, including creating an encrypted table and adding an encrypted column. Because the function is executed on the server, the client cannot determine whether the cache needs to be refreshed. The column can be encrypted only after the client is disconnected or the cache of the encrypted column on the client is refreshed.

7.4 MOT Management

7.4.1 Deleting/Creating an MOT

Step 1 The following describes how to grant a database user the permission to access the MOT storage engine. This operation is performed only once for each database user typically during initial configuration. To enable a user to create and access MOTs (through DDL, DML, and SELECT operations), execute the following statement only once:

```
openGauss=# create user cy password 'openGauss@123';
openGauss=# GRANT USAGE ON FOREIGN SERVER mot_server TO cy;
```

It is easy to create an MOT. Except for the differences in the statements for creating and deleting MOTs compared to disk-based tables in openGauss, the syntax for all other SELECT, DML, and DDL commands is identical for both MOTs and disk-based tables. Before creating an MOT, set **enable_incremental_checkpoint** to **off**.

Step 2 Disable incremental checkpointing.

```
openGauss=# alter system set enable_incremental_checkpoint='off';
```

Step 3 Restart the system for the modification to take effect.

```
openGauss=# \q
[omm@opengauss01 root]$ gs_om -t restart
[omm@opengauss01 root]$ gsql -p 26000    postgres -r
openGauss=# show enable_incremental_checkpoint;
 enable_incremental_checkpoint
-------------------------------
 off
(1 row)
```

Step 4 Create an MOT.

```
openGauss=# create foreign table mot_order (
    o_id              integer         not null,
    o_d_id            integer         not null,
    o_w_id            integer         not null,
    o_c_id            integer not null,
    o_entry_d         timestamp,
        o_carrier_id integer,
    o_ol_cnt          integer,
    o_all_local    integer,
    primary key (o_w_id, o_d_id, o_id)
 );
NOTICE:   CREATE FOREIGN TABLE / PRIMARY KEY will create constraint "mot_order_pkey" for
foreign table "mot_order"
CREATE FOREIGN TABLE
```

Step 5 Generate test data.

```
openGauss=# insert into mot_order select number, (number % 20) +
1,ceil(random()*20000+1),ceil(random()*96000+1),'2020-12-02 00:00:00'::timestamp + number *
interval '1 second',(number % 10),(number % 10)+5, ceil(random()*2) FROM
generate_series(1,200000) as number;
INSERT 0 200000
```

Step 6 Query and view the execution plan.

```
openGauss=# explain select * from mot_order where o_w_id=2 limit 4;
                                    QUERY PLAN
-------------------------------------------------------------------------
 Limit  (cost=0.00..0.02 rows=4 width=36)
   ->  Foreign Scan on mot_order  (cost=0.00..4.26 rows=1000 width=36)
         ->  Memory Engine returned rows: 0
               ->  Index Scan on: mot_order_pkey
                     Index Cond: (mot_order.o_w_id = 2)
(5 rows)
```

Step 7 Drop the MOT named **mot_order**.

```
openGauss=# drop foreign table mot_order;
DROP FOREIGN TABLE
```

In the preceding statement:

Always use the FOREIGN keyword to reference an MOT.

When creating an MOT, **[server mot_server]** is optional because the MOT is an integrated engine, not a standalone server.

In the preceding example, an MOT named **test** (containing an integer column named **x**) is created. Another example is provided in the next section "Creating an Index for an MOT."

If incremental checkpointing is enabled in **postgresql.conf**, MOTs cannot be created. Therefore, set **enable_incremental_checkpoint** to **off** before creating an MOT.

7.4.2 Creating an Index for an MOT

It is not necessary to specify the FOREIGN keyword before the MOT name because it is used only in the commands for creating and deleting tables. Create an **ORDER** table for TPC-C benchmark testing and create an index.

```
create FOREIGN table bmsql_oorder (
  o_w_id       integer      not null,
  o_d_id       integer      not null,
  o_id         integer      not null,
  o_c_id       integer not null,
  o_carrier_id integer,
  o_ol_cnt     integer,
  o_all_local  integer,
  o_entry_d    timestamp,
  primary key (o_w_id, o_d_id, o_id)
```

```
);
openGauss=# create index   bmsql_oorder_index1 on bmsql_oorder(o_w_id, o_d_id, o_c_id, o_id) ;
CREATE INDEX
```

7.4.3 Converting a Disk-based Table into an MOT

Direct conversion from disk-based tables to MOTs is not yet possible, meaning that no ALTER TABLE statement is available for converting a disk-based table into an MOT. The following describes how to manually convert a disk-based table into an MOT, as well as how to use the gs_dump and gs_restore tools to export data and import data, respectively.

Pre-checks:

Check whether the schema of the disk-based table to be converted into an MOT contains all required columns.

If certain columns are not supported, create a secondary disk-based table with an updated schema. This schema is the same as the original one, except that all the unsupported types have been converted into supported types.

Afterwards, use the following script to export this disk-based table and then import it into an MOT.

Conversion example:

Let's say that you have a database named **benchmarksql** and a table named **customer** (which is a disk-based table) to be migrated to an MOT.

To migrate the **customer** table to an MOT, perform the following:

Step 1 Check your source table column types.

```
openGauss=# create table customer(x integer,y integer);
CREATE TABLE
openGauss=# insert into customer values (1,2),(3,4);
openGauss=# \d+ customer
                              Table "public.customer"
 Column |   Type    | Modifiers | Storage | Stats target | Description
--------+---------+-----------+---------+--------------+-------------
 x      | integer |           | plain   |              |
 y      | integer |           | plain   |              |
Has OIDs: no
Options: orientation=row, compression=no, storage_type=USTORE
```

Step 2 Check your source table data.

```
openGauss=# select * from customer;
 x | y
---+---
 1 | 2
 3 | 4
(2 rows)
```

Step 3 Dump table data only by using gs_dump.

```
openGauss=# \q
[omm@opengauss01 root]$ cd ~
[omm@opengauss01 ~]$    gs_dump -Fc postgres -a --table customer -f customer.dump -p 26000
gs_dump[port='26000'][postgres][20XX-03-27 15:38:30]: dump database postgres successfully
gs_dump[port='26000'][postgres][20XX-03-27 15:38:30]: total time: 1485    ms
```

Step 4 Rename the source table.

```
[omm@opengauss01 ~]$ gsql -p 26000    postgres -r
openGauss=#    alter table customer rename to customer_bk;
ALTER TABLE
```

Step 5 Create an MOT to be the same as the source table.

```
openGauss=# create foreign table customer (x int, y int);
CREATE FOREIGN TABLE
openGauss=# select * from customer;
 x | y
---+---
(0 rows)
```

Step 6 Import the source dump data into the new MOT.

```
[omm@opengauss01 ~]$ gs_restore -C -d postgres -p 26000 customer.dump
start restore operation ...
table customer complete data imported !
Finish reading 3 SQL statements!
end restore operation ...
restore operation successful
total time: 21    ms

openGauss=# select * from customer;
 x | y
---+---
 1 | 2
 3 | 4
(2 rows)

openGauss=# \d
                                    List of relations
 Schema |      Name       |       Type         | Owner   |                  Storage
--------+-----------------+--------------------+---------+-----------------------------------
 public | customer        | foreign table | aharon |
 public | customer_bk | table              | aharon | {orientation=row,compression=no}
(2 rows)
```

Step 7 Check the size of the MOT global memory, mainly data and indexes.

```
select * from mot_global_memory_detail();
```

The result is as follows:

```
numa_node   | reserved_size            | used_size
```

```
----------------+----------------+-------------
-1              | 194716368896   | 25908215808
0               | 446693376      | 446693376
1               | 452984832      | 452984832
2               | 452984832      | 452984832
3               | 452984832      | 452984832
4               | 452984832      | 452984832
5               | 364904448      | 364904448
6               | 301989888      | 301989888
7               | 301989888      | 301989888
```

numa_node: indicates the NUMA node ID. NUMA is a computer architecture in which processors can access local memory associated with them more quickly. The node ID starts from 0. The value **–1** may indicate the summary statistics of all nodes or the memory statistics of non-specific nodes.

reserved_size is the total memory reserved for the MOT engine on the NUMA node, in bytes. For example, about 194716368896 bytes (approximate to 184 GB) are reserved for the **–1** node, and tens of millions to hundreds of millions of bytes are reserved for other nodes.

used_size indicates the memory used by the MOT engine on the NUMA node, in bytes. For example, the memory used by the **–1** node is about 25908215808 bytes (approximate to 24 GB), and the memory used by other nodes is the same as or close to the reserved memory.

By querying the mot_global_memory_detail table, you can learn about the memory allocation and usage of the MOT engine on each NUMA node. As shown in the output, a large amount of memory space is reserved for the **–1** node and a certain amount of memory space is used. A small amount of memory space is reserved for other NUMA nodes, and almost all the memory space is used up.

Step 8 Check the size of the MOT local memory, including the session memory.

```
select * from mot_local_memory_detail();
numa_node | reserved_size | used_size
-----------+---------------+-----------
          -1 |      20971520 |   20971520
           0 |      20971520 |   20971520
(2 rows)
```

Details are as follows:

The value **–1** indicates the total memory.

0–7 identify NUMA memory nodes.

reserved_size: total local memory reserved for the MOT engine on the NUMA node, in bytes. The command output shows that 20971520 bytes, that is, 20 MB memory, are pre-allocated to nodes **–1** and **0**.

used_size: local memory used by the MOT engine on the NUMA node, in bytes. The command output shows that the **–1** and **0** nodes have used all the pre-allocated memory, that is, 20971520 bytes (approximate to 20 MB).

Step 9 You can query the memory usage of all active sessions (connections) as follows:

```
select * from mot_session_memory_detail();
-------------------------------------------------------------------------
sessid                          | total_size | free_size | used_size
--------------------------------+------------+-----------+-----------
1591175063.139755603855104 | 6291456      | 1800704     | 4490752
```

total_size: memory allocated to the session.

free_size: unused memory.

used_size: used memory.

Step 10 DBAs can determine the status of the local memory used by the current session through the following query:

```
select * from mot_session_memory_detail()    where sessid = pg_current_sessionid();
                sessid                 | total_size | free_size | used_size
---------------------------+------------+-----------+-----------
 1712892896.281466191649136 |      2097152 |      492598 |     1604554
(1 row)
```

sessid: session ID. In this example, the value is **1712892896.281466191649136**, indicating the database session that is being queried.

total_size: total memory size allocated to the MOT in the current session. In this example, the value is **2097152**, in bytes (approximate to 2 MB).

free_size: unused MOT memory in the current session. In this example, the value is **492598**, in bytes (approximate to 482 KB).

used_size: used MOT memory in the current session. In this example, the value is **1604554**, in bytes (approximate to 1566 KB).

To sum up, the query result shows that about 2 MB memory is allocated to the current database session in the MOT engine. About 1566 KB memory is used, and about 482 KB memory is not used. The information helps you understand the MOT memory usage of the current session and provides reference for database performance analysis and optimization.

8 Troubleshooting

8.1 Introduction

8.1.1 About This Lab

This lab offers practical exercises for *Chapter 8 Troubleshooting*.

This section describes how to reproduce common openGauss problems, handle these problems, and demonstrate the entire troubleshooting process.

8.1.2 Objectives

- Understand common troubleshooting methods, covering techniques for reproducing and addressing issues that may arise with the OS.
- Understand common troubleshooting cases that address issues such as memory stalling, disk full errors, and memory shortages that may occur during TPC-C runtime.

8.2 Tasks

8.2.1 Task 1: An Error Is Reported or Threads Cannot Be Created in High Concurrency Scenarios

This task is performed to reproduce and solve the problem that the error message "too many clients already" is displayed or threads cannot be created.

Reproduction method:

1. Modify database parameters: Change the maximum number of connections to the minimum (that is, set **max_connections** to **10**) and the number of WAL sender threads to a value less than the maximum number of connections (that is, set **max_wal_senders** to **6**).
2. Use the sysbench tool to perform a pressure test on openGauss and simulate a high-concurrency scenario.

Troubleshooting method:

1. Run **ulimit -u** to obtain the value of **max user processes** in the OS.
2. Calculate the minimum value by using a formula and modify the **/etc/security/limits.conf** file.

3. Temporarily set **max_connections** to **5000** or a value less than or equal to the upper limit (that is, **262143**).
4. (Optional) In high concurrency scenarios, enable the thread pool to control thread resources in the database.

The procedure is as follows:

Step 1 Download the sysbench pressure test tool.

Log in to the server as the **root** user, download the sysbench pressure test tool, and run the following command in the **/root** directory:

```
[root@standalone ~]# wget https://github.com/akopytov/sysbench/archive/1.0.20.zip
```

After the download is complete, you can find the **1.0.20.zip** package in the **/root** directory, which is the sysbench installation package.

Step 2 Install the sysbench pressure test tool.

Perform the following operations as the **root** user:

Before installing sysbench, you need to install the dependency tool. Before installing the dependency tool, you need to modify the **openEuler.repo** configuration file of yum by changing **gpgcheck=1** to **gpgcheck=0**. Run the following command to complete the modification:

```
[root@standalone ~]# sed -i 's/gpgcheck=1/gpgcheck=0/g' /etc/yum.repos.d/openEuler.repo
```

Install the dependency tool.

```
[root@standalone ~]# yum -y install make automake libtool pkgconfig unzip libaio-devel
[root@standalone ~]# yum -y install postgresql-devel postgresql
```

Decompress the sysbench package.

```
[root@standalone ~]# unzip 1.0.20.zip
```

Compile and install sysbench. Go to the **/root/sysbench-1.0.20** directory.

```
[root@standalone ~]# cd sysbench-1.0.20/
```

Run the **autogen.sh** script.

```
[root@standalone sysbench-1.0.20]# ./autogen.sh
./autogen.sh: running `libtoolize --copy --force'
libtoolize: putting auxiliary files in AC_CONFIG_AUX_DIR, 'config'.
libtoolize: copying file 'config/ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'm4'.
libtoolize: copying file 'm4/libtool.m4'
libtoolize: copying file 'm4/ltoptions.m4'
libtoolize: copying file 'm4/ltsugar.m4'
libtoolize: copying file 'm4/ltversion.m4'
libtoolize: copying file 'm4/lt~obsolete.m4'
```

```
./autogen.sh: running `aclocal -I m4'
./autogen.sh: running `autoheader'
./autogen.sh: running `automake -c --foreign --add-missing'
configure.ac:59: installing 'config/ar-lib'
configure.ac:40: installing 'config/compile'
configure.ac:27: installing 'config/config.guess'
configure.ac:27: installing 'config/config.sub'
configure.ac:32: installing 'config/install-sh'
configure.ac:32: installing 'config/missing'
src/Makefile.am: installing 'config/depcomp'
parallel-tests: installing 'config/test-driver'
./autogen.sh: running `autoconf'
Libtoolized with: libtoolize (GNU libtool) 2.4.6
Automade with: automake (GNU automake) 1.16.2
Configured with: autoconf (GNU Autoconf) 2.69
[root@standalone sysbench-1.0.20]#
```

This script generates a **configure** script in the current directory (**sysbench-1.0.20**), as shown in the following:

```
[root@standalone sysbench-1.0.20]# ls -l configure
-rwxr-xr-x 1 root root 637398 Apr 12 15:09 configure
[root@standalone sysbench-1.0.20]#
```

Run the **configure** script.

```
[root@standalone sysbench-1.0.20]# ./configure --prefix=/sysbench --with-pgsql --without-mysql
```

If the **configure** script is executed successfully, the following information is displayed:

```
...
config.status: executing libtool commands
==========================================================================
sysbench version     : 1.0.20
CC                   : gcc
CFLAGS               : -O2 -funroll-loops   -Wall -Wextra -Wpointer-arith -Wbad-function-cast -
Wstrict-prototypes -Wnested-externs -Wno-format-zero-length -Wundef -Wstrict-prototypes -
Wmissing-prototypes -Wmissing-declarations -Wredundant-decls -Wcast-align -Wvla     -pthread
CPPFLAGS             : -D_GNU_SOURCE   -DCK_MD_CACHELINE=128   -I$(top_srcdir)/src -
I$(abs_top_builddir)/third_party/luajit/inc -I$(abs_top_builddir)/third_party/concurrency_kit/include
LDFLAGS              :
LIBS                 : -laio -lm
EXTRA_LDFLAGS        :

prefix               : /sysbench
bindir               : ${prefix}/bin
libexecdir           : ${prefix}/libexec
mandir               : ${prefix}/share/man
datadir              : ${prefix}/share

MySQL support        : no
Drizzle support      : no
AttachSQL support    : no
Oracle support       : no
```

```
PostgreSQL support : yes

LuaJIT                : bundled
LUAJIT_CFLAGS          : -I$(abs_top_builddir)/third_party/luajit/inc
LUAJIT_LIBS           : $(abs_top_builddir)/third_party/luajit/lib/libluajit-5.1.a -ldl
LUAJIT_LDFLAGS         : -rdynamic

Concurrency Kit       : bundled
CK_CFLAGS              : -I$(abs_top_builddir)/third_party/concurrency_kit/include
CK_LIBS               : $(abs_top_builddir)/third_party/concurrency_kit/lib/libck.a
configure flags       :
================================================================================
```

After the **configure** script is executed normally, a makefile is generated.

```
[root@standalone sysbench-1.0.20]# ls -l Makefile
-rw-r--r-- 1 root root 30512 Apr 12 15:19 Makefile
[root@standalone sysbench-1.0.20]#
```

Run the **make** and **make install** commands to start compilation and installation.

```
[root@standalone sysbench-1.0.20]# make && make install
```

If the **make** and **make install** commands are executed successfully, the following information is displayed:

```
...
make[2]: Leaving directory '/root/sysbench-1.0.20/tests'
make[1]: Leaving directory '/root/sysbench-1.0.20/tests'
make[1]: Entering directory '/root/sysbench-1.0.20'
make[2]: Entering directory '/root/sysbench-1.0.20'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/root/sysbench-1.0.20'
make[1]: Leaving directory '/root/sysbench-1.0.20'
[root@standalone sysbench-1.0.20]#
```

If no error information is displayed, sysbench is successfully installed.

Step 3 Configure environment variables.

Add the following code to the end of the **/etc/profile** file, save the file, and exit:

```
export PATH=/sysbench/bin:$PATH
```

Make the configured environment variables take effect.

```
[root@standalone ~]# source /etc/profile
```

Check whether the **sysbench** command can be executed properly.

```
[root@standalone ~]# sysbench --version
sysbench 1.0.20
```

The sysbench pressure test tool has been successfully installed and the **sysbench** command has been configured in the environment variable configuration file.

Step 4 Modify the database configuration file.

Switch to user **omm**, go to the directory where the database data is stored, and modify the **postgresql.conf** file.

```
[root@standalone ~]# su - omm # Switch to user omm.
[omm@standalone ~]$ cd /opt/huawei/install/data/dn/      # Go to the directory where the configuration file is stored.
[omm@standalone dn]$ vim postgresql.conf   # Edit the configuration file.
```

In the **postgresql.conf** file, set **listen_addresses** in line 68 to *, **max_connections** in line 74 to **10**, and **max_wal_senders** in line 257 to **6**.

```
listen_addresses = '*'                  # what IP address(es) to listen on;
...
max_connections = 10                    # (change requires restart)
...
max_wal_senders = 6                     # (change requires restart)
...
```

Save the modification and exit.

Modify the **pg_hba.conf** file in the current directory.

```
[omm@standalone dn]$ vim /opt/huawei/install/data/dn/pg_hba.conf
```

To allow remote access, add the following code under "# IPv4 local connections":

```
host    all     all     0.0.0.0/0       sha256
```

After the configuration is complete, information similar to the following is displayed.

```
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
host    all     all     192.168.56.28/32    sha256
host    all     all     0.0.0.0/0    sha256
# IPv6 local connections:
```

Save the modification and exit.

Step 5 Restart the database and create a database and user for the pressure test.

Switch to user **omm** and restart the database.

```
[omm@standalone dn]$ gs_om -t restart
Stopping cluster.
=========================================
Successfully stopped cluster.
=========================================
End stop cluster.
Starting cluster.
=========================================
```

```
[SUCCESS] standalone
20XX-XX-12 15:40:24.501 6618e568.1 [unknown] 281460602372112 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
20XX-XX-12 15:40:24.502 6618e568.1 [unknown] 281460602372112 [unknown] 0 dn_6001 01000   0
[BACKEND] WARNING:   could not create any HA TCP/IP sockets
========================================
Successfully started.
```

Ensure that the restart is normal before performing the following operations. If the restart fails, rectify the fault based on the error information.

Log in to the database, create the **sbtest** database and the **sbtest** user, and grant permissions to the user.

```
[omm@standalone ~]$ gsql -d postgres -p 26000 -r
openGauss=# create user sbtest Sysadmin password 'openGauss@1234';
openGauss=# GRANT ALL PRIVILEGES TO sbtest;
openGauss=# create database sbtest;
```

After the creation is complete, exit the database.

Step 6 Reproduce the issue.

Perform the following operations as the **omm** user:

Use the sysbench tool to perform a pressure test on openGauss. The pressure test consists of two phases:

1. Prepare phase: Create a table and generate test data. First, replace **192.168.56.28** in the following command with the private IP address of your server (to check it, run the **ifconfig** command).

```
[omm@standalone ~]$ sysbench /sysbench/share/sysbench/oltp_read_write.lua --db-driver=pgsql --
pgsql-host=192.168.56.28 --pgsql-port=26000   --pgsql-user=sbtest --pgsql-
password='openGauss@1234' --pgsql-db=sbtest --tables=10 --table-size=1000000 --threads=4 --
time=300   --report-interval=10 prepare
```

After the command is executed successfully, 10 tables are generated in the **sbtest** database. Each table contains 1000000 records (about 212 MB).

```
Initializing worker threads...
Creating table 'sbtest2'...
Inserting 1000000 records into 'sbtest2'
Creating table 'sbtest3'...
Creating table 'sbtest4'...
Inserting 1000000 records into 'sbtest3'
Inserting 1000000 records into 'sbtest4'
Creating table 'sbtest1'...
Inserting 1000000 records into 'sbtest1'
Creating a secondary index on 'sbtest3'...
Creating a secondary index on 'sbtest2'...
Creating a secondary index on 'sbtest1'...
Creating a secondary index on 'sbtest4'...
Creating table 'sbtest7'...
Inserting 1000000 records into 'sbtest7'
```

```
Creating table 'sbtest6'...
Inserting 1000000 records into 'sbtest6'
Creating table 'sbtest5'...
Inserting 1000000 records into 'sbtest5'
Creating table 'sbtest8'...
Inserting 1000000 records into 'sbtest8'
Creating a secondary index on 'sbtest7'...
Creating a secondary index on 'sbtest5'...
Creating a secondary index on 'sbtest6'...
Creating a secondary index on 'sbtest8'...
Creating table 'sbtest9'...
Inserting 1000000 records into 'sbtest9'
Creating table 'sbtest10'...
Inserting 1000000 records into 'sbtest10'
Creating a secondary index on 'sbtest9'...
Creating a secondary index on 'sbtest10'...
```

If you log in to the database and run the **\dt** command, the following information is displayed:

```
[omm@standalone ~]$ gsql -d sbtest -p 26000 -r
sbtest=# \dt+
                                    List of relations
 Schema |    Name    | Type  | Owner |  Size  |                  Storage                  | Description
--------+------------+-------+-------+--------+-------------------------------------------+-------------
 public | sbtest1    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest10 | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest2    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest3    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest4    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest5    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest6    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest7    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest8    | table | sbtest | 212 MB | {orientation=row,compression=no} |
 public | sbtest9    | table | sbtest | 212 MB | {orientation=row,compression=no} |
(10 rows)
```

2. Run phase: Start the pressure test.

```
[omm@standalone sysbench]$ sysbench /sysbench/share/sysbench/oltp_point_select.lua --db-
driver=pgsql --pgsql-host=192.168.56.28 --pgsql-port=26000   --pgsql-user=sbtest --pgsql-
password='openGauss@1234' --pgsql-db=sbtest --tables=10 --table-size=1000000 --threads=20 --
time=300   --report-interval=10 run
```

oltp_point_select.lua indicates the pressure test service scenario and **point_selects** indicates the service content, that is, point query. An error is reported during the command execution. The execution result is as follows:

```
Running the test with following options:
Number of threads: 20
Report intermediate results every 10 second(s)
Initializing random number generator from current time
```

```
Initializing worker threads...
FATAL: Connection to database failed: FATAL:  Too many clients already, current: 13, max_connections/reserved: 10/3.
...
```

In this case, the fault reporting the error message "too many clients already" or that threads cannot be created is reproduced.

Step 7 Rectify the fault.

Based on the lab environment, the solution is as follows:

1. Run **ulimit -u** to obtain the value of **max user processes** in the OS.

```
[root@standalone ~]# ulimit -u
unlimited
```

2. Use the following formula to calculate the minimum value and modify the **/etc/security/limits.conf** file:

Formula:

```
value=max(32768, Number of instances x 8192)
```

The number of instances is the number of database instances on the current server. In this example, the number is 1. Therefore, **value** is **32768**. Add the following two lines to the end of the **/etc/security/limits.conf** file:

```
* hard nproc 32768
* soft nproc 32768
```

3. Temporarily set **max_connections** to **5000** or a value less than or equal to the upper limit (that is, **262143**) and set **max_wal_senders** to **16**. Repeat Step 4 to modify the **postgresql.conf** file.
4. Restart the server.

```
[root@standalone ~]# reboot
```

5. Restart the database.

```
[omm@standalone ~]# gs_om -t start
```

6. Perform the sysbench pressure test again and verify the result.

```
[omm@standalone sysbench]$ sysbench /sysbench/share/sysbench/oltp_point_select.lua --db-driver=pgsql --pgsql-host=192.168.56.28 --pgsql-port=26000  --pgsql-user=sbtest --pgsql-password='openGauss@1234' --pgsql-db=sbtest --tables=10 --table-size=1000000 --threads=20 --time=300  --report-interval=10 run
```

In this case, the **sysbench** command is executed successfully. The command output is as follows:

```
Running the test with following options:
Number of threads: 20
Report intermediate results every 10 second(s)
Initializing random number generator from current time


Initializing worker threads...

Threads started!

[ 10s ] thds: 20 tps: 10848.37 qps: 10848.37 (r/w/o: 10848.37/0.00/0.00) lat (ms,95%): 4.18 err/s: 0.00 reconn/s: 0.00
[ 20s ] thds: 20 tps: 10760.10 qps: 10760.10 (r/w/o: 10760.10/0.00/0.00) lat (ms,95%): 4.25 err/s: 0.00 reconn/s: 0.00
[ 30s ] thds: 20 tps: 9451.52 qps: 9451.52 (r/w/o: 9451.52/0.00/0.00) lat (ms,95%): 5.09 err/s: 0.00 reconn/s: 0.00
[ 40s ] thds: 20 tps: 8072.83 qps: 8072.83 (r/w/o: 8072.83/0.00/0.00) lat (ms,95%): 5.88 err/s: 0.00 reconn/s: 0.00
[ 50s ] thds: 20 tps: 8884.81 qps: 8884.81 (r/w/o: 8884.81/0.00/0.00) lat (ms,95%): 5.09 err/s: 0.00 reconn/s: 0.00
...
```

8.2.2 Task 2: Failing to Connect to and Start Up a Database Due to a Network Fault

The following tries to reproduce the database startup failure and report the following network exception error: could not create listen socket for "localhost:26000".

Reproduction method: To reproduce the fault, install Nginx, change its port number to the database port number **26000**, stop the openGauss database, start the Nginx service, and then start the database. The database cannot be started or connected.

Troubleshooting method: Find the process that occupies the database port, change the process port to ensure that the process is normal, and restart the database service.

The procedure is as follows:

Step 1 Install Nginx.

Run the following command as the **root** user to install Nginx:

```
[root@standalone ~]# yum install -y nginx
```

Step 2 Change the Nginx port number to **26000**.

Go to the **/etc/nginx/** directory and change the port number to **26000** in the **nginx.conf** file. After the modification is successful, the following information is displayed:

```
server {
    listen       26000 default_server;
    listen       [::]:26000 default_server;
    server_name  _;
    root         /usr/share/nginx/html;
```

Step 3 Shut down the openGauss database.

Switch to user **omm** and run the following command to stop the database:

```
[omm@standalone ~]# gs_om -t stop
```

Step 4 Start the Nginx service.

Run the following command as the **root** user to start the Nginx service:

```
[root@standalone ~]# /usr/sbin/nginx -c /etc/nginx/nginx.conf
```

Check whether the Nginx process is started properly.

```
[root@standalone sbin]# ps -ef| grep nginx
root        58520        1   0 22:47 ?        00:00:00 nginx: master process /usr/sbin/nginx -c
/etc/nginx/nginx.conf
nginx       58521    58520   0 22:47 ?         00:00:00 nginx: worker process
nginx       58522    58520   0 22:47 ?         00:00:00 nginx: worker process
root        58655    17498   0 22:48 pts/2     00:00:00 grep --color=auto nginx
```

Check the port number occupied by Nginx.

```
[root@standalone sbin]# netstat -anop|grep 26000
tcp        0      0 0.0.0.0:26000           0.0.0.0:*               LISTEN      58520/nginx:
master   off (0.00/0/0)
tcp6       0      0 :::26000                :::*                    LISTEN      58520/nginx:
master   off (0.00/0/0)
```

Step 5 Start the database.

Switch to user **omm** and restart the database.

```
[omm@standalone ~]# gs_om -t start
```

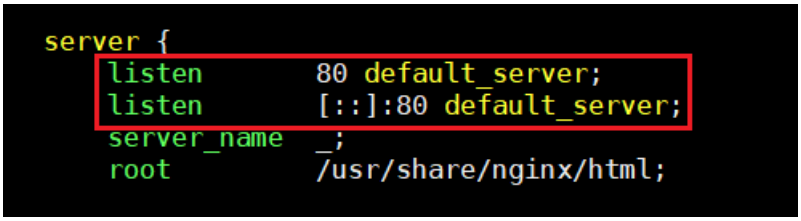
The startup fails. Some exception information is displayed as follows:

```
[omm@standalone ~]$ gs_om -t start
Starting cluster.
=========================================
[GAUSS-53600]: Can not start the database, the cmd is source /home/omm/.bashrc; python3
'/opt/huawei/install/om/script/local/StartInstance.py' -U omm -R /opt
/huawei/install/app -t 300 --security-mode=off,    Error:
[FAILURE] standalone:
[GAUSS-51607] : Failed to start instance. Error: Please check the gs_ctl log for failure details.
[20XX-XX-27 22:52:56.102][60744][][gs_ctl]: gs_ctl started,datadir is /opt/huawei/install/data/dn
[20XX-XX-27 22:52:56.116][60744][][gs_ctl]: port:26000 already in use. /proc/net/tcp:
...
20XX-XX-27 22:53:05.464 660432c8.1 [unknown] 140075594932160 [unknown] 0 dn_6001 42809    0
[BACKEND] FATAL:    could not create listen socket for "localhost:26000"
[20XX-XX-27 22:53:06.241][60744][][gs_ctl]: waitpid 60750 failed, exitstatus is 256, ret is 2
[20XX-XX-27 22:53:06.241][60744][][gs_ctl]: stopped waiting
[20XX-XX-27 22:53:06.241][60744][][gs_ctl]: could not start server
```

The troubleshooting method is as follows:

Step 6 Change the port number in the **nginx.conf** file to **80**.

Go to the **/etc/nginx/** directory and change the port number to **80** in the **nginx.conf** file as the **root** user. After the modification is successful, the following information is displayed:



Step 7 Reload the **nginx.conf** file.

Run the following command as user **root**:

```
[root@standalone ~]$ /usr/sbin/nginx -s reload
```

Check whether the Nginx port number is **80**. The result is as follows:

```
[root@standalone sbin]# netstat -anop|grep 80
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      58520/nginx:
master  off (0.00/0/0)
tcp6       0      0 :::80                   :::*                    LISTEN      58520/nginx:
master  off (0.00/0/0)
```

Step 8 Restart the database.

Switch to user **omm** and run the following command to start the database:

```
[omm@standalone ~]$ gs_om -t start
Starting cluster.
=========================================
[SUCCESS] standalone
...
Successfully started.
[omm@standalone ~]$
```

In this case, the database is started normally.

8.2.3 Task 3: An Error Reporting Insufficient Space Is Reported When TPC-H Data Is Imported Using a Script

This task tries to reproduce the tmpfs error "No space left on device" reported when TPC-H data is imported to the database using a script. After this error is reported, remaining data cannot be imported to the openGauss database. However, the gaussdb process will not be killed.

To solve this problem, temporarily increase the disk space of tmpfs or modify the import script. The procedure is as follows:

Step 1 Install Git as the **root** user.

```
[root@standalone sysbench-1.0.20]# yum install -y git
```

After the installation is successful, the following information is displayed:

```
...
Installed:
  git-2.27.0-3.oe1.aarch64              perl-TermReadKey-2.38-2.oe1.aarch64              perl-Error-
1:0.17028-1.oe1.noarch              perl-Git-2.27.0-3.oe1.noarch

Complete!
```

Step 2 Switch to user **omm** and download the test dataset from the **/home/omm/** directory.

```
git clone https://gitee.com/xzp-blog/tpch-kit.git
```

After the download is complete, the **tpch-kit** directory is displayed.

Step 3 Go to the **dbgen** directory and generate the makefile.

```
cd /home/omm/tpch-kit/dbgen/
make -f Makefile
```

Step 4 Connect to the openGauss database and create a **tpch** database.

```
[omm@standalone ~] $ gsql -d postgres -p 26000 -r
openGauss=# CREATE DATABASE tpch;
openGauss=# \q
```

Step 5 Create eight test tables.

```
gsql -d tpch -f dss.ddl -p 26000
```

If the command is successfully executed, the following information is displayed:

```
[omm@standalone dbgen]$ gsql -d tpch -f dss.ddl -p 26000
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
total time: 30  ms
```

After logging in to the **tpch** database, you can see the following eight tables:

```
        List of relations
 Schema |     Name     | Type  | Owner |               Storage
```

```
--------+----------+-------+-------+--------------------------------
 public | customer | table | omm   | {orientation=row,compression=no}
 public | lineitem | table | omm   | {orientation=row,compression=no}
 public | nation   | table | omm   | {orientation=row,compression=no}
 public | orders   | table | omm   | {orientation=row,compression=no}
 public | part     | table | omm   | {orientation=row,compression=no}
 public | partsupp | table | omm   | {orientation=row,compression=no}
 public | region   | table | omm   | {orientation=row,compression=no}
 public | supplier | table | omm   | {orientation=row,compression=no}
```

Step 6 Generate test data for the eight tables.

This step is critical to reproducing the error "No space left on device". The total disk size in my lab environment is 40 GB, and some of them has been occupied by the database. Therefore, only 10 GB test data is generated. You can change the value of **-s** in the following command to generate a test dataset based on your lab environment. The value of **-s** is an integer, for example, **1** indicates 1 GB and **10** indicates 10 GB.

```
cd /home/omm/tpch-kit/dbgen/
./dbgen -vf -s 10
```

After the command is successfully executed, the following information is displayed:

```
TPC-H Population Generator (Version 2.17.3)
Copyright Transaction Processing Performance Council 1994 - 2010
Generating data for suppliers table/
Preloading text ... 100%
done.
Generating data for customers tabledone.
Generating data for orders/lineitem tablesdone.
Generating data for part/partsupplier tablesdone.
Generating data for nation tabledone.
Generating data for region tabledone.
```

Step 7 Compile the data import script **LoadData.sh**.

Go to the **/home/omm/tpch-kit/dbgen** directory, create the **LoadData.sh** file, and add the following content to the file:

```
for i in `ls *.tbl`; do
  table=${i/.tbl/}
  echo "Loading $table..."
  sed 's/|$//' $i > /tmp/$i
  gsql tpch -p 26000 -q -c "TRUNCATE $table"
  gsql tpch -p 26000 -c "\\copy $table FROM '/tmp/$i' CSV DELIMITER '|'"
done
```

Save the modification and exit. Grant the execute permission on the **LoadData.sh** file and run the file.

```
[omm@standalone dbgen]$ chmod +x LoadData.sh   #d
[omm@standalone dbgen]$ sh LoadData.sh
```

Note: During the command execution, the error message "No space left on device" is displayed. The details are as follows:

```
[omm@standalone dbgen]$ sh LoadData.sh
Loading customer...
Loading lineitem...
sed: couldn't write 144 items to stdout: No space left on device
ERROR:  missing data for column "l_shipmode"
CONTEXT:  COPY lineitem, line 12154283:
"12157060|683089|33102|3|16|17152.80|0.05|0.03|N|O|1998-07-08|1998-06-18|1998-07-14|TAKE BACK
"
Loading nation...
sed: couldn't flush stdout: No space left on device
Loading orders...
sed: couldn't write 134 items to stdout: No space left on device
Loading partsupp...
...
sed: couldn't write 171 items to stdout: No space left on device
```

Step 8 Check the disk space.

Create an SSH session, log in to the system as the **root** user, and run the **df -h** command to check the current disk space usage.

```
[root@standalone ~]# df -h
Filesystem                Size  Used Avail Use% Mounted on
devtmpfs                  1.7G     0  1.7G   0% /dev
tmpfs                     1.7G   12K  1.7G   1% /dev/shm
tmpfs                     1.7G   33M  1.7G   2% /run
tmpfs                     1.7G     0  1.7G   0% /sys/fs/cgroup
/dev/mapper/openeuler-root  35G   19G   15G  57% /
tmpfs                     1.7G  1.7G     0 100% /tmp
/dev/sda1                 976M  146M  763M  17% /boot
tmpfs                     343M     0  343M   0% /run/user/0
[root@standalone ~]#
```

In the preceding information, the total disk space of the **/tmp** directory is 1.7 GB, the used disk space is 1.7 GB, and the usage is 100%. Therefore, you may need to reallocate disk space to the **/tmp** directory or add disks to solve the problem. In the test environment, the resources are limited. Therefore, you can change the size of the **/tmp** directory and create the **NewLoadData.sh** script to solve the problem. Perform either Step 9 or Step 10 to solve the problem.

Step 9 Method 1: Create data import script **NewLoadData.sh**.

Switch to user **omm**, create the **NewLoadData.sh** script, and add the following content:

```
#!/bin/bash
```

```
for i in `ls *.tbl`; do
  table=${i/.tbl/}
  echo "Loading $table..."
  gsql tpch -p 26000 -q -c "TRUNCATE $table"
  gsql tpch -p 26000 -c "\\copy $table FROM './$i' CSV DELIMITER '|'"
  rm -rf ./$i
done
```

Run the script again. The command output is as follows:

```
[omm@standalone dbgen]$ sh NewLoadData.sh
Loading customer...
Loading lineitem...
Loading nation...
Loading orders...
Loading partsupp...
Loading part...
Loading region...
...
```

Log in to the **tpch** database and check the size of each table. The details are as follows:

```
[omm@standalone ~]$ gsql -d tpch -p 26000 -r
tpch=# \d+
                                                List of relations
 Schema |     Name     | Type  | Owner |      Size      |                  Storage                   |
Description
--------+----------+-------+-------+------------+-----------------------------------+-------------
 public | customer | table | omm     | 281 MB       | {orientation=row,compression=no} |
 public | lineitem   | table | omm     | 9734 MB      | {orientation=row,compression=no} |
 public | nation     | table | omm     | 8192 bytes   | {orientation=row,compression=no} |
 public | orders     | table | omm     | 2159 MB      | {orientation=row,compression=no} |
 public | part       | table | omm     | 321 MB       | {orientation=row,compression=no} |
 public | partsupp   | table | omm     | 1365 MB      | {orientation=row,compression=no} |
 public | region     | table | omm     | 8192 bytes   | {orientation=row,compression=no} |
 public | supplier   | table | omm     | 17 MB        | {orientation=row,compression=no} |
(8 rows)
```

Step 10 Method 2: Reallocate the disk space of the **/tmp** directory and restart the server.

Switch to the **root** user and clear the **/tmp** directory.

```
[root@standalone ~]# rm -rf /tmp/*
```

Run the following command to edit the **/etc/fstab** file:

```
[root@standalone ~]# vim /etc/fstab
```

Add the following content to the end of the file to reallocate space (15 GB) to the **/tmp** directory:

```
tmpfs   /tmp    tmpfs   nodev,nosuid,size=15G   0   0
```

Change the space of the **/tmp** file to 15 GB. This operation takes effect only a restart.

Step 11 After the server is restarted, modify the **LoadData.sh** script in Step 7 and run the script.

After the server is restarted, run the following command to check the size of the **/tmp** directory:

```
df -h
```

The result is as follows:

```
[root@standalone ~]# df -h
Filesystem                Size  Used Avail Use% Mounted on
devtmpfs                  1.7G     0  1.7G   0% /dev
tmpfs                     1.7G     0  1.7G   0% /dev/shm
tmpfs                     1.7G  8.7M  1.7G   1% /run
tmpfs                     1.7G     0  1.7G   0% /sys/fs/cgroup
/dev/mapper/openeuler-root 35G   19G   15G  57% /
tmpfs                      15G     0   15G   0% /tmp
/dev/sda1                 976M  146M  763M  17% /boot
tmpfs                     343M     0  343M   0% /run/user/0
```

As shown in the command output, the size of the **/tmp** directory has been changed to 15 GB.

Switch to user **omm** and modify the **LoadData.sh** script as follows:

```
for i in `ls *.tbl`; do
  table=${i/.tbl/}
  echo "Loading $table..."
  mv ./$i /tmp/$i
  gsql tpch -p 26000 -q -c "TRUNCATE $table"
  gsql tpch -p 26000 -c "\\copy $table FROM '/tmp/$i' CSV DELIMITER '|'"
  rm -rf /tmp/$i
done
```

Run the script.

```
[omm@standalone dbgen]$ sh LoadData.sh
```

The execution result is the same as that in Step 9, and the size of the tables in the **tpch** database is the same as that in Step 9.

8.2.4 Task 4: Core Dump Occurs Due to Insufficient Disk Space During Write

This task tries to reproduce the core dump occurred in openGauss due to insufficient disk space or disk usage reaching 100% during write of a large amount of data to the database. As a result, the gaussdb process is killed, the session that is connecting to the

database is disconnected, and a file similar to **core-gaussdb-14734-2024_03_28_09_57_39-bbox.lz4** is generated in the DN directory.

When this problem occurs, you can clear useless data or add disks to solve it. This task clears the useless data to save the disk space.

Step 1 Repeat Step 5 in task 3 to generate about 10 GB test data.

Switch to user **omm** and run the following command to generate test data:

```
cd /home/omm/tpch-kit/dbgen/
./dbgen -vf -s 10
```

Step 2 Log in to the database, create the **t1** table, and import data. The problem recurs.

```
[omm@standalone dbgen]$ gsql -d tpch -p 26000 -r          # Log in to the database.
tpch=# create table t1 as select * from lineitem where 1 < 0;   # Create the t1 table.
tpch=# \copy t1 FROM '/home/omm/tpch-kit/dbgen/lineitem.tbl' CSV DELIMITER '|';     # Import data.
```

During data import, the following exception occurs:

```
ERROR:  could not extend file "base/32774/65624.5": wrote only 4096 of 8192 bytes at block 752374
HINT:  Check free disk space.
CONTEXT:  COPY t1, line 6234051: "6233319|775978|25979|2|34|69833.96|0.00|0.01|N|O|1997-11-22|1998-01-07|1997-11-30|COLLECT COD|AIR|nts use furiously furiously f"
tpch=# select * from t1;
no connection to the server
The connection to the server was lost. Attempting reset: Failed.
```

The **core-gaussdb-14734-2024_03_28_09_57_39-bbox.lz4** file is generated in the **/opt/huawei/install/data/dn** directory. That it, the disk space of the directory where the database data is stored is used up during data import. As a result, the gaussdb process is killed.

If you run the startup command to restart the database, the following error will be displayed:

```
[omm@standalone dn]$ gs_om -t start
[GAUSS-50205] : Failed to write log file /var/log/omm/omm/om/gs_om-2024-03-28_100603.log.
Error:
[Errno 28] No space left on device
```

The disk space usage of the directory where the database data is stored is 100%, as shown in the following:

```
[omm@standalone dn]$ df -h
Filesystem                  Size  Used Avail Use% Mounted on
devtmpfs                    3.7G     0  3.7G   0% /dev
tmpfs                       3.7G   12K  3.7G   1% /dev/shm
tmpfs                       3.7G   17M  3.7G   1% /run
tmpfs                       3.7G     0  3.7G   0% /sys/fs/cgroup
/dev/mapper/openeuler-root   35G   33G     0 100% /
tmpfs                       15G     0   15G   0% /tmp
/dev/sda1                  976M  146M  763M  17% /boot
```

```
tmpfs                    747M     0  747M   0% /run/user/0
```

Step 3 Clear useless data to ensure that the database can be restored.

The database cannot be started because the disk space is used up. In this case, clear the test data generated in Step 2. Go to the **/home/omm/tpch-kit/dbgen** directory where the test data is stored and delete the data.

```
[omm@standalone ~]$ cd /home/omm/tpch-kit/dbgen
[omm@standalone dbgen]$ rm -rf *.tbl     # Delete all generated test data.
```

The disk usage of the directory where the database data is stored is 85%, as shown in the following:

```
[omm@standalone dbgen]$ df -h
Filesystem                Size  Used Avail Use% Mounted on
devtmpfs                  3.7G     0  3.7G   0% /dev
tmpfs                     3.7G   12K  3.7G   1% /dev/shm
tmpfs                     3.7G   17M  3.7G   1% /run
tmpfs                     3.7G     0  3.7G   0% /sys/fs/cgroup
/dev/mapper/openeuler-root 35G   28G  5.2G  85% /
tmpfs                      15G     0   15G   0% /tmp
/dev/sda1                 976M  146M  763M  17% /boot
tmpfs                     747M     0  747M   0% /run/user/0
```

Run the following command to restart the database:

```
[omm@standalone dbgen]$ gs_om -t start
Starting cluster.
=========================================
[SUCCESS] standalone
...
=========================================
Successfully started.
```

The database can be started normally. Log in to the database and perform simple addition, deletion, and modification operations.

```
[omm@standalone dbgen]$ gsql -d tpch -p 26000 -r
gsql ((openGauss 5.0.0 build a07d57c3) compiled at 2023-03-29 03:09:38 commit 0 last mr   )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

tpch=# create table t3(id int,name varchar(50));
CREATE TABLE
tpch=#
tpch=# insert into t3 values(1,'test');
INSERT 0 1
tpch=# update t3 set name='hello' where id=1;
UPDATE 1
tpch=#
```

No core dump occurs. The problem caused by insufficient disk space is solved.

-End-